

MMALS_Prototype_1_1_RC2h_RC1b_Anchored_Conservative_Selector_

May 26, 2026

1 MMALS Prototype v1.1-RC2h — RC1b-Anchored Conservative Selector Harness

Mycelium-Inspired Mutualistic Adaptive Learning Systems

Status: strict no-leakage policy-selection harness with Hydro/fluid metrics preserved as audit-only, and an RC1b-anchored conservative selector. RC2H treats the historically robust RC1b context-gap policy as the default deployable anchor and allows overrides only when validation-memory evidence is strong enough.

1.1 1. Why RC2H exists

RC2G evidence showed that the fungal backbone and the fixed RC1b / context-gap policy remain strong, but a free validation selector can still choose weaker policies. RC2H therefore changes the selection question.

RC2G asked:

Which candidate has the highest validation-memory proxy score?

RC2H asks:

Is there enough no-leakage validation evidence to override the known-good RC1b anchor?

The default deployable anchor is the RC1b guarded context-gap family, represented by `context_gap_selected` and, as a secondary compatible anchor, `context_temp_blend_selected_head_guard_035`. Another policy may replace the anchor only if it beats it by a meaningful validation margin and passes weak-class, min-task, tri-boundary, zero-class and no-repair safety checks. Hydro remains audit-only by default.

1.2 RC2H change log from RC2G

- Renamed package/output paths to `MMALS_Prototype_1_1_RC2h_RC1b_Anchored_Conservative_Selector_`
- Default profile is `RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT`.
- Preserved strict no-leakage candidate selection.
- Kept Drive persistence under a clean RC2H version/run folder.
- Kept Hydro as audit-only by default.
- Preserved RC2G tri-boundary diagnostics and readout calibration infrastructure.

- Added RC1b anchor selection logic: keep `context_gap_selected` / guarded context-gap by default.
- Added conservative override gates: score margin, accuracy margin, class-2/class-4/class-5 floors, min-task/min-class floors, tri-boundary minimum, zero-seen-class rejection and no-repair veto.
- Added explicit audit columns for anchor kept, override applied, override rejected reason, anchor score and override margin.

1.3 0. Optional install cell for Colab

Most Colab runtimes already include the required packages. Uncomment only if needed.

```
[1]: # !pip -q install torch torchvision pandas numpy matplotlib
```

1.4 1. Imports and reproducibility

This block imports the scientific Python stack, selects CPU/GPU, and defines reproducibility helpers.

```
[2]: import math
import copy
import random
import gc
import time
import shutil
import sys
import os
import json
import hashlib
import uuid
import platform
import contextlib
from datetime import datetime, timezone
from pathlib import Path
from typing import Dict, List, Tuple, Optional, Any

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import TensorDataset, DataLoader
from IPython.display import display

# Robust torchvision import. Some runtimes have torch/torchvision NMS mismatch.
try:
```

```

    from torchvision import datasets, transforms
except RuntimeError as e:
    if "torchvision::nms" not in str(e):
        raise
    for name in list(sys.modules):
        if name.startswith("torchvision"):
            del sys.modules[name]
    try:
        lib = torch.library.Library("torchvision", "DEF")
        lib.define("nms(Tensor dets, Tensor scores, float iou_threshold) ->␣
↳Tensor")
    except Exception:
        pass
    from torchvision import datasets, transforms

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("DEVICE:", DEVICE)
if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))

def set_seed(seed: int):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)

def clear_memory():
    gc.collect()
    if torch.cuda.is_available():
        torch.cuda.empty_cache()

def make_loader(ds, batch_size=128, shuffle=True):
    return DataLoader(ds, batch_size=batch_size, shuffle=shuffle,␣
↳num_workers=0, pin_memory=False)

def count_parameters(model):
    return sum(p.numel() for p in model.parameters() if p.requires_grad)

def total_parameters(model):
    return sum(p.numel() for p in model.parameters())

def model_size_mb(model, bytes_per_param=4, trainable_only=True):
    n = count_parameters(model) if trainable_only else total_parameters(model)
    return n * bytes_per_param / (1024 ** 2)

```

DEVICE: cpu

1.5 2. Configuration

Default mode is `smoke` for first execution checks. Switch to `robust` after smoke passes. This notebook keeps the existing options `MNIST`, `FashionMNIST`, and `RotatedMNIST`, and adds the new `PermutedMNIST` option without removing the previous ones.

```
[3]: RUN_MODE = "evidence" # first validate execution; set to "robust" after smoke
    ↪ passes
    DATASET_NAME = "FashionMNIST" # "MNIST", "FashionMNIST", "RotatedMNIST", or
    ↪ "PermutedMNIST"
    DATA_ROOT = "./data"

# =====
# RC2h branch control
# =====
# Choose exactly one profile.
# Recommended default: RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT.
#
# 1) RC2B_STRICT
#   - Reproduce the previous strict no-leakage policy-selection branch.
#   - Hydro disabled.
#   - RC2c risk-aware selector disabled.
#
# 2) RC2H_RC1B_ANCHORED_CONSERVATIVE_NO_HYDRO
#   - RC2H RC1b-anchored conservative selector enabled.
#   - Hydro disabled.
#   - Use this to isolate selector effects.
#
# 3) RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT [DEFAULT]
#   - RC2H RC1b-anchored conservative selector enabled.
#   - Hydro metrics are computed on validation memory only.
#   - No Hydro training loss and no Hydro selection penalty.
#
# 4) RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_STABILITY_ABLATION [EXPERIMENTAL]
#   - RC2H selector enabled.
#   - Hydro audit enabled.
#   - Validation selection score includes a small drift/turbulence penalty.
#   - Report separately; not the default deployable claim.
EXPERIMENT_PROFILE = "RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT"
VALID_EXPERIMENT_PROFILES = [
    "RC2B_STRICT",
    "RC2H_RC1B_ANCHORED_CONSERVATIVE_NO_HYDRO",
    "RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT",
    "RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_STABILITY_ABLATION",
    "RC2G_TRI_BOUNDARY_EQUILIBRIUM_NO_HYDRO",
    "RC2G_TRI_BOUNDARY_EQUILIBRIUM_HYDRO_AUDIT",
    "RC2G_TRI_BOUNDARY_EQUILIBRIUM_HYDRO_STABILITY_ABLATION",
    "RC2D_GATE_ALIGNED_NO_HYDRO",
```

```

"RC2D_GATE_ALIGNED_HYDRO_AUDIT",
"RC2D_GATE_ALIGNED_HYDRO_STABILITY_ABLATION",
"RC2C_RISK_AWARE_NO_HYDRO",
"RC2C_RISK_AWARE_HYDRO_AUDIT",
"RC2C_RISK_AWARE_HYDRO_STABILITY_ABLATION",
# Backward-compatible aliases from the merged Hydro branch.
"RC3_HYDRO_AUDIT",
"RC3_HYDRO_STABILITY_SELECTED",
]
if EXPERIMENT_PROFILE not in VALID_EXPERIMENT_PROFILES:
    raise ValueError(f"EXPERIMENT_PROFILE must be one of_
↳{VALID_EXPERIMENT_PROFILES}")

# Low-level manual override.
# Keep AUTO_CONFIGURE_FROM_PROFILE=True unless you are deliberately doing an_
↳ablation.
AUTO_CONFIGURE_FROM_PROFILE = True

# Manual values are used only if AUTO_CONFIGURE_FROM_PROFILE=False.
POLICY_BRANCH = "RC2h" # "RC2b", "RC2c", "RC2d", "RC2e",_
↳"RC2g", or "RC2h"
ACTIVATE_RC2C_RISK_AWARE_SELECTION = True
ACTIVATE_HYDRO_AUDIT = True # compute audit metrics
ACTIVATE_HYDRO_REGULARIZER = False # add viscosity loss during training
ACTIVATE_HYDRO_SELECTION = False # use Hydro penalty in validation_
↳selection
HYDRO_REGULARIZER_MODE = "off" # "off", "viscous",_
↳"high_gain_viscous"
HYDRO_POLICY_MODE = "audit_only" # "off", "audit_only", or_
↳"stability_aware"

if AUTO_CONFIGURE_FROM_PROFILE:
    if EXPERIMENT_PROFILE == "RC2B_STRICT":
        POLICY_BRANCH = "RC2b"
        ACTIVATE_RC2C_RISK_AWARE_SELECTION = False
        ACTIVATE_HYDRO_AUDIT = False
        ACTIVATE_HYDRO_REGULARIZER = False
        ACTIVATE_HYDRO_SELECTION = False
        HYDRO_REGULARIZER_MODE = "off"
        HYDRO_POLICY_MODE = "off"
    elif EXPERIMENT_PROFILE == "RC2H_RC1B_ANCHORED_CONSERVATIVE_NO_HYDRO":
        POLICY_BRANCH = "RC2h"
        ACTIVATE_RC2C_RISK_AWARE_SELECTION = True
        ACTIVATE_HYDRO_AUDIT = False
        ACTIVATE_HYDRO_REGULARIZER = False
        ACTIVATE_HYDRO_SELECTION = False

```

```

HYDRO_REGULARIZER_MODE = "off"
HYDRO_POLICY_MODE = "off"
elif EXPERIMENT_PROFILE == "RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT":
    POLICY_BRANCH = "RC2h"
    ACTIVATE_RC2C_RISK_AWARE_SELECTION = True
    ACTIVATE_HYDRO_AUDIT = True
    ACTIVATE_HYDRO_REGULARIZER = False
    ACTIVATE_HYDRO_SELECTION = False
    HYDRO_REGULARIZER_MODE = "off"
    HYDRO_POLICY_MODE = "audit_only"
elif EXPERIMENT_PROFILE == □
↳"RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_STABILITY_ABLATION":
    POLICY_BRANCH = "RC2h"
    ACTIVATE_RC2C_RISK_AWARE_SELECTION = True
    ACTIVATE_HYDRO_AUDIT = True
    ACTIVATE_HYDRO_REGULARIZER = False
    ACTIVATE_HYDRO_SELECTION = True
    HYDRO_REGULARIZER_MODE = "off"
    HYDRO_POLICY_MODE = "stability_aware"
elif EXPERIMENT_PROFILE == "RC2G_TRI_BOUNDARY_EQUILIBRIUM_NO_HYDRO":
    POLICY_BRANCH = "RC2g"
    ACTIVATE_RC2C_RISK_AWARE_SELECTION = True
    ACTIVATE_HYDRO_AUDIT = False
    ACTIVATE_HYDRO_REGULARIZER = False
    ACTIVATE_HYDRO_SELECTION = False
    HYDRO_REGULARIZER_MODE = "off"
    HYDRO_POLICY_MODE = "off"
elif EXPERIMENT_PROFILE == "RC2G_TRI_BOUNDARY_EQUILIBRIUM_HYDRO_AUDIT":
    POLICY_BRANCH = "RC2g"
    ACTIVATE_RC2C_RISK_AWARE_SELECTION = True
    ACTIVATE_HYDRO_AUDIT = True
    ACTIVATE_HYDRO_REGULARIZER = False
    ACTIVATE_HYDRO_SELECTION = False
    HYDRO_REGULARIZER_MODE = "off"
    HYDRO_POLICY_MODE = "audit_only"
elif EXPERIMENT_PROFILE == □
↳"RC2G_TRI_BOUNDARY_EQUILIBRIUM_HYDRO_STABILITY_ABLATION":
    POLICY_BRANCH = "RC2g"
    ACTIVATE_RC2C_RISK_AWARE_SELECTION = True
    ACTIVATE_HYDRO_AUDIT = True
    ACTIVATE_HYDRO_REGULARIZER = False
    ACTIVATE_HYDRO_SELECTION = True
    HYDRO_REGULARIZER_MODE = "off"
    HYDRO_POLICY_MODE = "stability_aware"
elif EXPERIMENT_PROFILE == "RC2D_GATE_ALIGNED_NO_HYDRO":
    POLICY_BRANCH = "RC2d"
    ACTIVATE_RC2C_RISK_AWARE_SELECTION = True

```

```

    ACTIVATE_HYDRO_AUDIT = False
    ACTIVATE_HYDRO_REGULARIZER = False
    ACTIVATE_HYDRO_SELECTION = False
    HYDRO_REGULARIZER_MODE = "off"
    HYDRO_POLICY_MODE = "off"
elif EXPERIMENT_PROFILE == "RC2D_GATE_ALIGNED_HYDRO_AUDIT":
    POLICY_BRANCH = "RC2d"
    ACTIVATE_RC2C_RISK_AWARE_SELECTION = True
    ACTIVATE_HYDRO_AUDIT = True
    ACTIVATE_HYDRO_REGULARIZER = False
    ACTIVATE_HYDRO_SELECTION = False
    HYDRO_REGULARIZER_MODE = "off"
    HYDRO_POLICY_MODE = "audit_only"
elif EXPERIMENT_PROFILE == "RC2D_GATE_ALIGNED_HYDRO_STABILITY_ABLATION":
    POLICY_BRANCH = "RC2d"
    ACTIVATE_RC2C_RISK_AWARE_SELECTION = True
    ACTIVATE_HYDRO_AUDIT = True
    ACTIVATE_HYDRO_REGULARIZER = False
    ACTIVATE_HYDRO_SELECTION = True
    HYDRO_REGULARIZER_MODE = "off"
    HYDRO_POLICY_MODE = "stability_aware"
elif EXPERIMENT_PROFILE == "RC2C_RISK_AWARE_NO_HYDRO":
    POLICY_BRANCH = "RC2c"
    ACTIVATE_RC2C_RISK_AWARE_SELECTION = True
    ACTIVATE_HYDRO_AUDIT = False
    ACTIVATE_HYDRO_REGULARIZER = False
    ACTIVATE_HYDRO_SELECTION = False
    HYDRO_REGULARIZER_MODE = "off"
    HYDRO_POLICY_MODE = "off"
elif EXPERIMENT_PROFILE == "RC2C_RISK_AWARE_HYDRO_AUDIT":
    POLICY_BRANCH = "RC2c"
    ACTIVATE_RC2C_RISK_AWARE_SELECTION = True
    ACTIVATE_HYDRO_AUDIT = True
    ACTIVATE_HYDRO_REGULARIZER = False
    ACTIVATE_HYDRO_SELECTION = False
    HYDRO_REGULARIZER_MODE = "off"
    HYDRO_POLICY_MODE = "audit_only"
elif EXPERIMENT_PROFILE == "RC2C_RISK_AWARE_HYDRO_STABILITY_ABLATION":
    POLICY_BRANCH = "RC2c"
    ACTIVATE_RC2C_RISK_AWARE_SELECTION = True
    ACTIVATE_HYDRO_AUDIT = True
    ACTIVATE_HYDRO_REGULARIZER = False
    ACTIVATE_HYDRO_SELECTION = True
    HYDRO_REGULARIZER_MODE = "off"
    HYDRO_POLICY_MODE = "stability_aware"
elif EXPERIMENT_PROFILE == "RC3_HYDRO_AUDIT":
    POLICY_BRANCH = "RC2g"

```

```

    ACTIVATE_RC2C_RISK_AWARE_SELECTION = True
    ACTIVATE_HYDRO_AUDIT = True
    ACTIVATE_HYDRO_REGULARIZER = False
    ACTIVATE_HYDRO_SELECTION = False
    HYDRO_REGULARIZER_MODE = "off"
    HYDRO_POLICY_MODE = "audit_only"
elif EXPERIMENT_PROFILE == "RC3_HYDRO_STABILITY_SELECTED":
    POLICY_BRANCH = "RC2g"
    ACTIVATE_RC2C_RISK_AWARE_SELECTION = True
    ACTIVATE_HYDRO_AUDIT = True
    ACTIVATE_HYDRO_REGULARIZER = False
    ACTIVATE_HYDRO_SELECTION = True
    HYDRO_REGULARIZER_MODE = "off"
    HYDRO_POLICY_MODE = "stability_aware"

# Hydro branch constants. Selection weights are used only in STABILITY_SELECTED.
HYDRO_REGULARIZER_COEFF = 0.10
HYDRO_HIGH_GAIN_IMPORTANCE_THRESHOLD = 0.55
HYDRO_SELECTION_TURBULENCE_WEIGHT = 0.05 if ACTIVATE_HYDRO_SELECTION else 0.0
HYDRO_SELECTION_LATENT_DRIFT_WEIGHT = 0.20 if ACTIVATE_HYDRO_SELECTION else 0.0
HYDRO_SELECTION_ROUTE_DRIFT_WEIGHT = 0.05 if ACTIVATE_HYDRO_SELECTION else 0.0

NOTEBOOK_BASENAME = _
↳"MMALS_Prototype_1_1_RC2h_RC1b_Anchored_Conservative_Selector_Harness_Colab"
NOTEBOOK_FILENAME = f"{NOTEBOOK_BASENAME}.ipynb"
RESULTS_DIR = Path("./
↳mmals_v11_rc2h_rc1b_anchored_conservative_selector_results")
RESULTS_DIR.mkdir(parents=True, exist_ok=True)

if RUN_MODE == "smoke":
    TRAIN_PER_CLASS = 200
    TEST_PER_CLASS = 100
    SEEDS = [0]
    EPOCHS_PER_TASK = 2
    MEMORY_PER_TASK = 96
    MEMORY_PER_CLASS = 96
    BATCH_SIZE = 128
    LABEL_NOISE_RATE = 0.05
    INPUT_NOISE_STD = 0.03
    CONTEXT_NOISE_RATE = 0.15
    PROBE_EPOCHS = 10
    PROBE_MAX_PER_TASK = 600
    READOUT_TRAIN_EPOCHS = 2
    READOUT_TRAIN_STEPS = 10
elif RUN_MODE == "evidence":
    TRAIN_PER_CLASS = 1200
    TEST_PER_CLASS = 400

```



```

SEEDS = [0, 1, 2]
EPOCHS_PER_TASK = 8
MEMORY_PER_TASK = 256
MEMORY_PER_CLASS = 256
BATCH_SIZE = 128
LABEL_NOISE_RATE = 0.10
INPUT_NOISE_STD = 0.05
CONTEXT_NOISE_RATE = 0.20
PROBE_EPOCHS = 25
PROBE_MAX_PER_TASK = 2000
READOUT_TRAIN_EPOCHS = 5
READOUT_TRAIN_STEPS = 50
elif RUN_MODE == "robust":
    TRAIN_PER_CLASS = 1200
    TEST_PER_CLASS = 400
    SEEDS = list(range(5))
    EPOCHS_PER_TASK = 8
    MEMORY_PER_TASK = 256
    MEMORY_PER_CLASS = 256
    BATCH_SIZE = 128
    LABEL_NOISE_RATE = 0.10
    INPUT_NOISE_STD = 0.05
    CONTEXT_NOISE_RATE = 0.20
    PROBE_EPOCHS = 25
    PROBE_MAX_PER_TASK = 2000
    READOUT_TRAIN_EPOCHS = 5
    READOUT_TRAIN_STEPS = 50
elif RUN_MODE == "benchmark":
    TRAIN_PER_CLASS = 1200
    TEST_PER_CLASS = 400
    SEEDS = list(range(5))
    EPOCHS_PER_TASK = 8
    MEMORY_PER_TASK = 256
    MEMORY_PER_CLASS = 256
    BATCH_SIZE = 128
    LABEL_NOISE_RATE = 0.10
    INPUT_NOISE_STD = 0.05
    CONTEXT_NOISE_RATE = 0.20
    PROBE_EPOCHS = 25
    PROBE_MAX_PER_TASK = 2000
    READOUT_TRAIN_EPOCHS = 5
    READOUT_TRAIN_STEPS = 50
elif RUN_MODE == "paper":
    TRAIN_PER_CLASS = 3000
    TEST_PER_CLASS = 1000
    SEEDS = list(range(10))
    EPOCHS_PER_TASK = 15

```

```

MEMORY_PER_TASK = 512
MEMORY_PER_CLASS = 512
BATCH_SIZE = 128
LABEL_NOISE_RATE = 0.10
INPUT_NOISE_STD = 0.05
CONTEXT_NOISE_RATE = 0.20
PROBE_EPOCHS = 50
PROBE_MAX_PER_TASK = 5000
READOUT_TRAIN_EPOCHS = 10
READOUT_TRAIN_STEPS = 120
else:
    raise ValueError("RUN_MODE must be smoke, evidence, robust, benchmark, or_
↪paper")

PAIR_MODE = "overlap_chain"
N_TASKS = 5
ROTATION_ANGLES = [-30, -15, 0, 15, 30]
PERMUTATION_SEEDS = [100, 101, 102, 103, 104]
N_CONTEXTS = N_TASKS
N_HOSTS = 5
N_CLASSES = 6
INPUT_DIM = 28 * 28
HIDDEN_DIM = 256
LATENT_DIM = 64
ROUTE_HIDDEN_DIM = 128
TASK_EMB_DIM = 16
CONTEXT_FEAT_DIM = 32

LR = 1e-3
PROBE_LR = 1e-2
READOUT_LR = 3e-3
TEMPERATURE = 0.7
DISTILL_TEMPERATURE = 2.0
PROBE_DISTILL_TEMPERATURE = 2.0
CONTEXT_TEMPERATURE = 0.45
FRECHET_TEMPERATURE = 0.20
LATENT_FRECHET_TEMPERATURE = 0.35
CENTER_TEMPERATURE = 0.20

# Context calibration inherited from v0.14.x.
CONTEXT_SUP_START = 1.00
CONTEXT_SUP_END = 0.15
CONTEXT_SUP_GAMMA = 1.25
CONTEXT_LABEL_SMOOTHING = 0.03
LAMBDA_CONTEXT_CENTER = 0.35
LAMBDA_CONTEXT_ENTROPY = 0.010
LAMBDA_CONTEXT_MARGIN = 0.020

```

```

CONTEXT_MARGIN_TARGET = 0.35
LAMBDA_CONTEXT_REPLAY_CE = 0.50
LAMBDA_CONTEXT_REPLAY_CENTER = 0.20
LAMBDA_CONTEXT_TEACHER_KL = 0.30
LAMBDA_CONTEXT_FEATURE_DRIFT = 0.10
CONTEXT_DISTILL_TEMPERATURE = 2.0
CONTEXT_REPLAY_BATCH = 64
CONTEXT_TEMP_GRID = [0.25, 0.35, 0.45, 0.60, 0.80, 1.00, 1.25, 1.50, 2.00]
TEMP_CALIB_MAX_ITEMS = 128

# Prototype-first context posterior.
BATCH_BLEND_SINGLE = 0.30
BATCH_BLEND_LATENT = 0.55
BATCH_BLEND_FEATURE = 0.15
PROTO_FIRST_BLEND_SINGLE = 0.05
PROTO_FIRST_BLEND_LATENT = 0.75
PROTO_FIRST_BLEND_FEATURE = 0.20

# Replay and functional-memory controls.
BALANCED_REPLAY_WEIGHT = 1.00
BALANCED_REPLAY_BATCH = 128
BALANCED_REPLAY_USE_EVAL_CONTEXT = True
LAMBDA_ENTROPY = 0.002
LAMBDA_METABOLIC = 1e-4
PROTOTYPE_MAX_ITEMS = 256

# Readout/head audit controls.
HEAD_REPAIR_BATCH = 128
HEAD_MARGIN_TARGET = 0.25
HEAD_MARGIN_LAMBDA = 0.30
HARD_CLASSES = [2]
REFERENCE_CLASS_FOR_MARGIN = 4
READOUT_KL_WEIGHT = 0.35
READOUT_TEMP = 0.25
COSINE_SCALE_INIT = 12.0
VALIDATION_SPLIT_FRACTION = 0.25
VALIDATION_MAX_ITEMS_PER_CLASS = 128
VALIDATION_SCORE_CLASS2_WEIGHT = 0.20
VALIDATION_SCORE_MINCLASS_WEIGHT = 0.20
VALIDATION_SCORE_DAMAGE_WEIGHT = 1.00
VALIDATION_ACCEPTANCE_TOL = 0.005

# RC2c configurable strict no-leakage policy-selection controls.
# Compatibility note: the historical RC2B_* and RC3_* names are retained so
↳ inherited helper
# functions do not need to be renamed.
if POLICY_BRANCH == "RC2b":

```

```

    RC2C_METHOD = "paired_rc2b_validation_selected_policy"
elif POLICY_BRANCH == "RC2c":
    RC2C_METHOD = "paired_rc2c_risk_aware_policy"
elif POLICY_BRANCH == "RC2d":
    RC2C_METHOD = "paired_rc2d_gate_aligned_policy"
elif POLICY_BRANCH == "RC2g":
    RC2C_METHOD = "paired_rc2g_tri_boundary_equilibrium_policy"
elif POLICY_BRANCH == "RC2h":
    RC2C_METHOD = "paired_rc2h_rc1b_anchored_conservative_policy"
else:
    RC2C_METHOD = "paired_rc2g_tri_boundary_equilibrium_policy"

RC3_METHOD = RC2C_METHOD
RC2B_METHOD = RC2C_METHOD
RC2B_STRICT_NO_FINAL_TEST_SELECTION = True
RC2B_EVALUATE_ALL_CANDIDATES_AFTER_SELECTION = True
RC2B_POLICY_REPAIR_FRACTION = 0.50
RC2B_CONTEXT_SELECTION_FRACTION = 0.25
RC2B_POLICY_VALIDATION_FRACTION = 0.25
RC2B_SELECTION_TIE_TOL = 0.002
RC2B_COMPLEXITY_PENALTY = 0.001
RC2B_CLASS2_FLOOR = 0.80
RC2B_CLASS5_FLOOR = 0.90
RC2B_MIN_CLASS_FLOOR = 0.80
RC2B_VALIDATION_SCORE_CLASS5_WEIGHT = 0.05
RC2B_VALIDATION_SCORE_FLOOR_PENALTY = 2.00

# RC2c risk-aware selector controls.
RC2C_RISK_AWARE_SELECTOR_ENABLED = bool(ACTIVATE_RC2C_RISK_AWARE_SELECTION)
RC2C_CLASS2_FLOOR = 0.82
RC2C_MIN_CLASS_FLOOR = 0.82
RC2C_MIN_TASK_FLOOR = 0.82
RC2C_OLD_TASK_FLOOR = 0.82
RC2C_HARD_CLASS2_FLOOR = 0.80
RC2C_HARD_MIN_TASK_FLOOR = 0.80
RC2C_HARD_MIN_CLASS_FLOOR = 0.80
RC2C_CLASS2_FLOOR_PENALTY = 5.00
RC2C_MIN_CLASS_FLOOR_PENALTY = 4.00
RC2C_MIN_TASK_FLOOR_PENALTY = 4.00
RC2C_OLD_TASK_FLOOR_PENALTY = 3.00
RC2C_TASK_DISPERSION_PENALTY = 0.25
RC2C_NO_REPAIR_VETO_AFTER_TASK = 2
RC2C_NO_REPAIR_REQUIRED_MARGIN = 0.015
RC2C_NO_REPAIR_SOFT_PENALTY = 0.010
RC2C_GUARDED_HYSTERESIS_MARGIN = 0.003
RC2C_GUARDED_CONTEXT_BONUS = 0.001
RC2C_FLOOR_RISK_FALLBACK_ENABLED = True

```

```

RC2C_GUARDED_REFERENCE_VARIANTS = [
    "context_temp_blend_selected_head_guard_035",
    "context_gap_selected",
]

# RC2d gate-alignment selector controls.
# These are validation-memory-only controls; they do not touch final test data.
RC2D_SELECTOR_VERSION = "RC2d_gate_aligned_v1"
RC2D_GATE_ALIGNMENT_ENABLED = True
RC2D_AUX_PROXY_WEIGHT = 0.35          # auxiliary validation split score to
    ↪reduce one-split noise
RC2D_PRIMARY_PROXY_WEIGHT = 0.65
RC2D_PROXY_ACC_FLOOR = 0.68          # smoke guard; robust runs should exceed
    ↪this comfortably
RC2D_PROXY_ACC_FLOOR_PENALTY = 3.00
RC2D_CLASS5_FLOOR = 0.90
RC2D_HARD_CLASS5_FLOOR = 0.80
RC2D_CLASS5_FLOOR_PENALTY = 5.00
RC2D_CLASS2_RESCUE_MARGIN = 0.08
RC2D_CLASS5_RESCUE_MARGIN = 0.08
RC2D_TRUE_GUARD_RESCUE_SCORE_MARGIN = 0.020
RC2D_TRUE_GUARD_CLASS2_MARGIN = 0.05
RC2D_TRUE_GUARD_ACC_MARGIN = 0.010
RC2D_TRUE_GUARD_BONUS = 0.003
RC2D_CONTEXT_GUARD_BONUS = 0.001     # lower than RC2c to avoid
    ↪over-conservative context-gap selection
RC2D_GUARDED_HYSTERESIS_MARGIN = 0.003
RC2D_SELECTOR_FINAL_ALIGNMENT_TOL = 0.005
RC2D_CLASS_BALANCE_WEIGHT = 0.35
RC2D_MIN_TASK_WEIGHT = 0.35
RC2D_OLD_TASK_WEIGHT = 0.20
RC2D_TASK_STD_WEIGHT = 0.25

# RC2g NaN-safe / class-balanced selector controls.
# These are validation-memory / repair-memory-only controls.
RC2G_SELECTOR_VERSION = "RC2g_nan_safe_class_balanced_v1"
RC2G_NAN_SAFE_SCORING_ENABLED = True
RC2G_CLASS_AVAILABILITY_GATING_ENABLED = True
RC2G_EMERGENCY_SCORE_IF_ALL_NAN = True
RC2G_CLASS_BALANCED_READOUT_CALIBRATION_ENABLED = True
RC2G_CLASS_BALANCED_CALIB_STEPS = 12 if RUN_MODE == "smoke" else 30
RC2G_CLASS_BALANCED_CALIB_LR = 1.5e-3
RC2G_WEAK_CLASS_WEIGHT = 2.50
RC2G_MAX_CLASS_WEIGHT = 4.00
RC2G_CLASS_BALANCED_MARGIN_WEIGHT = 0.35
RC2G_SCORE_NEUTRAL_VALUE = 0.0

```

```

# RC2g tri-boundary equilibrium readout controls.
# These still use repair/policy-validation memory only.
RC2G_SELECTOR_VERSION = "RC2g_tri_boundary_class_balanced_v1"
RC2G_BOUNDARY_CLASSES = [2, 3, 4, 5]
RC2G_PROTECTED_CLASSES = [2, 4, 5]
RC2G_CLASS4_FLOOR = 0.45
RC2G_HARD_CLASS4_FLOOR = 0.05
RC2G_CLASS4_FLOOR_PENALTY = 6.00
RC2G_ZERO_SEEN_CLASS_REJECT_ENABLED = True
RC2G_ZERO_SEEN_CLASS_PENALTY = 4.00
RC2G_ZERO_CLASS_RESCUE_SCORE_MARGIN = 0.08
RC2G_BOUNDARY_MARGIN_TARGET = 0.20
RC2G_BOUNDARY_MARGIN_WEIGHT = 0.55
RC2G_CLASS4_WEIGHT = 3.50
RC2G_BOUNDARY_CLASS_WEIGHT = 2.75
RC2G_BOUNDARY_CALIB_STEPS = 18 if RUN_MODE == "smoke" else 40
RC2G_BOUNDARY_CALIB_LR = 1.2e-3

# Hydro audit/selection controls inherited from the v1.1 Hydro suite.
# Defaults are set through EXPERIMENT_PROFILE above.
RC3_HYDRO_AUDIT_ENABLED = bool(ACTIVATE_HYDRO_AUDIT)
RC3_HYDRO_REGULARIZER_ENABLED = bool(ACTIVATE_HYDRO_REGULARIZER)
RC3_HYDRO_SELECTION_ENABLED = bool(ACTIVATE_HYDRO_SELECTION)
RC3_HYDRO_REGULARIZER_MODE = HYDRO_REGULARIZER_MODE
RC3_HYDRO_POLICY_MODE = HYDRO_POLICY_MODE # "off", "audit_only", or ↵
↳ "stability_aware"
RC3_HYDRO_SELECTION_TURBULENCE_WEIGHT = HYDRO_SELECTION_TURBULENCE_WEIGHT
RC3_HYDRO_SELECTION_LATENT_DRIFT_WEIGHT = HYDRO_SELECTION_LATENT_DRIFT_WEIGHT
RC3_HYDRO_SELECTION_ROUTE_DRIFT_WEIGHT = HYDRO_SELECTION_ROUTE_DRIFT_WEIGHT
RC3_HYDRO_REGULARIZER_COEFF = HYDRO_REGULARIZER_COEFF
RC3_HYDRO_HIGH_GAIN_IMPORTANCE_THRESHOLD = HYDRO_HIGH_GAIN_IMPORTANCE_THRESHOLD
RC3_HYDRO_BASE_VISCOSITY = 0.10
RC3_HYDRO_MAX_VISCOSITY = 2.00
RC3_HYDRO_TURBULENCE_EPS = 1e-8
RC3_HYDRO_MAX_ITEMS_PER_CLASS = VALIDATION_MAX_ITEMS_PER_CLASS
RC3_HYDRO_MECHANISTIC_TURBULENCE_WARNING = 0.20
RC3_HYDRO_MECHANISTIC_LATENT_WARNING = 0.05

RC2B_CANDIDATE_VARIANTS = [
    "proto_global_no_repair",
    "context_blend_selected_global",
    "context_temp_blend_selected_global",
    "proto_global_head_ce_kl_guard_035",
    "context_temp_blend_selected_head_guard_035",
    "context_gap_selected",
]

```

```

# Patch missing inherited RC1b context-selection constants.
# Run this once, then rerun the failed "Run experiment" cell.

CONTEXT_TEMP_SELECTION_GRID = [0.35, 0.60, 0.80, 1.00, 1.25, 1.50]
CONTEXT_BLEND_SELECTION_GRID = [
    ("base_proto_first", 0.05, 0.75, 0.20),
    ("latent_only", 0.00, 1.00, 0.00),
    ("feature_only", 0.00, 0.00, 1.00),
    ("latent_dominant", 0.05, 0.90, 0.05),
    ("single_mid", 0.30, 0.55, 0.15),
    ("balanced_all", 0.34, 0.33, 0.33),
]

CONTEXT_SELECTION_SCORE_CONTEXT_WEIGHT = 0.20
CONTEXT_SELECTION_SCORE_CLASS2_WEIGHT = 0.10
CONTEXT_SELECTION_SCORE_MINCLASS_WEIGHT = 0.10
CONTEXT_SELECTION_SCORE_ENTROPY_PENALTY = 0.02

# Backward-compatible names used by inherited v0.14.x helper functions.
HEAD_REFRESH_EPOCHS = READOUT_TRAIN_EPOCHS
HEAD_REFRESH_STEPS = READOUT_TRAIN_STEPS
HEAD_REFRESH_LR = READOUT_LR

# Optional old helper compatibility constants.
PROBE_DISTILL_EPOCHS = 2
PROBE_DISTILL_LR = 2e-3
BIAS_CALIB_STEPS = 8
BIAS_CALIB_LR = 5e-2
PROBE_DISTILL_ALPHA_CE = 0.50
PROBE_DISTILL_ALPHA_KL = 0.50
BIAS_CALIB_REG = 1e-3

LOCAL_ADAPTER_KL_GRID = [0.05, 0.15, 0.35, 0.60]
LOCAL_ADAPTER_MARGIN_WEIGHT = HEAD_MARGIN_LAMBDA

SELECTED_LOCAL_HEAD_VARIANTS = [
    "no_repair_global_head",
    "kl_only_true_noop",
    "global_head_ce_kl_guard_035",
    "local_proto_cosine",
    "local_proto_euclidean",
    "local_trainable_proto_ce_kl_035",
    "local_diag_adapter_ce_only",
    "local_diag_adapter_ce_kl_005",
    "local_diag_adapter_ce_kl_015",
    "local_diag_adapter_ce_kl_035",
    "local_diag_adapter_ce_kl_060",

```

```

        "local_diag_adapter_ce_kl_margin_035",
        "local_adapter_validation_selected",
    ]

    # Keep the historical variants for control rows and final candidate comparison.
    SELECTED_CONTEXT_HEAD_VARIANTS = [
        "proto_global_no_repair",
        "proto_kl_only_true_noop",
        "proto_global_head_ce_kl_guard_035",
        "oracle_global_no_repair_reference",
        "oracle_global_head_ce_kl_guard_035_reference",
        "context_blend_selected_global",
        "context_temp_blend_selected_global",
        "context_blend_selected_head_guard_035",
        "context_temp_blend_selected_head_guard_035",
        "context_gap_selected",
    ]

    # Comparative baselines for the benchmark harness.
    if RUN_MODE == "smoke":
        SELECTED_BASELINE_METHODS = [
            "finetune",
            "experience_replay",
            "balanced_replay",
            "joint_upper_bound",
        ]
    else:
        SELECTED_BASELINE_METHODS = [
            "finetune",
            "ewc",
            "lwf",
            "experience_replay",
            "balanced_replay",
            "sparse_moe",
            "pnn",
            "joint_upper_bound",
        ]

    EWC_LAMBDA = 25.0
    LWF_LAMBDA = 1.0
    BASELINE_REPLAY_WEIGHT = 1.0
    SPARSE_MOE_ENTROPY_WEIGHT = 0.002

    RUN_FINAL_PROBES = True
    PROBE_CONTEXT_MODES = ["proto", "oracle"]
    HOST_PROBE_MODE = "oracle"

```



```

COMMON_MLP = dict(
    model_family="mlp",
    strategy="none",
    balanced_replay_ce=False,
    balanced_replay_weight=0.0,
    output_distill=False,
    route=0.0,
    latent=0.0,
    output=0.0,
    context_calibrated=False,
    context_replay=False,
    temp_calibration=False,
    context_replay_scale=0.0,
    head_margin_repair=False,
    post_head_refresh=False,
    post_probe_distill=False,
    post_bias_calibration=False,
)
COMMON_MMALS = dict(
    model_family="mmals",
    route=0.5,
    latent=1.0,
    output=1.0,
    output_distill=True,
    balanced_replay_ce=True,
    balanced_replay_weight=BALANCED_REPLAY_WEIGHT,
    context_calibrated=False,
    context_replay=False,
    temp_calibration=False,
    context_replay_scale=0.0,
    batch_blend_single=PROTO_FIRST_BLEND_SINGLE,
    batch_blend_latent=PROTO_FIRST_BLEND_LATENT,
    batch_blend_feature=PROTO_FIRST_BLEND_FEATURE,
    head_margin_repair=False,
    post_head_refresh=False,
    post_probe_distill=False,
    post_bias_calibration=False,
)
COMMON_PROTO_FIRST = {
    **COMMON_MMALS,
    "strategy": "proto_first",
    "context_calibrated": True,
    "context_replay": True,
    "temp_calibration": True,
    "context_replay_scale": 1.0,
    "route": 0.5,
    "latent": 1.0,
}

```

```

    "balanced_replay_ce": True,
    "balanced_replay_weight": BALANCED_REPLAY_WEIGHT,
    "output_distill": True,
    "output": 1.0,
}
METHOD_CONFIGS = {
    "mlp_finetune": {**COMMON_MLP},
    "mlp_ewc": {**COMMON_MLP},
    "mlp_lwf": {**COMMON_MLP},
    "mlp_experience_replay": {**COMMON_MLP},
    "mlp_balanced_replay": {**COMMON_MLP},
    "sparse_moe": {**COMMON_MLP, "model_family": "sparse_moe"},
    "pnn": {**COMMON_MLP, "model_family": "pnn"},
    "mlp_joint_upper_bound": {**COMMON_MLP},
    "mmals_balanced_replay": {
        **COMMON_MMALS,
        "strategy": "task_emb",
        "balanced_replay_ce": True,
        "balanced_replay_weight": BALANCED_REPLAY_WEIGHT,
        "output_distill": True,
        "output": 1.0,
    },
    "mmals_uniform_balanced_replay": {
        **COMMON_MMALS,
        "strategy": "uniform",
    },
    "mmals_proto_first_balanced_replay": {
        **COMMON_PROTO_FIRST,
    },
    "mmals_oracle_context_balanced_replay": {
        **COMMON_MMALS,
        "strategy": "oracle",
    },
}

print("EXPERIMENT_PROFILE:", EXPERIMENT_PROFILE)
print("POLICY_BRANCH:", POLICY_BRANCH)
print("ACTIVATE_RC2C_RISK_AWARE_SELECTION:", ACTIVATE_RC2C_RISK_AWARE_SELECTION)
print("ACTIVATE_HYDRO_AUDIT:", ACTIVATE_HYDRO_AUDIT)
print("ACTIVATE_HYDRO_REGULARIZER:", ACTIVATE_HYDRO_REGULARIZER, "mode=",
      ↪HYDRO_REGULARIZER_MODE)
print("ACTIVATE_HYDRO_SELECTION:", ACTIVATE_HYDRO_SELECTION, "mode=",
      ↪HYDRO_POLICY_MODE)
print("RUN_MODE:", RUN_MODE)
print("DATASET_NAME:", DATASET_NAME)
print("SEEDS:", SEEDS)
print("RC2C_METHOD:", RC2C_METHOD)

```

```

print("RC3_HYDRO_POLICY_MODE:", RC3_HYDRO_POLICY_MODE)
print("RC2B_CANDIDATE_VARIANTS / RC3 candidate pool:", RC2B_CANDIDATE_VARIANTS)
print("SELECTED_BASELINE_METHODS:", SELECTED_BASELINE_METHODS)
print("v1.1-RC2h RC1b-anchored conservative selector harness: RC2b strict_
↳baseline and RC2G tri-boundary controls are preserved; RC1b is the default_
↳anchor and overrides require strong no-leakage validation evidence.")
print("ROTATION_ANGLES:", ROTATION_ANGLES)
print("PERMUTATION_SEEDS:", PERMUTATION_SEEDS)

```

```

EXPERIMENT_PROFILE: RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT
POLICY_BRANCH: RC2h
ACTIVATE_RC2C_RISK_AWARE_SELECTION: True
ACTIVATE_HYDRO_AUDIT: True
ACTIVATE_HYDRO_REGULARIZER: False mode= off
ACTIVATE_HYDRO_SELECTION: False mode= audit_only
RUN_MODE: evidence
DATASET_NAME: FashionMNIST
SEEDS: [0, 1, 2]
RC2C_METHOD: paired_rc2h_rc1b_anchored_conservative_policy
RC3_HYDRO_POLICY_MODE: audit_only
RC2B_CANDIDATE_VARIANTS / RC3 candidate pool: ['proto_global_no_repair',
'context_blend_selected_global', 'context_temp_blend_selected_global',
'proto_global_head_ce_kl_guard_035',
'context_temp_blend_selected_head_guard_035', 'context_gap_selected']
SELECTED_BASELINE_METHODS: ['finetune', 'ewc', 'lwf', 'experience_replay',
'balanced_replay', 'sparse_moe', 'pnn', 'joint_upper_bound']
v1.1-RC2h RC1b-anchored conservative selector harness: RC2b strict baseline and
RC2G tri-boundary controls are preserved; RC1b is the default anchor and
overrides require strong no-leakage validation evidence.
ROTATION_ANGLES: [-30, -15, 0, 15, 30]
PERMUTATION_SEEDS: [100, 101, 102, 103, 104]

```

1.6 2a. How to activate or deactivate RC2H / Hydro

Use EXPERIMENT_PROFILE in the configuration cell.

Recommended default:

EXPERIMENT_PROFILE = "RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT"

Available RC2H profiles:

1. RC2B_STRICT
Reproduces the older RC2b strict selector. Hydro disabled. RC2H selector disabled.
2. RC2H_RC1B_ANCHORED_CONSERVATIVE_NO_HYDRO
RC2H anchored selector enabled. Hydro disabled. Use this to isolate selector effects.
3. RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT
Default. RC2H anchored selector enabled. Hydro metrics logged as audit only.

4. RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_STABILITY_ABLATION

Experimental. RC2H selector enabled and Hydro drift/turbulence penalty added to validation selection. Report separately.

Default scientific interpretation:

context_gap_selected / guarded RC1b context-gap is the deployable anchor. Another candidate can replace it only with strong validation-memory evidence and safety gates. Hydro remains an observability/audit contribution unless the Hydro stability ablation is explicitly selected.

1.7 2b. Persistence and OTEL-style traceability layer

This cell creates a persistent artifact directory. In Colab it can mount Google Drive and write results there; outside Colab it falls back to a local directory.

The trace format is **OTEL-style**, not a full external OTLP export by default. It writes JSONL events and span-like rows that can later be ingested by Jaeger, Grafana Tempo, Data-dog, OpenSearch, or an internal STRAT-Q observability pipeline.

RC2H Drive archive path when PERSIST_TO_GOOGLE_DRIVE = True:

/content/drive/MyDrive/MMALS/v1_1_RC2h_rc1b_anchored_conservative_selector/<EXPERIMENT_PROFILE>

```
[4]: # -----
# v1.1-RC2h RC1b-Anchored Conservative Selector Harness
# -----
# Set to False if you do not want Colab to mount Google Drive.
PERSIST_TO_GOOGLE_DRIVE = True
GOOGLE_DRIVE_PROJECT_DIR = f"MMALS/
    ↪v1_1_RC2h_rc1b_anchored_conservative_selector/{EXPERIMENT_PROFILE}"

RUN_TIMESTAMP_UTC = datetime.now(timezone.utc).strftime("%Y%m%dT%H%M%SZ")
RUN_ID =
    ↪f"MMALS_v11_{EXPERIMENT_PROFILE}_{DATASET_NAME}_{RUN_MODE}_{RUN_TIMESTAMP_UTC}_seeds{len(SE
TRACE_ID = hashlib.sha256(RUN_ID.encode("utf-8")).hexdigest()[:16]

# Notebook/PDF evidence-export names. The helper cell later stores a
    ↪run-summary PDF
# and optionally a full notebook PDF in RESULTS_DIR/notebook_exports.
NOTEBOOK_BASENAME = globals().get("NOTEBOOK_BASENAME",
    ↪f"MMALS_Prototype_1_1_RC2h_RC1b_Anchored_Conservative_Selector_Harness_Colab")
NOTEBOOK_FILENAME = globals().get("NOTEBOOK_FILENAME", f"{NOTEBOOK_BASENAME}.
    ↪ipynb")

DRIVE_AVAILABLE = False
if PERSIST_TO_GOOGLE_DRIVE:
    try:
        from google.colab import drive # type: ignore
        drive.mount("/content/drive", force_remount=False)
        DRIVE_AVAILABLE = True
```

```

except Exception as e:
    print("Google Drive mount skipped; falling back to local persistence.")
    print("Reason:", repr(e))

if DRIVE_AVAILABLE:
    PERSISTENCE_ROOT = Path("/content/drive/MyDrive") / GOOGLE_DRIVE_PROJECT_DIR
else:
    PERSISTENCE_ROOT = Path("./
↳mmals_v11_rc2h_rc1b_anchored_conservative_selector_results")

RESULTS_DIR = PERSISTENCE_ROOT / RUN_ID
TRACE_DIR = RESULTS_DIR / "traces"
AUDIT_DIR = RESULTS_DIR / "audit"
PLOTS_DIR = RESULTS_DIR / "plots"
RAW_LOG_DIR = RESULTS_DIR / "raw_logs"
NOTEBOOK_EXPORT_DIR = RESULTS_DIR / "notebook_exports"
TRUSTED_EVIDENCE_DIR = RESULTS_DIR / "TrustedEvidence"
for d in [RESULTS_DIR, TRUSTED_EVIDENCE_DIR, TRACE_DIR, AUDIT_DIR, PLOTS_DIR,
↳RAW_LOG_DIR, NOTEBOOK_EXPORT_DIR]:
    d.mkdir(parents=True, exist_ok=True)

def ensure_persistence_dirs():
    """Re-create Drive/local artifact directories if Colab/Drive sync
↳temporarily drops them.

    This is intentionally called before every file write because Google Drive
↳mounted
    folders can disappear after remounts, manual moves, or transient sync
↳issues.
    It does not change RUN_ID or any scientific result.
    """
    for d in [RESULTS_DIR, TRUSTED_EVIDENCE_DIR, TRACE_DIR, AUDIT_DIR,
↳PLOTS_DIR, RAW_LOG_DIR, NOTEBOOK_EXPORT_DIR]:
        Path(d).mkdir(parents=True, exist_ok=True)

TRACE_JSONL_PATH = TRACE_DIR / f"otel_style_events_{RUN_MODE}.jsonl"
TRACE_SPANS_PATH = TRACE_DIR / f"otel_style_spans_{RUN_MODE}.csv"
RAW_CONSOLE_LOG_PATH = RAW_LOG_DIR / f"console_{RUN_MODE}.log"
MANIFEST_PATH = RESULTS_DIR / f"run_manifest_{RUN_MODE}.json"

TRACE_EVENTS = []
TRACE_SPANS = []
_ORIGINAL_STDOUT = sys.stdout
_ORIGINAL_STDERR = sys.stderr
_LOG_FILE_HANDLE = None
_LOGGING_ACTIVE = False

```

```

def safe_jsonable(obj):
    """Convert numpy/torch/Pandas objects to JSON-safe values without dumping,
    ↪ large tensors."""
    if obj is None:
        return None
    if isinstance(obj, (str, int, float, bool)):
        if isinstance(obj, float) and (math.isnan(obj) or math.isinf(obj)):
            return None
        return obj
    if isinstance(obj, (np.integer,)):
        return int(obj)
    if isinstance(obj, (np.floating,)):
        v = float(obj)
        return None if math.isnan(v) or math.isinf(v) else v
    if isinstance(obj, (np.ndarray,)):
        if obj.size <= 16:
            return obj.tolist()
        return {"type": "ndarray", "shape": list(obj.shape), "mean": float(np.
    ↪ nanmean(obj)), "std": float(np.nanstd(obj))}
    if torch.is_tensor(obj):
        t = obj.detach().cpu()
        if t.numel() == 1:
            return safe_jsonable(t.item())
        if t.numel() <= 16:
            return t.tolist()
        tf = t.float()
        return {"type": "tensor", "shape": list(t.shape), "mean": float(tf.
    ↪ mean()), "std": float(tf.std())}
    if isinstance(obj, dict):
        return {str(k): safe_jsonable(v) for k, v in obj.items()}
    if isinstance(obj, (list, tuple)):
        return [safe_jsonable(x) for x in obj]
    return str(obj)

def config_snapshot():
    keys = [
        "RUN_MODE", "DATASET_NAME", "PAIR_MODE", "N_TASKS", "N_CONTEXTS",
    ↪ "N_HOSTS", "N_CLASSES",
        "TRAIN_PER_CLASS", "TEST_PER_CLASS", "SEEDS", "EPOCHS_PER_TASK",
    ↪ "MEMORY_PER_TASK", "MEMORY_PER_CLASS",
        "LABEL_NOISE_RATE", "INPUT_NOISE_STD", "CONTEXT_NOISE_RATE",
    ↪ "LATENT_DIM", "HIDDEN_DIM",
        "READOUT_TRAIN_EPOCHS", "READOUT_TRAIN_STEPS",
    ↪ "CONTEXT_TEMP_SELECTION_GRID", "CONTEXT_BLEND_SELECTION_GRID",

```

```

        "SELECTED_CONTEXT_HEAD_VARIANTS", "SELECTED_BASELINE_METHODS",
        ↪ "RC2B_CANDIDATE_VARIANTS", "RC2B_METHOD", "RC3_METHOD",
        ↪ "RC3_HYDRO_AUDIT_ENABLED", "RC3_HYDRO_POLICY_MODE",
        ↪ "RC3_HYDRO_SELECTION_TURBULENCE_WEIGHT",
        ↪ "RC3_HYDRO_SELECTION_LATENT_DRIFT_WEIGHT",
        ↪ "RC2B_STRICT_NO_FINAL_TEST_SELECTION", "NOTEBOOK_FILENAME",
        ↪ "ROTATION_ANGLES", "PERMUTATION_SEEDS",
    ]
    snap = {}
    for k in keys:
        if k in globals():
            snap[k] = safe_jsonable(globals()[k])
    return snap

def stable_hash(obj) -> str:
    payload = json.dumps(safe_jsonable(obj), sort_keys=True,
        ↪ ensure_ascii=False).encode("utf-8")
    return hashlib.sha256(payload).hexdigest()

CONFIG_HASH = stable_hash(config_snapshot())

def log_event(event_type: str, seed=None, task_id=None, method=None,
    ↪ variant=None, phase=None, payload=None, level="INFO"):
    event = {
        "trace_id": TRACE_ID,
        "run_id": RUN_ID,
        "timestamp_utc": datetime.now(timezone.utc).isoformat(),
        "level": level,
        "event_type": event_type,
        "dataset": DATASET_NAME,
        "run_mode": RUN_MODE,
        "seed": safe_jsonable(seed),
        "task_id": safe_jsonable(task_id),
        "method": safe_jsonable(method),
        "variant": safe_jsonable(variant),
        "phase": safe_jsonable(phase),
        "config_hash": CONFIG_HASH,
        "payload": safe_jsonable(payload or {}),
    }
    TRACE_EVENTS.append(event)
    ensure_persistence_dirs()
    with open(TRACE_JSONL_PATH, "a", encoding="utf-8") as f:
        f.write(json.dumps(event, ensure_ascii=False) + "\n")
    return event

class TeeStream:
    def __init__(self, *streams):

```

```

        self.streams = streams
    def write(self, data):
        for s in self.streams:
            try:
                s.write(data)
                s.flush()
            except Exception:
                pass
    def flush(self):
        for s in self.streams:
            try:
                s.flush()
            except Exception:
                pass

def start_persistent_logging():
    global _LOG_FILE_HANDLE, _LOGGING_ACTIVE
    if _LOGGING_ACTIVE:
        return
    ensure_persistence_dirs()
    _LOG_FILE_HANDLE = open(RAW_CONSOLE_LOG_PATH, "a", buffering=1,
↪encoding="utf-8")
    sys.stdout = TeeStream(_ORIGINAL_STDOUT, _LOG_FILE_HANDLE)
    sys.stderr = TeeStream(_ORIGINAL_STDERR, _LOG_FILE_HANDLE)
    _LOGGING_ACTIVE = True
    log_event("run_logging_started", payload={"raw_console_log":
↪str(RAW_CONSOLE_LOG_PATH)})

def stop_persistent_logging():
    global _LOG_FILE_HANDLE, _LOGGING_ACTIVE
    if not _LOGGING_ACTIVE:
        return
    log_event("run_logging_stopped", payload={"raw_console_log":
↪str(RAW_CONSOLE_LOG_PATH)})
    sys.stdout = _ORIGINAL_STDOUT
    sys.stderr = _ORIGINAL_STDERR
    try:
        _LOG_FILE_HANDLE.close()
    except Exception:
        pass
    _LOG_FILE_HANDLE = None
    _LOGGING_ACTIVE = False

def file_sha256(path: Path) -> str:
    h = hashlib.sha256()
    with open(path, "rb") as f:
        for chunk in iter(lambda: f.read(1024 * 1024), b""):

```



```

        h.update(chunk)
    return h.hexdigest()

def write_manifest(extra=None):
    ensure_persistence_dirs()
    artifacts = []
    if RESULTS_DIR.exists():
        for p in sorted(RESULTS_DIR.rglob("*")):
            if p.is_file() and p != MANIFEST_PATH:
                try:
                    artifacts.append({
                        "path": str(p.relative_to(RESULTS_DIR)),
                        "size_bytes": int(p.stat().st_size),
                        "sha256": file_sha256(p),
                    })
                except Exception as e:
                    artifacts.append({"path": str(p), "error": repr(e)})
    manifest = {
        "run_id": RUN_ID,
        "trace_id": TRACE_ID,
        "created_utc": datetime.now(timezone.utc).isoformat(),
        "notebook_version": "MMALS_v1.
↪1_RC2h_RC1b_Anchored_Conservative_Selector",
        "dataset": DATASET_NAME,
        "run_mode": RUN_MODE,
        "device": str(DEVICE),
        "python": sys.version,
        "platform": platform.platform(),
        "config_hash": CONFIG_HASH,
        "config": config_snapshot(),
        "persistence_root": str(PERSISTENCE_ROOT),
        "results_dir": str(RESULTS_DIR),
        "drive_available": DRIVE_AVAILABLE,
        "artifacts": artifacts,
        "extra": safe_jsonable(extra or {}),
    }
    ensure_persistence_dirs()
    with open(MANIFEST_PATH, "w", encoding="utf-8") as f:
        json.dump(manifest, f, indent=2, ensure_ascii=False)
    return manifest

log_event("run_initialized", payload={"results_dir": str(RESULTS_DIR),
↪ "drive_available": DRIVE_AVAILABLE})
print("RUN_ID:", RUN_ID)
print("TRACE_ID:", TRACE_ID)
print("RESULTS_DIR:", RESULTS_DIR)
print("TRACE_JSONL_PATH:", TRACE_JSONL_PATH)

```

```
print("RAW_CONSOLE_LOG_PATH:", RAW_CONSOLE_LOG_PATH)
print("NOTEBOOK_EXPORT_DIR:", NOTEBOOK_EXPORT_DIR)
```

Mounted at /content/drive

RUN_ID: MMALS_v11_RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT_FashionMNIST_evidence_20260526T183427Z_seeds3

TRACE_ID: 49d3678f2bbf0046

RESULTS_DIR: /content/drive/MyDrive/MMALS/v1_1_RC2h_rc1b_anchored_conservative_selector/RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT/MMALS_v11_RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT_FashionMNIST_evidence_20260526T183427Z_seeds3

TRACE_JSONL_PATH: /content/drive/MyDrive/MMALS/v1_1_RC2h_rc1b_anchored_conservative_selector/RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT/MMALS_v11_RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT_FashionMNIST_evidence_20260526T183427Z_seeds3/traces/otel_style_events_evidence.jsonl

RAW_CONSOLE_LOG_PATH: /content/drive/MyDrive/MMALS/v1_1_RC2h_rc1b_anchored_conservative_selector/RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT/MMALS_v11_RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT_FashionMNIST_evidence_20260526T183427Z_seeds3/raw_logs/console_evidence.log

NOTEBOOK_EXPORT_DIR: /content/drive/MyDrive/MMALS/v1_1_RC2h_rc1b_anchored_conservative_selector/RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT/MMALS_v11_RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT_FashionMNIST_evidence_20260526T183427Z_seeds3/notebook_exports

1.8 3. Dataset builder: MNIST-family domain-shift overlap-chain with global labels

The benchmark remains the overlapping chain:

$(0,1) \rightarrow (1,2) \rightarrow (2,3) \rightarrow (3,4) \rightarrow (4,5)$

Labels are global class IDs, not local binary task labels.

Supported DATASET_NAME values:

- MNIST: plain MNIST overlap-chain;
- FashionMNIST: plain FashionMNIST overlap-chain;
- RotatedMNIST: MNIST overlap-chain with deterministic task rotations;
- PermutedMNIST: MNIST overlap-chain with deterministic task-specific pixel permutations.

For DATASET_NAME = "RotatedMNIST", each task receives a deterministic rotation:

Task	Classes	Rotation
0	0 vs 1	-30°
1	1 vs 2	-15°
2	2 vs 3	0°
3	3 vs 4	+15°
4	4 vs 5	+30°

For DATASET_NAME = "PermutedMNIST", each task receives a deterministic pixel permutation:

Task	Classes	Permutation seed
0	0 vs 1	100
1	1 vs 2	101
2	2 vs 3	102
3	3 vs 4	103
4	4 vs 5	104

This keeps the original class-incremental overlap-chain stress while adding explicit domain/context shift.

```
[5]: from torchvision.transforms import functional as TF
from torchvision.transforms import InterpolationMode

_PERMUTATION_CACHE = {}

def get_base_transform(name):
    # Do not flatten here. Rotated-MNIST must rotate [1, 28, 28] tensors first.
    # Permuted-MNIST flattens only after applying a deterministic task
    ↪ permutation.
    return transforms.ToTensor()

def load_base_dataset(name):
    transform = get_base_transform(name)
    if name in ["MNIST", "RotatedMNIST", "PermutedMNIST"]:
        train = datasets.MNIST(DATA_ROOT, train=True, download=True,
    ↪ transform=transform)
        test = datasets.MNIST(DATA_ROOT, train=False, download=True,
    ↪ transform=transform)
    elif name == "FashionMNIST":
        train = datasets.FashionMNIST(DATA_ROOT, train=True, download=True,
    ↪ transform=transform)
        test = datasets.FashionMNIST(DATA_ROOT, train=False, download=True,
    ↪ transform=transform)
    else:
        raise ValueError("DATASET_NAME must be MNIST, FashionMNIST,
    ↪ RotatedMNIST, or PermutedMNIST")
    return train, test

def get_labels(base_dataset):
    if hasattr(base_dataset, "targets"):
        return np.array(base_dataset.targets)
    raise ValueError("Dataset has no targets attribute")

def task_pairs(pair_mode: str):
    if pair_mode == "disjoint_pairs":
        return [(0, 1), (2, 3), (4, 5), (6, 7), (8, 9)]
```

```

    if pair_mode == "overlap_chain":
        return [(0, 1), (1, 2), (2, 3), (3, 4), (4, 5)]
    raise ValueError("Unknown PAIR_MODE")

def seen_classes_until(task_id: int):
    classes = set()
    for a, b in task_pairs(PAIR_MODE)[: task_id + 1]:
        classes.add(a)
        classes.add(b)
    return sorted(c for c in classes if c < N_CLASSES)

def task_rotation_angle(task_id: int, dataset_name: str = DATASET_NAME) -> float:
    if dataset_name == "RotatedMNIST":
        if len(ROTATION_ANGLES) != N_TASKS:
            raise ValueError(f"ROTATION_ANGLES must have length {N_TASKS}, got {len(ROTATION_ANGLES)}")
        return float(ROTATION_ANGLES[task_id])
    return 0.0

def task_permutation_seed(task_id: int, dataset_name: str = DATASET_NAME) -> Optional[int]:
    if dataset_name == "PermutedMNIST":
        if len(PERMUTATION_SEEDS) != N_TASKS:
            raise ValueError(f"PERMUTATION_SEEDS must have length {N_TASKS}, got {len(PERMUTATION_SEEDS)}")
        return int(PERMUTATION_SEEDS[task_id])
    return None

def get_task_permutation(task_id: int, input_dim: int = INPUT_DIM, dataset_name: str = DATASET_NAME):
    seed = task_permutation_seed(task_id, dataset_name)
    if seed is None:
        return None
    key = (dataset_name, task_id, int(input_dim), int(seed))
    if key not in _PERMUTATION_CACHE:
        g = torch.Generator()
        g.manual_seed(seed)
        _PERMUTATION_CACHE[key] = torch.randperm(input_dim, generator=g)
    return _PERMUTATION_CACHE[key]

def apply_task_transform(x, task_id: int, dataset_name: str):
    # x arrives as [1, 28, 28]. Return flattened [784].
    if dataset_name == "RotatedMNIST":
        x = TF.rotate(
            x,
            angle=task_rotation_angle(task_id, dataset_name),

```

```

        interpolation=InterpolationMode.BILINEAR,
        fill=0.0,
    )
    return x.view(-1).float()

    if dataset_name == "PermutedMNIST":
        x_flat = x.view(-1).float()
        perm = get_task_permutation(task_id, input_dim=x_flat.numel(),
↪dataset_name=dataset_name)
        return x_flat[perm].float()

    return x.view(-1).float()

def subset_pair_dataset_global(
    base_dataset,
    a,
    b,
    task_id: int,
    dataset_name: str,
    max_per_class=None,
    seed=42,
    label_noise=0.0,
    input_noise_std=0.0,
):
    rng = np.random.default_rng(seed)
    labels = get_labels(base_dataset)
    idx_a = np.where(labels == a)[0]
    idx_b = np.where(labels == b)[0]

    if max_per_class is not None:
        idx_a = rng.choice(idx_a, size=min(max_per_class, len(idx_a)),
↪replace=False)
        idx_b = rng.choice(idx_b, size=min(max_per_class, len(idx_b)),
↪replace=False)

    indices = np.concatenate([idx_a, idx_b])
    rng.shuffle(indices)

    xs, ys = [], []
    for idx in indices:
        x, y = base_dataset[int(idx)]
        y = int(y)
        if y >= N_CLASSES:
            continue

    x = apply_task_transform(x, task_id=task_id, dataset_name=dataset_name)

```

```

        if input_noise_std > 0:
            x = torch.clamp(x + torch.randn_like(x) * input_noise_std, 0.0, 1.0)

        xs.append(x)
        ys.append(y)

X = torch.stack(xs).float()
y = torch.tensor(ys).long()

if label_noise > 0:
    flip_mask = torch.rand(len(y)) < label_noise
    y[flip_mask & (y == a)] = b
    y[flip_mask & (y == b)] = a

return TensorDataset(X, y)

def build_stress_tasks(name=DATASET_NAME, seed=42):
    train_base, test_base = load_base_dataset(name)
    pairs = task_pairs(PAIR_MODE)
    tasks = []

    for task_id, (a, b) in enumerate(pairs):
        train_ds = subset_pair_dataset_global(
            train_base, a, b,
            task_id=task_id,
            dataset_name=name,
            max_per_class=TRAIN_PER_CLASS,
            seed=seed + task_id,
            label_noise=LABEL_NOISE_RATE,
            input_noise_std=INPUT_NOISE_STD,
        )
        test_ds = subset_pair_dataset_global(
            test_base, a, b,
            task_id=task_id,
            dataset_name=name,
            max_per_class=TEST_PER_CLASS,
            seed=seed + 100 + task_id,
            label_noise=0.0,
            input_noise_std=INPUT_NOISE_STD,
        )

        angle = task_rotation_angle(task_id, name)
        perm_seed = task_permutation_seed(task_id, name)
        task_name = f"{a} vs {b}"
        if name == "RotatedMNIST":
            task_name += f" @ {angle:+.0f}deg"
        elif name == "PermutedMNIST":

```

```

        task_name += f" @ perm_seed={perm_seed}"

    tasks.append({
        "task_id": task_id,
        "name": task_name,
        "pair": (a, b),
        "rotation_angle": angle,
        "permutation_seed": perm_seed,
        "train": train_ds,
        "test": test_ds,
    })

    return tasks

def sample_context_id(task_id: int, n_contexts: int = N_CONTEXTS, noise_rate:
    float = CONTEXT_NOISE_RATE):
    if random.random() >= noise_rate:
        return task_id
    candidates = [i for i in range(n_contexts) if i != task_id]
    return random.choice(candidates)

tasks_preview = build_stress_tasks(DATASET_NAME, seed=0)
[
    (
        t["task_id"],
        t["name"],
        t["pair"],
        t["rotation_angle"],
        t.get("permutation_seed"),
        len(t["train"]),
        len(t["test"]),
        seen_classes_until(t["task_id"]),
    )
    for t in tasks_preview
]

```

```

100%|      | 26.4M/26.4M [00:02<00:00, 11.0MB/s]
100%|      | 29.5k/29.5k [00:00<00:00, 161kB/s]
100%|      | 4.42M/4.42M [00:01<00:00, 3.06MB/s]
100%|      | 5.15k/5.15k [00:00<00:00, 29.8MB/s]

```

```

[5]: [(0, '0 vs 1', (0, 1), 0.0, None, 2400, 800, [0, 1]),
      (1, '1 vs 2', (1, 2), 0.0, None, 2400, 800, [0, 1, 2]),
      (2, '2 vs 3', (2, 3), 0.0, None, 2400, 800, [0, 1, 2, 3]),
      (3, '3 vs 4', (3, 4), 0.0, None, 2400, 800, [0, 1, 2, 3, 4]),
      (4, '4 vs 5', (4, 5), 0.0, None, 2400, 800, [0, 1, 2, 3, 4, 5])]

```

1.9 4. Model definitions: MLP baseline and MMALS fungal medium

MMALS keeps hosts, transformations, a context encoder, a router, and a mediated latent state $\mathbf{zf}$.

This domain-shift notebook does not change the fungal backbone. It freezes the robustly selected RC1b candidate and tests it under rotation-induced or permutation-induced context/domain shift.

```
[6]: class MLPBaseline(nn.Module):
    def __init__(self, input_dim=INPUT_DIM, hidden_dim=HIDDEN_DIM,
        latent_dim=LATENT_DIM, n_classes=N_CLASSES):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, latent_dim),
            nn.ReLU(),
        )
        self.global_head = nn.Linear(latent_dim, n_classes)

    def forward(self, x, return_cache=False):
        z = self.encoder(x)
        logits = self.global_head(z)
        if not return_cache:
            return logits
        q = torch.full((x.shape[0], N_CONTEXTS), 1.0 / N_CONTEXTS, device=x.
            device)
        cache = {
            "zf": z,
            "routes": None,
            "transformed": None,
            "host_z": None,
            "q_context": q,
            "q_single": q,
            "context_logits": torch.zeros((x.shape[0], N_CONTEXTS), device=x.
                device),
            "context_center_logits": torch.zeros((x.shape[0], N_CONTEXTS),
                device=x.device),
            "context_feat": torch.zeros((x.shape[0], CONTEXT_FEAT_DIM),
                device=x.device),
            "active_context_mode": "mlp_none",
        }
        return logits, cache

class MLPBlock(nn.Module):
    def __init__(self, input_dim, hidden_dim, latent_dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
```



```

        nn.ReLU(),
        nn.Linear(hidden_dim, latent_dim),
        nn.ReLU(),
    )
    def forward(self, x):
        return self.net(x)

class ContextEncoder(nn.Module):
    def __init__(self, input_dim=INPUT_DIM, hidden_dim=128,
        ↪ feat_dim=CONTEXT_FEAT_DIM, n_contexts=N_CONTEXTS):
        super().__init__()
        self.feature_net = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, feat_dim),
            nn.ReLU(),
        )
        self.logit_head = nn.Linear(feat_dim, n_contexts)
    def forward(self, x):
        feat = self.feature_net(x)
        logits = self.logit_head(feat)
        return feat, logits

class MMALSClassIncremental(nn.Module):
    def __init__(self):
        super().__init__()
        self.n_tasks = N_TASKS
        self.n_contexts = N_CONTEXTS
        self.n_hosts = N_HOSTS
        self.n_classes = N_CLASSES
        self.temperature = TEMPERATURE
        self.context_temperature = CONTEXT_TEMPERATURE
        self.hosts = nn.ModuleList([MLPBlock(INPUT_DIM, HIDDEN_DIM, LATENT_DIM)
        ↪ for _ in range(N_HOSTS)])
        self.transforms = nn.ModuleList([
            nn.Sequential(nn.Linear(LATENT_DIM, LATENT_DIM), nn.ReLU(), nn.
        ↪ Linear(LATENT_DIM, LATENT_DIM))
            for _ in range(N_HOSTS)
        ])
        self.context_encoder = ContextEncoder()
        self.context_centers = nn.Parameter(torch.randn(N_CONTEXTS,
        ↪ CONTEXT_FEAT_DIM) * 0.05)
        self.context_embedding = nn.Embedding(N_CONTEXTS, TASK_EMB_DIM)
        self.router = nn.Sequential(
            nn.Linear(INPUT_DIM + TASK_EMB_DIM, ROUTE_HIDDEN_DIM),
            nn.ReLU(),
            nn.Linear(ROUTE_HIDDEN_DIM, N_HOSTS),

```

```

    )
    self.global_head = nn.Linear(LATENT_DIM, N_CLASSES)

    def transform_from_host_latents(self, host_z):
        transformed = [self.transforms[i](host_z[:, i, :]) for i in range(self.
↪n_hosts)]
        return torch.stack(transformed, dim=1)

    def context_probs_from_x(self, x):
        feat, logits = self.context_encoder(x)
        q = F.softmax(logits / self.context_temperature, dim=-1)
        return feat, logits, q

    def context_center_logits(self, feat):
        feat_n = F.normalize(feat, dim=-1)
        centers_n = F.normalize(self.context_centers, dim=-1)
        return (feat_n @ centers_n.T) / CENTER_TEMPERATURE

    def explicit_context_probs(self, batch_size, context_id, device):
        q = torch.zeros(batch_size, self.n_contexts, device=device)
        q[:, context_id] = 1.0
        return q

    def uniform_context_probs(self, batch_size, device):
        return torch.full((batch_size, self.n_contexts), 1.0 / self.n_contexts,
↪device=device)

    def latent_from_context_probs(self, x, q_context):
        context_emb = q_context @ self.context_embedding.weight
        host_z = torch.stack([host(x) for host in self.hosts], dim=1)
        transformed = self.transform_from_host_latents(host_z)
        route_input = torch.cat([x, context_emb], dim=1)
        route_logits = self.router(route_input)
        routes = F.softmax(route_logits / self.temperature, dim=-1)
        zf = (routes.unsqueeze(-1) * transformed).sum(dim=1)
        return zf, routes, route_logits, host_z, transformed

    def forward(self, x, task_id: int, context_mode: str = "oracle", context_id:
↪ Optional[int] = None,
                batch_context_probs: Optional[torch.Tensor] = None,
↪ return_cache: bool = False):
        b = x.shape[0]
        device = x.device
        feat, ctx_logits, q_single = self.context_probs_from_x(x)
        if context_mode in ["oracle", "noisy"]:
            cid = task_id if context_id is None else context_id
            q_context = self.explicit_context_probs(b, cid, device)

```

```

        elif context_mode == "uniform":
            q_context = self.uniform_context_probs(b, device)
        elif context_mode == "inferred_single":
            q_context = q_single
        elif context_mode == "inferred_batch":
            q_context = batch_context_probs if batch_context_probs is not None
    else q_single
    else:
        raise ValueError("context_mode must be oracle, noisy, uniform,
    inferred_single, or inferred_batch")
    zf, routes, route_logits, host_z, transformed = self.
    latent_from_context_probs(x, q_context)
    logits = self.global_head(zf)
    if not return_cache:
        return logits
    return logits, {
        "zf": zf,
        "routes": routes,
        "route_logits": route_logits,
        "host_z": host_z,
        "transformed": transformed,
        "context_feat": feat,
        "context_logits": ctx_logits,
        "context_center_logits": self.context_center_logits(feat),
        "q_context": q_context,
        "q_single": q_single,
    }

def build_model(config):
    if config["model_family"] == "mlp":
        return MLPBaseline().to(DEVICE)
    if config["model_family"] == "mmals":
        return MMALSClazzIncremental().to(DEVICE)
    raise ValueError(f"Unknown model_family: {config['model_family']}")

for family in ["mlp", "mmals"]:
    model_preview = build_model({"model_family": family})
    print(family, "trainable parameters:", count_parameters(model_preview),
    "total parameters:", total_parameters(model_preview), "approx MB total:",
    round(model_size_mb(model_preview, trainable_only=False), 3))
    del model_preview
clear_memory()

```

mlp trainable parameters: 217798 total parameters: 217798 approx MB total: 0.831
mmals trainable parameters: 1337216 total parameters: 1337216 approx MB total:
5.101

1.10 5. Context prototypes and prototype-first inference

Prototype-first context inference is preserved from v0.14.x and v1.0c. The RC1b candidate changes only validation-selected temperature/blend context posterior parameters and a guarded global read-out; it does not use explicit task IDs at inference.

```
[7]: def entropy_from_probs(q, eps=1e-8):
    return -(q * torch.log(q + eps)).sum(dim=-1)

def margin_from_probs(q):
    top2 = torch.topk(q, k=2, dim=-1).values
    return top2[:, 0] - top2[:, 1]

def brier_from_probs(q, target: int):
    y = torch.full((q.shape[0],), target, dtype=torch.long, device=q.device)
    onehot = F.one_hot(y, num_classes=q.shape[1]).float()
    return ((q - onehot) ** 2).sum(dim=-1)

def ece_from_probs(q, target: int, n_bins: int = 10):
    y = torch.full((q.shape[0],), target, dtype=torch.long, device=q.device)
    conf, pred = torch.max(q, dim=-1)
    correct = (pred == y).float()
    ece = torch.zeros((), device=q.device)
    for lo in torch.linspace(0, 1, n_bins + 1, device=q.device)[:n_bins]:
        hi = lo + 1.0 / n_bins
        mask = (conf > lo) & (conf <= hi)
        if mask.any():
            ece = ece + mask.float().mean() * torch.abs(conf[mask].mean() -
↪correct[mask].mean())
    return ece

@torch.no_grad()
def gaussian_stats_from_tensor(Z):
    if Z.shape[0] < 2:
        return None
    return {"mean": Z.mean(dim=0).to(DEVICE), "var": Z.var(dim=0,
↪unbiased=False).clamp_min(1e-6).to(DEVICE), "n": Z.shape[0]}

def diag_frechet_distance_from_stats(batch_values, proto_stats):
    mu = batch_values.mean(dim=0)
    var = batch_values.var(dim=0, unbiased=False).clamp_min(1e-6)
    mu_k = proto_stats["mean"].to(batch_values.device)
    var_k = proto_stats["var"].to(batch_values.device)
    return (mu - mu_k).pow(2).sum() + (torch.sqrt(var) - torch.sqrt(var_k)).
↪pow(2).sum()

def frechet_posterior_from_distances(distances, temperature):
    return F.softmax(-distances / temperature, dim=0)
```

```

def distance_margin_and_top(distances):
    sorted_d = torch.sort(distances).values
    margin = sorted_d[1] - sorted_d[0] if len(sorted_d) >= 2 else torch.
    ↪tensor(float("nan"), device=distances.device)
    top_context = torch.argmax(distances)
    return margin, top_context

@torch.no_grad()
def features_and_latents_for_dataset(model, dataset, context_id: int, ↵
    ↪max_items=PROTOTYPE_MAX_ITEMS, batch_size=BATCH_SIZE):
    model.eval()
    X, _ = dataset.tensors
    if max_items is not None and len(X) > max_items:
        idx = torch.randperm(len(X))[:max_items]
        X = X[idx]
    feats, latents = [], []
    loader = make_loader(TensorDataset(X, torch.zeros(len(X), dtype=torch.
    ↪long)), batch_size=batch_size, shuffle=False)
    for xb, _ in loader:
        xb = xb.to(DEVICE)
        feat, _, _ = model.context_probs_from_x(xb)
        q_oracle = model.explicit_context_probs(xb.shape[0], context_id, xb.
    ↪device)
        zf, _, _, _, _ = model.latent_from_context_probs(xb, q_oracle)
        feats.append(feat.detach().cpu())
        latents.append(zf.detach().cpu())
    return torch.cat(feats, dim=0), torch.cat(latents, dim=0)

@torch.no_grad()
def refresh_context_prototypes(model, memory_buffers: Dict[int, TensorDataset]):
    if not isinstance(model, MMALSClazzIncremental):
        return {}
    prototypes = {}
    for context_id, mem_ds in memory_buffers.items():
        feats, latents = features_and_latents_for_dataset(model, mem_ds, ↵
    ↪context_id=context_id)
        feat_stats = gaussian_stats_from_tensor(feats)
        latent_stats = gaussian_stats_from_tensor(latents)
        if feat_stats is not None and latent_stats is not None:
            prototypes[context_id] = {"feature": feat_stats, "latent": ↵
    ↪latent_stats}
    return prototypes

def infer_batch_context_probs(model, x, prototypes, config=None):
    config = config or {}

```

```

blend_single = config.get("batch_blend_single", BATCH_BLEND_SINGLE)
blend_latent = config.get("batch_blend_latent", BATCH_BLEND_LATENT)
blend_feature = config.get("batch_blend_feature", BATCH_BLEND_FEATURE)
context_feat, _, q_single = model.context_probs_from_x(x)
if not prototypes:
    return q_single, {
        "frechet_margin": torch.full((x.shape[0],), float("nan"), device=x.
↪device),
        "latent_frechet_margin": torch.full((x.shape[0],), float("nan"), ↪
↪device=x.device),
        "feature_frechet_margin": torch.full((x.shape[0],), float("nan"), ↪
↪device=x.device),
        "prototype_weight": torch.full((x.shape[0],), 0.0, device=x.device),
    }
zf_prelim, _, _, _ = model.latent_from_context_probs(x, q_single)
feature_distances = torch.full((model.n_contexts,), 1e6, device=x.device)
latent_distances = torch.full((model.n_contexts,), 1e6, device=x.device)
for cid, proto in prototypes.items():
    feature_distances[cid] = diag_frechet_distance_from_stats(context_feat, ↪
↪proto["feature"])
    latent_distances[cid] = diag_frechet_distance_from_stats(zf_prelim, ↪
↪proto["latent"])
    q_feature = frechet_posterior_from_distances(feature_distances, ↪
↪FRECHET_TEMPERATURE)
    q_latent = frechet_posterior_from_distances(latent_distances, ↪
↪LATENT_FRECHET_TEMPERATURE)
    proto_mass = max(blend_latent + blend_feature, 1e-8)
    q_proto = (blend_feature * q_feature + blend_latent * q_latent) / proto_mass
    q_batch = q_proto.unsqueeze(0).expand(x.shape[0], -1)
    q = blend_single * q_single + (1.0 - blend_single) * q_batch
    q = q / q.sum(dim=-1, keepdim=True).clamp_min(1e-8)
    f_margin, _ = distance_margin_and_top(feature_distances)
    l_margin, _ = distance_margin_and_top(latent_distances)
    return q, {
        "frechet_margin": l_margin.expand(x.shape[0]),
        "latent_frechet_margin": l_margin.expand(x.shape[0]),
        "feature_frechet_margin": f_margin.expand(x.shape[0]),
        "prototype_weight": torch.full((x.shape[0],), 1.0 - blend_single, ↪
↪device=x.device),
    }

```

1.11 6. Routing helpers

This preserves uniform, oracle and proto-first routes. The robust validation harness uses the same zf extraction protocol for every cloned variant.

```

[8]: def q_uniform_like(model, x):
    return model.uniform_context_probs(x.shape[0], x.device)

def normalize_probs(q, eps=1e-8):
    return q / q.sum(dim=-1, keepdim=True).clamp_min(eps)

def logits_from_context_probs(model, x, q_context):
    zf, routes, route_logits, host_z, transformed = model.
    ↪latent_from_context_probs(x, q_context)
    logits = model.global_head(zf)
    cache = {
        "zf": zf,
        "routes": routes,
        "route_logits": route_logits,
        "host_z": host_z,
        "transformed": transformed,
        "q_context": q_context,
    }
    return logits, cache

def method_context_mode_and_probs(model, x, task_id, config, prototypes=None,
    ↪train=False, noisy_context_id=None, y=None):
    if config["model_family"] == "mlp":
        return "mlp_none", None, None, {}
    strategy = config["strategy"]
    if strategy == "oracle":
        return "oracle", None, task_id, {}
    if strategy == "noisy":
        cid = noisy_context_id if noisy_context_id is not None else task_id
        return "noisy", None, cid, {}
    if strategy == "uniform":
        return "uniform", None, None, {}
    if strategy == "inferred_single":
        return "inferred_single", None, None, {}
    if strategy in ["inferred_batch", "proto_first"]:
        if train:
            return "inferred_single", None, None, {}
        q_batch, batch_info = infer_batch_context_probs(model, x, prototypes or
        ↪{}, config=config)
        return "inferred_batch", q_batch, None, batch_info
        raise ValueError(f"Unknown strategy: {strategy}")

def forward_by_method(model, x, task_id, config, prototypes=None, train=False,
    ↪noisy_context_id=None, y=None, return_cache=True):
    if config["model_family"] == "mlp":
        logits, cache = model(x, return_cache=True)
        return (logits, cache) if return_cache else logits

```

```

mode, batch_probs, context_id, batch_info = method_context_mode_and_probs(
    model, x, task_id, config, prototypes=prototypes, train=train,
    noisy_context_id=noisy_context_id, y=y
)
logits, cache = model(
    x,
    task_id,
    context_mode=mode,
    context_id=context_id,
    batch_context_probs=batch_probs,
    return_cache=True,
)
cache.update(batch_info)
cache["active_context_mode"] = mode
return (logits, cache) if return_cache else logits

def probe_config_from_mode(context_mode: str):
    if context_mode == "uniform":
        return {**COMMON_PROTO_FIRST, "strategy": "uniform"}
    if context_mode == "proto":
        return {**COMMON_PROTO_FIRST, "strategy": "proto_first"}
    if context_mode == "oracle":
        return {**COMMON_MMALS, "strategy": "oracle"}
    raise ValueError("context_mode must be uniform, proto, or oracle")

```

1.12 7. Training losses, balanced replay and functional memory

This block retains the v0.14.x continual-learning losses. v1.0-RC1b candidate-freeze validation does not change the base training objective.

```

[9]: def entropy_loss(routes, eps=1e-8):
    if routes is None:
        return torch.zeros(), device=DEVICE
    return -(routes * torch.log(routes + eps)).sum(dim=1).mean()

def metabolic_loss(transformed, routes):
    if transformed is None or routes is None:
        return torch.zeros(), device=DEVICE
    return (routes.unsqueeze(-1) * transformed.pow(2)).sum(dim=(1, 2)).mean()

def kl_distill_loss(logits_current, logits_teacher,
    ↪temperature=DISTILL_TEMPERATURE):
    log_p_current = F.log_softmax(logits_current / temperature, dim=-1)
    p_teacher = F.softmax(logits_teacher / temperature, dim=-1)
    return (temperature ** 2) * F.kl_div(log_p_current, p_teacher,
    ↪reduction="batchmean")

```



```

def clone_teacher(model):
    teacher = copy.deepcopy(model).to(DEVICE)
    teacher.eval()
    for p in teacher.parameters():
        p.requires_grad_(False)
    return teacher

def build_memory_subset(dataset: TensorDataset, max_items: int, seed: int):
    X, y = dataset.tensors
    rng = np.random.default_rng(seed)
    idx = rng.choice(np.arange(len(X)), size=min(max_items, len(X)),
    ↪replace=False)
    return TensorDataset(X[idx].clone(), y[idx].clone())

def sample_from_tensor_dataset(dataset: TensorDataset, batch_size: int):
    X, y = dataset.tensors
    idx = torch.randint(0, len(X), (min(batch_size, len(X)),))
    return X[idx].to(DEVICE), y[idx].to(DEVICE)

def update_class_memory(class_memory: Dict[int, Dict[str, torch.Tensor]],
    ↪dataset: TensorDataset, task_id: int, max_per_class: int, seed: int):
    X, y = dataset.tensors
    rng = np.random.default_rng(seed)
    new_memory = {k: {kk: vv.clone() for kk, vv in v.items()} for k, v in
    ↪class_memory.items()}
    for cls in sorted(torch.unique(y).tolist()):
        cls = int(cls)
        idx = torch.where(y == cls)[0]
        if len(idx) == 0:
            continue
        X_new = X[idx].clone()
        y_new = y[idx].clone()
        t_new = torch.full((len(idx),), task_id, dtype=torch.long)
        if cls in new_memory:
            X_cat = torch.cat([new_memory[cls]["X"], X_new], dim=0)
            y_cat = torch.cat([new_memory[cls]["y"], y_new], dim=0)
            t_cat = torch.cat([new_memory[cls]["task_id"], t_new], dim=0)
        else:
            X_cat, y_cat, t_cat = X_new, y_new, t_new
        if len(X_cat) > max_per_class:
            keep = rng.choice(np.arange(len(X_cat)), size=max_per_class,
            ↪replace=False)
            keep = torch.tensor(keep, dtype=torch.long)
            X_cat, y_cat, t_cat = X_cat[keep], y_cat[keep], t_cat[keep]
        new_memory[cls] = {"X": X_cat, "y": y_cat, "task_id": t_cat}
    return new_memory

```

```

def sample_balanced_class_memory(class_memory: Dict[int, Dict[str, torch.
    ↪Tensor]], batch_size: int):
    active_classes = sorted([int(c) for c, data in class_memory.items() if
    ↪len(data["y"]) > 0])
    if not active_classes:
        return None, None, None
    per_class = max(1, math.ceil(batch_size / len(active_classes)))
    xs, ys, ts = [], [], []
    for cls in active_classes:
        data = class_memory[cls]
        n = len(data["y"])
        k = min(per_class, n)
        idx = torch.randint(0, n, (k,))
        xs.append(data["X"][idx])
        ys.append(data["y"][idx])
        ts.append(data["task_id"][idx])
    X = torch.cat(xs, dim=0)
    y = torch.cat(ys, dim=0)
    t = torch.cat(ts, dim=0)
    perm = torch.randperm(len(y))[: min(batch_size, len(y))]
    return X[perm].to(DEVICE), y[perm].to(DEVICE), t[perm].to(DEVICE)

def balanced_replay_ce_loss(model, class_memory, config, prototypes=None,
    ↪batch_size=BALANCED_REPLAY_BATCH):
    zero = torch.zeros((), device=DEVICE)
    if not config.get("balanced_replay_ce", False) or not class_memory:
        return zero, {"balanced_replay_ce_raw": 0.0, "balanced_replay_classes":
    ↪0, "balanced_replay_items": 0}
    xb, yb, tb = sample_balanced_class_memory(class_memory,
    ↪batch_size=batch_size)
    if xb is None:
        return zero, {"balanced_replay_ce_raw": 0.0, "balanced_replay_classes":
    ↪0, "balanced_replay_items": 0}
    total_loss = zero
    n_groups = 0
    for src_task_id in torch.unique(tb).detach().cpu().tolist():
        src_task_id = int(src_task_id)
        mask = (tb == src_task_id)
        if not mask.any():
            continue
        logits, _ = forward_by_method(
            model,
            xb[mask],
            src_task_id,
            config,
            prototypes=prototypes,

```

```

        train=not BALANCED_REPLAY_USE_EVAL_CONTEXT,
        y=yb[mask],
        return_cache=True,
    )
    total_loss = total_loss + F.cross_entropy(logits, yb[mask])
    n_groups += 1
    raw = total_loss / max(n_groups, 1)
    weight = float(config.get("balanced_replay_weight", BALANCED_REPLAY_WEIGHT))
    return weight * raw, {
        "balanced_replay_ce_raw": float(raw.detach().cpu()),
        "balanced_replay_classes": len(class_memory),
        "balanced_replay_items": int(sum(len(v["y"]) for v in class_memory.
↪values()))),
    }

def context_supervision_weight(config, task_id: int, epoch: int):
    if not config.get("context_calibrated", False):
        return 0.0
    total_steps = max(N_TASKS * EPOCHS_PER_TASK - 1, 1)
    step = task_id * EPOCHS_PER_TASK + epoch
    progress = min(max(step / total_steps, 0.0), 1.0)
    return CONTEXT_SUP_END + (CONTEXT_SUP_START - CONTEXT_SUP_END) * ((1.0 -
↪progress) ** CONTEXT_SUP_GAMMA)

def context_calibration_loss(model, cache, task_id: int, config, epoch: int):
    zero = torch.zeros((), device=DEVICE)
    if not config.get("context_calibrated", False) or config["model_family"] !=
↪"mmals":
        return zero, {"ctx_weight": 0.0, "ctx_ce": 0.0}
    target = torch.full((cache["context_logits"].shape[0],), task_id,
↪dtype=torch.long, device=DEVICE)
    weight = context_supervision_weight(config, task_id, epoch)
    ce = F.cross_entropy(cache["context_logits"], target,
↪label_smoothing=CONTEXT_LABEL_SMOOTHING)
    center_ce = F.cross_entropy(model.
↪context_center_logits(cache["context_feat"]), target,
↪label_smoothing=CONTEXT_LABEL_SMOOTHING)
    entropy = entropy_from_probs(cache["q_single"]).mean()
    margin_penalty = F.relu(CONTEXT_MARGIN_TARGET -
↪margin_from_probs(cache["q_single"])).mean()
    loss = weight * (ce + LAMBDA_CONTEXT_CENTER * center_ce) +
↪LAMBDA_CONTEXT_ENTROPY * entropy + LAMBDA_CONTEXT_MARGIN * margin_penalty
    return loss, {"ctx_weight": float(weight), "ctx_ce": float(ce.detach().
↪cpu())}

```

```

def context_replay_loss(model, teacher, memory_buffers, config,
    ↪batch_size=CONTEXT_REPLAY_BATCH):
    zero = torch.zeros(), device=DEVICE
    if config["model_family"] != "mmals" or not config.get("context_replay",
    ↪False) or not memory_buffers:
        return zero, {"ctx_replay_raw": 0.0, "ctx_replay_scale": config.
    ↪get("context_replay_scale", 0.0)}
    ce_total = zero
    center_total = zero
    kl_total = zero
    feat_total = zero
    n_terms = 0
    for old_context_id, mem_ds in memory_buffers.items():
        xb, _ = sample_from_tensor_dataset(mem_ds, batch_size=batch_size)
        feat_cur, logits_cur, _ = model.context_probs_from_x(xb)
        target = torch.full((xb.shape[0],), old_context_id, dtype=torch.long,
    ↪device=DEVICE)
        ce_total = ce_total + F.cross_entropy(logits_cur, target,
    ↪label_smoothing=CONTEXT_LABEL_SMOOTHING)
        center_total = center_total + F.cross_entropy(model.
    ↪context_center_logits(feat_cur), target,
    ↪label_smoothing=CONTEXT_LABEL_SMOOTHING)
        if teacher is not None and isinstance(teacher, MMALSClazzIncremental):
            with torch.no_grad():
                feat_t, logits_t, _ = teacher.context_probs_from_x(xb)
                kl_total = kl_total + kl_distill_loss(logits_cur, logits_t,
    ↪temperature=CONTEXT_DISTILL_TEMPERATURE)
                feat_total = feat_total + F.mse_loss(feat_cur, feat_t)
            n_terms += 1
        ce_loss = ce_total / max(n_terms, 1)
        center_loss = center_total / max(n_terms, 1)
        kl_loss = kl_total / max(n_terms, 1) if teacher is not None and
    ↪isinstance(teacher, MMALSClazzIncremental) else zero
        feat_loss = feat_total / max(n_terms, 1) if teacher is not None and
    ↪isinstance(teacher, MMALSClazzIncremental) else zero
        raw = LAMBDA_CONTEXT_REPLAY_CE * ce_loss + LAMBDA_CONTEXT_REPLAY_CENTER *
    ↪center_loss + LAMBDA_CONTEXT_TEACHER_KL * kl_loss +
    ↪LAMBDA_CONTEXT_FEATURE_DRIFT * feat_loss
        scale = float(config.get("context_replay_scale", 1.0))
        return scale * raw, {"ctx_replay_raw": float(raw.detach().cpu()),
    ↪"ctx_replay_scale": scale}

def rc3_training_hydro_importance(reference_logits, reference_routes):

```

```

    """Importance signal used only by optional RC2c Hydro training regularizer.
    ↪ """
    probs = F.softmax(reference_logits, dim=-1)
    confidence = probs.max(dim=-1).values.mean()
    if reference_routes is None:
        dominance = torch.tensor(1.0, device=reference_logits.device)
    else:
        dominance = reference_routes.max(dim=-1).values.mean()
    return torch.clamp(confidence * dominance, 0.0, 1.0)

def rc3_training_hydro_viscosity(importance):
    return torch.clamp(
        RC3_HYDRO_BASE_VISCOSITY + (RC3_HYDRO_MAX_VISCOSITY -
    ↪ RC3_HYDRO_BASE_VISCOSITY) * importance,
        RC3_HYDRO_BASE_VISCOSITY,
        RC3_HYDRO_MAX_VISCOSITY,
    )

def old_memory_losses(model, teacher, memory_buffers, config, batch_size,
    ↪ prototypes=None):
    zero = torch.zeros((), device=DEVICE)
    if teacher is None or not memory_buffers:
        return zero, {"route": 0.0, "latent": 0.0, "output": 0.0,
    ↪ "hydro_viscosity": 0.0}
    route_total = zero
    latent_total = zero
    output_total = zero
    hydro_total = zero
    n_terms = 0
    for old_task_id, mem_ds in memory_buffers.items():
        xb, yb = sample_from_tensor_dataset(mem_ds, batch_size=batch_size)
        logits_cur, cache_cur = forward_by_method(model, xb, old_task_id,
    ↪ config, prototypes=prototypes, train=False, y=yb, return_cache=True)
        with torch.no_grad():
            logits_t, cache_t = forward_by_method(teacher, xb, old_task_id,
    ↪ config, prototypes=prototypes, train=False, y=yb, return_cache=True)
            if config["model_family"] == "mmals":
                route_total = route_total + F.mse_loss(cache_cur["routes"],
    ↪ cache_t["routes"])
                latent_total = latent_total + F.mse_loss(cache_cur["zf"],
    ↪ cache_t["zf"])
            if config.get("output_distill", False) or config.get("output", 0.0) > 0:
                output_total = output_total + kl_distill_loss(logits_cur, logits_t)
            if config["model_family"] == "mmals" and globals().
    ↪ get("RC3_HYDRO_REGULARIZER_ENABLED", False):

```

```

        # Optional Hydro viscosity regularizer. It is disabled by default.
        # It reuses existing old-memory teacher comparisons and adds no
        ↪final-test leakage.
        importance = rc3_training_hydro_importance(logits_t.detach(),
        ↪cache_t.get("routes"))
        if globals().get("RC3_HYDRO_REGULARIZER_MODE", "off") ==
        ↪"high_gain_viscous" and float(importance.detach().cpu()) <
        ↪RC3_HYDRO_HIGH_GAIN_IMPORTANCE_THRESHOLD:
            hydro_component = zero
        else:
            nu = rc3_training_hydro_viscosity(importance)
            # Use existing route/latent/output preservation terms as a
            ↪viscosity-weighted stabilizer.
            hydro_component = nu.detach() * (
                0.25 * F.mse_loss(cache_cur["routes"], cache_t["routes"].
        ↪detach())
                + F.mse_loss(cache_cur["zf"], cache_t["zf"].detach())
                + 0.25 * kl_distill_loss(logits_cur, logits_t.detach())
            )
            hydro_total = hydro_total + hydro_component
        n_terms += 1
        route_loss = route_total / max(n_terms, 1)
        latent_loss = latent_total / max(n_terms, 1)
        output_loss = output_total / max(n_terms, 1)
        hydro_loss = hydro_total / max(n_terms, 1)
        loss = config.get("route", 0.0) * route_loss + config.get("latent", 0.0) *
        ↪latent_loss + config.get("output", 0.0) * output_loss
        if config["model_family"] == "mmals" and globals().
        ↪get("RC3_HYDRO_REGULARIZER_ENABLED", False):
            loss = loss + RC3_HYDRO_REGULARIZER_COEFF * hydro_loss
        if config["model_family"] == "mlp" and config.get("output_distill", False):
            loss = output_loss
        return loss, {"route": float(route_loss.detach().cpu()), "latent":
        ↪float(latent_loss.detach().cpu()), "output": float(output_loss.detach().
        ↪cpu()), "hydro_viscosity": float(hydro_loss.detach().cpu())}

@torch.no_grad()
def calibrate_context_temperature(model, memory_buffers, config,
        ↪candidate_temps=CONTEXT_TEMP_GRID, max_items=TEMP_CALIB_MAX_ITEMS):
    if config["model_family"] != "mmals" or not config.get("temp_calibration",
        ↪False) or not memory_buffers:
        return float(getattr(model, "context_temperature", float("nan")))
    saved_temp = float(model.context_temperature)
    logits_all, targets_all = [], []
    model.eval()
    for cid, mem_ds in memory_buffers.items():

```

```

X, _ = mem_ds.tensors
if len(X) > max_items:
    X = X[torch.randperm(len(X))[:max_items]]
loader = make_loader(TensorDataset(X, torch.zeros(len(X), dtype=torch.
↳long)), batch_size=BATCH_SIZE, shuffle=False)
for xb, _ in loader:
    xb = xb.to(DEVICE)
    _, logits = model.context_encoder(xb)
    logits_all.append(logits.detach())
    targets_all.append(torch.full((xb.shape[0],), cid, dtype=torch.
↳long, device=DEVICE))
if not logits_all:
    return saved_temp
logits_all = torch.cat(logits_all, dim=0)
targets_all = torch.cat(targets_all, dim=0)
best_temp, best_loss = saved_temp, float("inf")
for temp in candidate_temps:
    loss = F.cross_entropy(logits_all / temp, targets_all).item()
    if loss < best_loss:
        best_loss = loss
        best_temp = float(temp)
model.context_temperature = best_temp
return best_temp

# -----
# v0.14.8 head-repair utilities
# -----

def head_margin_repair_loss(logits, y, target_margin=HEAD_MARGIN_TARGET,
↳hard_classes=HARD_CLASSES, ref_class=REFERENCE_CLASS_FOR_MARGIN):
    """Penalize hard-class logits when they fall below the reference class
↳logit.

    This is intentionally head-focused: it does not change routes or latent
↳states.
    """
    if logits is None or y is None:
        return torch.zeros((), device=DEVICE)
    losses = []
    for cls in hard_classes:
        if cls >= logits.shape[1] or ref_class >= logits.shape[1]:
            continue
        mask = (y == cls)
        if mask.any():
            margin = logits[mask, cls] - logits[mask, ref_class]
            losses.append(F.relu(target_margin - margin).mean())
    if not losses:

```

```

        return torch.zeros(), device=logits.device)
    return torch.stack(losses).mean()

def class_memory_to_tensor_dataset(class_memory: Dict[int, Dict[str, torch.
    ↪Tensor]]):
    if not class_memory:
        return None
    xs, ys, ts = [], [], []
    for cls, data in sorted(class_memory.items()):
        if len(data["y"]) == 0:
            continue
        xs.append(data["X"])
        ys.append(data["y"])
        ts.append(data["task_id"])
    if not xs:
        return None
    return TensorDataset(torch.cat(xs, dim=0), torch.cat(ys, dim=0), torch.
    ↪cat(ts, dim=0))

@torch.no_grad()
def extract_zf_from_batch_by_task(model, xb, tb, config, prototypes=None):
    """Extract frozen z_f for a mixed replay batch containing items from
    ↪several source tasks."""
    zf_out = torch.zeros((xb.shape[0], LATENT_DIM), device=xb.device)
    for src_task_id in torch.unique(tb).detach().cpu().tolist():
        src_task_id = int(src_task_id)
        mask = (tb == src_task_id)
        if not mask.any():
            continue
        _, cache = forward_by_method(
            model,
            xb[mask],
            src_task_id,
            config,
            prototypes=prototypes,
            train=False,
            y=None,
            return_cache=True,
        )
        zf_out[mask] = cache["zf"].detach()
    return zf_out

def sample_repair_batch(class_memory, batch_size=HEAD_REPAIR_BATCH):

```



```

    xb, yb, tb = sample_balanced_class_memory(class_memory,
↪batch_size=batch_size)
    return xb, yb, tb

def train_head_refresh_phase(model, class_memory, config, prototypes=None,
↪steps=HEAD_REFRESH_STEPS, epochs=HEAD_REFRESH_EPOCHS, lr=HEAD_REFRESH_LR):
    """Freeze the fungal backbone and update only the deployed global head
↪using balanced memory."""
    rows = []
    if config["model_family"] != "mmals" or not class_memory:
        return rows
    model.eval()
    opt = torch.optim.Adam(model.global_head.parameters(), lr=lr)
    for epoch in range(epochs):
        sums = {"head_refresh_loss": 0.0, "head_refresh_ce": 0.0,
↪"head_refresh_margin": 0.0}
        n = 0
        for _ in range(steps):
            xb, yb, tb = sample_repair_batch(class_memory)
            if xb is None:
                continue
            with torch.no_grad():
                zf = extract_zf_from_batch_by_task(model, xb, tb, config,
↪prototypes=prototypes)
            opt.zero_grad(set_to_none=True)
            logits = model.global_head(zf)
            ce = F.cross_entropy(logits, yb)
            margin = head_margin_repair_loss(logits, yb)
            loss = ce + HEAD_MARGIN_LAMBDA * margin
            loss.backward()
            opt.step()
            sums["head_refresh_loss"] += float(loss.detach().cpu())
            sums["head_refresh_ce"] += float(ce.detach().cpu())
            sums["head_refresh_margin"] += float(margin.detach().cpu())
            n += 1
        if n > 0:
            rows.append({k: v / n for k, v in sums.items()} | {"repair_epoch":
↪epoch, "repair_type": "head_refresh"})
    return rows

def build_memory_feature_dataset(model, class_memory, config, prototypes=None,
↪max_items=None):
    mem_ds = class_memory_to_tensor_dataset(class_memory)
    if mem_ds is None:

```

```

        return None
    X, y, t = mem_ds.tensors
    if max_items is not None and len(y) > max_items:
        idx = torch.randperm(len(y))[:max_items]
        X, y, t = X[idx], y[idx], t[idx]
    feats, labels = [], []
    loader = make_loader(TensorDataset(X, y, t), batch_size=BATCH_SIZE,
↪shuffle=False)
    model.eval()
    for xb, yb, tb in loader:
        xb, yb, tb = xb.to(DEVICE), yb.to(DEVICE), tb.to(DEVICE)
        with torch.no_grad():
            zf = extract_zf_from_batch_by_task(model, xb, tb, config,
↪prototypes=prototypes)
            feats.append(zf.detach().cpu())
            labels.append(yb.detach().cpu())
    return TensorDataset(torch.cat(feats, dim=0).float(), torch.cat(labels,
↪dim=0).long())

def train_temporary_zf_probe(feature_ds, seed=0, epochs=PROBE_EPOCHS):
    if feature_ds is None:
        return None, []
    set_seed(seed + 81000)
    probe = LinearProbe(input_dim=LATENT_DIM).to(DEVICE)
    opt = torch.optim.Adam(probe.parameters(), lr=PROBE_LR)
    loader = make_loader(feature_ds, batch_size=BATCH_SIZE, shuffle=True)
    rows = []
    for epoch in range(epochs):
        total, n = 0.0, 0
        probe.train()
        for zf, yb in loader:
            zf, yb = zf.to(DEVICE), yb.to(DEVICE)
            opt.zero_grad(set_to_none=True)
            loss = F.cross_entropy(probe(zf), yb)
            loss.backward()
            opt.step()
            total += float(loss.detach().cpu())
            n += 1
        rows.append({"repair_epoch": epoch, "repair_type": "temporary_probe",
↪"probe_loss": total / max(n, 1)})
    probe.eval()
    for p in probe.parameters():
        p.requires_grad_(False)
    return probe, rows

```

```

def train_probe_distilled_head_phase(model, class_memory, config,
    ↪prototypes=None, seed=0, steps=HEAD_REFRESH_STEPS,
    ↪epochs=PROBE_DISTILL_EPOCHS, lr=PROBE_DISTILL_LR):
    """Train a temporary linear probe on frozen z_f and distill it into the
    ↪deployed global head."""
    rows = []
    if config["model_family"] != "mmals" or not class_memory:
        return rows
    feature_ds = build_memory_feature_dataset(model, class_memory, config,
    ↪prototypes=prototypes)
    probe, probe_rows = train_temporary_zf_probe(feature_ds, seed=seed,
    ↪epochs=max(5, PROBE_EPOCHS // 2))
    rows.extend(probe_rows)
    if probe is None:
        return rows
    model.eval()
    opt = torch.optim.Adam(model.global_head.parameters(), lr=lr)
    for epoch in range(epochs):
        sums = {"probe_distill_loss": 0.0, "probe_distill_ce": 0.0,
    ↪"probe_distill_kl": 0.0, "probe_distill_margin": 0.0}
        n = 0
        for _ in range(steps):
            xb, yb, tb = sample_repair_batch(class_memory)
            if xb is None:
                continue
            with torch.no_grad():
                zf = extract_zf_from_batch_by_task(model, xb, tb, config,
    ↪prototypes=prototypes)
                probe_logits = probe(zf)
                opt.zero_grad(set_to_none=True)
                logits = model.global_head(zf)
                ce = F.cross_entropy(logits, yb)
                kl = kl_distill_loss(logits, probe_logits,
    ↪temperature=PROBE_DISTILL_TEMPERATURE)
                margin = head_margin_repair_loss(logits, yb)
                loss = PROBE_DISTILL_ALPHA_CE * ce + PROBE_DISTILL_ALPHA_KL * kl +
    ↪HEAD_MARGIN_LAMBDA * margin
                loss.backward()
                opt.step()
                sums["probe_distill_loss"] += float(loss.detach().cpu())
                sums["probe_distill_ce"] += float(ce.detach().cpu())
                sums["probe_distill_kl"] += float(kl.detach().cpu())
                sums["probe_distill_margin"] += float(margin.detach().cpu())
                n += 1
        if n > 0:

```

```

        rows.append({k: v / n for k, v in sums.items()} | {"repair_epoch": epoch, "repair_type": "probe_distill"})
    return rows

def train_bias_calibration_phase(model, class_memory, config, prototypes=None, steps=BIAS_CALIB_STEPS, lr=BIAS_CALIB_LR):
    """Learn a tiny post-hoc class-bias correction on balanced memory and fold it into the global head bias."""
    rows = []
    if config["model_family"] != "mmals" or not class_memory:
        return rows
    delta = nn.Parameter(torch.zeros(N_CLASSES, device=DEVICE))
    opt = torch.optim.Adam([delta], lr=lr)
    for step in range(steps):
        xb, yb, tb = sample_repair_batch(class_memory)
        if xb is None:
            continue
        with torch.no_grad():
            zf = extract_zf_from_batch_by_task(model, xb, tb, config, prototypes=prototypes)
            base_logits = model.global_head(zf).detach()
            opt.zero_grad(set_to_none=True)
            logits = base_logits + delta.unsqueeze(0)
            ce = F.cross_entropy(logits, yb)
            margin = head_margin_repair_loss(logits, yb)
            reg = BIAS_CALIB_REG * delta.pow(2).mean()
            loss = ce + HEAD_MARGIN_LAMBDA * margin + reg
            loss.backward()
            opt.step()
            rows.append({
                "repair_epoch": step,
                "repair_type": "bias_calibration",
                "bias_calib_loss": float(loss.detach().cpu()),
                "bias_calib_ce": float(ce.detach().cpu()),
                "bias_calib_margin": float(margin.detach().cpu()),
            })
        with torch.no_grad():
            if model.global_head.bias is not None:
                model.global_head.bias.add_(delta.detach())
    return rows

def maybe_run_head_repair_phases(model, class_memory, config, prototypes=None, seed=0):
    rows = []
    if config["model_family"] != "mmals":

```

```

        return rows
    if config.get("post_head_refresh", False):
        rows.extend(train_head_refresh_phase(model, class_memory, config,
        ↪prototypes=prototypes))
    if config.get("post_probe_distill", False):
        rows.extend(train_probe_distilled_head_phase(model, class_memory,
        ↪config, prototypes=prototypes, seed=seed))
    if config.get("post_bias_calibration", False):
        rows.extend(train_bias_calibration_phase(model, class_memory, config,
        ↪prototypes=prototypes))
    return rows

```

1.13 8. Readout and v1.0-RC1b context-posterior utilities

The local readout classes from v1.0a/v1.0b are kept as reusable infrastructure, but the RC1b validation target is **not** a local-head branch.

The object of validation is the inferred-context posterior used to extract **zf**, plus a true CE+KL guarded global readout. Deployable variants remain task-free. Oracle variants are diagnostic references only.

```

[10]: def clone_tensor_dataset(ds: TensorDataset):
        tensors = tuple(t.clone().detach().cpu() for t in ds.tensors)
        return TensorDataset(*tensors)

    def clone_memory_buffers(memory_buffers: Dict[int, TensorDataset]):
        return {int(k): clone_tensor_dataset(v) for k, v in memory_buffers.items()}

    def clone_class_memory(class_memory):
        return {int(k): {kk: vv.clone().detach().cpu() for kk, vv in v.items()} for
        ↪k, v in class_memory.items()}

    def clone_prototypes(prototypes):
        out = {}
        for cid, proto in prototypes.items():
            out[int(cid)] = {}
            for kind, stats in proto.items():
                out[int(cid)][kind] = {}
                for sk, sv in stats.items():
                    if torch.is_tensor(sv):
                        out[int(cid)][kind][sk] = sv.clone().detach().cpu()
                    else:
                        out[int(cid)][kind][sk] = copy.deepcopy(sv)
        return out

    def cpu_state_dict(model):
        return {k: v.detach().cpu().clone() for k, v in model.state_dict().items()}

```

```

def restore_base_model_from_checkpoint(checkpoint, config):
    model = build_model(config)
    model.load_state_dict(checkpoint["model_state"])
    if hasattr(model, "context_temperature"):
        model.context_temperature = float(checkpoint.get("context_temperature",
↪CONTEXT_TEMPERATURE))
    model.to(DEVICE)
    return model

def split_class_memory_for_readout_validation(class_memory,
↪val_fraction=VALIDATION_SPLIT_FRACTION, seed=0):
    if not class_memory:
        return {}, {}
    rng = np.random.default_rng(seed)
    repair_memory, validation_memory = {}, {}
    for cls, data in class_memory.items():
        n = len(data["y"])
        if n == 0:
            continue
        idx = np.arange(n)
        rng.shuffle(idx)
        n_val = int(round(n * val_fraction))
        n_val = min(max(n_val, 1 if n >= 4 else 0), max(n - 1, 0))
        val_idx = torch.tensor(idx[:n_val], dtype=torch.long)
        train_idx = torch.tensor(idx[n_val:], dtype=torch.long)
        if len(train_idx) > 0:
            repair_memory[int(cls)] = {k: v[train_idx].clone() for k, v in data.
↪items()}
        if len(val_idx) > 0:
            validation_memory[int(cls)] = {k: v[val_idx].clone() for k, v in
↪data.items()}
    if not repair_memory:
        repair_memory = clone_class_memory(class_memory)
    if not validation_memory:
        validation_memory = clone_class_memory(class_memory)
    return repair_memory, validation_memory

@torch.no_grad()
def collect_zf_memory(model, class_memory, config, prototypes=None,
↪max_items=None):
    mem_ds = class_memory_to_tensor_dataset(class_memory)
    if mem_ds is None:
        return None
    X, y, t = mem_ds.tensors
    if max_items is not None and len(y) > max_items:
        idx = torch.randperm(len(y))[:max_items]

```

```

        X, y, t = X[idx], y[idx], t[idx]
        Zs, Ys, Ts, Qs = [], [], [], []
        loader = make_loader(TensorDataset(X, y, t), batch_size=BATCH_SIZE,
↪shuffle=False)
        model.eval()
        for xb, yb, tb in loader:
            xb, yb, tb = xb.to(DEVICE), yb.to(DEVICE), tb.to(DEVICE)
            for src_task_id in torch.unique(tb).detach().cpu().tolist():
                src_task_id = int(src_task_id)
                mask = (tb == src_task_id)
                if not mask.any():
                    continue
                _, cache = forward_by_method(
                    model, xb[mask], src_task_id, config,
                    prototypes=prototypes, train=False, y=yb[mask],
↪return_cache=True
                )
                Zs.append(cache["zf"].detach().cpu())
                Ys.append(yb[mask].detach().cpu())
                Ts.append(tb[mask].detach().cpu())
                Qs.append(cache["q_context"].detach().cpu())
        if not Zs:
            return None
        return {
            "Z": torch.cat(Zs, dim=0).float(),
            "y": torch.cat(Ys, dim=0).long(),
            "task_id": torch.cat(Ts, dim=0).long(),
            "q_context": torch.cat(Qs, dim=0).float(),
        }

def compute_class_prototypes(memory_features, n_classes=N_CLASSES, eps=1e-8):
    Z = memory_features["Z"]
    y = memory_features["y"]
    mu = torch.zeros(n_classes, Z.shape[1])
    counts = torch.zeros(n_classes)
    valid = torch.zeros(n_classes, dtype=torch.bool)
    for cls in range(n_classes):
        mask = (y == cls)
        if mask.any():
            mu[cls] = Z[mask].mean(dim=0)
            counts[cls] = float(mask.sum())
            valid[cls] = True
    return mu, valid, counts

def compute_context_class_prototypes(memory_features, n_contexts=N_CONTEXTS,
↪n_classes=N_CLASSES):
    Z = memory_features["Z"]

```

```

y = memory_features["y"]
t = memory_features["task_id"]
global_mu, global_valid, _ = compute_class_prototypes(memory_features,
↪n_classes=n_classes)
mu = torch.zeros(n_contexts, n_classes, Z.shape[1])
valid = torch.zeros(n_contexts, n_classes, dtype=torch.bool)
for cid in range(n_contexts):
    for cls in range(n_classes):
        mask = (t == cid) & (y == cls)
        if mask.any():
            mu[cid, cls] = Z[mask].mean(dim=0)
            valid[cid, cls] = True
        elif global_valid[cls]:
            mu[cid, cls] = global_mu[cls]
            valid[cid, cls] = True
    return mu, valid, global_mu, global_valid

class GlobalLinearReadout(nn.Module):
    def __init__(self, base_head):
        super().__init__()
        self.linear = copy.deepcopy(base_head)
    def forward(self, zf, q_context=None):
        return self.linear(zf)

class PrototypeCosineReadout(nn.Module):
    def __init__(self, prototypes, valid, scale=COSINE_SCALE_INIT, bias=None):
        super().__init__()
        self.register_buffer("prototypes", F.normalize(prototypes.float(),
↪dim=-1))
        self.register_buffer("valid", valid.bool())
        self.scale = nn.Parameter(torch.tensor(float(scale)))
        self.bias = nn.Parameter(torch.zeros(N_CLASSES) if bias is None else
↪bias.float())
    def forward(self, zf, q_context=None):
        z = F.normalize(zf, dim=-1)
        logits = self.scale.clamp(1.0, 80.0) * (z @ self.prototypes.T) + self.
↪bias
        logits = logits.masked_fill(~self.valid.unsqueeze(0), -1e9)
        return logits

class PrototypeEuclideanReadout(nn.Module):
    def __init__(self, prototypes, valid, temperature=READOUT_TEMP, bias=None):
        super().__init__()
        self.register_buffer("prototypes", prototypes.float())
        self.register_buffer("valid", valid.bool())
        self.log_temperature = nn.Parameter(torch.log(torch.
↪tensor(float(temperature))))

```



```

        self.bias = nn.Parameter(torch.zeros(N_CLASSES) if bias is None else
↪bias.float())
    def forward(self, zf, q_context=None):
        diff = zf.unsqueeze(1) - self.prototypes.unsqueeze(0)
        dist = diff.pow(2).mean(dim=-1)
        temp = torch.exp(self.log_temperature).clamp(0.03, 5.0)
        logits = -dist / temp + self.bias
        logits = logits.masked_fill(~self.valid.unsqueeze(0), -1e9)
        return logits

class TrainablePrototypeEuclideanReadout(nn.Module):
    """Trainable prototype readout without per-dimension adapter.

    This separates the effect of learning class centers/bias/temperature from
↪the
    stronger diagonal local adapter.
    """
    def __init__(self, prototypes, valid, temperature=READOUT_TEMP):
        super().__init__()
        self.mu = nn.Parameter(prototypes.float().clone())
        self.bias = nn.Parameter(torch.zeros(N_CLASSES))
        self.log_temperature = nn.Parameter(torch.log(torch.
↪tensor(float(temperature))))
        self.register_buffer("valid", valid.bool())
    def forward(self, zf, q_context=None):
        diff = zf.unsqueeze(1) - self.mu.unsqueeze(0)
        dist = diff.pow(2).mean(dim=-1)
        temp = torch.exp(self.log_temperature).clamp(0.03, 5.0)
        logits = -dist / temp + self.bias
        logits = logits.masked_fill(~self.valid.unsqueeze(0), -1e9)
        return logits

class LocalAdapterPrototypeReadout(nn.Module):
    def __init__(self, prototypes, valid, temperature=READOUT_TEMP):
        super().__init__()
        self.mu = nn.Parameter(prototypes.float().clone())
        self.log_diag = nn.Parameter(torch.zeros_like(prototypes.float()))
        self.bias = nn.Parameter(torch.zeros(N_CLASSES))
        self.log_temperature = nn.Parameter(torch.log(torch.
↪tensor(float(temperature))))
        self.register_buffer("valid", valid.bool())
    def forward(self, zf, q_context=None):
        alpha = torch.exp(self.log_diag.clamp(-2.0, 2.0))
        diff = (zf.unsqueeze(1) - self.mu.unsqueeze(0)) * alpha.unsqueeze(0)
        dist = diff.pow(2).mean(dim=-1)
        temp = torch.exp(self.log_temperature).clamp(0.03, 5.0)
        logits = -dist / temp + self.bias

```

```

        logits = logits.masked_fill(~self.valid.unsqueeze(0), -1e9)
        return logits

class ContextPrototypeCosineReadout(nn.Module):
    def __init__(self, context_prototypes, context_valid,
        ↪scale=COSINE_SCALE_INIT):
        super().__init__()
        self.register_buffer("context_prototypes", F.
        ↪normalize(context_prototypes.float(), dim=-1))
        self.register_buffer("context_valid", context_valid.bool())
        self.scale = nn.Parameter(torch.tensor(float(scale)))
        self.bias = nn.Parameter(torch.zeros(N_CLASSES))
    def forward(self, zf, q_context=None):
        if q_context is None:
            q_context = torch.full((zf.shape[0], N_CONTEXTS), 1.0 / N_CONTEXTS,
        ↪device=zf.device)
            z = F.normalize(zf, dim=-1)
            # scores: batch x context x class
            scores = torch.einsum("bd,cyd->bcy", z, self.context_prototypes.to(zf.
        ↪device))
            scores = scores.masked_fill(~self.context_valid.to(zf.device).
        ↪unsqueeze(0), -1e9)
            logits = (q_context.unsqueeze(-1) * scores).sum(dim=1)
            logits = self.scale.clamp(1.0, 80.0) * logits + self.bias
            return logits

def build_readout_from_memory(model, class_memory, config, prototypes, variant,
    ↪seed=0):
    set_seed(seed)
    feats = collect_zf_memory(model, class_memory, config,
    ↪prototypes=prototypes)
    if feats is None:
        return GlobalLinearReadout(model.global_head), [{"repair_type":
    ↪"fallback_global_empty_memory"}]
    class_mu, class_valid, counts = compute_class_prototypes(feats)
    rows = [{
        "repair_type": "prototype_build",
        "variant": variant,
        "memory_items": int(len(feats["y"])),
        "valid_classes": int(class_valid.sum().item()),
    }]
    if variant == "local_proto_cosine":
        return PrototypeCosineReadout(class_mu, class_valid).to(DEVICE), rows
    if variant == "local_proto_euclidean":
        return PrototypeEuclideanReadout(class_mu, class_valid).to(DEVICE), rows
    if variant == "local_trainable_proto_ce_kl_035":

```

```

        return TrainablePrototypeEuclideanReadout(class_mu, class_valid).
↳to(DEVICE), rows
        if variant.startswith("local_diag_adapter") or variant == "
↳"local_adapter_proto_ce_kl_035":
            return LocalAdapterPrototypeReadout(class_mu, class_valid).to(DEVICE),
↳rows
        if variant == "context_local_proto_cosine":
            ctx_mu, ctx_valid, _gm, _gv = compute_context_class_prototypes(feats)
            return ContextPrototypeCosineReadout(ctx_mu, ctx_valid).to(DEVICE), rows
        raise ValueError(f"Unknown local readout variant: {variant}")

def sample_repair_batch_v10b(class_memory, batch_size=HEAD_REPAIR_BATCH):
    if not class_memory:
        return None, None, None
    return sample_balanced_class_memory(class_memory, batch_size)

def readout_ce_kl_loss(readout, zf, q_context, yb, pre_global_head=None,
↳kl_weight=READOUT_KL_WEIGHT, use_kl=True, margin_weight=0.0):
    logits = readout(zf, q_context)
    ce = F.cross_entropy(logits, yb)
    kl = logits.sum() * 0.0
    if use_kl and pre_global_head is not None and kl_weight > 0:
        with torch.no_grad():
            pre_logits = pre_global_head(zf)
            kl = kl_distill_loss(logits, pre_logits,
↳temperature=PROBE_DISTILL_TEMPERATURE)
            margin = head_margin_repair_loss(logits, yb) if margin_weight > 0 else
↳logits.sum() * 0.0
    return ce + kl_weight * kl + margin_weight * margin, {"ce": ce, "kl": kl,
↳"margin": margin}, logits

def train_readout_on_memory(readout, model, class_memory, config,
↳prototypes=None, pre_global_head=None,
                                epochs=READOUT_TRAIN_EPOCHS,
↳steps=READOUT_TRAIN_STEPS,
                                lr=READOUT_LR, kl_weight=READOUT_KL_WEIGHT,
↳use_kl=True,
                                margin_weight=0.0, variant="readout_train"):
    rows = []
    if not class_memory:
        return rows
    # Only train readout parameters. The fungal backbone remains frozen.
    model.eval()
    for p in model.parameters():
        p.requires_grad_(False)
    params = [p for p in readout.parameters() if p.requires_grad]

```

```

if not params:
    return rows
opt = torch.optim.Adam(params, lr=lr)
for epoch in range(epochs):
    sums = {"loss": 0.0, "ce": 0.0, "kl": 0.0, "margin": 0.0}
    n = 0
    for _ in range(steps):
        xb, yb, tb = sample_repair_batch_v10b(class_memory)
        if xb is None:
            continue
        xb, yb, tb = xb.to(DEVICE), yb.to(DEVICE), tb.to(DEVICE)
        zf_list, q_list, y_list = [], [], []
        with torch.no_grad():
            for src_task_id in torch.unique(tb).detach().cpu().tolist():
                src_task_id = int(src_task_id)
                mask = (tb == src_task_id)
                if not mask.any():
                    continue
                _, cache = forward_by_method(model, xb[mask], src_task_id,
↪config, prototypes=prototypes, train=False, y=yb[mask], return_cache=True)
                zf_list.append(cache["zf"].detach())
                q_list.append(cache["q_context"].detach())
                y_list.append(yb[mask].detach())
            if not zf_list:
                continue
            zf = torch.cat(zf_list, dim=0)
            q_context = torch.cat(q_list, dim=0)
            y_mix = torch.cat(y_list, dim=0)
            opt.zero_grad(set_to_none=True)
            loss, details, _ = readout_ce_kl_loss(readout, zf, q_context,
↪y_mix, pre_global_head=pre_global_head, kl_weight=kl_weight, use_kl=use_kl,
↪margin_weight=margin_weight)
            loss.backward()
            torch.nn.utils.clip_grad_norm_(params, max_norm=5.0)
            opt.step()
            sums["loss"] += float(loss.detach().cpu())
            sums["ce"] += float(details["ce"].detach().cpu())
            sums["kl"] += float(details["kl"].detach().cpu())
            sums["margin"] += float(details["margin"].detach().cpu())
            n += 1
    if n > 0:
        rows.append({
            "repair_epoch": epoch,
            "repair_type": variant,
            "readout_train_loss": sums["loss"] / n,
            "readout_train_ce": sums["ce"] / n,
            "readout_train_kl": sums["kl"] / n,

```

```

        "readout_train_margin": sums["margin"] / n,
        "kl_weight": kl_weight,
        "margin_weight": margin_weight,
    })
    return rows

@torch.no_grad()
def proxy_metrics_for_readout(model, readout, class_memory, config,
    ⇨ prototypes=None, max_items_per_class=VALIDATION_MAX_ITEMS_PER_CLASS):
    if not class_memory:
        return {"proxy_acc": np.nan, "proxy_class2": np.nan, "proxy_class5": np.
    ⇨ nan, "proxy_min_class": np.nan}
    correct_by_cls = {c: 0 for c in range(N_CLASSES)}
    total_by_cls = {c: 0 for c in range(N_CLASSES)}
    model.eval(); readout.eval()
    for cls, data in sorted(class_memory.items()):
        X, y, t = data["X"], data["y"], data["task_id"]
        if len(y) == 0:
            continue
        if len(y) > max_items_per_class:
            idx = torch.randperm(len(y))[:max_items_per_class]
            X, y, t = X[idx], y[idx], t[idx]
        loader = make_loader(TensorDataset(X, y, t), batch_size=BATCH_SIZE,
    ⇨ shuffle=False)
        for xb, yb, tb in loader:
            xb, yb, tb = xb.to(DEVICE), yb.to(DEVICE), tb.to(DEVICE)
            for src_task_id in torch.unique(tb).detach().cpu().tolist():
                src_task_id = int(src_task_id)
                mask = (tb == src_task_id)
                if not mask.any():
                    continue
                _, cache = forward_by_method(model, xb[mask], src_task_id,
    ⇨ config, prototypes=prototypes, train=False, y=yb[mask], return_cache=True)
                logits = readout(cache["zf"], cache["q_context"])
                seen = seen_classes_until(src_task_id)
                pred = mask_logits_to_seen_classes(logits, seen).argmax(dim=-1)
                yy = yb[mask]
                for c in torch.unique(yy).detach().cpu().tolist():
                    c = int(c)
                    m = (yy == c)
                    total_by_cls[c] += int(m.sum().item())
                    correct_by_cls[c] += int((pred[m] == yy[m]).sum().item())
            per_cls = {c: (correct_by_cls[c] / max(total_by_cls[c], 1) if
    ⇨ total_by_cls[c] > 0 else np.nan) for c in range(N_CLASSES)}
            total = sum(total_by_cls.values())
            correct = sum(correct_by_cls.values())
            valid_accs = [v for v in per_cls.values() if not np.isnan(v)]

```

```

    return {
        "proxy_acc": correct / max(total, 1),
        "proxy_class2": per_cls.get(2, np.nan),
        "proxy_class5": per_cls.get(5, np.nan),
        "proxy_min_class": float(np.min(valid_accs)) if valid_accs else np.nan,
    }

def validation_score(candidate, baseline):
    if candidate is None:
        return -1e9
    acc = candidate.get("proxy_acc", np.nan)
    c2 = candidate.get("proxy_class2", np.nan)
    minc = candidate.get("proxy_min_class", np.nan)
    base_acc = baseline.get("proxy_acc", np.nan)
    if np.isnan(acc):
        return -1e9
    damage = max(0.0, (base_acc if not np.isnan(base_acc) else acc) - acc)
    return float(
        acc
        + VALIDATION_SCORE_CLASS2_WEIGHT * (0.0 if np.isnan(c2) else c2)
        + VALIDATION_SCORE_MINCLASS_WEIGHT * (0.0 if np.isnan(minc) else minc)
        - VALIDATION_SCORE_DAMAGE_WEIGHT * damage
    )

def build_and_train_readout_variant(model, class_memory, config, prototypes,
    variant, seed=0):
    """Return a readout and logs for one paired local-head variant."""
    set_seed(seed)
    repair_memory, validation_memory =
    split_class_memory_for_readout_validation(class_memory, seed=seed + 100)
    pre_global_head = copy.deepcopy(model.global_head).to(DEVICE)
    pre_global_head.eval()
    for p in pre_global_head.parameters():
        p.requires_grad_(False)
    rows = []

    if variant in ["no_repair_global_head", "kl_only_true_noop"]:
        readout = GlobalLinearReadout(model.global_head).to(DEVICE)
        rows.append({"repair_type": "no_op", "variant": variant})
        return readout, rows

    if variant == "global_head_ce_kl_guard_035":
        readout = GlobalLinearReadout(model.global_head).to(DEVICE)
        rows.extend(train_readout_on_memory(readout, model, repair_memory,
    config, prototypes=prototypes, pre_global_head=pre_global_head,
        kl_weight=0.35, use_kl=True,
    variant="global_head_ce_kl_guard_035"))

```

```

    return readout, rows

    if variant in ["local_proto_cosine", "local_proto_euclidean",
↪ "context_local_proto_cosine"]:
        readout, build_rows = build_readout_from_memory(model, repair_memory,
↪ config, prototypes, variant, seed=seed)
        rows.extend(build_rows)
        return readout, rows

    if variant == "local_adapter_proto_ce_kl_035":
        # Backward-compatible alias from v1.0a.
        readout, build_rows = build_readout_from_memory(model, repair_memory,
↪ config, prototypes, "local_diag_adapter_ce_kl_035", seed=seed)
        rows.extend(build_rows)
        rows.extend(train_readout_on_memory(readout, model, repair_memory,
↪ config, prototypes=prototypes, pre_global_head=pre_global_head,
                                                kl_weight=0.35, use_kl=True,
↪ margin_weight=0.0, variant="local_adapter_proto_ce_kl_035"))
        return readout, rows

    if variant == "local_trainable_proto_ce_kl_035":
        readout, build_rows = build_readout_from_memory(model, repair_memory,
↪ config, prototypes, variant, seed=seed)
        rows.extend(build_rows)
        rows.extend(train_readout_on_memory(readout, model, repair_memory,
↪ config, prototypes=prototypes, pre_global_head=pre_global_head,
                                                kl_weight=0.35, use_kl=True,
↪ margin_weight=0.0, variant=variant))
        return readout, rows

    if variant.startswith("local_diag_adapter_ce"):
        readout, build_rows = build_readout_from_memory(model, repair_memory,
↪ config, prototypes, variant, seed=seed)
        rows.extend(build_rows)
        if variant == "local_diag_adapter_ce_only":
            kl_weight, use_kl, margin_weight = 0.0, False, 0.0
        elif variant == "local_diag_adapter_ce_kl_005":
            kl_weight, use_kl, margin_weight = 0.05, True, 0.0
        elif variant == "local_diag_adapter_ce_kl_015":
            kl_weight, use_kl, margin_weight = 0.15, True, 0.0
        elif variant == "local_diag_adapter_ce_kl_035":
            kl_weight, use_kl, margin_weight = 0.35, True, 0.0
        elif variant == "local_diag_adapter_ce_kl_060":
            kl_weight, use_kl, margin_weight = 0.60, True, 0.0
        elif variant == "local_diag_adapter_ce_kl_margin_035":

```

```

        kl_weight, use_kl, margin_weight = 0.35, True,
↪LOCAL_ADAPTER_MARGIN_WEIGHT
    else:
        raise ValueError(f"Unknown local diagonal adapter variant:
↪{variant}")
        rows.extend(train_readout_on_memory(readout, model, repair_memory,
↪config, prototypes=prototypes, pre_global_head=pre_global_head,
                                                kl_weight=kl_weight, use_kl=use_kl,
↪margin_weight=margin_weight, variant=variant))
        return readout, rows

    if variant in ["local_mycorrhizal_selected",
↪"local_adapter_validation_selected"]:
        baseline_readout = GlobalLinearReadout(model.global_head).to(DEVICE)
        baseline_proxy = proxy_metrics_for_readout(model, baseline_readout,
↪validation_memory, config, prototypes=prototypes)
        candidates = [
            "global_head_ce_kl_guard_035",
            "local_proto_euclidean",
            "local_trainable_proto_ce_kl_035",
            "local_diag_adapter_ce_only",
            "local_diag_adapter_ce_kl_005",
            "local_diag_adapter_ce_kl_015",
            "local_diag_adapter_ce_kl_035",
            "local_diag_adapter_ce_kl_060",
            "local_diag_adapter_ce_kl_margin_035",
        ]
        best_readout = baseline_readout
        best_name = "no_repair_global_head"
        best_proxy = baseline_proxy
        best_score = validation_score(baseline_proxy, baseline_proxy)
        rows.append({"repair_type": "selector_baseline", "candidate":
↪best_name, **{f"baseline_{k}": v for k, v in baseline_proxy.items()}},
↪{"validation_score": best_score})
        for cand in candidates:
            cand_readout, cand_rows = build_and_train_readout_variant(model,
↪repair_memory, config, prototypes, cand, seed=seed + 17)
            proxy = proxy_metrics_for_readout(model, cand_readout,
↪validation_memory, config, prototypes=prototypes)
            score = validation_score(proxy, baseline_proxy)
            rows.extend([**r, "candidate": cand} for r in cand_rows])
            rows.append({
                "repair_type": "selector_candidate",
                "candidate": cand,
                "proxy_acc": proxy.get("proxy_acc", np.nan),
                "proxy_class2": proxy.get("proxy_class2", np.nan),

```



```

        "proxy_class5": proxy.get("proxy_class5", np.nan),
        "proxy_min_class": proxy.get("proxy_min_class", np.nan),
        "validation_score": score,
    })
    if score > best_score + VALIDATION_ACCEPTANCE_TOL:
        best_score = score
        best_readout = cand_readout
        best_name = cand
        best_proxy = proxy
    rows.append({
        "repair_type": "selector_selected",
        "selected_candidate": best_name,
        "selected_score": best_score,
        "selected_proxy_acc": best_proxy.get("proxy_acc", np.nan),
        "selected_proxy_class2": best_proxy.get("proxy_class2", np.nan),
        "selected_proxy_class5": best_proxy.get("proxy_class5", np.nan),
        "selected_proxy_min_class": best_proxy.get("proxy_min_class", np.
↪nan),
    })
    return best_readout, rows

    raise ValueError(f"Unknown readout variant: {variant}")

# -----
# v1.0c inferred-context / proto-to-oracle gap closure utilities
# -----

def method_name_for_context_variant(variant: str):
    return f"paired_{variant}"

def make_context_config(base_strategy="proto_first", temp=None,
↪blend_tuple=None):
    """Create an evaluation/training config for a context posterior variant.

    temp is applied to model.context_temperature outside this config.
    blend_tuple is (name, single_weight, latent_weight, feature_weight).
    """
    if base_strategy == "oracle":
        cfg = copy.
↪deepcopy(METHOD_CONFIGS["mmals_oracle_context_balanced_replay"])
        return cfg
    cfg = copy.deepcopy(METHOD_CONFIGS["mmals_proto_first_balanced_replay"])
    cfg["strategy"] = "proto_first"
    if blend_tuple is not None:
        _name, bs, bl, bf = blend_tuple
        s = max(float(bs) + float(bl) + float(bf), 1e-8)

```

```

        cfg["batch_blend_single"] = float(bs) / s
        cfg["batch_blend_latent"] = float(bl) / s
        cfg["batch_blend_feature"] = float(bf) / s
        cfg["context_blend_name"] = str(_name)
    if temp is not None:
        cfg["selected_context_temperature"] = float(temp)
    return cfg

def set_model_context_temperature(model, temp):
    if hasattr(model, "context_temperature") and temp is not None:
        model.context_temperature = float(temp)

@torch.no_grad()
def proxy_metrics_for_context_config(model, class_memory, config,
    ↪ prototypes=None, readout=None,
    ↪
    ↪ max_items_per_class=VALIDATION_MAX_ITEMS_PER_CLASS):
    """Score a context-posterior config on memory validation, without final
    ↪ test data.

    This combines readout accuracy on memory, context top-1 against stored
    ↪ source task,
    context entropy/margin, class-2 accuracy, class-5 preservation and
    ↪ min-class robustness.
    """
    if not class_memory:
        return {
            "proxy_acc": np.nan,
            "proxy_context_top1": np.nan,
            "proxy_context_entropy": np.nan,
            "proxy_context_margin": np.nan,
            "proxy_class2": np.nan,
            "proxy_class5": np.nan,
            "proxy_min_class": np.nan,
            "proxy_min_task": np.nan,
            "proxy_old_task_min": np.nan,
            "proxy_task_acc_std": np.nan,
            "proxy_score": -1e9,
        }
    if readout is None:
        readout = GlobalLinearReadout(model.global_head).to(DEVICE)
    model.eval(); readout.eval()
    correct_by_cls = {c: 0 for c in range(N_CLASSES)}
    total_by_cls = {c: 0 for c in range(N_CLASSES)}
    correct_by_task = {t: 0 for t in range(N_TASKS)}

```

```

total_by_task = {t: 0 for t in range(N_TASKS)}
correct_total, total = 0, 0
top1_values, ent_values, margin_values = [], [], []
for cls, data in sorted(class_memory.items()):
    X, y, t = data["X"], data["y"], data["task_id"]
    if len(y) == 0:
        continue
    if len(y) > max_items_per_class:
        idx = torch.randperm(len(y))[:max_items_per_class]
        X, y, t = X[idx], y[idx], t[idx]
    loader = make_loader(TensorDataset(X, y, t), batch_size=BATCH_SIZE,
↳shuffle=False)
    for xb, yb, tb in loader:
        xb, yb, tb = xb.to(DEVICE), yb.to(DEVICE), tb.to(DEVICE)
        for src_task_id in torch.unique(tb).detach().cpu().tolist():
            src_task_id = int(src_task_id)
            mask = (tb == src_task_id)
            if not mask.any():
                continue
            _, cache = forward_by_method(
                model, xb[mask], src_task_id, config,
                prototypes=prototypes, train=False, y=yb[mask],
↳return_cache=True
            )
            logits = readout(cache["zf"], cache["q_context"])
            seen = seen_classes_until(src_task_id)
            pred = mask_logits_to_seen_classes(logits, seen).argmax(dim=-1)
            yy = yb[mask]
            correct = (pred == yy)
            correct_total += int(correct.sum().item())
            total += int(yy.numel())
            correct_by_task[src_task_id] += int(correct.sum().item())
            total_by_task[src_task_id] += int(yy.numel())
            q = cache["q_context"]
            top1_values.append((q.argmax(dim=-1) == src_task_id).float().
↳detach().cpu())
            ent_values.append(entropy_from_probs(q).detach().cpu())
            margin_values.append(margin_from_probs(q).detach().cpu())
            for c in torch.unique(yy).detach().cpu().tolist():
                c = int(c)
                m = (yy == c)
                total_by_cls[c] += int(m.sum().item())
                correct_by_cls[c] += int((pred[m] == yy[m]).sum().item())
        per_cls = {c: (correct_by_cls[c] / max(total_by_cls[c], 1) if
↳total_by_cls[c] > 0 else np.nan) for c in range(N_CLASSES)}
        valid_accs = [v for v in per_cls.values() if not np.isnan(v)]
        acc = correct_total / max(total, 1)

```

```

ctx_top1 = float(torch.cat(top1_values).mean()) if top1_values else np.nan
ctx_entropy = float(torch.cat(ent_values).mean()) if ent_values else np.nan
ctx_margin = float(torch.cat(margin_values).mean()) if margin_values else
↪ np.nan
min_class = float(np.min(valid_accs)) if valid_accs else np.nan
per_task = {t: (correct_by_task[t] / max(total_by_task[t], 1) if
↪ total_by_task[t] > 0 else np.nan) for t in range(N_TASKS)}
valid_task_accs = [v for v in per_task.values() if not np.isnan(v)]
min_task = float(np.min(valid_task_accs)) if valid_task_accs else np.nan
active_tasks = [t for t, v in per_task.items() if not np.isnan(v)]
max_active_task = max(active_tasks) if active_tasks else -1
old_task_accs = [per_task[t] for t in active_tasks if t < max_active_task]
old_task_min = float(np.min(old_task_accs)) if old_task_accs else min_task
task_acc_std = float(np.std(valid_task_accs)) if valid_task_accs else np.nan
c2 = per_cls.get(2, np.nan)
c5 = per_cls.get(5, np.nan)
score = float(
    acc
    + CONTEXT_SELECTION_SCORE_CONTEXT_WEIGHT * (0.0 if np.isnan(ctx_top1)
↪ else ctx_top1)
    + CONTEXT_SELECTION_SCORE_CLASS2_WEIGHT * (0.0 if np.isnan(c2) else c2)
    + CONTEXT_SELECTION_SCORE_MINCLASS_WEIGHT * (0.0 if np.isnan(min_class)
↪ else min_class)
    + 0.05 * (0.0 if np.isnan(min_task) else min_task)
    - CONTEXT_SELECTION_SCORE_ENTROPY_PENALTY * (0.0 if np.
↪ isnan(ctx_entropy) else ctx_entropy)
)
return {
    "proxy_acc": acc,
    "proxy_context_top1": ctx_top1,
    "proxy_context_entropy": ctx_entropy,
    "proxy_context_margin": ctx_margin,
    "proxy_class2": c2,
    "proxy_class5": c5,
    "proxy_min_class": min_class,
    "proxy_min_task": min_task,
    "proxy_old_task_min": old_task_min,
    "proxy_task_acc_std": task_acc_std,
    "proxy_score": score,
}

def candidate_context_configs(selection_mode="temp_blend"):
    """Return candidate (name, temp, blend_tuple) triples for context selection.
    ↪ """

```

```

    base_blend = ("base_proto_first", PROTO_FIRST_BLEND_SINGLE,
↳PROTO_FIRST_BLEND_LATENT, PROTO_FIRST_BLEND_FEATURE)
    if selection_mode == "temp":
        return [(f"temp_{t}", t, base_blend) for t in
↳CONTEXT_TEMP_SELECTION_GRID]
    if selection_mode == "blend":
        return [(f"blend_{name}", CONTEXT_TEMPERATURE, (name, bs, bl, bf)) for
↳name, bs, bl, bf in CONTEXT_BLEND_SELECTION_GRID]
    if selection_mode == "temp_blend":
        # Cross-product is deliberately compact to keep the audit tractable.
        temps = [0.60, 0.80, 1.00, 1.25]
        blends = CONTEXT_BLEND_SELECTION_GRID
        return [(f"temp_{t}__blend_{name}", t, (name, bs, bl, bf)) for t in
↳temps for name, bs, bl, bf in blends]
    raise ValueError("selection_mode must be temp, blend, or temp_blend")

def select_context_config_on_memory(model, class_memory, base_config,
↳prototypes, selection_mode="temp", seed=0):
    """Select a context posterior config on held-out memory, never final test
↳data."""
    set_seed(seed)
    _repair_memory, validation_memory =
↳split_class_memory_for_readout_validation(class_memory, seed=seed + 303)
    saved_temp = float(getattr(model, "context_temperature",
↳CONTEXT_TEMPERATURE))
    baseline_readout = GlobalLinearReadout(model.global_head).to(DEVICE)
    best = None
    rows = []
    for cand_name, temp, blend in candidate_context_configs(selection_mode):
        set_model_context_temperature(model, temp)
        cfg = make_context_config(base_strategy="proto_first", temp=temp,
↳blend_tuple=blend)
        proxy = proxy_metrics_for_context_config(model, validation_memory, cfg,
↳prototypes=prototypes, readout=baseline_readout)
        row = {
            "repair_type": "context_config_candidate",
            "candidate": cand_name,
            "selection_mode": selection_mode,
            "candidate_temp": float(temp),
            "candidate_blend_name": str(blend[0]),
            "candidate_blend_single": float(cfg.get("batch_blend_single", np.
↳nan)),
            "candidate_blend_latent": float(cfg.get("batch_blend_latent", np.
↳nan)),

```

```

        "candidate_blend_feature": float(cfg.get("batch_blend_feature", np.
↪nan)),
        **proxy,
    }
    rows.append(row)
    if best is None or proxy["proxy_score"] > best["proxy_score"]:
        best = {**row, "config": cfg, "temp": float(temp), "blend": blend}
    if best is None:
        set_model_context_temperature(model, saved_temp)
        return copy.deepcopy(base_config), [{"repair_type": ↵
↪"context_config_fallback", "selection_mode": selection_mode}]
    set_model_context_temperature(model, best["temp"])
    rows.append({
        "repair_type": "context_config_selected",
        "selection_mode": selection_mode,
        "selected_candidate": best["candidate"],
        "selected_temp": best["temp"],
        "selected_blend_name": best["candidate_blend_name"],
        "selected_blend_single": best["candidate_blend_single"],
        "selected_blend_latent": best["candidate_blend_latent"],
        "selected_blend_feature": best["candidate_blend_feature"],
        "selected_proxy_score": best["proxy_score"],
        "selected_proxy_acc": best["proxy_acc"],
        "selected_proxy_context_top1": best["proxy_context_top1"],
        "selected_proxy_class2": best["proxy_class2"],
        "selected_proxy_min_class": best["proxy_min_class"],
    })
    return best["config"], rows

```

1.14 9. Evaluation metrics and class diagnostics

The scorecard keeps the v0.14.x/v1.0 diagnostics: final average accuracy, minimum task accuracy, forgetting, class-2/class-4 ambiguity, class-5 preservation, predicted-class-4 bias, readout parameter cost, and probe gaps.

```

[11]: def mask_logits_to_seen_classes(logits, seen_classes):
    mask = torch.full_like(logits, -1e9)
    mask[:, seen_classes] = 0.0
    return logits + mask

@torch.no_grad()
def evaluate_task_with_readout(model, readout, dataset, task_id, config, ↵
↪prototypes=None, batch_size=BATCH_SIZE, seen_classes=None):
    model.eval(); readout.eval()
    loader = make_loader(dataset, batch_size=batch_size, shuffle=False)
    correct, total = 0, 0
    context_entropies, context_margins, context_top1 = [], [], []

```

```

context_briers, context_eces = [], []
frechet_margins, latent_frechet_margins, prototype_weights = [], [], []
class2_margin_24, class5_margin_54 = [], []
class2_logit_rank, class5_logit_rank = [], []
if seen_classes is None:
    seen_classes = list(range(N_CLASSES))
for xb, yb in loader:
    xb, yb = xb.to(DEVICE), yb.to(DEVICE)
    _, cache = forward_by_method(model, xb, task_id, config,
    ↪prototypes=prototypes, train=False, y=yb, return_cache=True)
    logits = readout(cache["zf"], cache["q_context"])
    logits_eval = mask_logits_to_seen_classes(logits, seen_classes)
    pred = logits_eval.argmax(dim=-1)
    correct += (pred == yb).sum().item()
    total += yb.numel()
    q = cache["q_context"]
    context_entropies.append(entropy_from_probs(q).detach().cpu())
    context_margins.append(margin_from_probs(q).detach().cpu())
    context_top1.append((q.argmax(dim=-1) == task_id).float().detach().
    ↪cpu())
    context_briers.append(brier_from_probs(q, task_id).detach().cpu())
    context_eces.append(torch.tensor([float(ece_from_probs(q, task_id).
    ↪detach().cpu()))))
    if "frechet_margin" in cache:
        frechet_margins.append(cache["frechet_margin"].detach().cpu())
    if "latent_frechet_margin" in cache:
        latent_frechet_margins.append(cache["latent_frechet_margin"].
    ↪detach().cpu())
    if "prototype_weight" in cache:
        prototype_weights.append(cache["prototype_weight"].detach().cpu())
    if (yb == 2).any():
        m = (yb == 2)
        class2_margin_24.append((logits_eval[m, 2] - logits_eval[m, 4]).
    ↪detach().cpu())
        ranks = (logits_eval[m] > logits_eval[m, 2].unsqueeze(-1)).
    ↪sum(dim=-1) + 1
        class2_logit_rank.append(ranks.float().detach().cpu())
    if (yb == 5).any():
        m = (yb == 5)
        class5_margin_54.append((logits_eval[m, 5] - logits_eval[m, 4]).
    ↪detach().cpu())
        ranks = (logits_eval[m] > logits_eval[m, 5].unsqueeze(-1)).
    ↪sum(dim=-1) + 1
        class5_logit_rank.append(ranks.float().detach().cpu())
    return {
        "acc": correct / max(total, 1),

```

```

        "context_entropy": float(torch.cat(context_entropies).mean()) if
↪context_entropies else np.nan,
        "context_margin": float(torch.cat(context_margins).mean()) if
↪context_margins else np.nan,
        "context_top1_acc": float(torch.cat(context_top1).mean()) if
↪context_top1 else np.nan,
        "context_brier": float(torch.cat(context_briers).mean()) if
↪context_briers else np.nan,
        "context_ece": float(torch.cat(context_eces).mean()) if context_eces
↪else np.nan,
        "latent_frechet_margin": float(torch.cat(latent_frechet_margins).
↪mean()) if latent_frechet_margins else np.nan,
        "prototype_weight": float(torch.cat(prototype_weights).mean()) if
↪prototype_weights else 0.0,
        "class2_margin_vs4": float(torch.cat(class2_margin_24).mean()) if
↪class2_margin_24 else np.nan,
        "class5_margin_vs4": float(torch.cat(class5_margin_54).mean()) if
↪class5_margin_54 else np.nan,
        "class2_logit_rank": float(torch.cat(class2_logit_rank).mean()) if
↪class2_logit_rank else np.nan,
        "class5_logit_rank": float(torch.cat(class5_logit_rank).mean()) if
↪class5_logit_rank else np.nan,
    }

@torch.no_grad()
def evaluate_seen_tasks_with_readout(model, readout, tasks, upto_task, config,
↪prototypes=None, batch_size=BATCH_SIZE):
    seen = seen_classes_until(upto_task)
    return [
        evaluate_task_with_readout(model, readout, tasks[tid]["test"], tid,
↪config, prototypes=prototypes, batch_size=batch_size, seen_classes=seen)
        for tid in range(upto_task + 1)
    ]

def forgetting_from_history(history_df: pd.DataFrame):
    rows = []
    for (method, seed), group in history_df.groupby(["method", "seed"]):
        for after_task in sorted(group.after_task.unique()):
            sub = group[group.after_task == after_task]
            fs = []
            for task_id in sorted(sub.task_id.unique()):
                task_hist = group[(group.task_id == task_id) & (group.
↪after_task <= after_task)]
                best = task_hist.acc.max()
                current = sub[sub.task_id == task_id].acc.iloc[0]
                if task_id < after_task:

```



```

        fs.append(best - current)
        rows.append({"method": method, "seed": seed, "after_task":
↪after_task, "avg_forgetting": float(np.mean(fs)) if fs else 0.0})
    return pd.DataFrame(rows)

@torch.no_grad()
def class_confusion_matrix_with_readout(model, readout, tasks, upto_task,
↪config, prototypes=None, batch_size=BATCH_SIZE):
    model.eval(); readout.eval()
    seen = seen_classes_until(upto_task)
    mat = torch.zeros(N_CLASSES, N_CLASSES, dtype=torch.long)
    for task_id in range(upto_task + 1):
        loader = make_loader(tasks[task_id]["test"], batch_size=batch_size,
↪shuffle=False)
        for xb, yb in loader:
            xb, yb = xb.to(DEVICE), yb.to(DEVICE)
            _, cache = forward_by_method(model, xb, task_id, config,
↪prototypes=prototypes, train=False, y=yb, return_cache=True)
            logits = readout(cache["zf"], cache["q_context"])
            logits_eval = mask_logits_to_seen_classes(logits, seen)
            pred = logits_eval.argmax(dim=-1).detach().cpu()
            yb_cpu = yb.detach().cpu()
            for yt, yp in zip(yb_cpu, pred):
                if int(yt) < N_CLASSES and int(yp) < N_CLASSES:
                    mat[int(yt), int(yp)] += 1

    rows = []
    for true_class in range(N_CLASSES):
        denom = mat[true_class].sum().item()
        for pred_class in range(N_CLASSES):
            rows.append({
                "true_class": true_class,
                "pred_class": pred_class,
                "count": int(mat[true_class, pred_class].item()),
                "rate": float(mat[true_class, pred_class].item() / max(denom,
↪1)),
            })
    return pd.DataFrame(rows)

def class_diagnostics_from_confusion(confusion_df: pd.DataFrame):
    if confusion_df is None or confusion_df.empty:
        return pd.DataFrame()
    rows = []
    for (method, seed), sub in confusion_df.groupby(["method", "seed"]):
        diag = {}
        for cls in range(N_CLASSES):
            m = sub[(sub.true_class == cls) & (sub.pred_class == cls)]
            diag[cls] = float(m.rate.iloc[0]) if not m.empty else np.nan

```

```

    pred4 = sub[sub.pred_class == 4].groupby("true_class")["rate"].sum()
    pred4_bias = float(pred4.reindex(range(N_CLASSES), fill_value=0.0).
    ↪mean())
    rows.append({
        "method": method,
        "seed": seed,
        "old_class_acc": float(np.nanmean([diag.get(0, np.nan), diag.get(1,
    ↪np.nan), diag.get(2, np.nan)])),
        "recent_class_acc": float(np.nanmean([diag.get(3, np.nan), diag.
    ↪get(4, np.nan), diag.get(5, np.nan)])),
        "class0_acc": diag.get(0, np.nan),
        "class1_acc": diag.get(1, np.nan),
        "class2_acc": diag.get(2, np.nan),
        "class3_acc": diag.get(3, np.nan),
        "class4_acc": diag.get(4, np.nan),
        "class5_acc": diag.get(5, np.nan),
        "min_class_acc": float(np.nanmin([diag.get(c, np.nan) for c in
    ↪range(N_CLASSES)])),
        "pred4_bias": pred4_bias,
    })
    return pd.DataFrame(rows)

```

1.15 10. Paired same-backbone training and RC1b validation protocol

The protocol is inherited from v0.14.8e, v0.14.9, v1.0a, v1.0b, v1.0c-control-fix and the v1.0-RC1 robust harness, but the goal is now candidate freeze:

1. Train one proto-first MMALS backbone per seed.
2. Save checkpoints after every task.
3. Clone the same checkpoint.
4. Keep the fungal backbone fixed.
5. Compare no-repair, no-op, true global guard, context-only, rejected original RC1 diagnostic, RC1b context+guard candidate, and oracle references.
6. Evaluate whether the RC1b candidate remains superior or clearly robustness-preferable across several seeds.

This is not a new full-training recipe. It is a corrected freeze/validate harness for the first v1.0 benchmark candidate.

```

[12]: def train_base_proto_first_checkpoints(seed: int):
    """Train one proto-first MMALS backbone and save checkpoints after every
    ↪task."""
    set_seed(seed)
    clear_memory()
    tasks = build_stress_tasks(DATASET_NAME, seed=seed)
    config = METHOD_CONFIGS["mmals_proto_first_balanced_replay"]
    model = build_model(config)
    memory_buffers: Dict[int, TensorDataset] = {}

```

```

class_memory: Dict[int, Dict[str, torch.Tensor]] = {}
context_prototypes: Dict[int, Dict[str, torch.Tensor]] = {}
teacher: Optional[nn.Module] = None
loss_rows, checkpoints = [], []
started = time.time()
for task_id, task in enumerate(tasks):
    opt = torch.optim.Adam(model.parameters(), lr=LR)
    loader = make_loader(task["train"], batch_size=BATCH_SIZE, shuffle=True)
    for epoch in range(EPOCHS_PER_TASK):
        model.train()
        sums = {"loss": 0.0, "task": 0.0, "memory": 0.0, "context": 0.0,
↪ "replay": 0.0, "balanced": 0.0, "fungal": 0.0}
        n_batches = 0
        for xb, yb in loader:
            xb, yb = xb.to(DEVICE), yb.to(DEVICE)
            noisy_context_id = sample_context_id(task_id)
            opt.zero_grad(set_to_none=True)
            logits, cache = forward_by_method(
                model, xb, task_id, config,
                prototypes=context_prototypes, train=True,
                noisy_context_id=noisy_context_id, y=yb, return_cache=True
            )
            task_loss = F.cross_entropy(logits, yb)
            fungal_loss = LAMBDA_ENTROPY * entropy_loss(cache["routes"]) +
↪ LAMBDA_METABOLIC * metabolic_loss(cache["transformed"], cache["routes"])
            context_loss, _ = context_calibration_loss(model, cache,
↪ task_id, config, epoch)
            replay_loss, _ = context_replay_loss(model, teacher,
↪ memory_buffers, config, batch_size=min(CONTEXT_REPLAY_BATCH,
↪ MEMORY_PER_TASK))
            memory_loss, _ = old_memory_losses(model, teacher,
↪ memory_buffers, config, batch_size=min(BATCH_SIZE, MEMORY_PER_TASK),
↪ prototypes=context_prototypes)
            balanced_loss, _ = balanced_replay_ce_loss(model, class_memory,
↪ config, prototypes=context_prototypes, batch_size=BALANCED_REPLAY_BATCH)
            loss = task_loss + fungal_loss + context_loss + replay_loss +
↪ memory_loss + balanced_loss
            loss.backward()
            torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=5.0)
            opt.step()
            sums["loss"] += float(loss.detach().cpu())
            sums["task"] += float(task_loss.detach().cpu())
            sums["memory"] += float(memory_loss.detach().cpu())
            sums["context"] += float(context_loss.detach().cpu())
            sums["replay"] += float(replay_loss.detach().cpu())
            sums["balanced"] += float(balanced_loss.detach().cpu())

```

```

        sums["fungal"] += float(fungal_loss.detach().cpu())
        n_batches += 1
    loss_rows.append({
        "method": "base_proto_first_training",
        "seed": seed,
        "task_id": task_id,
        "epoch": epoch,
        "loss": sums["loss"] / max(n_batches, 1),
        "task_loss": sums["task"] / max(n_batches, 1),
        "memory_loss": sums["memory"] / max(n_batches, 1),
        "context_loss": sums["context"] / max(n_batches, 1),
        "context_replay_loss": sums["replay"] / max(n_batches, 1),
        "balanced_replay_loss": sums["balanced"] / max(n_batches, 1),
        "fungal_loss": sums["fungal"] / max(n_batches, 1),
        "context_temperature": float(getattr(model,
↪ "context_temperature", np.nan)),
    })
    print(
        f"[base_proto seed={seed}] task={task_id} epoch={epoch+1}/
↪ {EPOCHS_PER_TASK} "
        f"loss={sums['loss']/max(n_batches,1):.4f} task={sums['task']/
↪ max(n_batches,1):.4f} "
        f"memory={sums['memory']/max(n_batches,1):.4f}
↪ balanced={sums['balanced']/max(n_batches,1):.4f}"
    )
    memory_buffers[task_id] = build_memory_subset(task["train"],
↪ MEMORY_PER_TASK, seed=seed + 1000 + task_id)
    class_memory = update_class_memory(class_memory, task["train"],
↪ task_id, MEMORY_PER_CLASS, seed=seed + 2000 + task_id)
    context_prototypes = refresh_context_prototypes(model, memory_buffers)
    chosen_temp = calibrate_context_temperature(model, memory_buffers,
↪ config)
    if config.get("temp_calibration", False):
        print(f"[base_proto seed={seed}] calibrated context temperature
↪ after task {task_id}: {chosen_temp:.2f}")
    checkpoints.append({
        "seed": seed,
        "after_task": task_id,
        "model_state": cpu_state_dict(model),
        "context_temperature": float(getattr(model, "context_temperature",
↪ CONTEXT_TEMPERATURE)),
        "memory_buffers": clone_memory_buffers(memory_buffers),
        "class_memory": clone_class_memory(class_memory),
        "context_prototypes": clone_prototypes(context_prototypes),
        "replay_memory_items": int(sum(len(v["y"]) for v in class_memory.
↪ values()))),

```

```

    })
    teacher = clone_teacher(model)
    clear_memory()
    print(f"[base_proto seed={seed}] completed in {time.time() - started:.1f}s")
    return {
        "seed": seed,
        "tasks": tasks,
        "config": config,
        "checkpoints": checkpoints,
        "loss_df": pd.DataFrame(loss_rows),
    }

def method_name_for_variant(variant: str):
    return f"paired_{variant}"

def apply_local_head_variant_to_checkpoint(base_run, checkpoint, variant: str):
    seed = int(checkpoint["seed"])
    after_task = int(checkpoint["after_task"])
    tasks = base_run["tasks"]
    config = copy.deepcopy(METHOD_CONFIGS["mmals_proto_first_balanced_replay"])
    method_name = method_name_for_variant(variant)
    model = restore_base_model_from_checkpoint(checkpoint, config)
    class_memory = clone_class_memory(checkpoint["class_memory"])
    context_prototypes = clone_prototypes(checkpoint["context_prototypes"])
    readout, repair_rows = build_and_train_readout_variant(
        model, class_memory, config, context_prototypes,
        variant=variant, seed=seed * 10_000 + after_task * 101
    )
    for rr in repair_rows:
        rr.update({
            "method": method_name,
            "seed": seed,
            "after_task": after_task,
            "task_id": after_task,
            "variant": variant,
            "paired_same_backbone": True,
            "local_mycorrhizal_head": not variant.startswith("global") and
↪variant not in ["no_repair_global_head", "kl_only_true_noop"],
        })
    evals = evaluate_seen_tasks_with_readout(model, readout, tasks,
↪upto_task=after_task, config=config, prototypes=context_prototypes)
    history_rows = []
    replay_memory_items = int(checkpoint.get("replay_memory_items", 0))
    readout_trainable = count_parameters(readout)
    readout_total = total_parameters(readout)
    for old_task_id, metrics in enumerate(evals):
        row = {

```

```

        "method": method_name,
        "seed": seed,
        "model_family": "mmals_local_readout",
        "after_task": after_task,
        "task_id": old_task_id,
        "task_name": tasks[old_task_id]["name"],
        "seen_classes": str(seen_classes_until(after_task)),
        "trainable_params": count_parameters(model) + readout_trainable,
        "total_params": total_parameters(model) + readout_total,
        "backbone_params": total_parameters(model),
        "readout_trainable_params": readout_trainable,
        "readout_total_params": readout_total,
        "model_size_mb_total": (total_parameters(model) + readout_total) * 4 / (1024 ** 2),
        "uses_balanced_replay": True,
        "uses_output_distill": True,
        "uses_local_mycorrhizal_head": not variant.startswith("global") and
        variant not in ["no_repair_global_head", "kl_only_true_noop"],
        "uses_global_head_refresh": variant == "global_head_ce_kl_guard_035",
        "probe_row": False,
        "backbone_source": "same_proto_first_checkpoint",
        "probe_feature": "original_backbone_zf_custom_readout",
        "probe_context_mode": "proto_first",
        "replay_memory_items": replay_memory_items,
        "memory_per_class": MEMORY_PER_CLASS,
        "memory_per_task": MEMORY_PER_TASK,
        "paired_same_backbone": True,
        "readout_variant": variant,
    }
    row.update(metrics)
    history_rows.append(row)
    confusion_rows = []
    if after_task == N_TASKS - 1:
        cm_df = class_confusion_matrix_with_readout(model, readout, tasks,
        upto_task=after_task, config=config, prototypes=context_prototypes)
        cm_df["method"] = method_name
        cm_df["seed"] = seed
        cm_df["after_task"] = after_task
        cm_df["backbone_source"] = "same_proto_first_checkpoint"
        cm_df["readout_variant"] = variant
        confusion_rows = cm_df.to_dict("records")
        print(f"[{method_name} seed={seed}] after task {after_task}:
        {[round(m['acc'], 3) for m in evals]}")
    return pd.DataFrame(history_rows), pd.DataFrame(confusion_rows), pd.
    DataFrame(repair_rows)

```

```

def final_probe_artifact_from_base_run(base_run):
    final_cp = base_run["checkpoints"][-1]
    model = restore_base_model_from_checkpoint(final_cp,
METHOD_CONFIGS["mmals_proto_first_balanced_replay"])
    return {
        "method": "mmals_proto_first_balanced_replay",
        "seed": int(final_cp["seed"]),
        "model": model,
        "tasks": base_run["tasks"],
        "config": METHOD_CONFIGS["mmals_proto_first_balanced_replay"],
        "memory_buffers": clone_memory_buffers(final_cp["memory_buffers"]),
        "class_memory": clone_class_memory(final_cp["class_memory"]),
        "context_prototypes": clone_prototypes(final_cp["context_prototypes"]),
        "replay_memory_items": int(final_cp.get("replay_memory_items", 0)),
    }

# -----
# v1.0c paired context-head variant application
# -----

def context_variant_selection_mode(variant: str):
    if "temp_blend" in variant or variant == "context_gap_selected":
        return "temp_blend"
    if "blend_selected" in variant:
        return "blend"
    if "temp_selected" in variant:
        return "temp"
    return None

def is_oracle_context_reference(variant: str):
    return variant.startswith("oracle_")

def variant_uses_guarded_head(variant: str):
    """Return True only for variants that must actually train a guarded global
readout.

    v1.0c evidence exposed a control issue: the control named
    `proto_global_head_ce_kl_guard_035` did not contain the exact substring
    `head_guard_035`, so the previous helper treated it as a no-op.

    This control-fix accepts both naming conventions:
    - `head_guard_035` for context-selected guarded variants;
    - `global_head_ce_kl_guard_035` for the true global guarded control and
readout reference.

```

```

        """
    return (
        "head_guard_035" in variant
        or "global_head_ce_kl_guard_035" in variant
        or variant == "context_gap_selected"
    )

def build_context_variant_config_and_logs(model, class_memory,
    ↪context_prototypes, variant, seed=0):
    """Return config/logs for one deployable or reference context variant."""
    if is_oracle_context_reference(variant):
        cfg = make_context_config(base_strategy="oracle")
        return cfg, [{"repair_type": "context_reference", "variant": variant,
    ↪"uses_oracle_context_reference": True}]
    cfg = copy.deepcopy(METHOD_CONFIGS["mmals_proto_first_balanced_replay"])
    mode = context_variant_selection_mode(variant)
    if mode is None:
        return cfg, [{"repair_type": "context_no_change", "variant": variant,
    ↪"uses_oracle_context_reference": False}]
    selected_cfg, rows = select_context_config_on_memory(model, class_memory,
    ↪cfg, context_prototypes, selection_mode=mode, seed=seed)
    for r in rows:
        r.update({"variant": variant, "uses_oracle_context_reference": False})
    return selected_cfg, rows

def apply_context_head_variant_to_checkpoint(base_run, checkpoint, variant:
    ↪str):
    seed = int(checkpoint["seed"])
    after_task = int(checkpoint["after_task"])
    tasks = base_run["tasks"]
    method_name = method_name_for_context_variant(variant)

    # Start from the exact same checkpoint.
    base_cfg = METHOD_CONFIGS["mmals_proto_first_balanced_replay"]
    model = restore_base_model_from_checkpoint(checkpoint, base_cfg)
    class_memory = clone_class_memory(checkpoint["class_memory"])
    context_prototypes = clone_prototypes(checkpoint["context_prototypes"])

    # Select / set the context posterior configuration.
    config, config_rows = build_context_variant_config_and_logs(
        model, class_memory, context_prototypes, variant=variant,
        seed=seed * 10_000 + after_task * 313
    )

```



```

    # Build readout. v1.0-RC1b primarily uses global readout controls; local
    ↪adapters are not the target here.
    readout = GlobalLinearReadout(model.global_head).to(DEVICE)
    repair_rows = list(config_rows)

    if variant_uses_guarded_head(variant):
        repair_memory, _validation_memory =
    ↪split_class_memory_for_readout_validation(class_memory, seed=seed * 10_000 +
    ↪after_task * 101)
        pre_global_head = copy.deepcopy(model.global_head).to(DEVICE)
        pre_global_head.eval()
        for p in pre_global_head.parameters():
            p.requires_grad_(False)
        repair_rows.extend(train_readout_on_memory(
            readout, model, repair_memory, config,
            prototypes=context_prototypes, pre_global_head=pre_global_head,
            kl_weight=0.35, use_kl=True, margin_weight=0.0,
            variant=f"{variant}_global_head_guard"
        ))

    for rr in repair_rows:
        rr.update({
            "method": method_name,
            "seed": seed,
            "after_task": after_task,
            "task_id": after_task,
            "variant": variant,
            "paired_same_backbone": True,
            "context_gap_closure_audit": True,
            "uses_oracle_context_reference":
    ↪is_oracle_context_reference(variant),
            "uses_context_selection": context_variant_selection_mode(variant),
    ↪is not None,
            "uses_guarded_global_head": variant_uses_guarded_head(variant),
            "true_guard_control_fixed": variant ==
    ↪"proto_global_head_ce_kl_guard_035",
        })

    evals = evaluate_seen_tasks_with_readout(
        model, readout, tasks, upto_task=after_task,
        config=config, prototypes=context_prototypes
    )

    history_rows = []
    replay_memory_items = int(checkpoint.get("replay_memory_items", 0))
    readout_trainable = count_parameters(readout)
    readout_total = total_parameters(readout)

```

```

for old_task_id, metrics in enumerate(evals):
    row = {
        "method": method_name,
        "seed": seed,
        "model_family": "mmals_context_readout",
        "after_task": after_task,
        "task_id": old_task_id,
        "task_name": tasks[old_task_id]["name"],
        "seen_classes": str(seen_classes_until(after_task)),
        "trainable_params": count_parameters(model) + readout_trainable,
        "total_params": total_parameters(model) + readout_total,
        "backbone_params": total_parameters(model),
        "readout_trainable_params": readout_trainable,
        "readout_total_params": readout_total,
        "model_size_mb_total": (total_parameters(model) + readout_total) * 4 / (1024 ** 2),
        "uses_balanced_replay": True,
        "uses_output_distill": True,
        "uses_local_mycorrhizal_head": False,
        "uses_global_head_refresh": variant_uses_guarded_head(variant),
        "uses_context_selection": context_variant_selection_mode(variant)
        is not None,
        "uses_oracle_context_reference":
        is_oracle_context_reference(variant),
        "probe_row": False,
        "backbone_source": "same_proto_first_checkpoint",
        "probe_feature": "original_backbone_zf_context_variant",
        "probe_context_mode": "oracle_reference" if
        is_oracle_context_reference(variant) else "proto_first_variant",
        "replay_memory_items": replay_memory_items,
        "memory_per_class": MEMORY_PER_CLASS,
        "memory_per_task": MEMORY_PER_TASK,
        "paired_same_backbone": True,
        "readout_variant": variant,
        "context_variant": variant,
        "context_temperature_used": float(getattr(model,
        "context_temperature", np.nan)),
        "batch_blend_single_used": float(config.get("batch_blend_single",
        np.nan)),
        "batch_blend_latent_used": float(config.get("batch_blend_latent",
        np.nan)),
        "batch_blend_feature_used": float(config.get("batch_blend_feature",
        np.nan)),
    }
    row.update(metrics)
    history_rows.append(row)

```

```

confusion_rows = []
if after_task == N_TASKS - 1:
    cm_df = class_confusion_matrix_with_readout(
        model, readout, tasks, upto_task=after_task,
        config=config, prototypes=context_prototypes
    )
    cm_df["method"] = method_name
    cm_df["seed"] = seed
    cm_df["after_task"] = after_task
    cm_df["backbone_source"] = "same_proto_first_checkpoint"
    cm_df["readout_variant"] = variant
    cm_df["context_variant"] = variant
    cm_df["uses_oracle_context_reference"] = _
    is_oracle_context_reference(variant)
    confusion_rows = cm_df.to_dict("records")

    print(f"[{method_name}] seed={seed} after task {after_task}: _
    {round(m['acc'], 3) for m in evals}")
    return pd.DataFrame(history_rows), pd.DataFrame(confusion_rows), pd.
    DataFrame(repair_rows)

```

1.16 10c. RC2c Hydro audit helpers

This block adds Hydro diagnostics to the strict no-leakage policy-selection harness.

Depending on EXPERIMENT_PROFILE, Hydro can be:

- disabled (RC2b_STRICT),
- audit-only (RC3_HYDRO_AUDIT, default),
- used as a training regularizer (RC3_HYDRO_CANDIDATE),
- used as a validation-time stability penalty (RC3_HYDRO_STABILITY_SELECTED).

The audit computes, on **policy-validation memory only**:

- route drift D_r ;
- latent drift D_z ;
- output drift D_y ;
- route speed and latent speed;
- adaptive viscosity estimate ν ;
- Reynolds-like instability;
- turbulence-like score;
- wave/context entropy.

By default these metrics are reported and exported. They do not alter selection unless the RC3_HYDRO_STABILITY_SELECTED profile is used.

```

[13]: # -----
      # RC3 Hydro audit layer: validation-memory-only drift/turbulence metrics.

```

```

# -----

def rc3_hydro_context_entropy(q, eps=1e-8):
    if q is None:
        return torch.tensor(float("nan"))
    return -(q * torch.log(q + eps)).sum(dim=-1)

def rc3_hydro_adaptive_viscosity(importance, base=None, max_v=None):
    base = RC3_HYDRO_BASE_VISCOSITY if base is None else float(base)
    max_v = RC3_HYDRO_MAX_VISCOSITY if max_v is None else float(max_v)
    return torch.clamp(base + (max_v - base) * importance, base, max_v)

@torch.no_grad()
def rc3_hydro_importance(reference_logits, reference_routes):
    probs = F.softmax(reference_logits, dim=-1)
    confidence = probs.max(dim=-1).values.mean()
    if reference_routes is None:
        dominance = torch.tensor(1.0, device=reference_logits.device)
    else:
        dominance = reference_routes.max(dim=-1).values.mean()
    return torch.clamp(confidence * dominance, 0.0, 1.0)

def rc3_hydro_kl(student_logits, teacher_logits,
    ↪temperature=DISTILL_TEMPERATURE):
    log_p = F.log_softmax(student_logits / temperature, dim=-1)
    q = F.softmax(teacher_logits / temperature, dim=-1)
    return (temperature ** 2) * F.kl_div(log_p, q, reduction="batchmean")

@torch.no_grad()
def rc3_eval_candidate_cache(candidate, xb, src_task_id, yb=None):
    model = candidate["model"]
    readout = candidate["readout"]
    config = candidate["config"]
    prototypes = candidate["context_prototypes"]
    model.eval(); readout.eval()
    _, cache = forward_by_method(
        model, xb, int(src_task_id), config,
        prototypes=prototypes, train=False, y=yb, return_cache=True
    )
    logits = readout(cache["zf"], cache.get("q_context"))
    return logits, cache

```

```

def rc3_pairwise_hydro_proxy(candidate, reference_candidate, class_memory,
    ↪max_items_per_class=None):
    """Compute Hydro drift of candidate versus a reference policy on validation
    ↪memory.

    The reference is usually the no-repair/global-head candidate for the same
    ↪checkpoint.
    This is a **selection-memory** audit, not a final-test audit.
    """

    if not RC3_HYDRO_AUDIT_ENABLED or not class_memory:
        return {}

    max_items_per_class = RC3_HYDRO_MAX_ITEMS_PER_CLASS if max_items_per_class
    ↪is None else int(max_items_per_class)

    rows = []
    for cls, data in sorted(class_memory.items()):
        X, y, t = data["X"], data["y"], data["task_id"]
        if len(y) == 0:
            continue
        if len(y) > max_items_per_class:
            # Deterministic compact sampling based on candidate and class.
            # Deterministic SHA-based sample; no Python hash().
            seed_int = int(hashlib.sha256(f"{candidate.get('variant')}:{cls}").
    ↪encode()).hexdigest()[:16], 16) % (2**31-1)
            gen = torch.Generator().manual_seed(seed_int)
            idx = torch.randperm(len(y), generator=gen)[:max_items_per_class]
            X, y, t = X[idx], y[idx], t[idx]
            loader = make_loader(TensorDataset(X, y, t), batch_size=BATCH_SIZE,
    ↪shuffle=False)
            for xb, yb, tb in loader:
                xb, yb, tb = xb.to(DEVICE), yb.to(DEVICE), tb.to(DEVICE)
                for src_task_id in torch.unique(tb).detach().cpu().tolist():
                    src_task_id = int(src_task_id)
                    mask = tb == src_task_id
                    if not mask.any():
                        continue
                    c_logits, c_cache = rc3_eval_candidate_cache(candidate,
    ↪xb[mask], src_task_id, yb[mask])
                    r_logits, r_cache =
    ↪rc3_eval_candidate_cache(reference_candidate, xb[mask], src_task_id,
    ↪yb[mask])

                    c_routes, r_routes = c_cache.get("routes"), r_cache.
    ↪get("routes")
                    if c_routes is None or r_routes is None:
                        route_drift = torch.tensor(0.0, device=xb.device)

```

```

        else:
            route_drift = F.mse_loss(c_routes, r_routes)
            latent_drift = F.mse_loss(c_cache["zf"], r_cache["zf"])
            output_drift = rc3_hydro_kl(c_logits, r_logits)
            importance = rc3_hydro_importance(r_logits, r_routes)
            nu = rc3_hydro_adaptive_viscosity(importance)
            route_speed = torch.sqrt(torch.clamp(route_drift, min=0.0))
            latent_speed = torch.sqrt(torch.clamp(latent_drift, min=0.0))
            reynolds_like = (route_speed + latent_speed) / (nu +
→RC3_HYDRO_TURBULENCE_EPS)
            turbulence_score = reynolds_like * (1.0 + output_drift.detach())
            q = c_cache.get("q_context")
            wave_entropy = rc3_hydro_context_entropy(q).mean() if q is not
→None else torch.tensor(float("nan"), device=xb.device)
            rows.append({
                "route_drift_Dr": float(route_drift.detach().cpu()),
                "latent_drift_Dz": float(latent_drift.detach().cpu()),
                "output_drift_Dy": float(output_drift.detach().cpu()),
                "teacher_importance": float(importance.detach().cpu()),
                "adaptive_viscosity_nu": float(nu.detach().cpu()),
                "route_speed": float(route_speed.detach().cpu()),
                "latent_speed": float(latent_speed.detach().cpu()),
                "reynolds_like": float(reynolds_like.detach().cpu()),
                "turbulence_score": float(turbulence_score.detach().cpu()),
                "wave_entropy": float(wave_entropy.detach().cpu()) if not
→torch.isnan(wave_entropy) else np.nan,
            })
        if not rows:
            return {}
        df = pd.DataFrame(rows)
        out = {}
        for col in df.columns:
            out[f"hydro_{col}"] = float(df[col].mean())
        out["hydro_audit_memory_only"] = True
        out["hydro_audit_reference_variant"] = reference_candidate.get("variant",
→"unknown")
        return out

def rc3_hydro_selection_penalty(proxy):
    """Optional Hydro penalty. Default weights are zero: audit-only mode."""
    if (not RC3_HYDRO_AUDIT_ENABLED) or (not globals().
→get("RC3_HYDRO_SELECTION_ENABLED", False)) or RC3_HYDRO_POLICY_MODE !=
→"stability_aware":
        return 0.0
    turb = proxy.get("hydro_turbulence_score", np.nan)
    lat = proxy.get("hydro_latent_drift_Dz", np.nan)

```

```

route = proxy.get("hydro_route_drift_Dr", np.nan)
penalty = 0.0
if not np.isnan(turb):
    penalty += RC3_HYDRO_SELECTION_TURBULENCE_WEIGHT * float(turb)
if not np.isnan(lat):
    penalty += RC3_HYDRO_SELECTION_LATENT_DRIFT_WEIGHT * float(lat)
if not np.isnan(route):
    penalty += RC3_HYDRO_SELECTION_ROUTE_DRIFT_WEIGHT * float(route)
return float(penalty)

```

1.17 10b. RC2c strict no-leakage policy-selection utilities

These functions implement the strict RC2c rule.

The important rule is:

Candidate policy selection uses only memory-derived repair/context/policy-validation partitions. Final test sets are evaluated only after the policy has already been selected.

For auditability, the notebook can still evaluate all candidate policies after the selection decision. Those candidate final-test metrics are diagnostic only and are not used by the selector.

```

[14]: # -----
# v1.1-RC3 Hydro-audited strict no-leakage policy-selection utilities.
# -----

def rc2b_candidate_method_name(variant: str) -> str:
    return method_name_for_context_variant(variant)

def rc2b_policy_family(variant: str) -> str:
    if variant in ["proto_global_no_repair", "proto_kl_only_true_noop"]:
        return "proto_no_repair"
    if variant in ["context_blend_selected_global", ↵
↵ "context_temp_blend_selected_global"]:
        return "context_only_global"
    if variant == "proto_global_head_ce_kl_guard_035":
        return "proto_true_global_guard"
    if variant in ["context_temp_blend_selected_head_guard_035", ↵
↵ "context_gap_selected"]:
        return "rc1b_guarded_context_gap"
    return "other"

def rc2b_policy_label(variant: str) -> str:
    labels = {
        "proto_no_repair": "proto-first no-repair / copied global head",
        "context_only_global": "validation-selected context-only global head",

```

```

        "proto_true_global_guard": "proto-first true CE/KL guarded global head",
        "rc1b_guarded_context_gap": "RC1b guarded context-gap",
        "other": f"other policy: {variant}",
    }
    return labels.get(rc2b_policy_family(variant), str(variant))

def rc2b_policy_complexity_rank(variant: str) -> int:
    # Lower is simpler. Used only as a tie-breaker after validation score.
    fam = rc2b_policy_family(variant)
    if fam == "proto_no_repair":
        return 0
    if fam == "context_only_global":
        return 1
    if fam == "proto_true_global_guard":
        return 2
    if fam == "rc1b_guarded_context_gap":
        return 3
    return 9

def split_class_memory_three_way(class_memory, seed=0,
                                repair_fraction=RC2B_POLICY_REPAIR_FRACTION,
                                context_fraction=RC2B_CONTEXT_SELECTION_FRACTION):
    """Split class memory into repair, context-selection and policy-validation
    memory.

    This is the core RC2b no-leakage guard:
    - repair_memory trains guarded heads;
    - context_selection_memory selects context temp/blend when needed;
    - policy_validation_memory selects the deployable policy;
    - final test is not touched until after selection.
    """
    rng = np.random.default_rng(seed)
    repair, context_select, policy_val = {}, {}, {}
    for cls, data in class_memory.items():
        X, y, t = data["X"], data["y"], data["task_id"]
        n = len(y)
        if n == 0:
            continue
        idx = np.arange(n)
        rng.shuffle(idx)
        n_repair = max(1, int(round(n * repair_fraction))) if n >= 3 else
        max(1, n - 1)
        n_context = max(1, int(round(n * context_fraction))) if n - n_repair >=
        2 else max(0, n - n_repair - 1)

```



```

n_repair = min(n_repair, n)
n_context = min(n_context, max(0, n - n_repair))
repair_idx = idx[:n_repair]
context_idx = idx[n_repair:n_repair + n_context]
policy_idx = idx[n_repair + n_context:]
if len(policy_idx) == 0:
    # Make sure policy validation exists, even in tiny smoke memory.
    if len(context_idx) > 0:
        policy_idx = context_idx[-1:]
        context_idx = context_idx[:-1]
    elif len(repair_idx) > 1:
        policy_idx = repair_idx[-1:]
        repair_idx = repair_idx[:-1]
def pack(sub_idx):
    if len(sub_idx) == 0:
        return None
    tidx = torch.as_tensor(sub_idx, dtype=torch.long)
    return {"X": X[tidx].clone(), "y": y[tidx].clone(), "task_id":
    ↪t[tidx].clone()}
    r = pack(repair_idx)
    c = pack(context_idx)
    p = pack(policy_idx)
    if r is not None:
        repair[int(cls)] = r
    if c is not None:
        context_select[int(cls)] = c
    if p is not None:
        policy_val[int(cls)] = p
    # Fallback: if context selection is empty because smoke memory is tiny, use
    ↪policy validation.
    if not context_select:
        context_select = clone_class_memory(policy_val)
    return repair, context_select, policy_val

def rc2c_is_finite(x):
    try:
        return np.isfinite(float(x))
    except Exception:
        return False

def rc2c_proxy_value(proxy, key, default=np.nan):
    try:
        v = proxy.get(key, default)
        return float(v)

```

```

    except Exception:
        return default

def rc2c_floor_deficit(value, floor):
    if not rc2c_is_finite(value):
        return 0.0
    return max(0.0, float(floor) - float(value))

def rc2c_is_guarded_context_candidate(variant: str) -> bool:
    return rc2b_policy_family(variant) == "rc1b_guarded_context_gap" or variant in RC2C_GUARDED_REFERENCE_VARIANTS

def rc2c_candidate_meets_hard_floors(candidate) -> bool:
    p = candidate.get("proxy", {})
    c2 = rc2c_proxy_value(p, "proxy_class2")
    min_task = rc2c_proxy_value(p, "proxy_min_task", rc2c_proxy_value(p, "proxy_min_class"))
    min_class = rc2c_proxy_value(p, "proxy_min_class")
    ok_c2 = (not rc2c_is_finite(c2)) or c2 >= RC2C_HARD_CLASS2_FLOOR
    ok_min_task = (not rc2c_is_finite(min_task)) or min_task >= RC2C_HARD_MIN_TASK_FLOOR
    ok_min_class = (not rc2c_is_finite(min_class)) or min_class >= RC2C_HARD_MIN_CLASS_FLOOR
    return bool(ok_c2 and ok_min_task and ok_min_class)

def rc2c_risk_components(proxy, baseline_proxy, variant: str, after_task=None):
    """Return validation-memory-only score components for RC2c risk-aware policy selection."""
    c2 = rc2c_proxy_value(proxy, "proxy_class2")
    c5 = rc2c_proxy_value(proxy, "proxy_class5")
    min_class = rc2c_proxy_value(proxy, "proxy_min_class")
    min_task = rc2c_proxy_value(proxy, "proxy_min_task", min_class)
    old_task_min = rc2c_proxy_value(proxy, "proxy_old_task_min", min_task)
    task_std = rc2c_proxy_value(proxy, "proxy_task_acc_std", 0.0)
    family = rc2b_policy_family(variant)

    floor_penalty = 0.0
    floor_penalty += RC2C_CLASS2_FLOOR_PENALTY * rc2c_floor_deficit(c2, RC2C_CLASS2_FLOOR)
    floor_penalty += RC2C_MIN_CLASS_FLOOR_PENALTY * rc2c_floor_deficit(min_class, RC2C_MIN_CLASS_FLOOR)

```

```

    floor_penalty += RC2C_MIN_TASK_FLOOR_PENALTY * rc2c_floor_deficit(min_task,
↪RC2C_MIN_TASK_FLOOR)
    floor_penalty += RC2C_OLD_TASK_FLOOR_PENALTY *
↪rc2c_floor_deficit(old_task_min, RC2C_OLD_TASK_FLOOR)

    dispersion_penalty = 0.0
    if rc2c_is_finite(task_std):
        dispersion_penalty = RC2C_TASK_DISPERSION_PENALTY * max(0.0, task_std)

    no_repair_soft_penalty = 0.0
    if after_task is not None and int(after_task) >=
↪RC2C_NO_REPAIR_VETO_AFTER_TASK and family == "proto_no_repair":
        no_repair_soft_penalty = RC2C_NO_REPAIR_SOFT_PENALTY

    guarded_bonus = RC2C_GUARDED_CONTEXT_BONUS if
↪rc2c_is_guarded_context_candidate(variant) else 0.0
    complexity_penalty = RC2B_COMPLEXITY_PENALTY *
↪rc2b_policy_complexity_rank(variant)
    class5_bonus = RC2B_VALIDATION_SCORE_CLASS5_WEIGHT * (0.0 if not
↪rc2c_is_finite(c5) else c5)
    hydro_penalty = rc3_hydro_selection_penalty(proxy)

    return {
        "class2_floor_penalty": RC2C_CLASS2_FLOOR_PENALTY *
↪rc2c_floor_deficit(c2, RC2C_CLASS2_FLOOR),
        "min_class_floor_penalty": RC2C_MIN_CLASS_FLOOR_PENALTY *
↪rc2c_floor_deficit(min_class, RC2C_MIN_CLASS_FLOOR),
        "min_task_floor_penalty": RC2C_MIN_TASK_FLOOR_PENALTY *
↪rc2c_floor_deficit(min_task, RC2C_MIN_TASK_FLOOR),
        "old_task_floor_penalty": RC2C_OLD_TASK_FLOOR_PENALTY *
↪rc2c_floor_deficit(old_task_min, RC2C_OLD_TASK_FLOOR),
        "floor_penalty": floor_penalty,
        "task_dispersion_penalty": dispersion_penalty,
        "no_repair_soft_penalty": no_repair_soft_penalty,
        "complexity_penalty": complexity_penalty,
        "class5_bonus": class5_bonus,
        "guarded_context_bonus": guarded_bonus,
        "hydro_selection_penalty": hydro_penalty,
        "total_risk_penalty": floor_penalty + dispersion_penalty +
↪no_repair_soft_penalty + complexity_penalty + hydro_penalty,
    }

def rc2b_validation_score(proxy, baseline_proxy, variant: str, after_task=None):
    """Policy-level validation score using memory validation only.

```

```

In RC2B_STRICT this reproduces the older selector.
In RC2c profiles this becomes a risk-aware selector:
- stronger weak-class / min-task penalties;
- trajectory-aware old-task validation-memory penalty;
- no-repair soft penalty after early tasks;
- guarded context-gap hysteresis support handled by rc2c_select_candidate.
"""

base_score = validation_score(proxy, baseline_proxy)
if base_score <= -1e8:
    return -1e9

if not RC2C_RISK_AWARE_SELECTOR_ENABLED:
    c2 = proxy.get("proxy_class2", np.nan)
    c5 = proxy.get("proxy_class5", np.nan)
    minc = proxy.get("proxy_min_class", np.nan)
    penalty = RC2B_COMPLEXITY_PENALTY * rc2b_policy_complexity_rank(variant)
    floor_penalty = 0.0
    if not np.isnan(c2) and c2 < RC2B_CLASS2_FLOOR:
        floor_penalty += RC2B_VALIDATION_SCORE_FLOOR_PENALTY *
↪(RC2B_CLASS2_FLOOR - c2)
    if not np.isnan(c5) and c5 < RC2B_CLASS5_FLOOR:
        floor_penalty += RC2B_VALIDATION_SCORE_FLOOR_PENALTY *
↪(RC2B_CLASS5_FLOOR - c5)
    if not np.isnan(minc) and minc < RC2B_MIN_CLASS_FLOOR:
        floor_penalty += RC2B_VALIDATION_SCORE_FLOOR_PENALTY *
↪(RC2B_MIN_CLASS_FLOOR - minc)
    class5_bonus = RC2B_VALIDATION_SCORE_CLASS5_WEIGHT * (0.0 if np.
↪isnan(c5) else c5)
    hydro_penalty = rc3_hydro_selection_penalty(proxy)
    return float(base_score + class5_bonus - penalty - floor_penalty -
↪hydro_penalty)

    comp = rc2c_risk_components(proxy, baseline_proxy, variant,
↪after_task=after_task)
    return float(
        base_score
        + comp["class5_bonus"]
        + comp["guarded_context_bonus"]
        - comp["total_risk_penalty"]
    )

def rc2c_select_candidate(candidates, after_task):
    """Risk-aware no-leakage selector.

```

```

    This function only consumes candidate proxy metrics computed on
    ↪policy-validation memory.
    It never reads final-test metrics.
    """
    scored = [c for c in candidates if rc2c_is_finite(c.get("selection_score",
    ↪-1e9))]
    if not scored:
        selected = candidates[0]
        return selected, {
            "raw_best_variant": selected["variant"],
            "selection_reason": "fallback_no_finite_scores",
            "no_repair_veto_applied": False,
            "guarded_hysteresis_applied": False,
            "risk_floor_fallback_applied": False,
        }

    sorted_by_score = sorted(
        scored,
        key=lambda c: (
            float(c.get("selection_score", -1e9)),
            -float(c["complexity_rank"]),
            float(c["proxy"].get("proxy_acc", -1e9) if not np.isnan(c["proxy"]).
    ↪get("proxy_acc", np.nan)) else -1e9),
        ),
        reverse=True,
    )
    raw_best = sorted_by_score[0]
    selected = raw_best
    reason = "direct_risk_aware_score_win"
    no_repair_veto = False
    hysteresis = False
    floor_fallback = False

    guarded_candidates = [c for c in scored if
    ↪rc2c_is_guarded_context_candidate(c["variant"])]
    best_guarded = max(guarded_candidates, key=lambda c: float(c.
    ↪get("selection_score", -1e9))) if guarded_candidates else None

    # 1) No-repair veto after early tasks unless it really beats the guarded
    ↪family.
    if (
        RC2C_RISK_AWARE_SELECTOR_ENABLED
        and best_guarded is not None
        and int(after_task) >= RC2C_NO_REPAIR_VETO_AFTER_TASK
        and rc2b_policy_family(raw_best["variant"]) == "proto_no_repair"
    ):

```

```

        margin = float(raw_best.get("selection_score", -1e9)) -
        float(best_guarded.get("selection_score", -1e9))
        if margin < RC2C_NO_REPAIR_REQUIRED_MARGIN:
            selected = best_guarded
            reason = f"no_repair_veto_guarded_fallback_margin_{margin:+.4f}"
            no_repair_veto = True

    # 2) Hard-floor fallback: prefer a floor-safe candidate when the current
    selected one is fragile.
    if RC2C_RISK_AWARE_SELECTOR_ENABLED and RC2C_FLOOR_RISK_FALLBACK_ENABLED:
        if not rc2c_candidate_meets_hard_floors(selected):
            floor_safe = [c for c in scored if
            rc2c_candidate_meets_hard_floors(c)]
            if floor_safe:
                best_floor_safe = max(floor_safe, key=lambda c: float(c.
                get("selection_score", -1e9)))
                if float(best_floor_safe.get("selection_score", -1e9)) >=
                float(selected.get("selection_score", -1e9)) -
                RC2C_GUARDED_HYSTERESIS_MARGIN:
                    selected = best_floor_safe
                    reason = "hard_floor_risk_fallback"
                    floor_fallback = True

    # 3) Guarded hysteresis: if close, prefer the historically robust guarded
    context-gap family.
    if (
        RC2C_RISK_AWARE_SELECTOR_ENABLED
        and best_guarded is not None
        and not rc2c_is_guarded_context_candidate(selected["variant"])
        and float(best_guarded.get("selection_score", -1e9)) >= float(selected.
        get("selection_score", -1e9)) - RC2C_GUARDED_HYSTERESIS_MARGIN
    ):
        selected = best_guarded
        reason = "guarded_context_gap_hysteresis"
        hysteresis = True

    return selected, {
        "raw_best_variant": raw_best["variant"],
        "raw_best_policy_family": raw_best["policy_family"],
        "raw_best_proxy_score": raw_best.get("selection_score", np.nan),
        "best_guarded_variant": best_guarded["variant"] if best_guarded is not
        None else "",
        "best_guarded_proxy_score": best_guarded.get("selection_score", np.nan)
        if best_guarded is not None else np.nan,
        "selection_reason": reason,
        "no_repair_veto_applied": no_repair_veto,
    }

```

```

        "guarded_hysteresis_applied": hysteresis,
        "risk_floor_fallback_applied": floor_fallback,
    }

def build_rc2b_candidate_from_checkpoint(base_run, checkpoint, variant: str,
                                       repair_memory,
    ↪ context_selection_memory, policy_validation_memory,
                                       seed=0):
    """Build one candidate policy and score it on policy-validation memory only.
    ↪ """
    candidate_seed = int(seed)
    cp_seed = int(checkpoint["seed"])
    after_task = int(checkpoint["after_task"])
    method_name = rc2b_candidate_method_name(variant)

    base_cfg = METHOD_CONFIGS["mmals_proto_first_balanced_replay"]
    model = restore_base_model_from_checkpoint(checkpoint, base_cfg)
    context_prototypes = clone_prototypes(checkpoint["context_prototypes"])

    config, config_rows = build_context_variant_config_and_logs(
        model, context_selection_memory, context_prototypes,
        variant=variant, seed=candidate_seed + 313,
    )

    readout = GlobalLinearReadout(model.global_head).to(DEVICE)
    repair_rows = list(config_rows)
    pre_global_head = copy.deepcopy(model.global_head).to(DEVICE)
    pre_global_head.eval()
    for p in pre_global_head.parameters():
        p.requires_grad_(False)

    if variant_uses_guarded_head(variant):
        repair_rows.extend(train_readout_on_memory(
            readout, model, repair_memory, config,
            prototypes=context_prototypes, pre_global_head=pre_global_head,
            kl_weight=0.35, use_kl=True, margin_weight=0.0,
            variant=f"{variant}_rc2b_guarded_policy_candidate",
        ))

    proxy = proxy_metrics_for_context_config(
        model, policy_validation_memory, config,
        prototypes=context_prototypes, readout=readout,
        max_items_per_class=VALIDATION_MAX_ITEMS_PER_CLASS,
    )

    for rr in repair_rows:
        rr.update({

```

```

        "method": method_name,
        "seed": cp_seed,
        "after_task": after_task,
        "task_id": after_task,
        "variant": variant,
        "repair_type": str(rr.get("repair_type", "")),
        "rc2b_strict_candidate": True,
        "rc2b_no_final_test_used_for_selection": True,
        "policy_family": rc2b_policy_family(variant),
        "uses_guarded_global_head": variant_uses_guarded_head(variant),
        "uses_context_selection": context_variant_selection_mode(variant)
    )
    if is_not_None(
        "uses_oracle_context_reference":
    ):
        is_oracle_context_reference(variant),
    })

    return {
        "variant": variant,
        "method": method_name,
        "policy_family": rc2b_policy_family(variant),
        "policy_label": rc2b_policy_label(variant),
        "complexity_rank": rc2b_policy_complexity_rank(variant),
        "model": model,
        "readout": readout,
        "config": config,
        "context_prototypes": context_prototypes,
        "proxy": proxy,
        "repair_rows": repair_rows,
    }

def rc2b_history_rows_from_evals(candidate, tasks, checkpoint,
    method_name=None, selected=False):
    seed = int(checkpoint["seed"])
    after_task = int(checkpoint["after_task"])
    method = method_name or candidate["method"]
    model = candidate["model"]
    readout = candidate["readout"]
    config = candidate["config"]
    context_prototypes = candidate["context_prototypes"]
    evals = evaluate_seen_tasks_with_readout(model, readout, tasks,
    upto_task=after_task, config=config, prototypes=context_prototypes)

    history_rows = []
    replay_memory_items = int(checkpoint.get("replay_memory_items", 0))
    readout_trainable = count_parameters(readout)
    readout_total = total_parameters(readout)

```



```

for old_task_id, metrics in enumerate(evals):
    row = {
        "method": method,
        "seed": seed,
        "model_family": "mmals_rc2b_policy" if selected else
↪ "mmals_rc2b_candidate",
        "after_task": after_task,
        "task_id": old_task_id,
        "task_name": tasks[old_task_id]["name"],
        "seen_classes": str(seen_classes_until(after_task)),
        "trainable_params": count_parameters(model) + readout_trainable,
        "total_params": total_parameters(model) + readout_total,
        "backbone_params": total_parameters(model),
        "readout_trainable_params": readout_trainable,
        "readout_total_params": readout_total,
        "model_size_mb_total": (total_parameters(model) + readout_total) *
↪ 4 / (1024 ** 2),
        "uses_balanced_replay": True,
        "uses_output_distill": True,
        "uses_local_mycorrhizal_head": False,
        "uses_global_head_refresh":
↪ variant_uses_guarded_head(candidate["variant"]),
        "uses_context_selection":
↪ context_variant_selection_mode(candidate["variant"]) is not None,
        "uses_oracle_context_reference": False,
        "probe_row": False,
        "backbone_source": "same_proto_first_checkpoint",
        "probe_feature": "original_backbone_zf_rc2b_policy",
        "probe_context_mode": "proto_first_rc2b_strict",
        "replay_memory_items": replay_memory_items,
        "memory_per_class": MEMORY_PER_CLASS,
        "memory_per_task": MEMORY_PER_TASK,
        "paired_same_backbone": True,
        "readout_variant": candidate["variant"],
        "context_variant": candidate["variant"],
        "selected_candidate_method": candidate["method"] if selected else
↪ None,
        "rc2b_selected_policy": bool(selected),
        "rc2b_candidate_policy_family": candidate["policy_family"],
        "rc2b_selection_proxy_acc": candidate["proxy"].get("proxy_acc", np.
↪ nan),
        "rc2b_selection_score": candidate.get("selection_score", np.nan),
        "rc2b_no_final_test_used_for_selection": True,
        "context_temperature_used": float(getattr(model,
↪ "context_temperature", np.nan)),

```

```

        "batch_blend_single_used": float(config.get("batch_blend_single",
↪np.nan)),
        "batch_blend_latent_used": float(config.get("batch_blend_latent",
↪np.nan)),
        "batch_blend_feature_used": float(config.get("batch_blend_feature",
↪np.nan)),
    }
    row.update(metrics)
    history_rows.append(row)
    return pd.DataFrame(history_rows)

def rc2b_confusion_rows(candidate, tasks, checkpoint, method_name=None,
↪selected=False):
    seed = int(checkpoint["seed"])
    after_task = int(checkpoint["after_task"])
    if after_task != N_TASKS - 1:
        return pd.DataFrame()
    method = method_name or candidate["method"]
    cm_df = class_confusion_matrix_with_readout(
        candidate["model"], candidate["readout"], tasks,
        upto_task=after_task, config=candidate["config"],
↪prototypes=candidate["context_prototypes"]
    )
    cm_df["method"] = method
    cm_df["seed"] = seed
    cm_df["after_task"] = after_task
    cm_df["backbone_source"] = "same_proto_first_checkpoint"
    cm_df["readout_variant"] = candidate["variant"]
    cm_df["context_variant"] = candidate["variant"]
    cm_df["rc2b_selected_policy"] = bool(selected)
    cm_df["selected_candidate_method"] = candidate["method"] if selected else
↪None
    return cm_df

def apply_rc2b_strict_policy_to_checkpoint(base_run, checkpoint):
    """Strictly select the deployable policy before final-test evaluation."""
    seed = int(checkpoint["seed"])
    after_task = int(checkpoint["after_task"])
    tasks = base_run["tasks"]
    class_memory = clone_class_memory(checkpoint["class_memory"])

    repair_memory, context_selection_memory, policy_validation_memory =
↪split_class_memory_three_way(
        class_memory,

```

```

        seed=seed * 100_000 + after_task * 10_000 + 777,
    )

    candidates = []
    for variant in RC2B_CANDIDATE_VARIANTS:
        cand = build_rc2b_candidate_from_checkpoint(
            base_run, checkpoint, variant,
            repair_memory=repair_memory,
            context_selection_memory=context_selection_memory,
            policy_validation_memory=policy_validation_memory,
            seed=seed * 10_000 + after_task * 313 +
↪rc2b_policy_complexity_rank(variant),
        )
        candidates.append(cand)

    baseline = next((c for c in candidates if c["variant"] ==
↪"proto_global_no_repair"), candidates[0])
    baseline_proxy = baseline["proxy"]

    # RC3 Hydro audit: compute candidate-vs-baseline drift/turbulence on
↪policy-validation memory only.
    # No final-test examples are used here.
    for cand in candidates:
        hydro_proxy = rc3_pairwise_hydro_proxy(cand, baseline,
↪policy_validation_memory)
        cand["hydro_proxy"] = hydro_proxy
        cand["proxy"].update(hydro_proxy)

    selection_rows = []
    for cand in candidates:
        score = rc2b_validation_score(cand["proxy"], baseline_proxy,
↪cand["variant"], after_task=after_task)
        cand["risk_components"] = rc2c_risk_components(cand["proxy"],
↪baseline_proxy, cand["variant"], after_task=after_task) if
↪RC2C_RISK_AWARE_SELECTOR_ENABLED else {}
        cand["selection_score"] = score
        p = cand["proxy"]
        selection_rows.append({
            "method": RC2B_METHOD,
            "seed": seed,
            "after_task": after_task,
            "task_id": after_task,
            "candidate_variant": cand["variant"],
            "candidate_method": cand["method"],
            "policy_family": cand["policy_family"],
            "policy_label": cand["policy_label"],

```

```

        "complexity_rank": cand["complexity_rank"],
        "proxy_acc": p.get("proxy_acc", np.nan),
        "proxy_class2": p.get("proxy_class2", np.nan),
        "proxy_class5": p.get("proxy_class5", np.nan),
        "proxy_min_class": p.get("proxy_min_class", np.nan),
        "proxy_min_task": p.get("proxy_min_task", np.nan),
        "proxy_old_task_min": p.get("proxy_old_task_min", np.nan),
        "proxy_task_acc_std": p.get("proxy_task_acc_std", np.nan),
        "proxy_score": score,
        "rc2c_risk_aware_selector_enabled": 1
    }
    RC2C_RISK_AWARE_SELECTOR_ENABLED,
        "rc2c_class2_floor_penalty": cand.get("risk_components", {}).
    get("class2_floor_penalty", np.nan),
        "rc2c_min_class_floor_penalty": cand.get("risk_components", {}).
    get("min_class_floor_penalty", np.nan),
        "rc2c_min_task_floor_penalty": cand.get("risk_components", {}).
    get("min_task_floor_penalty", np.nan),
        "rc2c_old_task_floor_penalty": cand.get("risk_components", {}).
    get("old_task_floor_penalty", np.nan),
        "rc2c_task_dispersion_penalty": cand.get("risk_components", {}).
    get("task_dispersion_penalty", np.nan),
        "rc2c_no_repair_soft_penalty": cand.get("risk_components", {}).
    get("no_repair_soft_penalty", np.nan),
        "rc2c_guarded_context_bonus": cand.get("risk_components", {}).
    get("guarded_context_bonus", np.nan),
        "hydro_route_drift_Dr": p.get("hydro_route_drift_Dr", np.nan),
        "hydro_latent_drift_Dz": p.get("hydro_latent_drift_Dz", np.nan),
        "hydro_output_drift_Dy": p.get("hydro_output_drift_Dy", np.nan),
        "hydro_adaptive_viscosity_nu": p.get("hydro_adaptive_viscosity_nu",
    np.nan),
        "hydro_reynolds_like": p.get("hydro_reynolds_like", np.nan),
        "hydro_turbulence_score": p.get("hydro_turbulence_score", np.nan),
        "hydro_wave_entropy": p.get("hydro_wave_entropy", np.nan),
        "hydro_selection_penalty": rc3_hydro_selection_penalty(p),
        "hydro_audit_memory_only": p.get("hydro_audit_memory_only", False),
        "hydro_audit_reference_variant": p.
    get("hydro_audit_reference_variant", ""),
        "selection_phase": "candidate_scored_before_final_test",
        "no_final_test_used_for_selection": True,
        "repair_memory_items": int(sum(len(v["y"]) for v in repair_memory.
    values())),
        "context_selection_memory_items": int(sum(len(v["y"]) for v in
    context_selection_memory.values())),
        "policy_validation_memory_items": int(sum(len(v["y"]) for v in
    policy_validation_memory.values())),
    })

```

```

    if RC2C_RISK_AWARE_SELECTOR_ENABLED:
        selected, selection_meta = rc2c_select_candidate(candidates,
        ↪after_task=after_task)
    else:
        # Deterministic tie-break: score descending, then simpler policy, then
        ↪proxy accuracy.
        candidates_sorted = sorted(
            candidates,
            key=lambda c: (
                float(c.get("selection_score", -1e9)),
                -float(c["complexity_rank"]),
                float(c["proxy"].get("proxy_acc", -1e9) if not np.
        ↪isnan(c["proxy"].get("proxy_acc", np.nan)) else -1e9),
            ),
            reverse=True,
        )
        best = candidates_sorted[0]
        # If multiple candidates are within the tie tolerance, choose the
        ↪simpler policy.
        near_best = [c for c in candidates if c.get("selection_score", -1e9) >=
        ↪best.get("selection_score", -1e9) - RC2B_SELECTION_TIE_TOL]
        selected = sorted(near_best, key=lambda c: (c["complexity_rank"],
        ↪-float(c["proxy"].get("proxy_acc", -1e9) if not np.isnan(c["proxy"]).
        ↪get("proxy_acc", np.nan)) else -1e9)))[0]
        selection_meta = {
            "raw_best_variant": best["variant"],
            "raw_best_policy_family": best["policy_family"],
            "raw_best_proxy_score": best.get("selection_score", np.nan),
            "best_guarded_variant": "",
            "best_guarded_proxy_score": np.nan,
            "selection_reason": "rc2b_strict_score_then_simplicity_tiebreak",
            "no_repair_veto_applied": False,
            "guarded_hysteresis_applied": False,
            "risk_floor_fallback_applied": False,
        }

    for r in selection_rows:
        r["selected_candidate_variant"] = selected["variant"]
        r["selected_candidate_method"] = selected["method"]
        r["selected_policy_family"] = selected["policy_family"]
        r["selected_policy_label"] = selected["policy_label"]
        r["selected_proxy_score"] = selected.get("selection_score", np.nan)
        r["selected_proxy_acc"] = selected["proxy"].get("proxy_acc", np.nan)
        r["is_selected_candidate"] = (r["candidate_variant"] ==
        ↪selected["variant"])

```

```

        r.update(selection_meta)

    candidate_history, candidate_confusions = [], []
    if RC2B_EVALUATE_ALL_CANDIDATES_AFTER_SELECTION:
        for cand in candidates:
            h = rc2b_history_rows_from_evals(cand, tasks, checkpoint,
↪method_name=cand["method"], selected=False)
            candidate_history.append(h)
            c = rc2b_confusion_rows(cand, tasks, checkpoint,
↪method_name=cand["method"], selected=False)
            if not c.empty:
                candidate_confusions.append(c)

    selected_history = rc2b_history_rows_from_evals(selected, tasks,
↪checkpoint, method_name=RC2B_METHOD, selected=True)
    selected_confusion = rc2b_confusion_rows(selected, tasks, checkpoint,
↪method_name=RC2B_METHOD, selected=True)

    repair_rows = []
    for cand in candidates:
        for rr in cand.get("repair_rows", []):
            rr = dict(rr)
            rr.update({
                "rc2b_candidate_variant": cand["variant"],
                "rc2b_selected_candidate_variant": selected["variant"],
                "rc2b_selected_candidate_method": selected["method"],
                "rc2b_is_selected_candidate": cand["variant"] ==
↪selected["variant"],
                "rc2b_no_final_test_used_for_selection": True,
            })
            repair_rows.append(rr)

    selection_df = pd.DataFrame(selection_rows)
    candidate_history_df = pd.concat(candidate_history, ignore_index=True) if
↪candidate_history else pd.DataFrame()
    candidate_confusion_df = pd.concat(candidate_confusions, ignore_index=True)
↪if candidate_confusions else pd.DataFrame()
    selected_confusion_df = selected_confusion if not selected_confusion.empty
↪else pd.DataFrame()
    repair_df = pd.DataFrame(repair_rows)

    print(
        f"[{POLICY_BRANCH} seed={seed}] after task {after_task}: selected
↪{selected['variant']} "
        f"proxy_acc={selected['proxy'].get('proxy_acc', np.nan):.4f} "
        f"score={selected.get('selection_score', np.nan):.4f}"
    )

```

```

    )
    return selected_history, selected_confusion_df, candidate_history_df,
    ↪candidate_confusion_df, repair_df, selection_df

```

```

[15]: # -----
# v1.1-RC2d gate-aligned selector overrides
# -----
# RC2d fixes the RC2c smoke failure modes:
# - performance gate failure;
# - class-2 robustness failure;
# - class-5 preservation failure;
# - partial selector/final-test alignment.
#
# All functions below still use validation-memory artifacts only.
# They intentionally override selected RC2c helper names so the existing
# experiment loop can remain unchanged.
# -----

RC2D_SELECTOR_VERSION = globals().get("RC2D_SELECTOR_VERSION",
    ↪"RC2d_gate_aligned_v1")
RC2D_GATE_ALIGNMENT_ENABLED = globals().get("RC2D_GATE_ALIGNMENT_ENABLED", True)
RC2D_AUX_PROXY_WEIGHT = globals().get("RC2D_AUX_PROXY_WEIGHT", 0.35)
RC2D_PRIMARY_PROXY_WEIGHT = globals().get("RC2D_PRIMARY_PROXY_WEIGHT", 0.65)
RC2D_PROXY_ACC_FLOOR = globals().get("RC2D_PROXY_ACC_FLOOR", 0.68)
RC2D_PROXY_ACC_FLOOR_PENALTY = globals().get("RC2D_PROXY_ACC_FLOOR_PENALTY", 3.
    ↪00)
RC2D_CLASS5_FLOOR = globals().get("RC2D_CLASS5_FLOOR", 0.90)
RC2D_HARD_CLASS5_FLOOR = globals().get("RC2D_HARD_CLASS5_FLOOR", 0.80)
RC2D_CLASS5_FLOOR_PENALTY = globals().get("RC2D_CLASS5_FLOOR_PENALTY", 5.00)
RC2D_CLASS2_RESCUE_MARGIN = globals().get("RC2D_CLASS2_RESCUE_MARGIN", 0.08)
RC2D_CLASS5_RESCUE_MARGIN = globals().get("RC2D_CLASS5_RESCUE_MARGIN", 0.08)
RC2D_TRUE_GUARD_RESCUE_SCORE_MARGIN = globals().
    ↪get("RC2D_TRUE_GUARD_RESCUE_SCORE_MARGIN", 0.020)
RC2D_TRUE_GUARD_CLASS2_MARGIN = globals().get("RC2D_TRUE_GUARD_CLASS2_MARGIN",
    ↪0.05)
RC2D_TRUE_GUARD_ACC_MARGIN = globals().get("RC2D_TRUE_GUARD_ACC_MARGIN", 0.010)
RC2D_TRUE_GUARD_BONUS = globals().get("RC2D_TRUE_GUARD_BONUS", 0.003)
RC2D_CONTEXT_GUARD_BONUS = globals().get("RC2D_CONTEXT_GUARD_BONUS", 0.001)
RC2D_GUARDED_HYSTERESIS_MARGIN = globals().
    ↪get("RC2D_GUARDED_HYSTERESIS_MARGIN", 0.003)
RC2D_SELECTOR_FINAL_ALIGNMENT_TOL = globals().
    ↪get("RC2D_SELECTOR_FINAL_ALIGNMENT_TOL", 0.005)
RC2D_CLASS_BALANCE_WEIGHT = globals().get("RC2D_CLASS_BALANCE_WEIGHT", 0.35)
RC2D_MIN_TASK_WEIGHT = globals().get("RC2D_MIN_TASK_WEIGHT", 0.35)
RC2D_OLD_TASK_WEIGHT = globals().get("RC2D_OLD_TASK_WEIGHT", 0.20)
RC2D_TASK_STD_WEIGHT = globals().get("RC2D_TASK_STD_WEIGHT", 0.25)

```

```

# Use the new deployed method name while preserving RC2B/RC3 compatibility
↳ aliases.
if globals().get("POLICY_BRANCH", "RC2d") == "RC2d":
    RC2C_METHOD = "paired_rc2d_gate_aligned_policy"
    RC2B_METHOD = RC2C_METHOD
    RC3_METHOD = RC2C_METHOD

def rc2d_finite_value(proxy, key, default=np.nan):
    try:
        v = proxy.get(key, default)
        return float(v)
    except Exception:
        return default

def rc2d_weighted_metric(primary, aux, key, default=np.nan):
    pv = rc2d_finite_value(primary, key, default)
    av = rc2d_finite_value(aux, key, np.nan) if aux is not None else np.nan
    if rc2c_is_finite(pv) and rc2c_is_finite(av):
        return RC2D_PRIMARY_PROXY_WEIGHT * pv + RC2D_AUX_PROXY_WEIGHT * av
    if rc2c_is_finite(pv):
        return pv
    if rc2c_is_finite(av):
        return av
    return default

def rc2d_merge_proxy_metrics(primary_proxy, auxiliary_proxy=None):
    """Merge primary and auxiliary validation-memory proxies.

    The primary split remains the policy-validation split. The auxiliary split
    ↳ is
    another memory-only view used to reduce selection noise. No final-test data
    ↳ is used.
    """
    if auxiliary_proxy is None:
        merged = dict(primary_proxy)
        merged["proxy_aux_available"] = False
        return merged
    merged = dict(primary_proxy)
    for key in [
        "proxy_acc", "proxy_context_top1", "proxy_context_entropy",
    ↳ "proxy_context_margin",
        "proxy_class2", "proxy_class5", "proxy_min_class", "proxy_min_task",
        "proxy_old_task_min", "proxy_task_acc_std", "proxy_score",

```



```

]:
    if key in primary_proxy or key in auxiliary_proxy:
        merged[key] = rc2d_weighted_metric(primary_proxy, auxiliary_proxy,
↪key)
        merged[f"primary_{key}"] = primary_proxy.get(key, np.nan)
        merged[f"aux_{key}"] = auxiliary_proxy.get(key, np.nan)
    merged["proxy_aux_available"] = True
    merged["proxy_aux_weight"] = RC2D_AUX_PROXY_WEIGHT
    return merged

def rc2c_candidate_meets_hard_floors(candidate) -> bool:
    """RC2d hard-floor check: class 2, class 5, min task, min class and proxy
↪accuracy."""
    p = candidate.get("proxy", {})
    c2 = rc2c_proxy_value(p, "proxy_class2")
    c5 = rc2c_proxy_value(p, "proxy_class5")
    min_task = rc2c_proxy_value(p, "proxy_min_task", rc2c_proxy_value(p,
↪"proxy_min_class"))
    min_class = rc2c_proxy_value(p, "proxy_min_class")
    acc = rc2c_proxy_value(p, "proxy_acc")
    ok_c2 = (not rc2c_is_finite(c2)) or c2 >= RC2C_HARD_CLASS2_FLOOR
    ok_c5 = (not rc2c_is_finite(c5)) or c5 >= RC2D_HARD_CLASS5_FLOOR
    ok_min_task = (not rc2c_is_finite(min_task)) or min_task >=
↪RC2C_HARD_MIN_TASK_FLOOR
    ok_min_class = (not rc2c_is_finite(min_class)) or min_class >=
↪RC2C_HARD_MIN_CLASS_FLOOR
    ok_acc = (not rc2c_is_finite(acc)) or acc >= max(0.50, RC2D_PROXY_ACC_FLOOR,
↪0.05)
    return bool(ok_c2 and ok_c5 and ok_min_task and ok_min_class and ok_acc)

def rc2c_risk_components(proxy, baseline_proxy, variant: str, after_task=None):
    """RC2d risk components, still computed from validation memory only."""
    acc = rc2c_proxy_value(proxy, "proxy_acc")
    c2 = rc2c_proxy_value(proxy, "proxy_class2")
    c5 = rc2c_proxy_value(proxy, "proxy_class5")
    min_class = rc2c_proxy_value(proxy, "proxy_min_class")
    min_task = rc2c_proxy_value(proxy, "proxy_min_task", min_class)
    old_task_min = rc2c_proxy_value(proxy, "proxy_old_task_min", min_task)
    task_std = rc2c_proxy_value(proxy, "proxy_task_acc_std", 0.0)
    family = rc2b_policy_family(variant)

    class2_penalty = RC2C_CLASS2_FLOOR_PENALTY * rc2c_floor_deficit(c2,
↪RC2C_CLASS2_FLOOR)

```

```

class5_penalty = RC2D_CLASS5_FLOOR_PENALTY * rc2c_floor_deficit(c5,
↪RC2D_CLASS5_FLOOR)
min_class_penalty = RC2C_MIN_CLASS_FLOOR_PENALTY *
↪rc2c_floor_deficit(min_class, RC2C_MIN_CLASS_FLOOR)
min_task_penalty = RC2C_MIN_TASK_FLOOR_PENALTY *
↪rc2c_floor_deficit(min_task, RC2C_MIN_TASK_FLOOR)
old_task_penalty = RC2C_OLD_TASK_FLOOR_PENALTY *
↪rc2c_floor_deficit(old_task_min, RC2C_OLD_TASK_FLOOR)
acc_penalty = RC2D_PROXY_ACC_FLOOR_PENALTY * rc2c_floor_deficit(acc,
↪RC2D_PROXY_ACC_FLOOR)
floor_penalty = class2_penalty + class5_penalty + min_class_penalty +
↪min_task_penalty + old_task_penalty + acc_penalty

dispersion_penalty = RC2D_TASK_STD_WEIGHT * max(0.0, task_std) if
↪rc2c_is_finite(task_std) else 0.0
no_repair_soft_penalty = 0.0
if after_task is not None and int(after_task) >=
↪RC2C_NO_REPAIR_VETO_AFTER_TASK and family == "proto_no_repair":
    no_repair_soft_penalty = RC2C_NO_REPAIR_SOFT_PENALTY

guarded_bonus = RC2D_CONTEXT_GUARD_BONUS if
↪rc2c_is_guarded_context_candidate(variant) else 0.0
true_guard_bonus = RC2D_TRUE_GUARD_BONUS if family ==
↪"proto_true_global_guard" else 0.0
complexity_penalty = RC2B_COMPLEXITY_PENALTY *
↪rc2b_policy_complexity_rank(variant)
class5_bonus = RC2B_VALIDATION_SCORE_CLASS5_WEIGHT * (0.0 if not
↪rc2c_is_finite(c5) else c5)
hydro_penalty = rc3_hydro_selection_penalty(proxy)

return {
    "proxy_acc_floor_penalty": acc_penalty,
    "class2_floor_penalty": class2_penalty,
    "class5_floor_penalty": class5_penalty,
    "min_class_floor_penalty": min_class_penalty,
    "min_task_floor_penalty": min_task_penalty,
    "old_task_floor_penalty": old_task_penalty,
    "floor_penalty": floor_penalty,
    "task_dispersion_penalty": dispersion_penalty,
    "no_repair_soft_penalty": no_repair_soft_penalty,
    "complexity_penalty": complexity_penalty,
    "class5_bonus": class5_bonus,
    "guarded_context_bonus": guarded_bonus,
    "true_guard_bonus": true_guard_bonus,
    "hydro_selection_penalty": hydro_penalty,

```

```

        "total_risk_penalty": floor_penalty + dispersion_penalty +
↪no_repair_soft_penalty + complexity_penalty + hydro_penalty,
    }

def rc2d_gate_aligned_score(proxy, baseline_proxy, variant: str,
↪after_task=None):
    """Gate-aligned validation score with explicit weak-class and
↪average-performance terms."""
    base_score = validation_score(proxy, baseline_proxy)
    if base_score <= -1e8:
        return -1e9

    acc = rc2c_proxy_value(proxy, "proxy_acc", 0.0)
    c2 = rc2c_proxy_value(proxy, "proxy_class2", 0.0)
    c5 = rc2c_proxy_value(proxy, "proxy_class5", 0.0)
    min_class = rc2c_proxy_value(proxy, "proxy_min_class", 0.0)
    min_task = rc2c_proxy_value(proxy, "proxy_min_task", min_class)
    old_task_min = rc2c_proxy_value(proxy, "proxy_old_task_min", min_task)
    task_std = rc2c_proxy_value(proxy, "proxy_task_acc_std", 0.0)

    comp = rc2c_risk_components(proxy, baseline_proxy, variant,
↪after_task=after_task)
    balance_score = (
        acc
        + RC2D_CLASS_BALANCE_WEIGHT * (0.5 * c2 + 0.5 * c5)
        + RC2D_MIN_TASK_WEIGHT * min_task
        + RC2D_OLD_TASK_WEIGHT * old_task_min
        - RC2D_TASK_STD_WEIGHT * max(0.0, task_std)
    )
    return float(
        0.50 * base_score
        + 0.50 * balance_score
        + comp["class5_bonus"]
        + comp["guarded_context_bonus"]
        + comp["true_guard_bonus"]
        - comp["total_risk_penalty"]
    )

def rc2b_validation_score(proxy, baseline_proxy, variant: str, after_task=None):
    """Override: RC2d gate-aligned selector score, with RC2b strict fallback."""
    if not RC2C_RISK_AWARE_SELECTOR_ENABLED or globals().get("POLICY_BRANCH")
↪== "RC2b":
        base_score = validation_score(proxy, baseline_proxy)
        if base_score <= -1e8:
            return -1e9

```

```

        c2 = proxy.get("proxy_class2", np.nan)
        c5 = proxy.get("proxy_class5", np.nan)
        minc = proxy.get("proxy_min_class", np.nan)
        penalty = RC2B_COMPLEXITY_PENALTY * rc2b_policy_complexity_rank(variant)
        floor_penalty = 0.0
        if not np.isnan(c2) and c2 < RC2B_CLASS2_FLOOR:
            floor_penalty += RC2B_VALIDATION_SCORE_FLOOR_PENALTY *
↪(RC2B_CLASS2_FLOOR - c2)
        if not np.isnan(c5) and c5 < RC2B_CLASS5_FLOOR:
            floor_penalty += RC2B_VALIDATION_SCORE_FLOOR_PENALTY *
↪(RC2B_CLASS5_FLOOR - c5)
        if not np.isnan(minc) and minc < RC2B_MIN_CLASS_FLOOR:
            floor_penalty += RC2B_VALIDATION_SCORE_FLOOR_PENALTY *
↪(RC2B_MIN_CLASS_FLOOR - minc)
        class5_bonus = RC2B_VALIDATION_SCORE_CLASS5_WEIGHT * (0.0 if np.
↪isnan(c5) else c5)
        hydro_penalty = rc3_hydro_selection_penalty(proxy)
        return float(base_score + class5_bonus - penalty - floor_penalty -
↪hydro_penalty)

    return rc2d_gate_aligned_score(proxy, baseline_proxy, variant,
↪after_task=after_task)

def rc2d_candidate_proxy_acc(c):
    return rc2c_proxy_value(c.get("proxy", {}), "proxy_acc", -1e9)

def rc2d_candidate_class2(c):
    return rc2c_proxy_value(c.get("proxy", {}), "proxy_class2", np.nan)

def rc2d_candidate_class5(c):
    return rc2c_proxy_value(c.get("proxy", {}), "proxy_class5", np.nan)

def rc2d_not_too_far(candidate, selected, margin):
    return float(candidate.get("selection_score", -1e9)) >= float(selected.
↪get("selection_score", -1e9)) - margin

def rc2c_select_candidate(candidates, after_task):
    """Override: RC2d gate-aligned selector.

Still uses only candidate proxies computed on validation-memory partitions.
    """

```

```

    scored = [c for c in candidates if rc2c_is_finite(c.get("selection_score", -1e9))]
    if not scored:
        selected = candidates[0]
        return selected, {
            "raw_best_variant": selected["variant"],
            "selection_reason": "fallback_no_finite_scores",
            "no_repair_veto_applied": False,
            "guarded_hysteresis_applied": False,
            "risk_floor_fallback_applied": False,
            "true_guard_rescue_applied": False,
            "class2_rescue_applied": False,
            "class5_rescue_applied": False,
            "rc2d_selector_version": RC2D_SELECTOR_VERSION,
        }

    sorted_by_score = sorted(
        scored,
        key=lambda c: (
            float(c.get("selection_score", -1e9)),
            float(c.get("proxy", {}).get("proxy_acc", -1e9) if not np.isnan(c.get("proxy", {}).get("proxy_acc", np.nan)) else -1e9),
            -float(c["complexity_rank"]),
        ),
        reverse=True,
    )
    raw_best = sorted_by_score[0]
    selected = raw_best
    reason = "direct_rc2d_gate_aligned_score_win"
    no_repair_veto = False
    hysteresis = False
    floor_fallback = False
    true_guard_rescue = False
    class2_rescue = False
    class5_rescue = False

    guarded_candidates = [c for c in scored if
        rc2c_is_guarded_context_candidate(c["variant"])]
    best_guarded = max(guarded_candidates, key=lambda c: float(c.get("selection_score", -1e9))) if guarded_candidates else None
    true_guard_candidates = [c for c in scored if
        rc2b_policy_family(c["variant"]) == "proto_true_global_guard"]
    best_true_guard = max(true_guard_candidates, key=lambda c: float(c.get("selection_score", -1e9))) if true_guard_candidates else None

```

```

    # 1) No-repair veto after early tasks, but do not force guarded if true
    ↪guard is better.
    if (
        best_guarded is not None
        and int(after_task) >= RC2C_NO_REPAIR_VETO_AFTER_TASK
        and rc2b_policy_family(raw_best["variant"]) == "proto_no_repair"
    ):
        margin = float(raw_best.get("selection_score", -1e9)) -
    ↪float(best_guarded.get("selection_score", -1e9))
        if margin < RC2C_NO_REPAIR_REQUIRED_MARGIN:
            selected = best_guarded
            reason = f"no_repair_veto_guarded_fallback_margin_{margin:+.4f}"
            no_repair_veto = True

    # 2) True global guard rescue: avoid over-selecting context-gap if the true
    ↪guard is close and better on class 2 or validation accuracy.
    if best_true_guard is not None and rc2d_not_too_far(best_true_guard,
    ↪selected, RC2D_TRUE_GUARD_RESCUE_SCORE_MARGIN):
        tg_c2 = rc2d_candidate_class2(best_true_guard)
        se_c2 = rc2d_candidate_class2(selected)
        tg_acc = rc2d_candidate_proxy_acc(best_true_guard)
        se_acc = rc2d_candidate_proxy_acc(selected)
        c2_gain = (rc2c_is_finite(tg_c2) and rc2c_is_finite(se_c2) and tg_c2 >=
    ↪se_c2 + RC2D_TRUE_GUARD_CLASS2_MARGIN)
        acc_gain = (rc2c_is_finite(tg_acc) and rc2c_is_finite(se_acc) and
    ↪tg_acc >= se_acc + RC2D_TRUE_GUARD_ACC_MARGIN)
        if (c2_gain or acc_gain) and
    ↪rc2c_candidate_meets_hard_floors(best_true_guard):
            selected = best_true_guard
            reason = "true_global_guard_rescue_for_class2_or_accuracy_alignment"
            true_guard_rescue = True

    # 3) Weak-class rescue for class 2.
    selected_c2 = rc2d_candidate_class2(selected)
    if rc2c_is_finite(selected_c2) and selected_c2 < RC2C_HARD_CLASS2_FLOOR:
        c2_candidates = [c for c in scored if
    ↪rc2c_is_finite(rc2d_candidate_class2(c))]
        if c2_candidates:
            best_c2 = max(c2_candidates, key=lambda c:
    ↪(rc2d_candidate_class2(c), float(c.get("selection_score", -1e9))))
            if rc2d_candidate_class2(best_c2) >= selected_c2 +
    ↪RC2D_CLASS2_RESCUE_MARGIN and rc2d_not_too_far(best_c2, selected,
    ↪RC2D_TRUE_GUARD_RESCUE_SCORE_MARGIN):
                selected = best_c2
                reason = "class2_hard_floor_rescue"
                class2_rescue = True

```

```

# 4) Weak-class rescue for class 5 preservation.
selected_c5 = rc2d_candidate_class5(selected)
if rc2c_is_finite(selected_c5) and selected_c5 < RC2D_HARD_CLASS5_FLOOR:
    c5_candidates = [c for c in scored if
↳rc2c_is_finite(rc2d_candidate_class5(c))]
    if c5_candidates:
        best_c5 = max(c5_candidates, key=lambda c:
↳(rc2d_candidate_class5(c), float(c.get("selection_score", -1e9))))
        if rc2d_candidate_class5(best_c5) >= selected_c5 +
↳RC2D_CLASS5_RESCUE_MARGIN and rc2d_not_too_far(best_c5, selected,
↳RC2D_TRUE_GUARD_RESCUE_SCORE_MARGIN):
            selected = best_c5
            reason = "class5_hard_floor_rescue"
            class5_rescue = True

# 5) Hard-floor fallback: prefer a floor-safe candidate if close.
if RC2C_FLOOR_RISK_FALLBACK_ENABLED and not
↳rc2c_candidate_meets_hard_floors(selected):
    floor_safe = [c for c in scored if rc2c_candidate_meets_hard_floors(c)]
    if floor_safe:
        best_floor_safe = max(floor_safe, key=lambda c: float(c.
↳get("selection_score", -1e9)))
        if rc2d_not_too_far(best_floor_safe, selected,
↳RC2D_GUARDED_HYSTERESIS_MARGIN):
            selected = best_floor_safe
            reason = "hard_floor_risk_fallback"
            floor_fallback = True

# 6) Reduced guarded hysteresis: apply only if guarded is floor-safe and
↳not weaker on class 2.
if (
    best_guarded is not None
    and not rc2c_is_guarded_context_candidate(selected["variant"])
    and rc2d_not_too_far(best_guarded, selected,
↳RC2D_GUARDED_HYSTERESIS_MARGIN)
    and rc2c_candidate_meets_hard_floors(best_guarded)
):
    bg_c2 = rc2d_candidate_class2(best_guarded)
    se_c2 = rc2d_candidate_class2(selected)
    not_c2_worse = (not rc2c_is_finite(bg_c2)) or (not
↳rc2c_is_finite(se_c2)) or bg_c2 >= se_c2 - 0.03
    if not_c2_worse:
        selected = best_guarded
        reason = "reduced_guarded_context_gap_hysteresis"
        hysteresis = True

```

```

    return selected, {
        "raw_best_variant": raw_best["variant"],
        "raw_best_policy_family": raw_best["policy_family"],
        "raw_best_proxy_score": raw_best.get("selection_score", np.nan),
        "best_guarded_variant": best_guarded["variant"] if best_guarded is not None
        else "",
        "best_guarded_proxy_score": best_guarded.get("selection_score", np.nan)
        if best_guarded is not None else np.nan,
        "best_true_guard_variant": best_true_guard["variant"] if
        best_true_guard is not None else "",
        "best_true_guard_proxy_score": best_true_guard.get("selection_score",
        np.nan) if best_true_guard is not None else np.nan,
        "selection_reason": reason,
        "no_repair_veto_applied": no_repair_veto,
        "guarded_hysteresis_applied": hysteresis,
        "risk_floor_fallback_applied": floor_fallback,
        "true_guard_rescue_applied": true_guard_rescue,
        "class2_rescue_applied": class2_rescue,
        "class5_rescue_applied": class5_rescue,
        "rc2d_selector_version": RC2D_SELECTOR_VERSION,
    }

def build_rc2b_candidate_from_checkpoint(base_run, checkpoint, variant: str,
                                       repair_memory,
                                       context_selection_memory, policy_validation_memory,
                                       seed=0):
    """Override: build one candidate and score it on memory-only primary +
    auxiliary validation splits."""
    candidate_seed = int(seed)
    cp_seed = int(checkpoint["seed"])
    after_task = int(checkpoint["after_task"])
    method_name = rc2b_candidate_method_name(variant)

    base_cfg = METHOD_CONFIGS["mmals_proto_first_balanced_replay"]
    model = restore_base_model_from_checkpoint(checkpoint, base_cfg)
    context_prototypes = clone_prototypes(checkpoint["context_prototypes"])

    config, config_rows = build_context_variant_config_and_logs(
        model, context_selection_memory, context_prototypes,
        variant=variant, seed=candidate_seed + 313,
    )

    readout = GlobalLinearReadout(model.global_head).to(DEVICE)
    repair_rows = list(config_rows)

```



```

pre_global_head = copy.deepcopy(model.global_head).to(DEVICE)
pre_global_head.eval()
for p in pre_global_head.parameters():
    p.requires_grad_(False)

if variant_uses_guarded_head(variant):
    repair_rows.extend(train_readout_on_memory(
        readout, model, repair_memory, config,
        prototypes=context_prototypes, pre_global_head=pre_global_head,
        kl_weight=0.35, use_kl=True, margin_weight=0.0,
        variant=f"{variant}_rc2d_guarded_policy_candidate",
    ))

primary_proxy = proxy_metrics_for_context_config(
    model, policy_validation_memory, config,
    prototypes=context_prototypes, readout=readout,
    max_items_per_class=VALIDATION_MAX_ITEMS_PER_CLASS,
)
auxiliary_proxy = proxy_metrics_for_context_config(
    model, context_selection_memory, config,
    prototypes=context_prototypes, readout=readout,
    max_items_per_class=VALIDATION_MAX_ITEMS_PER_CLASS,
) if RC2D_AUX_PROXY_WEIGHT > 0 else None
proxy = rc2d_merge_proxy_metrics(primary_proxy, auxiliary_proxy)

for rr in repair_rows:
    rr.update({
        "method": method_name,
        "seed": cp_seed,
        "after_task": after_task,
        "task_id": after_task,
        "variant": variant,
        "repair_type": str(rr.get("repair_type", "")),
        "rc2b_strict_candidate": True,
        "rc2d_gate_aligned_candidate": True,
        "rc2b_no_final_test_used_for_selection": True,
        "policy_family": rc2b_policy_family(variant),
        "uses_guarded_global_head": variant_uses_guarded_head(variant),
        "uses_context_selection": context_variant_selection_mode(variant)
        ↪is not None,
        "uses_oracle_context_reference":
        ↪is_oracle_context_reference(variant),
    })

return {
    "variant": variant,
    "method": method_name,

```

```

        "policy_family": rc2b_policy_family(variant),
        "policy_label": rc2b_policy_label(variant),
        "complexity_rank": rc2b_policy_complexity_rank(variant),
        "model": model,
        "readout": readout,
        "config": config,
        "context_prototypes": context_prototypes,
        "proxy": proxy,
        "primary_proxy": primary_proxy,
        "auxiliary_proxy": auxiliary_proxy or {},
        "repair_rows": repair_rows,
    }

print("RC2d gate-aligned selector overrides loaded:", RC2D_SELECTOR_VERSION)
print("RC2D_AUX_PROXY_WEIGHT:", RC2D_AUX_PROXY_WEIGHT)
print("RC2D_CLASS5_FLOOR:", RC2D_CLASS5_FLOOR, "RC2D_PROXY_ACC_FLOOR:", RC2D_PROXY_ACC_FLOOR)

```

RC2d gate-aligned selector overrides loaded: RC2d_gate_aligned_v1
RC2D_AUX_PROXY_WEIGHT: 0.35
RC2D_CLASS5_FLOOR: 0.9 RC2D_PROXY_ACC_FLOOR: 0.68

```

[16]: # -----
# v1.1-RC2g tri-boundary equilibrium selector/readout overrides
# -----
# RC2g patches the RC2D smoke failure mode without changing the backbone:
# - class-specific metrics are ignored until the class is available;
# - NaN metrics never poison the candidate score;
# - all-NaN selector fallback is replaced by a deterministic memory-only
  ↪ emergency score;
# - guarded readout candidates receive a small class-balanced calibration on
  ↪ repair memory.
# All operations remain strict: repair/context-selection/policy-validation
  ↪ memory only.
# -----

RC2G_SELECTOR_VERSION = globals().get("RC2G_SELECTOR_VERSION",
  ↪ "RC2g_nan_safe_class_balanced_v1")
RC2G_NAN_SAFE_SCORING_ENABLED = globals().get("RC2G_NAN_SAFE_SCORING_ENABLED",
  ↪ True)
RC2G_CLASS_AVAILABILITY_GATING_ENABLED = globals().
  ↪ get("RC2G_CLASS_AVAILABILITY_GATING_ENABLED", True)
RC2G_EMERGENCY_SCORE_IF_ALL_NAN = globals().
  ↪ get("RC2G_EMERGENCY_SCORE_IF_ALL_NAN", True)
RC2G_CLASS_BALANCED_READOUT_CALIBRATION_ENABLED = globals().
  ↪ get("RC2G_CLASS_BALANCED_READOUT_CALIBRATION_ENABLED", True)

```

```

RC2G_CLASS_BALANCED_CALIB_STEPS = globals().
    ↪get("RC2G_CLASS_BALANCED_CALIB_STEPS", 12 if RUN_MODE == "smoke" else 30)
RC2G_CLASS_BALANCED_CALIB_LR = globals().get("RC2G_CLASS_BALANCED_CALIB_LR", 1.
    ↪5e-3)
RC2G_WEAK_CLASS_WEIGHT = globals().get("RC2G_WEAK_CLASS_WEIGHT", 2.50)
RC2G_MAX_CLASS_WEIGHT = globals().get("RC2G_MAX_CLASS_WEIGHT", 4.00)
RC2G_CLASS_BALANCED_MARGIN_WEIGHT = globals().
    ↪get("RC2G_CLASS_BALANCED_MARGIN_WEIGHT", 0.35)
RC2G_SCORE_NEUTRAL_VALUE = globals().get("RC2G_SCORE_NEUTRAL_VALUE", 0.0)

if globals().get("POLICY_BRANCH", "RC2g") == "RC2g":
    RC2C_METHOD = "paired_rc2g_tri_boundary_equilibrium_policy"
    RC2B_METHOD = RC2C_METHOD
    RC3_METHOD = RC2C_METHOD

def rc2f_seen_classes(after_task=None):
    if after_task is None:
        return set(range(N_CLASSES))
    try:
        return set(seen_classes_until(int(after_task)))
    except Exception:
        return set(range(N_CLASSES))

def rc2f_class_applicable(class_id: int, after_task=None) -> bool:
    if not RC2G_CLASS_AVAILABILITY_GATING_ENABLED:
        return True
    return int(class_id) in rc2f_seen_classes(after_task)

def rc2f_finite(x) -> bool:
    return rc2c_is_finite(x)

def rc2f_metric(proxy, key, neutral=RC2G_SCORE_NEUTRAL_VALUE):
    value = rc2c_proxy_value(proxy, key, np.nan)
    return float(value) if rc2f_finite(value) else float(neutral)

def rc2f_class_metric(proxy, key, class_id: int, after_task=None,
    ↪neutral=RC2G_SCORE_NEUTRAL_VALUE):
    # If class is not yet part of the seen history, the metric is not
    ↪applicable.
    if not rc2f_class_applicable(class_id, after_task):
        return float(neutral), False
    value = rc2c_proxy_value(proxy, key, np.nan)

```

```

    if rc2f_finite(value):
        return float(value), True
    # Seen but absent from the small validation split: keep score finite and
    ↪audit as missing.
    return float(neutral), False

def rc2f_floor_deficit_metric(proxy, key, floor, class_id=None,
    ↪after_task=None):
    if class_id is not None and not rc2f_class_applicable(class_id, after_task):
        return 0.0
    value = rc2c_proxy_value(proxy, key, np.nan)
    if not rc2f_finite(value):
        return 0.0
    return max(0.0, float(floor) - float(value))

def rc2c_candidate_meets_hard_floors(candidate) -> bool:
    """RC2g class-aware hard-floor check.

    Class-specific floors are enforced only after the class has appeared in the
    benchmark history. Missing class metrics from tiny memory splits do not make
    the score non-finite and do not trigger accidental fallback.
    """
    p = candidate.get("proxy", {})
    after_task = candidate.get("after_task", None)
    c2 = rc2c_proxy_value(p, "proxy_class2")
    c5 = rc2c_proxy_value(p, "proxy_class5")
    min_task = rc2c_proxy_value(p, "proxy_min_task", rc2c_proxy_value(p,
    ↪"proxy_min_class"))
    min_class = rc2c_proxy_value(p, "proxy_min_class")
    acc = rc2c_proxy_value(p, "proxy_acc")
    ok_c2 = (not rc2f_class_applicable(2, after_task)) or (not rc2f_finite(c2))
    ↪or c2 >= RC2C_HARD_CLASS2_FLOOR
    ok_c5 = (not rc2f_class_applicable(5, after_task)) or (not rc2f_finite(c5))
    ↪or c5 >= RC2D_HARD_CLASS5_FLOOR
    ok_min_task = (not rc2f_finite(min_task)) or min_task >=
    ↪RC2C_HARD_MIN_TASK_FLOOR
    ok_min_class = (not rc2f_finite(min_class)) or min_class >=
    ↪RC2C_HARD_MIN_CLASS_FLOOR
    ok_acc = (not rc2f_finite(acc)) or acc >= max(0.50, RC2D_PROXY_ACC_FLOOR -
    ↪0.05)
    return bool(ok_c2 and ok_c5 and ok_min_task and ok_min_class and ok_acc)

def rc2c_risk_components(proxy, baseline_proxy, variant: str, after_task=None):

```

```

"""RC2g NaN-safe risk components, still validation-memory-only."""
acc = rc2f_metric(proxy, "proxy_acc", neutral=0.0)
c2, c2_available = rc2f_class_metric(proxy, "proxy_class2", 2,
↳after_task=after_task, neutral=0.0)
c5, c5_available = rc2f_class_metric(proxy, "proxy_class5", 5,
↳after_task=after_task, neutral=0.0)
min_class = rc2f_metric(proxy, "proxy_min_class", neutral=acc)
min_task = rc2f_metric(proxy, "proxy_min_task", neutral=min_class)
old_task_min = rc2f_metric(proxy, "proxy_old_task_min", neutral=min_task)
task_std = rc2f_metric(proxy, "proxy_task_acc_std", neutral=0.0)
family = rc2b_policy_family(variant)

class2_penalty = RC2C_CLASS2_FLOOR_PENALTY *
↳rc2f_floor_deficit_metric(proxy, "proxy_class2", RC2C_CLASS2_FLOOR,
↳class_id=2, after_task=after_task)
class5_penalty = RC2D_CLASS5_FLOOR_PENALTY *
↳rc2f_floor_deficit_metric(proxy, "proxy_class5", RC2D_CLASS5_FLOOR,
↳class_id=5, after_task=after_task)
min_class_penalty = RC2C_MIN_CLASS_FLOOR_PENALTY *
↳rc2f_floor_deficit_metric(proxy, "proxy_min_class", RC2C_MIN_CLASS_FLOOR,
↳after_task=after_task)
min_task_penalty = RC2C_MIN_TASK_FLOOR_PENALTY *
↳rc2f_floor_deficit_metric(proxy, "proxy_min_task", RC2C_MIN_TASK_FLOOR,
↳after_task=after_task)
old_task_penalty = RC2C_OLD_TASK_FLOOR_PENALTY *
↳rc2f_floor_deficit_metric(proxy, "proxy_old_task_min", RC2C_OLD_TASK_FLOOR,
↳after_task=after_task)
acc_penalty = RC2D_PROXY_ACC_FLOOR_PENALTY *
↳rc2f_floor_deficit_metric(proxy, "proxy_acc", RC2D_PROXY_ACC_FLOOR,
↳after_task=after_task)
floor_penalty = class2_penalty + class5_penalty + min_class_penalty +
↳min_task_penalty + old_task_penalty + acc_penalty

dispersion_penalty = RC2D_TASK_STD_WEIGHT * max(0.0, task_std)
no_repair_soft_penalty = 0.0
if after_task is not None and int(after_task) >=
↳RC2C_NO_REPAIR_VETO_AFTER_TASK and family == "proto_no_repair":
    no_repair_soft_penalty = RC2C_NO_REPAIR_SOFT_PENALTY

guarded_bonus = RC2D_CONTEXT_GUARD_BONUS if
↳rc2c_is_guarded_context_candidate(variant) else 0.0
true_guard_bonus = RC2D_TRUE_GUARD_BONUS if family ==
↳"proto_true_global_guard" else 0.0
complexity_penalty = RC2B_COMPLEXITY_PENALTY *
↳rc2b_policy_complexity_rank(variant)

```

```

class5_bonus = RC2B_VALIDATION_SCORE_CLASS5_WEIGHT * c5 if c5_available
↪else 0.0
hydro_penalty = rc3_hydro_selection_penalty(proxy)

return {
    "proxy_acc_floor_penalty": acc_penalty,
    "class2_floor_penalty": class2_penalty,
    "class5_floor_penalty": class5_penalty,
    "min_class_floor_penalty": min_class_penalty,
    "min_task_floor_penalty": min_task_penalty,
    "old_task_floor_penalty": old_task_penalty,
    "floor_penalty": floor_penalty,
    "task_dispersion_penalty": dispersion_penalty,
    "no_repair_soft_penalty": no_repair_soft_penalty,
    "complexity_penalty": complexity_penalty,
    "class5_bonus": class5_bonus,
    "guarded_context_bonus": guarded_bonus,
    "true_guard_bonus": true_guard_bonus,
    "hydro_selection_penalty": hydro_penalty,
    "total_risk_penalty": floor_penalty + dispersion_penalty +
↪no_repair_soft_penalty + complexity_penalty + hydro_penalty,
    "rc2f_class2_applicable": rc2f_class_applicable(2, after_task),
    "rc2f_class5_applicable": rc2f_class_applicable(5, after_task),
    "rc2f_class2_available_in_proxy": bool(c2_available),
    "rc2f_class5_available_in_proxy": bool(c5_available),
}

def rc2f_nan_safe_validation_base(proxy, baseline_proxy):
    base = validation_score(proxy, baseline_proxy)
    if rc2f_finite(base) and base > -1e8:
        return float(base)
    # Fallback to average accuracy if the composite score is non-finite.
    acc = rc2f_metric(proxy, "proxy_acc", neutral=0.0)
    min_class = rc2f_metric(proxy, "proxy_min_class", neutral=acc)
    return 0.80 * acc + 0.20 * min_class

def rc2d_gate_aligned_score(proxy, baseline_proxy, variant: str,
↪after_task=None):
    """RC2g NaN-safe gate-aligned score."""
    base_score = rc2f_nan_safe_validation_base(proxy, baseline_proxy)
    acc = rc2f_metric(proxy, "proxy_acc", neutral=0.0)
    c2, _ = rc2f_class_metric(proxy, "proxy_class2", 2, after_task=after_task,
↪neutral=0.0)
    c5, _ = rc2f_class_metric(proxy, "proxy_class5", 5, after_task=after_task,
↪neutral=0.0)

```

```

min_class = rc2f_metric(proxy, "proxy_min_class", neutral=acc)
min_task = rc2f_metric(proxy, "proxy_min_task", neutral=min_class)
old_task_min = rc2f_metric(proxy, "proxy_old_task_min", neutral=min_task)
task_std = rc2f_metric(proxy, "proxy_task_acc_std", neutral=0.0)
comp = rc2c_risk_components(proxy, baseline_proxy, variant,
↪after_task=after_task)

applicable_weak_classes = []
if rc2f_class_applicable(2, after_task):
    applicable_weak_classes.append(c2)
if rc2f_class_applicable(5, after_task):
    applicable_weak_classes.append(c5)
weak_class_score = float(np.mean(applicable_weak_classes)) if
↪applicable_weak_classes else 0.0

balance_score = (
    acc
    + RC2D_CLASS_BALANCE_WEIGHT * weak_class_score
    + RC2D_MIN_TASK_WEIGHT * min_task
    + RC2D_OLD_TASK_WEIGHT * old_task_min
    - RC2D_TASK_STD_WEIGHT * max(0.0, task_std)
)
score = float(
    0.50 * base_score
    + 0.50 * balance_score
    + comp["class5_bonus"]
    + comp["guarded_context_bonus"]
    + comp["true_guard_bonus"]
    - comp["total_risk_penalty"]
)
return score if rc2f_finite(score) else -1e9

def rc2b_validation_score(proxy, baseline_proxy, variant: str, after_task=None):
    """Override: RC2g NaN-safe score, with RC2b strict fallback."""
    if not RC2C_RISK_AWARE_SELECTOR_ENABLED or globals().get("POLICY_BRANCH")
↪== "RC2b":
        base_score = rc2f_nan_safe_validation_base(proxy, baseline_proxy)
        c2 = rc2c_proxy_value(proxy, "proxy_class2", np.nan)
        c5 = rc2c_proxy_value(proxy, "proxy_class5", np.nan)
        minc = rc2c_proxy_value(proxy, "proxy_min_class", np.nan)
        penalty = RC2B_COMPLEXITY_PENALTY * rc2b_policy_complexity_rank(variant)
        floor_penalty = 0.0
        if rc2f_class_applicable(2, after_task) and rc2f_finite(c2) and c2 <
↪RC2B_CLASS2_FLOOR:
            floor_penalty += RC2B_VALIDATION_SCORE_FLOOR_PENALTY *
↪(RC2B_CLASS2_FLOOR - c2)

```

```

        if rc2f_class_applicable(5, after_task) and rc2f_finite(c5) and c5 <
↪RC2B_CLASS5_FLOOR:
            floor_penalty += RC2B_VALIDATION_SCORE_FLOOR_PENALTY *
↪(RC2B_CLASS5_FLOOR - c5)
            if rc2f_finite(minc) and minc < RC2B_MIN_CLASS_FLOOR:
                floor_penalty += RC2B_VALIDATION_SCORE_FLOOR_PENALTY *
↪(RC2B_MIN_CLASS_FLOOR - minc)
            class5_bonus = RC2B_VALIDATION_SCORE_CLASS5_WEIGHT * (c5 if
↪rc2f_class_applicable(5, after_task) and rc2f_finite(c5) else 0.0)
            hydro_penalty = rc3_hydro_selection_penalty(proxy)
            score = float(base_score + class5_bonus - penalty - floor_penalty -
↪hydro_penalty)
            return score if rc2f_finite(score) else -1e9
        return rc2d_gate_aligned_score(proxy, baseline_proxy, variant,
↪after_task=after_task)

def rc2f_emergency_candidate_score(candidate, after_task=None):
    """Deterministic memory-only fallback when every candidate score is NaN."""
    p = candidate.get("proxy", {})
    acc = rc2f_metric(p, "proxy_acc", neutral=0.0)
    min_class = rc2f_metric(p, "proxy_min_class", neutral=acc)
    min_task = rc2f_metric(p, "proxy_min_task", neutral=min_class)
    c2, _ = rc2f_class_metric(p, "proxy_class2", 2, after_task=after_task,
↪neutral=0.0)
    c5, _ = rc2f_class_metric(p, "proxy_class5", 5, after_task=after_task,
↪neutral=0.0)
    weak_terms = []
    if rc2f_class_applicable(2, after_task):
        weak_terms.append(c2)
    if rc2f_class_applicable(5, after_task):
        weak_terms.append(c5)
    weak = float(np.mean(weak_terms)) if weak_terms else 0.0
    score = 0.55 * acc + 0.20 * min_task + 0.20 * min_class + 0.05 * weak
    score -= RC2B_COMPLEXITY_PENALTY * rc2b_policy_complexity_rank(candidate.
↪get("variant", ""))
    return float(score) if rc2f_finite(score) else -1e9

def rc2d_candidate_proxy_acc(c):
    return rc2f_metric(c.get("proxy", {}), "proxy_acc", neutral=-1e9)

def rc2d_candidate_class2(c):
    value, _ = rc2f_class_metric(c.get("proxy", {}), "proxy_class2", 2,
↪after_task=c.get("after_task"), neutral=np.nan)

```



```

    return value

def rc2d_candidate_class5(c):
    value, _ = rc2f_class_metric(c.get("proxy", {}), "proxy_class5", 5,
    ↪after_task=c.get("after_task"), neutral=np.nan)
    return value

def rc2c_select_candidate(candidates, after_task):
    """Override: RC2g tri-boundary equilibrium selector."""
    emergency_scoring_used = False
    scored = [c for c in candidates if rc2f_finite(c.get("selection_score",
    ↪-1e9))]
    if not scored and RC2G_EMERGENCY_SCORE_IF_ALL_NAN:
        for c in candidates:
            c["selection_score"] = rc2f_emergency_candidate_score(c,
    ↪after_task=after_task)
            c["rc2f_emergency_score_used"] = True
        scored = [c for c in candidates if rc2f_finite(c.get("selection_score",
    ↪-1e9))]
        emergency_scoring_used = True
    if not scored:
        selected = max(candidates, key=lambda c: (rc2d_candidate_proxy_acc(c),
    ↪-rc2b_policy_complexity_rank(c.get("variant", ""))))
        return selected, {
            "raw_best_variant": selected["variant"],
            "selection_reason": "fallback_no_finite_scores_proxy_acc_tiebreak",
            "no_repair_veto_applied": False,
            "guarded_hysteresis_applied": False,
            "risk_floor_fallback_applied": False,
            "true_guard_rescue_applied": False,
            "class2_rescue_applied": False,
            "class5_rescue_applied": False,
            "rc2d_selector_version": RC2G_SELECTOR_VERSION,
            "rc2f_selector_version": RC2G_SELECTOR_VERSION,
            "rc2f_emergency_scoring_used": True,
            "rc2f_no_finite_scores_after_emergency": True,
        }

    sorted_by_score = sorted(
        scored,
        key=lambda c: (
            float(c.get("selection_score", -1e9)),
            rc2d_candidate_proxy_acc(c),
            -float(c["complexity_rank"]),
        ),

```

```

        reverse=True,
    )
    raw_best = sorted_by_score[0]
    selected = raw_best
    reason = "direct_rc2f_nan_safe_class_balanced_score_win" if not
↪emergency_scoring_used else "rc2g_emergency_score_win"
    no_repair_veto = False
    hysteresis = False
    floor_fallback = False
    true_guard_rescue = False
    class2_rescue = False
    class5_rescue = False

    guarded_candidates = [c for c in scored if
↪rc2c_is_guarded_context_candidate(c["variant"])]
    best_guarded = max(guarded_candidates, key=lambda c: float(c.
↪get("selection_score", -1e9))) if guarded_candidates else None
    true_guard_candidates = [c for c in scored if
↪rc2b_policy_family(c["variant"]) == "proto_true_global_guard"]
    best_true_guard = max(true_guard_candidates, key=lambda c: float(c.
↪get("selection_score", -1e9))) if true_guard_candidates else None

    if (
        best_guarded is not None
        and int(after_task) >= RC2C_NO_REPAIR_VETO_AFTER_TASK
        and rc2b_policy_family(raw_best["variant"]) == "proto_no_repair"
    ):
        margin = float(raw_best.get("selection_score", -1e9)) -
↪float(best_guarded.get("selection_score", -1e9))
        if margin < RC2C_NO_REPAIR_REQUIRED_MARGIN:
            selected = best_guarded
            reason = f"no_repair_veto_guarded_fallback_margin_{margin:+.4f}"
            no_repair_veto = True

        if best_true_guard is not None and rc2d_not_too_far(best_true_guard,
↪selected, RC2D_TRUE_GUARD_RESCUE_SCORE_MARGIN):
            tg_c2 = rc2d_candidate_class2(best_true_guard)
            se_c2 = rc2d_candidate_class2(selected)
            tg_acc = rc2d_candidate_proxy_acc(best_true_guard)
            se_acc = rc2d_candidate_proxy_acc(selected)
            c2_gain = (rc2f_class_applicable(2, after_task) and rc2f_finite(tg_c2)
↪and rc2f_finite(se_c2) and tg_c2 >= se_c2 + RC2D_TRUE_GUARD_CLASS2_MARGIN)
            acc_gain = (rc2f_finite(tg_acc) and rc2f_finite(se_acc) and tg_acc >=
↪se_acc + RC2D_TRUE_GUARD_ACC_MARGIN)
            if (c2_gain or acc_gain) and
↪rc2c_candidate_meets_hard_floors(best_true_guard):

```

```

        selected = best_true_guard
        reason = "true_global_guard_rescue_for_class2_or_accuracy_alignment"
        true_guard_rescue = True

    if rc2f_class_applicable(2, after_task):
        selected_c2 = rc2d_candidate_class2(selected)
        if rc2f_finite(selected_c2) and selected_c2 < RC2C_HARD_CLASS2_FLOOR:
            c2_candidates = [c for c in scored if
↪rc2f_finite(rc2d_candidate_class2(c))]
            if c2_candidates:
                best_c2 = max(c2_candidates, key=lambda c:
↪(rc2d_candidate_class2(c), float(c.get("selection_score", -1e9))))
                if rc2d_candidate_class2(best_c2) >= selected_c2 +
↪RC2D_CLASS2_RESCUE_MARGIN and rc2d_not_too_far(best_c2, selected,
↪RC2D_TRUE_GUARD_RESCUE_SCORE_MARGIN):
                    selected = best_c2
                    reason = "class2_hard_floor_rescue"
                    class2_rescue = True

    if rc2f_class_applicable(5, after_task):
        selected_c5 = rc2d_candidate_class5(selected)
        if rc2f_finite(selected_c5) and selected_c5 < RC2D_HARD_CLASS5_FLOOR:
            c5_candidates = [c for c in scored if
↪rc2f_finite(rc2d_candidate_class5(c))]
            if c5_candidates:
                best_c5 = max(c5_candidates, key=lambda c:
↪(rc2d_candidate_class5(c), float(c.get("selection_score", -1e9))))
                if rc2d_candidate_class5(best_c5) >= selected_c5 +
↪RC2D_CLASS5_RESCUE_MARGIN and rc2d_not_too_far(best_c5, selected,
↪RC2D_TRUE_GUARD_RESCUE_SCORE_MARGIN):
                    selected = best_c5
                    reason = "class5_hard_floor_rescue"
                    class5_rescue = True

    if RC2C_FLOOR_RISK_FALLBACK_ENABLED and not
↪rc2c_candidate_meets_hard_floors(selected):
        floor_safe = [c for c in scored if rc2c_candidate_meets_hard_floors(c)]
        if floor_safe:
            best_floor_safe = max(floor_safe, key=lambda c: float(c.
↪get("selection_score", -1e9)))
            if rc2d_not_too_far(best_floor_safe, selected,
↪RC2D_GUARDED_HYSTERESIS_MARGIN):
                selected = best_floor_safe
                reason = "hard_floor_risk_fallback"
                floor_fallback = True

```

```

    if (
        best_guarded is not None
        and not rc2c_is_guarded_context_candidate(selected["variant"])
        and rc2d_not_too_far(best_guarded, selected,
↪RC2D_GUARDED_HYSTERESIS_MARGIN)
        and rc2c_candidate_meets_hard_floors(best_guarded)
    ):
        bg_c2 = rc2d_candidate_class2(best_guarded)
        se_c2 = rc2d_candidate_class2(selected)
        not_c2_worse = (not rc2f_class_applicable(2, after_task)) or (not
↪rc2f_finite(bg_c2)) or (not rc2f_finite(se_c2)) or bg_c2 >= se_c2 - 0.03
        if not_c2_worse:
            selected = best_guarded
            reason = "reduced_guarded_context_gap_hysteresis"
            hysteresis = True

    return selected, {
        "raw_best_variant": raw_best["variant"],
        "raw_best_policy_family": raw_best["policy_family"],
        "raw_best_proxy_score": raw_best.get("selection_score", np.nan),
        "best_guarded_variant": best_guarded["variant"] if best_guarded is not
↪None else "",
        "best_guarded_proxy_score": best_guarded.get("selection_score", np.nan)
↪if best_guarded is not None else np.nan,
        "best_true_guard_variant": best_true_guard["variant"] if
↪best_true_guard is not None else "",
        "best_true_guard_proxy_score": best_true_guard.get("selection_score",
↪np.nan) if best_true_guard is not None else np.nan,
        "selection_reason": reason,
        "no_repair_veto_applied": no_repair_veto,
        "guarded_hysteresis_applied": hysteresis,
        "risk_floor_fallback_applied": floor_fallback,
        "true_guard_rescue_applied": true_guard_rescue,
        "class2_rescue_applied": class2_rescue,
        "class5_rescue_applied": class5_rescue,
        "rc2d_selector_version": RC2G_SELECTOR_VERSION,
        "rc2f_selector_version": RC2G_SELECTOR_VERSION,
        "rc2f_emergency_scoring_used": emergency_scoring_used,
        "rc2f_no_finite_scores_after_emergency": False,
        "rc2f_class2_applicable": rc2f_class_applicable(2, after_task),
        "rc2f_class5_applicable": rc2f_class_applicable(5, after_task),
    }

def rc2f_memory_class_support(class_memory):
    support = {c: 0 for c in range(N_CLASSES)}

```

```

for cls, data in (class_memory or {}).items():
    try:
        support[int(cls)] += int(len(data["y"]))
    except Exception:
        pass
return support

def rc2f_class_weights_for_memory(class_memory, after_task=None):
    support = rc2f_memory_class_support(class_memory)
    weights = torch.ones(N_CLASSES, device=DEVICE)
    positive_counts = [v for v in support.values() if v > 0]
    max_count = max(positive_counts) if positive_counts else 1
    for cls, count in support.items():
        if count > 0:
            weights[int(cls)] = min(RC2G_MAX_CLASS_WEIGHT, math.sqrt(max_count /
↪ max(count, 1)))
        for weak_cls in [2, 5]:
            if support.get(weak_cls, 0) > 0 and rc2f_class_applicable(weak_cls,
↪ after_task):
                weights[weak_cls] = min(RC2G_MAX_CLASS_WEIGHT,
↪ max(float(weights[weak_cls]), RC2G_WEAK_CLASS_WEIGHT))
    return weights

def rc2f_readout_class_balanced_calibration(readout, model, class_memory,
↪ config, prototypes=None, after_task=None, steps=None, lr=None):
    """Small repair-memory-only calibration pass for guarded readouts.

    This targets the RC2C/RC2D smoke pattern where zf probes were stronger than
    the deployed global head. It does not touch final-test data.
    """
    rows = []
    if not RC2G_CLASS_BALANCED_READOUT_CALIBRATION_ENABLED or not class_memory:
        return rows
    support = rc2f_memory_class_support(class_memory)
    active_weak = [c for c in [2, 5] if support.get(c, 0) > 0 and
↪ rc2f_class_applicable(c, after_task)]
    if not active_weak:
        return rows
    params = [p for p in readout.parameters() if p.requires_grad]
    if not params:
        return rows
    steps = int(RC2G_CLASS_BALANCED_CALIB_STEPS if steps is None else steps)
    lr = float(RC2G_CLASS_BALANCED_CALIB_LR if lr is None else lr)
    weights = rc2f_class_weights_for_memory(class_memory, after_task=after_task)
    model.eval()

```

```

for p in model.parameters():
    p.requires_grad_(False)
readout.train()
opt = torch.optim.Adam(params, lr=lr)
for step in range(steps):
    xb, yb, tb = sample_repair_batch_v10b(class_memory,
↪batch_size=HEAD_REPAIR_BATCH)
    if xb is None:
        continue
    xb, yb, tb = xb.to(DEVICE), yb.to(DEVICE), tb.to(DEVICE)
    zf_list, q_list, y_list = [], [], []
    with torch.no_grad():
        for src_task_id in torch.unique(tb).detach().cpu().tolist():
            src_task_id = int(src_task_id)
            mask = (tb == src_task_id)
            if not mask.any():
                continue
            _, cache = forward_by_method(model, xb[mask], src_task_id,
↪config, prototypes=prototypes, train=False, y=yb[mask], return_cache=True)
            zf_list.append(cache["zf"].detach())
            q_list.append(cache["q_context"].detach())
            y_list.append(yb[mask].detach())
        if not zf_list:
            continue
    zf = torch.cat(zf_list, dim=0)
    q_context = torch.cat(q_list, dim=0)
    y_mix = torch.cat(y_list, dim=0)
    opt.zero_grad(set_to_none=True)
    logits = readout(zf, q_context)
    ce = F.cross_entropy(logits, y_mix, weight=weights)
    margin = head_margin_repair_loss(logits, y_mix,
↪hard_classes=active_weak)
    loss = ce + RC2G_CLASS_BALANCED_MARGIN_WEIGHT * margin
    loss.backward()
    torch.nn.utils.clip_grad_norm_(params, max_norm=5.0)
    opt.step()
    rows.append({
        "repair_epoch": step,
        "repair_type": "rc2f_class_balanced_readout_calibration",
        "rc2f_active_weak_classes": str(active_weak),
        "rc2f_class_balanced_loss": float(loss.detach().cpu()),
        "rc2f_class_balanced_ce": float(ce.detach().cpu()),
        "rc2f_class_balanced_margin": float(margin.detach().cpu()),
        "rc2f_class_weights": safe_jsonable(weights.detach().cpu()),
        "rc2f_selector_version": RC2G_SELECTOR_VERSION,
    })
readout.eval()

```

```

return rows

def build_rc2b_candidate_from_checkpoint(base_run, checkpoint, variant: str,
                                         repair_memory,
context_selection_memory, policy_validation_memory,
                                         seed=0):
    """Override: build one RC2G candidate with NaN-safe proxy and
class-balanced guarded readout calibration."""
    candidate_seed = int(seed)
    cp_seed = int(checkpoint["seed"])
    after_task = int(checkpoint["after_task"])
    method_name = rc2b_candidate_method_name(variant)

    base_cfg = METHOD_CONFIGS["mmals_proto_first_balanced_replay"]
    model = restore_base_model_from_checkpoint(checkpoint, base_cfg)
    context_prototypes = clone_prototypes(checkpoint["context_prototypes"])

    config, config_rows = build_context_variant_config_and_logs(
        model, context_selection_memory, context_prototypes,
        variant=variant, seed=candidate_seed + 313,
    )

    readout = GlobalLinearReadout(model.global_head).to(DEVICE)
    repair_rows = list(config_rows)
    pre_global_head = copy.deepcopy(model.global_head).to(DEVICE)
    pre_global_head.eval()
    for p in pre_global_head.parameters():
        p.requires_grad_(False)

    if variant_uses_guarded_head(variant):
        repair_rows.extend(train_readout_on_memory(
            readout, model, repair_memory, config,
            prototypes=context_prototypes, pre_global_head=pre_global_head,
            kl_weight=0.35, use_kl=True, margin_weight=0.0,
            variant=f"{variant}_rc2f_guarded_policy_candidate",
        ))
        repair_rows.extend(rc2f_readout_class_balanced_calibration(
            readout, model, repair_memory, config,
            prototypes=context_prototypes, after_task=after_task,
        ))

    primary_proxy = proxy_metrics_for_context_config(
        model, policy_validation_memory, config,
        prototypes=context_prototypes, readout=readout,
        max_items_per_class=VALIDATION_MAX_ITEMS_PER_CLASS,
    )

```

```

auxiliary_proxy = proxy_metrics_for_context_config(
    model, context_selection_memory, config,
    prototypes=context_prototypes, readout=readout,
    max_items_per_class=VALIDATION_MAX_ITEMS_PER_CLASS,
) if RC2D_AUX_PROXY_WEIGHT > 0 else None
proxy = rc2d_merge_proxy_metrics(primary_proxy, auxiliary_proxy)

for rr in repair_rows:
    rr.update({
        "method": method_name,
        "seed": cp_seed,
        "after_task": after_task,
        "task_id": after_task,
        "variant": variant,
        "repair_type": str(rr.get("repair_type", "")),
        "rc2b_strict_candidate": True,
        "rc2d_gate_aligned_candidate": True,
        "rc2f_nan_safe_class_balanced_candidate": True,
        "rc2b_no_final_test_used_for_selection": True,
        "policy_family": rc2b_policy_family(variant),
        "uses_guarded_global_head": variant_uses_guarded_head(variant),
        "uses_context_selection": context_variant_selection_mode(variant)
    is not None,
        "uses_oracle_context_reference":
    is_oracle_context_reference(variant),
        "rc2f_class2_applicable": rc2f_class_applicable(2, after_task),
        "rc2f_class5_applicable": rc2f_class_applicable(5, after_task),
    })

return {
    "variant": variant,
    "method": method_name,
    "policy_family": rc2b_policy_family(variant),
    "policy_label": rc2b_policy_label(variant),
    "complexity_rank": rc2b_policy_complexity_rank(variant),
    "model": model,
    "readout": readout,
    "config": config,
    "context_prototypes": context_prototypes,
    "proxy": proxy,
    "primary_proxy": primary_proxy,
    "auxiliary_proxy": auxiliary_proxy or {},
    "repair_rows": repair_rows,
    "after_task": after_task,
    "rc2f_selector_version": RC2G_SELECTOR_VERSION,
}

```



```

print("RC2g tri-boundary equilibrium overrides loaded:", RC2G_SELECTOR_VERSION)
print("RC2G_CLASS_BALANCED_READOUT_CALIBRATION_ENABLED:", ☐)
    ↪ RC2G_CLASS_BALANCED_READOUT_CALIBRATION_ENABLED)
print("RC2G_EMERGENCY_SCORE_IF_ALL_NAN:", RC2G_EMERGENCY_SCORE_IF_ALL_NAN)

```

```

RC2g tri-boundary equilibrium overrides loaded:
RC2g_tri_boundary_class_balanced_v1
RC2G_CLASS_BALANCED_READOUT_CALIBRATION_ENABLED: True
RC2G_EMERGENCY_SCORE_IF_ALL_NAN: True

```

```

[17]: # -----
# v1.1-RC2g tri-boundary equilibrium readout overrides
# -----
# RC2g addresses the RC2e smoke failure mode: class 2 recovered, but class 4
# collapsed and class 5 remained weak. The backbone and no-leakage protocol are
# unchanged. This patch modifies only validation-memory scoring and ☐
    ↪ repair-memory
# readout calibration.
#
# Boundary-safety principle:
# - class 2 must beat class 4 when the true label is 2;
# - class 3 must beat class 4 when the true label is 3;
# - class 4 must beat class 2, class 3 and class 5 when the true label is 4;
# - class 5 must beat class 4 when the true label is 5.
# -----

RC2G_SELECTOR_VERSION = globals().get("RC2G_SELECTOR_VERSION", ☐)
    ↪ "RC2g_tri_boundary_class_balanced_v1")
RC2G_BOUNDARY_CLASSES = globals().get("RC2G_BOUNDARY_CLASSES", [2, 3, 4, 5])
RC2G_PROTECTED_CLASSES = globals().get("RC2G_PROTECTED_CLASSES", [2, 4, 5])
RC2G_CLASS4_FLOOR = globals().get("RC2G_CLASS4_FLOOR", 0.45)
RC2G_HARD_CLASS4_FLOOR = globals().get("RC2G_HARD_CLASS4_FLOOR", 0.05)
RC2G_CLASS4_FLOOR_PENALTY = globals().get("RC2G_CLASS4_FLOOR_PENALTY", 6.00)
RC2G_ZERO_SEEN_CLASS_REJECT_ENABLED = globals().
    ↪ get("RC2G_ZERO_SEEN_CLASS_REJECT_ENABLED", True)
RC2G_ZERO_SEEN_CLASS_PENALTY = globals().get("RC2G_ZERO_SEEN_CLASS_PENALTY", 4.
    ↪ 00)
RC2G_ZERO_CLASS_RESCUE_SCORE_MARGIN = globals().
    ↪ get("RC2G_ZERO_CLASS_RESCUE_SCORE_MARGIN", 0.08)
RC2G_BOUNDARY_MARGIN_TARGET = globals().get("RC2G_BOUNDARY_MARGIN_TARGET", 0.20)
RC2G_BOUNDARY_MARGIN_WEIGHT = globals().get("RC2G_BOUNDARY_MARGIN_WEIGHT", 0.55)
RC2G_CLASS4_WEIGHT = globals().get("RC2G_CLASS4_WEIGHT", 3.50)
RC2G_BOUNDARY_CLASS_WEIGHT = globals().get("RC2G_BOUNDARY_CLASS_WEIGHT", 2.75)
RC2G_BOUNDARY_CALIB_STEPS = globals().get("RC2G_BOUNDARY_CALIB_STEPS", 18 if ☐
    ↪ RUN_MODE == "smoke" else 40)
RC2G_BOUNDARY_CALIB_LR = globals().get("RC2G_BOUNDARY_CALIB_LR", 1.2e-3)

```

```

if globals().get("POLICY_BRANCH", "RC2g") == "RC2g":
    RC2C_METHOD = "paired_rc2g_tri_boundary_equilibrium_policy"
    RC2B_METHOD = RC2C_METHOD
    RC3_METHOD = RC2C_METHOD

def rc2f_class_key(class_id: int) -> str:
    return f"proxy_class{int(class_id)}"

def rc2f_candidate_class(candidate, class_id: int, neutral=np.nan):
    value, _ = rc2f_class_metric(candidate.get("proxy", {}),
    ↪rc2f_class_key(class_id), int(class_id), after_task=candidate.
    ↪get("after_task"), neutral=neutral)
    return value

def rc2f_candidate_zero_seen_class_count(candidate) -> int:
    try:
        return int(candidate.get("proxy", {})).
    ↪get("proxy_zero_seen_class_count", 0) or 0)
    except Exception:
        return 0

def rc2f_seen_pred_count(candidate, class_id: int) -> int:
    try:
        return int(candidate.get("proxy", {})).
    ↪get(f"proxy_pred_count_class{int(class_id)}", 0) or 0)
    except Exception:
        return 0

def rc2f_boundary_margin_loss(logits, y, target_margin=None):
    """Symmetric boundary margin for the 2/3/4/5 overlap edge.

    This replaces the one-way class2-vs-class4 pressure that caused RC2e to
    recover class 2 while sacrificing class 4. It is applied only during the
    repair-memory calibration pass for guarded readouts.
    """

    if logits is None or y is None:
        return torch.zeros((), device=DEVICE)
    target_margin = RC2G_BOUNDARY_MARGIN_TARGET if target_margin is None else
    ↪float(target_margin)
    terms = []
    constraints = [
        (2, 4),          # true 2 should outrank 4

```

```

        (3, 4),          # true 3 should outrank 4
        (4, 2),          # true 4 should outrank 2
        (4, 3),          # true 4 should outrank 3
        (4, 5),          # true 4 should outrank 5
        (5, 4),          # true 5 should outrank 4
    ]
    for true_cls, rival_cls in constraints:
        if true_cls >= logits.shape[1] or rival_cls >= logits.shape[1]:
            continue
        mask = (y == int(true_cls))
        if mask.any():
            margin = logits[mask, true_cls] - logits[mask, rival_cls]
            terms.append(F.relu(target_margin - margin).mean())
    if not terms:
        return torch.zeros((), device=logits.device)
    return torch.stack(terms).mean()

@torch.no_grad()
def proxy_metrics_for_context_config(model, class_memory, config,
    ↪prototypes=None, readout=None,
    ↪
    ↪max_items_per_class=VALIDATION_MAX_ITEMS_PER_CLASS):
    """RC2g proxy metrics with boundary-class and never-predicted audits.

    Still memory-only: this is used for context/policy validation and never
    ↪reads
    final test data before policy selection.
    """
    if not class_memory:
        out = {
            "proxy_acc": np.nan,
            "proxy_context_top1": np.nan,
            "proxy_context_entropy": np.nan,
            "proxy_context_margin": np.nan,
            "proxy_min_class": np.nan,
            "proxy_min_task": np.nan,
            "proxy_old_task_min": np.nan,
            "proxy_task_acc_std": np.nan,
            "proxy_score": -1e9,
            "proxy_zero_seen_class_count": np.nan,
            "proxy_never_predicted_seen_classes": "",
        }
    for c in range(N_CLASSES):
        out[f"proxy_class{c}"] = np.nan
        out[f"proxy_pred_count_class{c}"] = 0
        out[f"proxy_pred_rate_class{c}"] = np.nan

```

```

    return out

if readout is None:
    readout = GlobalLinearReadout(model.global_head).to(DEVICE)
model.eval(); readout.eval()

correct_by_cls = {c: 0 for c in range(N_CLASSES)}
total_by_cls = {c: 0 for c in range(N_CLASSES)}
pred_by_cls = {c: 0 for c in range(N_CLASSES)}
correct_by_task = {t: 0 for t in range(N_TASKS)}
total_by_task = {t: 0 for t in range(N_TASKS)}
correct_total, total = 0, 0
top1_values, ent_values, margin_values = [], [], []

for cls, data in sorted(class_memory.items()):
    X, y, t = data["X"], data["y"], data["task_id"]
    if len(y) == 0:
        continue
    if len(y) > max_items_per_class:
        idx = torch.randperm(len(y))[:max_items_per_class]
        X, y, t = X[idx], y[idx], t[idx]
    loader = make_loader(TensorDataset(X, y, t), batch_size=BATCH_SIZE,
↪shuffle=False)
    for xb, yb, tb in loader:
        xb, yb, tb = xb.to(DEVICE), yb.to(DEVICE), tb.to(DEVICE)
        for src_task_id in torch.unique(tb).detach().cpu().tolist():
            src_task_id = int(src_task_id)
            mask = (tb == src_task_id)
            if not mask.any():
                continue
            _, cache = forward_by_method(
                model, xb[mask], src_task_id, config,
                prototypes=prototypes, train=False, y=yb[mask],
↪return_cache=True
            )
            logits = readout(cache["zf"], cache["q_context"])
            seen = seen_classes_until(src_task_id)
            pred = mask_logits_to_seen_classes(logits, seen).argmax(dim=-1)
            yy = yb[mask]
            correct = (pred == yy)
            correct_total += int(correct.sum().item())
            total += int(yy.numel())
            correct_by_task[src_task_id] += int(correct.sum().item())
            total_by_task[src_task_id] += int(yy.numel())

            q = cache["q_context"]

```

```

        top1_values.append((q.argmax(dim=-1) == src_task_id).float().
↪detach().cpu())
        ent_values.append(entropy_from_probs(q).detach().cpu())
        margin_values.append(margin_from_probs(q).detach().cpu())

        for c in torch.unique(yy).detach().cpu().tolist():
            c = int(c)
            m = (yy == c)
            total_by_cls[c] += int(m.sum().item())
            correct_by_cls[c] += int((pred[m] == yy[m]).sum().item())
        for pcls in torch.unique(pred).detach().cpu().tolist():
            pcls = int(pcls)
            if 0 <= pcls < N_CLASSES:
                pred_by_cls[pcls] += int((pred == pcls).sum().item())

    per_cls = {c: (correct_by_cls[c] / max(total_by_cls[c], 1) if
↪total_by_cls[c] > 0 else np.nan) for c in range(N_CLASSES)}
    valid_accs = [v for v in per_cls.values() if not np.isnan(v)]
    acc = correct_total / max(total, 1)
    ctx_top1 = float(torch.cat(top1_values).mean()) if top1_values else np.nan
    ctx_entropy = float(torch.cat(ent_values).mean()) if ent_values else np.nan
    ctx_margin = float(torch.cat(margin_values).mean()) if margin_values else
↪np.nan
    min_class = float(np.min(valid_accs)) if valid_accs else np.nan
    per_task = {tid: (correct_by_task[tid] / max(total_by_task[tid], 1) if
↪total_by_task[tid] > 0 else np.nan) for tid in range(N_TASKS)}
    valid_task_accs = [v for v in per_task.values() if not np.isnan(v)]
    min_task = float(np.min(valid_task_accs)) if valid_task_accs else np.nan
    active_tasks = [tid for tid, v in per_task.items() if not np.isnan(v)]
    max_active_task = max(active_tasks) if active_tasks else -1
    old_task_accs = [per_task[tid] for tid in active_tasks if tid <
↪max_active_task]
    old_task_min = float(np.min(old_task_accs)) if old_task_accs else min_task
    task_acc_std = float(np.std(valid_task_accs)) if valid_task_accs else np.nan

    true_seen_classes = [c for c in range(N_CLASSES) if total_by_cls[c] > 0]
    never_predicted_seen = [c for c in true_seen_classes if pred_by_cls[c] == 0]
    zero_seen_class_count = len(never_predicted_seen)
    pred_rates = {c: (pred_by_cls[c] / max(total, 1) if total > 0 else np.nan)
↪for c in range(N_CLASSES)}

    # Score remains compact; selection scoring below applies stronger gates.
    c2 = per_cls.get(2, np.nan)
    c4 = per_cls.get(4, np.nan)
    c5 = per_cls.get(5, np.nan)
    protected_terms = [x for x in [c2, c4, c5] if not np.isnan(x)]

```

```

protected_mean = float(np.mean(protected_terms)) if protected_terms else 0.0
score = float(
    acc
    + CONTEXT_SELECTION_SCORE_CONTEXT_WEIGHT * (0.0 if np.isnan(ctx_top1)
↪ else ctx_top1)
    + CONTEXT_SELECTION_SCORE_MINCLASS_WEIGHT * (0.0 if np.isnan(min_class)
↪ else min_class)
    + 0.08 * protected_mean
    + 0.05 * (0.0 if np.isnan(min_task) else min_task)
    - CONTEXT_SELECTION_SCORE_ENTROPY_PENALTY * (0.0 if np.
↪ isnan(ctx_entropy) else ctx_entropy)
    - 0.05 * zero_seen_class_count
)

out = {
    "proxy_acc": acc,
    "proxy_context_top1": ctx_top1,
    "proxy_context_entropy": ctx_entropy,
    "proxy_context_margin": ctx_margin,
    "proxy_min_class": min_class,
    "proxy_min_task": min_task,
    "proxy_old_task_min": old_task_min,
    "proxy_task_acc_std": task_acc_std,
    "proxy_score": score,
    "proxy_zero_seen_class_count": zero_seen_class_count,
    "proxy_never_predicted_seen_classes": str(never_predicted_seen),
}

for c in range(N_CLASSES):
    out[f"proxy_class{c}"] = per_cls.get(c, np.nan)
    out[f"proxy_pred_count_class{c}"] = int(pred_by_cls[c])
    out[f"proxy_pred_rate_class{c}"] = pred_rates[c]
    # Backward-compatible aliases used by older gates.
    out["proxy_class2"] = out.get("proxy_class2", np.nan)
    out["proxy_class4"] = out.get("proxy_class4", np.nan)
    out["proxy_class5"] = out.get("proxy_class5", np.nan)
return out

def rc2f_class_weights_for_memory(class_memory, after_task=None):
    support = rc2f_memory_class_support(class_memory)
    weights = torch.ones(N_CLASSES, device=DEVICE)
    positive_counts = [v for v in support.values() if v > 0]
    max_count = max(positive_counts) if positive_counts else 1
    for cls, count in support.items():
        if count > 0:
            weights[int(cls)] = min(RC2G_MAX_CLASS_WEIGHT, math.sqrt(max_count /
↪ max(count, 1)))

```

```

    for cls in RC2G_PROTECTED_CLASSES:
        if support.get(cls, 0) > 0 and rc2f_class_applicable(cls, after_task):
            target_weight = RC2G_CLASS4_WEIGHT if int(cls) == 4 else
↪RC2G_BOUNDARY_CLASS_WEIGHT
            weights[int(cls)] = min(RC2G_MAX_CLASS_WEIGHT,
↪max(float(weights[int(cls)]), float(target_weight)))
        return weights

# Override the RC2g class weight helper name expected by the previous cell.
rc2f_class_weights_for_memory = rc2f_class_weights_for_memory

def rc2f_readout_class_balanced_calibration(readout, model, class_memory,
↪config, prototypes=None, after_task=None, steps=None, lr=None):
    """Boundary-safe repair-memory-only calibration for guarded readouts."""
    rows = []
    if not RC2G_CLASS_BALANCED_READOUT_CALIBRATION_ENABLED or not class_memory:
        return rows
    support = rc2f_memory_class_support(class_memory)
    active_boundary = [c for c in RC2G_BOUNDARY_CLASSES if support.get(c, 0) >
↪0 and rc2f_class_applicable(c, after_task)]
    if not active_boundary:
        return rows
    params = [p for p in readout.parameters() if p.requires_grad]
    if not params:
        return rows
    steps = int(RC2G_BOUNDARY_CALIB_STEPS if steps is None else steps)
    lr = float(RC2G_BOUNDARY_CALIB_LR if lr is None else lr)
    weights = rc2f_class_weights_for_memory(class_memory, after_task=after_task)
    model.eval()
    for p in model.parameters():
        p.requires_grad_(False)
    readout.train()
    opt = torch.optim.Adam(params, lr=lr)
    for step in range(steps):
        xb, yb, tb = sample_repair_batch_v10b(class_memory,
↪batch_size=HEAD_REPAIR_BATCH)
        if xb is None:
            continue
        xb, yb, tb = xb.to(DEVICE), yb.to(DEVICE), tb.to(DEVICE)
        zf_list, q_list, y_list = [], [], []
        with torch.no_grad():
            for src_task_id in torch.unique(tb).detach().cpu().tolist():
                src_task_id = int(src_task_id)
                mask = (tb == src_task_id)
                if not mask.any():

```

```

        continue
        _, cache = forward_by_method(model, xb[mask], src_task_id,
↪config, prototypes=prototypes, train=False, y=yb[mask], return_cache=True)
        zf_list.append(cache["zf"].detach())
        q_list.append(cache["q_context"].detach())
        y_list.append(yb[mask].detach())
    if not zf_list:
        continue
    zf = torch.cat(zf_list, dim=0)
    q_context = torch.cat(q_list, dim=0)
    y_mix = torch.cat(y_list, dim=0)
    opt.zero_grad(set_to_none=True)
    logits = readout(zf, q_context)
    ce = F.cross_entropy(logits, y_mix, weight=weights)
    boundary = rc2f_boundary_margin_loss(logits, y_mix)
    loss = ce + RC2G_BOUNDARY_MARGIN_WEIGHT * boundary
    loss.backward()
    torch.nn.utils.clip_grad_norm_(params, max_norm=5.0)
    opt.step()
    rows.append({
        "repair_epoch": step,
        "repair_type": "rc2g_tri_boundary_equilibrium_readout_calibration",
        "rc2f_active_boundary_classes": str(active_boundary),
        "rc2g_tri_boundary_loss": float(loss.detach().cpu()),
        "rc2g_tri_boundary_ce": float(ce.detach().cpu()),
        "rc2f_boundary_margin": float(boundary.detach().cpu()),
        "rc2f_class_weights": safe_jsonable(weights.detach().cpu()),
        "rc2f_selector_version": RC2G_SELECTOR_VERSION,
    })
    readout.eval()
    return rows

def rc2c_candidate_meets_hard_floors(candidate) -> bool:
    """RC2g class-aware hard floor check, including class-4 tri-boundaryty."""
    p = candidate.get("proxy", {})
    after_task = candidate.get("after_task", None)
    acc = rc2c_proxy_value(p, "proxy_acc")
    c2 = rc2c_proxy_value(p, "proxy_class2")
    c4 = rc2c_proxy_value(p, "proxy_class4")
    c5 = rc2c_proxy_value(p, "proxy_class5")
    min_task = rc2c_proxy_value(p, "proxy_min_task", rc2c_proxy_value(p,
↪"proxy_min_class"))
    min_class = rc2c_proxy_value(p, "proxy_min_class")
    zero_count = int(p.get("proxy_zero_seen_class_count", 0) or 0)
    ok_c2 = (not rc2f_class_applicable(2, after_task)) or (not rc2f_finite(c2))
↪or c2 >= RC2C_HARD_CLASS2_FLOOR

```



```

    ok_c4 = (not rc2f_class_applicable(4, after_task)) or (not rc2f_finite(c4))
    or c4 >= RC2G_HARD_CLASS4_FLOOR
    ok_c5 = (not rc2f_class_applicable(5, after_task)) or (not rc2f_finite(c5))
    or c5 >= RC2D_HARD_CLASS5_FLOOR
    ok_min_task = (not rc2f_finite(min_task)) or min_task >=
    RC2C_HARD_MIN_TASK_FLOOR
    ok_min_class = (not rc2f_finite(min_class)) or min_class >=
    RC2C_HARD_MIN_CLASS_FLOOR
    ok_acc = (not rc2f_finite(acc)) or acc >= max(0.50, RC2D_PROXY_ACC_FLOOR -
    0.05)
    ok_zero = (not RC2G_ZERO_SEEN_CLASS_REJECT_ENABLED) or zero_count == 0
    return bool(ok_c2 and ok_c4 and ok_c5 and ok_min_task and ok_min_class and
    ok_acc and ok_zero)

def rc2c_risk_components(proxy, baseline_proxy, variant: str, after_task=None):
    """RC2g tri-boundary equilibrium risk components, still
    validation-memory-only."""
    acc = rc2f_metric(proxy, "proxy_acc", neutral=0.0)
    c2, c2_available = rc2f_class_metric(proxy, "proxy_class2", 2,
    after_task=after_task, neutral=0.0)
    c4, c4_available = rc2f_class_metric(proxy, "proxy_class4", 4,
    after_task=after_task, neutral=0.0)
    c5, c5_available = rc2f_class_metric(proxy, "proxy_class5", 5,
    after_task=after_task, neutral=0.0)
    min_class = rc2f_metric(proxy, "proxy_min_class", neutral=acc)
    min_task = rc2f_metric(proxy, "proxy_min_task", neutral=min_class)
    old_task_min = rc2f_metric(proxy, "proxy_old_task_min", neutral=min_task)
    task_std = rc2f_metric(proxy, "proxy_task_acc_std", neutral=0.0)
    family = rc2b_policy_family(variant)
    zero_count = int(proxy.get("proxy_zero_seen_class_count", 0) or 0)

    class2_penalty = RC2C_CLASS2_FLOOR_PENALTY *
    rc2f_floor_deficit_metric(proxy, "proxy_class2", RC2C_CLASS2_FLOOR,
    class_id=2, after_task=after_task)
    class4_penalty = RC2G_CLASS4_FLOOR_PENALTY *
    rc2f_floor_deficit_metric(proxy, "proxy_class4", RC2G_CLASS4_FLOOR,
    class_id=4, after_task=after_task)
    class5_penalty = RC2D_CLASS5_FLOOR_PENALTY *
    rc2f_floor_deficit_metric(proxy, "proxy_class5", RC2D_CLASS5_FLOOR,
    class_id=5, after_task=after_task)
    min_class_penalty = RC2C_MIN_CLASS_FLOOR_PENALTY *
    rc2f_floor_deficit_metric(proxy, "proxy_min_class", RC2C_MIN_CLASS_FLOOR,
    after_task=after_task)

```

```

    min_task_penalty = RC2C_MIN_TASK_FLOOR_PENALTY *␣
↪rc2f_floor_deficit_metric(proxy, "proxy_min_task", RC2C_MIN_TASK_FLOOR,␣
↪after_task=after_task)
    old_task_penalty = RC2C_OLD_TASK_FLOOR_PENALTY *␣
↪rc2f_floor_deficit_metric(proxy, "proxy_old_task_min", RC2C_OLD_TASK_FLOOR,␣
↪after_task=after_task)
    acc_penalty = RC2D_PROXY_ACC_FLOOR_PENALTY *␣
↪rc2f_floor_deficit_metric(proxy, "proxy_acc", RC2D_PROXY_ACC_FLOOR,␣
↪after_task=after_task)
    zero_penalty = RC2G_ZERO_SEEN_CLASS_PENALTY * max(0, zero_count)
    floor_penalty = class2_penalty + class4_penalty + class5_penalty +␣
↪min_class_penalty + min_task_penalty + old_task_penalty + acc_penalty +␣
↪zero_penalty

    dispersion_penalty = RC2D_TASK_STD_WEIGHT * max(0.0, task_std)
    no_repair_soft_penalty = 0.0
    if after_task is not None and int(after_task) >=␣
↪RC2C_NO_REPAIR_VETO_AFTER_TASK and family == "proto_no_repair":
        no_repair_soft_penalty = RC2C_NO_REPAIR_SOFT_PENALTY
    guarded_bonus = RC2D_CONTEXT_GUARD_BONUS if␣
↪rc2c_is_guarded_context_candidate(variant) else 0.0
    true_guard_bonus = RC2D_TRUE_GUARD_BONUS if family ==␣
↪"proto_true_global_guard" else 0.0
    complexity_penalty = RC2B_COMPLEXITY_PENALTY *␣
↪rc2b_policy_complexity_rank(variant)
    class5_bonus = RC2B_VALIDATION_SCORE_CLASS5_WEIGHT * c5 if c5_available␣
↪else 0.0
    hydro_penalty = rc3_hydro_selection_penalty(proxy)

    return {
        "proxy_acc_floor_penalty": acc_penalty,
        "class2_floor_penalty": class2_penalty,
        "class4_floor_penalty": class4_penalty,
        "class5_floor_penalty": class5_penalty,
        "min_class_floor_penalty": min_class_penalty,
        "min_task_floor_penalty": min_task_penalty,
        "old_task_floor_penalty": old_task_penalty,
        "zero_seen_class_penalty": zero_penalty,
        "floor_penalty": floor_penalty,
        "task_dispersion_penalty": dispersion_penalty,
        "no_repair_soft_penalty": no_repair_soft_penalty,
        "complexity_penalty": complexity_penalty,
        "class5_bonus": class5_bonus,
        "guarded_context_bonus": guarded_bonus,
        "true_guard_bonus": true_guard_bonus,
        "hydro_selection_penalty": hydro_penalty,
    }

```

```

        "total_risk_penalty": floor_penalty + dispersion_penalty +
↪no_repair_soft_penalty + complexity_penalty + hydro_penalty,
        "rc2f_class2_applicable": rc2f_class_applicable(2, after_task),
        "rc2f_class4_applicable": rc2f_class_applicable(4, after_task),
        "rc2f_class5_applicable": rc2f_class_applicable(5, after_task),
        "rc2f_class2_available_in_proxy": bool(c2_available),
        "rc2f_class4_available_in_proxy": bool(c4_available),
        "rc2f_class5_available_in_proxy": bool(c5_available),
        "rc2f_zero_seen_class_count": zero_count,
    }

def rc2d_gate_aligned_score(proxy, baseline_proxy, variant: str,
↪after_task=None):
    """RC2g tri-boundary equilibrium gate-aligned score."""
    base_score = rc2f_nan_safe_validation_base(proxy, baseline_proxy)
    acc = rc2f_metric(proxy, "proxy_acc", neutral=0.0)
    min_class = rc2f_metric(proxy, "proxy_min_class", neutral=acc)
    min_task = rc2f_metric(proxy, "proxy_min_task", neutral=min_class)
    old_task_min = rc2f_metric(proxy, "proxy_old_task_min", neutral=min_task)
    task_std = rc2f_metric(proxy, "proxy_task_acc_std", neutral=0.0)
    comp = rc2c_risk_components(proxy, baseline_proxy, variant,
↪after_task=after_task)

    protected = []
    for cls in RC2G_PROTECTED_CLASSES:
        if rc2f_class_applicable(cls, after_task):
            v, _ = rc2f_class_metric(proxy, rc2f_class_key(cls), cls,
↪after_task=after_task, neutral=0.0)
            protected.append(v)
    protected_score = float(np.mean(protected)) if protected else 0.0
    c4, _ = rc2f_class_metric(proxy, "proxy_class4", 4, after_task=after_task,
↪neutral=0.0)
    zero_count = int(proxy.get("proxy_zero_seen_class_count", 0) or 0)

    balance_score = (
        acc
        + RC2D_CLASS_BALANCE_WEIGHT * protected_score
        + 0.15 * (c4 if rc2f_class_applicable(4, after_task) else 0.0)
        + RC2D_MIN_TASK_WEIGHT * min_task
        + RC2D_OLD_TASK_WEIGHT * old_task_min
        - RC2D_TASK_STD_WEIGHT * max(0.0, task_std)
        - 0.15 * zero_count
    )
    score = float(
        0.45 * base_score
        + 0.55 * balance_score

```

```

        + comp["class5_bonus"]
        + comp["guarded_context_bonus"]
        + comp["true_guard_bonus"]
        - comp["total_risk_penalty"]
    )
    return score if rc2f_finite(score) else -1e9

def rc2b_validation_score(proxy, baseline_proxy, variant: str, after_task=None):
    """Override: RC2g tri-boundary equilibrium score, with RC2b strict fallback.
    ↪ """
    if not RC2C_RISK_AWARE_SELECTOR_ENABLED or globals().get("POLICY_BRANCH")_
    ↪ == "RC2b":
        base_score = rc2f_nan_safe_validation_base(proxy, baseline_proxy)
        penalty = RC2B_COMPLEXITY_PENALTY * rc2b_policy_complexity_rank(variant)
        floor_penalty = 0.0
        for cls, floor in [(2, RC2B_CLASS2_FLOOR), (4, RC2G_CLASS4_FLOOR), (5,
    ↪ RC2B_CLASS5_FLOOR)]:
            val = rc2c_proxy_value(proxy, rc2f_class_key(cls), np.nan)
            if rc2f_class_applicable(cls, after_task) and rc2f_finite(val) and_
    ↪ val < floor:
                floor_penalty += RC2B_VALIDATION_SCORE_FLOOR_PENALTY * (floor -
    ↪ val)
            minc = rc2c_proxy_value(proxy, "proxy_min_class", np.nan)
            if rc2f_finite(minc) and minc < RC2B_MIN_CLASS_FLOOR:
                floor_penalty += RC2B_VALIDATION_SCORE_FLOOR_PENALTY *_
    ↪ (RC2B_MIN_CLASS_FLOOR - minc)
            hydro_penalty = rc3_hydro_selection_penalty(proxy)
            score = float(base_score - penalty - floor_penalty - hydro_penalty)
            return score if rc2f_finite(score) else -1e9
        return rc2d_gate_aligned_score(proxy, baseline_proxy, variant,
    ↪ after_task=after_task)

def rc2f_emergency_candidate_score(candidate, after_task=None):
    p = candidate.get("proxy", {})
    acc = rc2f_metric(p, "proxy_acc", neutral=0.0)
    min_class = rc2f_metric(p, "proxy_min_class", neutral=acc)
    min_task = rc2f_metric(p, "proxy_min_task", neutral=min_class)
    protected = []
    for cls in RC2G_PROTECTED_CLASSES:
        if rc2f_class_applicable(cls, after_task):
            v, _ = rc2f_class_metric(p, rc2f_class_key(cls), cls,
    ↪ after_task=after_task, neutral=0.0)
            protected.append(v)
    protected_mean = float(np.mean(protected)) if protected else 0.0

```

```

    zero_count = int(p.get("proxy_zero_seen_class_count", 0) or 0)
    score = 0.50 * acc + 0.20 * min_task + 0.15 * min_class + 0.15 * _
    ↪protected_mean - 0.20 * zero_count
    score -= RC2B_COMPLEXITY_PENALTY * rc2b_policy_complexity_rank(candidate.
    ↪get("variant", ""))
    return float(score) if rc2f_finite(score) else -1e9

def rc2c_select_candidate(candidates, after_task):
    """Override: RC2g tri-boundary equilibrium selector with zero-class_
    ↪rejection."""
    emergency_scoring_used = False
    scored = [c for c in candidates if rc2f_finite(c.get("selection_score", _
    ↪-1e9))]
    if not scored and RC2G_EMERGENCY_SCORE_IF_ALL_NAN:
        for c in candidates:
            c["selection_score"] = rc2f_emergency_candidate_score(c, _
    ↪after_task=after_task)
            c["rc2f_emergency_score_used"] = True
        scored = [c for c in candidates if rc2f_finite(c.get("selection_score", _
    ↪-1e9))]
        emergency_scoring_used = True
    if not scored:
        selected = max(candidates, key=lambda c: (rc2d_candidate_proxy_acc(c), _
    ↪-rc2b_policy_complexity_rank(c.get("variant", ""))))
        return selected, {
            "raw_best_variant": selected["variant"],
            "selection_reason": "fallback_no_finite_scores_proxy_acc_tiebreak",
            "no_repair_veto_applied": False,
            "guarded_hysteresis_applied": False,
            "risk_floor_fallback_applied": False,
            "true_guard_rescue_applied": False,
            "class2_rescue_applied": False,
            "class4_rescue_applied": False,
            "class5_rescue_applied": False,
            "zero_seen_class_reject_applied": False,
            "rc2f_selector_version": RC2G_SELECTOR_VERSION,
            "rc2f_emergency_scoring_used": True,
        }

    sorted_by_score = sorted(
        scored,
        key=lambda c: (
            float(c.get("selection_score", -1e9)),
            -rc2f_candidate_zero_seen_class_count(c),
            rc2d_candidate_proxy_acc(c),

```

```

        -float(c["complexity_rank"]),
    ),
    reverse=True,
)
raw_best = sorted_by_score[0]
selected = raw_best
reason = "direct_rc2g_tri_boundary_score_win" if not emergency_scoring_used_
↪else "rc2g_emergency_score_win"
    no_repair_veto = False
    hysteresis = False
    floor_fallback = False
    true_guard_rescue = False
    class2_rescue = False
    class4_rescue = False
    class5_rescue = False
    zero_reject = False

    guarded_candidates = [c for c in scored if_
↪rc2c_is_guarded_context_candidate(c["variant"])]
    best_guarded = max(guarded_candidates, key=lambda c: float(c.
↪get("selection_score", -1e9))) if guarded_candidates else None
    true_guard_candidates = [c for c in scored if_
↪rc2b_policy_family(c["variant"]) == "proto_true_global_guard"]
    best_true_guard = max(true_guard_candidates, key=lambda c: float(c.
↪get("selection_score", -1e9))) if true_guard_candidates else None

    if (
        best_guarded is not None
        and int(after_task) >= RC2C_NO_REPAIR_VETO_AFTER_TASK
        and rc2b_policy_family(raw_best["variant"]) == "proto_no_repair"
    ):
        margin = float(raw_best.get("selection_score", -1e9)) -_
↪float(best_guarded.get("selection_score", -1e9))
        if margin < RC2C_NO_REPAIR_REQUIRED_MARGIN:
            selected = best_guarded
            reason = f"no_repair_veto_guarded_fallback_margin_{margin:+.4f}"
            no_repair_veto = True

    # Hard zero-class rejection: if a candidate never predicts a seen class,
    # use the best non-zero alternative unless no such alternative exists.
    if RC2G_ZERO_SEEN_CLASS_REJECT_ENABLED and_
↪rc2f_candidate_zero_seen_class_count(selected) > 0:
        safe_candidates = [c for c in scored if_
↪rc2f_candidate_zero_seen_class_count(c) == 0]
        if safe_candidates:

```

```

        best_safe = max(safe_candidates, key=lambda c: float(c.
↪get("selection_score", -1e9)))
        selected = best_safe
        reason = "zero_seen_class_reject_rescue"
        zero_reject = True

    if best_true_guard is not None and rc2d_not_too_far(best_true_guard,
↪selected, RC2D_TRUE_GUARD_RESCUE_SCORE_MARGIN):
        tg_c2 = rc2f_candidate_class(best_true_guard, 2)
        se_c2 = rc2f_candidate_class(selected, 2)
        tg_c4 = rc2f_candidate_class(best_true_guard, 4)
        se_c4 = rc2f_candidate_class(selected, 4)
        tg_acc = rc2d_candidate_proxy_acc(best_true_guard)
        se_acc = rc2d_candidate_proxy_acc(selected)
        c2_gain = (rc2f_class_applicable(2, after_task) and rc2f_finite(tg_c2)
↪and rc2f_finite(se_c2) and tg_c2 >= se_c2 + RC2D_TRUE_GUARD_CLASS2_MARGIN)
        c4_gain = (rc2f_class_applicable(4, after_task) and rc2f_finite(tg_c4)
↪and rc2f_finite(se_c4) and tg_c4 >= se_c4 + 0.05)
        acc_gain = (rc2f_finite(tg_acc) and rc2f_finite(se_acc) and tg_acc >=
↪se_acc + RC2D_TRUE_GUARD_ACC_MARGIN)
        if (c2_gain or c4_gain or acc_gain) and
↪rc2c_candidate_meets_hard_floors(best_true_guard):
            selected = best_true_guard
            reason =
↪"true_global_guard_rescue_for_boundary_or_accuracy_alignment"
            true_guard_rescue = True

    for cls, hard_floor, rescue_margin, flag_name in [
        (2, RC2C_HARD_CLASS2_FLOOR, RC2D_CLASS2_RESCUE_MARGIN, "class2"),
        (4, RC2G_HARD_CLASS4_FLOOR, 0.05, "class4"),
        (5, RC2D_HARD_CLASS5_FLOOR, RC2D_CLASS5_RESCUE_MARGIN, "class5"),
    ]:
        if not rc2f_class_applicable(cls, after_task):
            continue
        selected_val = rc2f_candidate_class(selected, cls)
        if rc2f_finite(selected_val) and selected_val < hard_floor:
            cls_candidates = [c for c in scored if
↪rc2f_finite(rc2f_candidate_class(c, cls))]
            if cls_candidates:
                best_cls = max(cls_candidates, key=lambda c:
↪(rc2f_candidate_class(c, cls), float(c.get("selection_score", -1e9))))
                if rc2f_candidate_class(best_cls, cls) >= selected_val +
↪rescue_margin and rc2d_not_too_far(best_cls, selected,
↪RC2D_TRUE_GUARD_RESCUE_SCORE_MARGIN + 0.03):
                    selected = best_cls
                    reason = f"{flag_name}_hard_floor_rescue"

```

```

        if cls == 2:
            class2_rescue = True
        elif cls == 4:
            class4_rescue = True
        elif cls == 5:
            class5_rescue = True

    if RC2C_FLOOR_RISK_FALLBACK_ENABLED and not ␣
↪rc2c_candidate_meets_hard_floors(selected):
        floor_safe = [c for c in scored if rc2c_candidate_meets_hard_floors(c)]
        if floor_safe:
            best_floor_safe = max(floor_safe, key=lambda c: float(c.
↪get("selection_score", -1e9)))
            selected = best_floor_safe
            reason = "hard_floor_risk_fallback"
            floor_fallback = True

    if (
        best_guarded is not None
        and not rc2c_is_guarded_context_candidate(selected["variant"])
        and rc2d_not_too_far(best_guarded, selected, ␣
↪RC2D_GUARDED_HYSTERESIS_MARGIN)
        and rc2c_candidate_meets_hard_floors(best_guarded)
    ):
        bg_c2 = rc2f_candidate_class(best_guarded, 2)
        se_c2 = rc2f_candidate_class(selected, 2)
        bg_c4 = rc2f_candidate_class(best_guarded, 4)
        se_c4 = rc2f_candidate_class(selected, 4)
        not_boundary_worse = True
        if rc2f_class_applicable(2, after_task) and rc2f_finite(bg_c2) and ␣
↪rc2f_finite(se_c2):
            not_boundary_worse = not_boundary_worse and bg_c2 >= se_c2 - 0.03
        if rc2f_class_applicable(4, after_task) and rc2f_finite(bg_c4) and ␣
↪rc2f_finite(se_c4):
            not_boundary_worse = not_boundary_worse and bg_c4 >= se_c4 - 0.03
        if not_boundary_worse:
            selected = best_guarded
            reason = "tri_boundary_guarded_context_hysteresis"
            hysteresis = True

    return selected, {
        "raw_best_variant": raw_best["variant"],
        "raw_best_policy_family": raw_best["policy_family"],
        "raw_best_proxy_score": raw_best.get("selection_score", np.nan),
        "best_guarded_variant": best_guarded["variant"] if best_guarded is not ␣
↪None else "",

```



```

        "best_guarded_proxy_score": best_guarded.get("selection_score", np.nan)
    if best_guarded is not None else np.nan,
        "best_true_guard_variant": best_true_guard["variant"] if
    best_true_guard is not None else "",
        "best_true_guard_proxy_score": best_true_guard.get("selection_score",
    np.nan) if best_true_guard is not None else np.nan,
        "selection_reason": reason,
        "no_repair_veto_applied": no_repair_veto,
        "guarded_hysteresis_applied": hysteresis,
        "risk_floor_fallback_applied": floor_fallback,
        "true_guard_rescue_applied": true_guard_rescue,
        "class2_rescue_applied": class2_rescue,
        "class4_rescue_applied": class4_rescue,
        "class5_rescue_applied": class5_rescue,
        "zero_seen_class_reject_applied": zero_reject,
        "rc2f_selector_version": RC2G_SELECTOR_VERSION,
        "rc2f_emergency_scoring_used": emergency_scoring_used,
        "rc2f_class2_applicable": rc2f_class_applicable(2, after_task),
        "rc2f_class4_applicable": rc2f_class_applicable(4, after_task),
        "rc2f_class5_applicable": rc2f_class_applicable(5, after_task),
    }

print("RC2g tri-boundary equilibrium readout overrides loaded:",
    RC2G_SELECTOR_VERSION)
print("RC2G_ZERO_SEEN_CLASS_REJECT_ENABLED:",
    RC2G_ZERO_SEEN_CLASS_REJECT_ENABLED)
print("RC2G_PROTECTED_CLASSES:", RC2G_PROTECTED_CLASSES)

```

RC2g tri-boundary equilibrium readout overrides loaded:
RC2g_tri_boundary_class_balanced_v1
RC2G_ZERO_SEEN_CLASS_REJECT_ENABLED: True
RC2G_PROTECTED_CLASSES: [2, 4, 5]

```

[18]: # -----
# v1.1-RC2g tri-boundary equilibrium overrides
# -----
# RC2G is a small patch over RC2F. It does not change the backbone, dataset,
# memory split, Hydro mode, or final-test no-leakage discipline.
#
# RC2F fixed class 4 but weakened class 2 again. RC2G therefore optimizes
# the equilibrium among classes 2, 4 and 5 rather than protecting one boundary
# class in isolation.
# -----

RC2G_SELECTOR_VERSION = "RC2g_tri_boundary_equilibrium_v1"
RC2G_TRI_CLASSES = [2, 4, 5]
RC2G_BRIDGE_CLASSES = [3]

```

```

RC2G_PROTECTED_CLASSES = [2, 4, 5]

# Smoke-friendly soft floors; robust/paper modes keep stricter pressure.
RC2G_CLASS2_FLOOR = 0.55 if RUN_MODE == "smoke" else 0.80
RC2G_CLASS4_FLOOR = 0.45 if RUN_MODE == "smoke" else 0.75
RC2G_CLASS5_FLOOR = 0.60 if RUN_MODE == "smoke" else 0.80
RC2G_TRI_MIN_FLOOR = 0.45 if RUN_MODE == "smoke" else 0.75

RC2G_CLASS2_FLOOR_PENALTY = 7.00
RC2G_CLASS4_FLOOR_PENALTY = 4.00
RC2G_CLASS5_FLOOR_PENALTY = 6.00
RC2G_TRI_MIN_FLOOR_PENALTY = 7.00
RC2G_TRI_BALANCE_PENALTY = 1.50
RC2G_TRI_MIN_SCORE_WEIGHT = 0.55
RC2G_TRI_MEAN_SCORE_WEIGHT = 0.25
RC2G_TRI_BALANCE_SCORE_WEIGHT = 0.20

# Class-4 is protected, but RC2G also prevents the selector/readout from solving
# class 4 by overpredicting it. This is validation-memory only.
RC2G_PRED4_RATE_TARGET = 0.32 if RUN_MODE == "smoke" else 0.24
RC2G_PRED4_BIAS_PENALTY = 1.25

# Calibration: class 2 and class 5 receive slightly more pressure than class 4,
# because RC2F already showed class 4 can be recovered strongly.
RC2G_CLASS2_WEIGHT = 3.40
RC2G_CLASS4_WEIGHT = 2.40
RC2G_CLASS5_WEIGHT = 3.20
RC2G_BRIDGE_CLASS3_WEIGHT = 1.70
RC2G_EQUILIBRIUM_MARGIN_TARGET = 0.18
RC2G_EQUILIBRIUM_MARGIN_WEIGHT = 0.45
RC2G_PRED4_CALIB_PENALTY = 0.25
RC2G_EQUILIBRIUM_CALIB_STEPS = 28 if RUN_MODE == "smoke" else 55
RC2G_EQUILIBRIUM_CALIB_LR = 1.1e-3

if globals().get("POLICY_BRANCH", "RC2g") == "RC2g":
    RC2C_METHOD = "paired_rc2g_tri_boundary_equilibrium_policy"
    RC2B_METHOD = RC2C_METHOD
    RC3_METHOD = RC2C_METHOD

def rc2g_class_value(proxy, cls: int, after_task=None, neutral=0.0):
    v, available = rc2f_class_metric(proxy, rc2f_class_key(cls), cls,
    ↪after_task=after_task, neutral=neutral)
    return float(v), bool(available)

def rc2g_tri_values(proxy, after_task=None, neutral=0.0):

```

```

vals, available = [], []
for cls in RC2G_TRI_CLASSES:
    if rc2f_class_applicable(cls, after_task):
        v, ok = rc2g_class_value(proxy, cls, after_task=after_task,
↪neutral=neutral)
        vals.append(float(v))
        available.append(bool(ok))
return vals, available

def rc2g_tri_stats(proxy, after_task=None):
    vals, avail = rc2g_tri_values(proxy, after_task=after_task, neutral=0.0)
    if not vals:
        return {
            "tri_min": 0.0,
            "tri_mean": 0.0,
            "tri_gap": 0.0,
            "tri_available_count": 0,
            "tri_values": [],
        }
    arr = np.asarray(vals, dtype=float)
    return {
        "tri_min": float(np.min(arr)),
        "tri_mean": float(np.mean(arr)),
        "tri_gap": float(np.max(arr) - np.min(arr)),
        "tri_available_count": int(sum(avail)),
        "tri_values": vals,
    }

def rc2g_pred4_rate(proxy):
    return rc2f_metric(proxy, "proxy_pred_rate_class4", neutral=0.0)

def rc2g_pred4_bias_penalty(proxy, after_task=None):
    if not rc2f_class_applicable(4, after_task):
        return 0.0
    pred4 = rc2g_pred4_rate(proxy)
    return RC2G_PRED4_BIAS_PENALTY * max(0.0, pred4 - RC2G_PRED4_RATE_TARGET)

def rc2f_boundary_margin_loss(logits, y, target_margin=None):
    """RC2G equilibrium margin for the 2/3/4/5 boundary.

    This supersedes the RC2F boundary loss. It still protects class 4, but also
    explicitly protects class 2 against classes 1/3/4 and class 5 against class_
↪4.

```

```

"""
if logits is None or y is None:
    return torch.zeros(), device=DEVICE)
target_margin = RC2G_EQUILIBRIUM_MARGIN_TARGET if target_margin is None
else float(target_margin)
constraints = [
    (2, 1), (2, 3), (2, 4),
    (3, 2), (3, 4),
    (4, 2), (4, 3), (4, 5),
    (5, 4),
]
terms = []
for true_cls, rival_cls in constraints:
    if true_cls >= logits.shape[1] or rival_cls >= logits.shape[1]:
        continue
    mask = (y == int(true_cls))
    if mask.any():
        margin = logits[mask, true_cls] - logits[mask, rival_cls]
        terms.append(F.relu(target_margin - margin).mean())
if not terms:
    return torch.zeros(), device=logits.device)
return torch.stack(terms).mean()

def rc2f_class_weights_for_memory(class_memory, after_task=None):
    """RC2G class weights: tri-boundary equilibrium, not class-4 dominance."""
    support = rc2f_memory_class_support(class_memory)
    weights = torch.ones(N_CLASSES, device=DEVICE)
    positive_counts = [v for v in support.values() if v > 0]
    max_count = max(positive_counts) if positive_counts else 1
    for cls, count in support.items():
        if count > 0:
            weights[int(cls)] = min(RC2G_MAX_CLASS_WEIGHT if
"RC2G_MAX_CLASS_WEIGHT" in globals() else RC2G_CLASS2_WEIGHT,
                                math.sqrt(max_count / max(count, 1)))
    if support.get(2, 0) > 0 and rc2f_class_applicable(2, after_task):
        weights[2] = max(float(weights[2]), RC2G_CLASS2_WEIGHT)
    if support.get(3, 0) > 0 and rc2f_class_applicable(3, after_task):
        weights[3] = max(float(weights[3]), RC2G_BRIDGE_CLASS3_WEIGHT)
    if support.get(4, 0) > 0 and rc2f_class_applicable(4, after_task):
        weights[4] = max(float(weights[4]), RC2G_CLASS4_WEIGHT)
    if support.get(5, 0) > 0 and rc2f_class_applicable(5, after_task):
        weights[5] = max(float(weights[5]), RC2G_CLASS5_WEIGHT)
    max_w = max(RC2G_CLASS2_WEIGHT, RC2G_CLASS4_WEIGHT, RC2G_CLASS5_WEIGHT, 4.0)
    return weights.clamp(1.0, max_w)

```

```

def rc2f_readout_class_balanced_calibration(readout, model, class_memory,
↪config, prototypes=None, after_task=None, steps=None, lr=None):
    """RC2G repair-memory-only tri-boundary equilibrium calibration."""
    rows = []
    if not RC2G_CLASS_BALANCED_READOUT_CALIBRATION_ENABLED or not class_memory:
        return rows
    support = rc2f_memory_class_support(class_memory)
    active_tri = [c for c in RC2G_TRI_CLASSES if support.get(c, 0) > 0 and
↪rc2f_class_applicable(c, after_task)]
    if not active_tri:
        return rows
    params = [p for p in readout.parameters() if p.requires_grad]
    if not params:
        return rows
    steps = int(RC2G_EQUILIBRIUM_CALIB_STEPS if steps is None else steps)
    lr = float(RC2G_EQUILIBRIUM_CALIB_LR if lr is None else lr)
    weights = rc2f_class_weights_for_memory(class_memory, after_task=after_task)
    model.eval()
    for p in model.parameters():
        p.requires_grad_(False)
    readout.train()
    opt = torch.optim.Adam(params, lr=lr)
    for step in range(steps):
        xb, yb, tb = sample_repair_batch_v10b(class_memory,
↪batch_size=HEAD_REPAIR_BATCH)
        if xb is None:
            continue
        xb, yb, tb = xb.to(DEVICE), yb.to(DEVICE), tb.to(DEVICE)
        zf_list, q_list, y_list = [], [], []
        with torch.no_grad():
            for src_task_id in torch.unique(tb).detach().cpu().tolist():
                src_task_id = int(src_task_id)
                mask = (tb == src_task_id)
                if not mask.any():
                    continue
                _, cache = forward_by_method(model, xb[mask], src_task_id,
↪config, prototypes=prototypes, train=False, y=yb[mask], return_cache=True)
                zf_list.append(cache["zf"].detach())
                q_list.append(cache["q_context"].detach())
                y_list.append(yb[mask].detach())
        if not zf_list:
            continue
        zf = torch.cat(zf_list, dim=0)
        q_context = torch.cat(q_list, dim=0)
        y_mix = torch.cat(y_list, dim=0)
        opt.zero_grad(set_to_none=True)
        logits = readout(zf, q_context)

```

```

        ce = F.cross_entropy(logits, y_mix, weight=weights)
        margin = rc2f_boundary_margin_loss(logits, y_mix)
        probs = F.softmax(logits, dim=-1)
        true4_rate = (y_mix == 4).float().mean() if y_mix.numel() else torch.
↳tensor(0.0, device=logits.device)
        pred4_prob = probs[:, 4].mean() if logits.shape[1] > 4 else torch.
↳tensor(0.0, device=logits.device)
        pred4_bias = F.relu(pred4_prob - (true4_rate + 0.10)).pow(2)
        loss = ce + RC2G_EQUILIBRIUM_MARGIN_WEIGHT * margin +
↳RC2G_PRED4_CALIB_PENALTY * pred4_bias
        loss.backward()
        torch.nn.utils.clip_grad_norm_(params, max_norm=5.0)
        opt.step()
        rows.append({
            "repair_epoch": step,
            "repair_type": "rc2g_tri_boundary_equilibrium_readout_calibration",
            "rc2g_active_tri_classes": str(active_tri),
            "rc2g_equilibrium_loss": float(loss.detach().cpu()),
            "rc2g_equilibrium_ce": float(ce.detach().cpu()),
            "rc2g_equilibrium_margin": float(margin.detach().cpu()),
            "rc2g_pred4_bias_loss": float(pred4_bias.detach().cpu()),
            "rc2g_class_weights": safe_jsonable(weights.detach().cpu()),
            "rc2g_selector_version": RC2G_SELECTOR_VERSION,
        })
    readout.eval()
    return rows

def rc2c_candidate_meets_hard_floors(candidate) -> bool:
    """RC2G hard floors: allow smoke diagnostics but reject zero-class/
↳pathological policies."""
    p = candidate.get("proxy", {})
    after_task = candidate.get("after_task", None)
    acc = rc2c_proxy_value(p, "proxy_acc")
    c2 = rc2c_proxy_value(p, "proxy_class2")
    c4 = rc2c_proxy_value(p, "proxy_class4")
    c5 = rc2c_proxy_value(p, "proxy_class5")
    min_task = rc2c_proxy_value(p, "proxy_min_task", rc2c_proxy_value(p,
↳"proxy_min_class"))
    zero_count = int(p.get("proxy_zero_seen_class_count", 0) or 0)
    tri = rc2g_tri_stats(p, after_task=after_task)
    ok_acc = (not rc2f_finite(acc)) or acc >= max(0.50, RC2D_PROXY_ACC_FLOOR -
↳0.08)
    ok_c2 = (not rc2f_class_applicable(2, after_task)) or (not rc2f_finite(c2))
↳or c2 >= max(0.20, RC2G_CLASS2_FLOOR - 0.30)

```

```

    ok_c4 = (not rc2f_class_applicable(4, after_task)) or (not rc2f_finite(c4))
    or c4 >= max(0.10, RC2G_CLASS4_FLOOR - 0.30)
    ok_c5 = (not rc2f_class_applicable(5, after_task)) or (not rc2f_finite(c5))
    or c5 >= max(0.20, RC2G_CLASS5_FLOOR - 0.35)
    ok_min_task = (not rc2f_finite(min_task)) or min_task >= 0.25
    ok_tri = tri["tri_min"] >= 0.20 if tri["tri_values"] else True
    ok_zero = (not RC2G_ZERO_SEEN_CLASS_REJECT_ENABLED) or zero_count == 0
    return bool(ok_acc and ok_c2 and ok_c4 and ok_c5 and ok_min_task and ok_tri
    and ok_zero)

def rc2c_risk_components(proxy, baseline_proxy, variant: str, after_task=None):
    """RC2G tri-boundary risk components, validation-memory-only."""
    acc = rc2f_metric(proxy, "proxy_acc", neutral=0.0)
    min_class = rc2f_metric(proxy, "proxy_min_class", neutral=acc)
    min_task = rc2f_metric(proxy, "proxy_min_task", neutral=min_class)
    old_task_min = rc2f_metric(proxy, "proxy_old_task_min", neutral=min_task)
    task_std = rc2f_metric(proxy, "proxy_task_acc_std", neutral=0.0)
    family = rc2b_policy_family(variant)
    zero_count = int(proxy.get("proxy_zero_seen_class_count", 0) or 0)
    tri = rc2g_tri_stats(proxy, after_task=after_task)

    c2_pen = RC2G_CLASS2_FLOOR_PENALTY * rc2f_floor_deficit_metric(proxy,
    "proxy_class2", RC2G_CLASS2_FLOOR, class_id=2, after_task=after_task)
    c4_pen = RC2G_CLASS4_FLOOR_PENALTY * rc2f_floor_deficit_metric(proxy,
    "proxy_class4", RC2G_CLASS4_FLOOR, class_id=4, after_task=after_task)
    c5_pen = RC2G_CLASS5_FLOOR_PENALTY * rc2f_floor_deficit_metric(proxy,
    "proxy_class5", RC2G_CLASS5_FLOOR, class_id=5, after_task=after_task)
    tri_min_pen = RC2G_TRI_MIN_FLOOR_PENALTY * max(0.0, RC2G_TRI_MIN_FLOOR -
    tri["tri_min"]) if tri["tri_values"] else 0.0
    tri_gap_pen = RC2G_TRI_BALANCE_PENALTY * tri["tri_gap"] if
    tri["tri_values"] else 0.0
    pred4_pen = rc2g_pred4_bias_penalty(proxy, after_task=after_task)
    min_task_pen = RC2C_MIN_TASK_FLOOR_PENALTY *
    rc2f_floor_deficit_metric(proxy, "proxy_min_task", RC2C_MIN_TASK_FLOOR,
    after_task=after_task)
    old_task_pen = RC2C_OLD_TASK_FLOOR_PENALTY *
    rc2f_floor_deficit_metric(proxy, "proxy_old_task_min", RC2C_OLD_TASK_FLOOR,
    after_task=after_task)
    acc_pen = RC2D_PROXY_ACC_FLOOR_PENALTY * rc2f_floor_deficit_metric(proxy,
    "proxy_acc", RC2D_PROXY_ACC_FLOOR, after_task=after_task)
    zero_pen = RC2G_ZERO_SEEN_CLASS_PENALTY * max(0, zero_count)
    dispersion_pen = RC2D_TASK_STD_WEIGHT * max(0.0, task_std)
    no_repair_pen = RC2C_NO_REPAIR_SOFT_PENALTY if after_task is not None and
    int(after_task) >= RC2C_NO_REPAIR_VETO_AFTER_TASK and family ==
    "proto_no_repair" else 0.0

```

```

    complexity_pen = RC2B_COMPLEXITY_PENALTY *␣
↪rc2b_policy_complexity_rank(variant)
    hydro_pen = rc3_hydro_selection_penalty(proxy)
    guarded_bonus = RC2D_CONTEXT_GUARD_BONUS if␣
↪rc2c_is_guarded_context_candidate(variant) else 0.0
    true_guard_bonus = RC2D_TRUE_GUARD_BONUS if family ==␣
↪"proto_true_global_guard" else 0.0

    total_pen = c2_pen + c4_pen + c5_pen + tri_min_pen + tri_gap_pen +␣
↪pred4_pen + min_task_pen + old_task_pen + acc_pen + zero_pen +␣
↪dispersion_pen + no_repair_pen + complexity_pen + hydro_pen
    return {
        "proxy_acc_floor_penalty": acc_pen,
        "class2_floor_penalty": c2_pen,
        "class4_floor_penalty": c4_pen,
        "class5_floor_penalty": c5_pen,
        "tri_min_floor_penalty": tri_min_pen,
        "tri_balance_penalty": tri_gap_pen,
        "pred4_bias_penalty": pred4_pen,
        "zero_seen_class_penalty": zero_pen,
        "min_task_floor_penalty": min_task_pen,
        "old_task_floor_penalty": old_task_pen,
        "task_dispersion_penalty": dispersion_pen,
        "no_repair_soft_penalty": no_repair_pen,
        "complexity_penalty": complexity_pen,
        "guarded_context_bonus": guarded_bonus,
        "true_guard_bonus": true_guard_bonus,
        "hydro_selection_penalty": hydro_pen,
        "total_risk_penalty": total_pen,
        "rc2g_tri_min": tri["tri_min"],
        "rc2g_tri_mean": tri["tri_mean"],
        "rc2g_tri_gap": tri["tri_gap"],
        "rc2g_pred4_rate": rc2g_pred4_rate(proxy),
        "rc2g_selector_version": RC2G_SELECTOR_VERSION,
    }

def rc2d_gate_aligned_score(proxy, baseline_proxy, variant: str,␣
↪after_task=None):
    """RC2G tri-boundary equilibrium validation score."""
    base_score = rc2f_nan_safe_validation_base(proxy, baseline_proxy)
    acc = rc2f_metric(proxy, "proxy_acc", neutral=0.0)
    min_task = rc2f_metric(proxy, "proxy_min_task", neutral=acc)
    old_task_min = rc2f_metric(proxy, "proxy_old_task_min", neutral=min_task)
    task_std = rc2f_metric(proxy, "proxy_task_acc_std", neutral=0.0)
    tri = rc2g_tri_stats(proxy, after_task=after_task)

```



```

    comp = rc2c_risk_components(proxy, baseline_proxy, variant,
↪after_task=after_task)
    tri_score = (
        RC2G_TRI_MIN_SCORE_WEIGHT * tri["tri_min"]
        + RC2G_TRI_MEAN_SCORE_WEIGHT * tri["tri_mean"]
        - RC2G_TRI_BALANCE_SCORE_WEIGHT * tri["tri_gap"]
    ) if tri["tri_values"] else 0.0
    operational_score = (
        acc
        + 0.30 * min_task
        + 0.15 * old_task_min
        + tri_score
        - 0.20 * max(0.0, task_std)
        - rc2g_pred4_bias_penalty(proxy, after_task=after_task)
    )
    score = float(
        0.42 * base_score
        + 0.58 * operational_score
        + comp["guarded_context_bonus"]
        + comp["true_guard_bonus"]
        - comp["total_risk_penalty"]
    )
    return score if rc2f_finite(score) else -1e9

def rc2g_candidate_tri_min(candidate, after_task=None):
    return rc2g_tri_stats(candidate.get("proxy", {}),
↪after_task=after_task)["tri_min"]

def rc2g_candidate_tri_gap(candidate, after_task=None):
    return rc2g_tri_stats(candidate.get("proxy", {}),
↪after_task=after_task)["tri_gap"]

def rc2b_validation_score(proxy, baseline_proxy, variant: str, after_task=None):
    """Override: RC2G tri-boundary equilibrium score, with RC2b strict fallback.
↪"""
    if not RC2C_RISK_AWARE_SELECTOR_ENABLED or globals().get("POLICY_BRANCH")
↪== "RC2b":
        base_score = rc2f_nan_safe_validation_base(proxy, baseline_proxy)
        tri = rc2g_tri_stats(proxy, after_task=after_task)
        penalty = RC2B_COMPLEXITY_PENALTY * rc2b_policy_complexity_rank(variant)
        score = base_score + 0.20 * tri["tri_min"] - 0.10 * tri["tri_gap"] -
↪penalty - rc2g_pred4_bias_penalty(proxy, after_task=after_task)
        return float(score) if rc2f_finite(score) else -1e9

```

```

    return rc2d_gate_aligned_score(proxy, baseline_proxy, variant,
    ↪after_task=after_task)

def rc2f_emergency_candidate_score(candidate, after_task=None):
    p = candidate.get("proxy", {})
    acc = rc2f_metric(p, "proxy_acc", neutral=0.0)
    min_task = rc2f_metric(p, "proxy_min_task", neutral=acc)
    tri = rc2g_tri_stats(p, after_task=after_task)
    zero_count = int(p.get("proxy_zero_seen_class_count", 0) or 0)
    score = 0.45 * acc + 0.20 * min_task + 0.30 * tri["tri_min"] - 0.15 *
    ↪tri["tri_gap"] - 0.20 * zero_count
    score -= RC2B_COMPLEXITY_PENALTY * rc2b_policy_complexity_rank(candidate.
    ↪get("variant", ""))
    return float(score) if rc2f_finite(score) else -1e9

def rc2c_select_candidate(candidates, after_task):
    """RC2G selector: accuracy + tri-boundary equilibrium + no-leakage gates."""
    emergency_scoring_used = False
    scored = [c for c in candidates if rc2f_finite(c.get("selection_score",
    ↪-1e9))]
    if not scored and RC2G_EMERGENCY_SCORE_IF_ALL_NAN:
        for c in candidates:
            c["selection_score"] = rc2f_emergency_candidate_score(c,
    ↪after_task=after_task)
            c["rc2g_emergency_score_used"] = True
        scored = [c for c in candidates if rc2f_finite(c.get("selection_score",
    ↪-1e9))]
        emergency_scoring_used = True
    if not scored:
        selected = max(candidates, key=lambda c: (rc2d_candidate_proxy_acc(c),
    ↪rc2b_policy_complexity_rank(c.get("variant", ""))))
        return selected, {
            "raw_best_variant": selected["variant"],
            "selection_reason": "fallback_no_finite_scores_proxy_acc_tiebreak",
            "rc2g_selector_version": RC2G_SELECTOR_VERSION,
            "rc2g_emergency_scoring_used": True,
        }

    sorted_by_score = sorted(
        scored,
        key=lambda c: (
            float(c.get("selection_score", -1e9)),
            rc2g_candidate_tri_min(c, after_task),
            -rc2g_candidate_tri_gap(c, after_task),

```

```

        rc2d_candidate_proxy_acc(c),
        -rc2f_candidate_zero_seen_class_count(c),
        -float(c["complexity_rank"]),
    ),
    reverse=True,
)
raw_best = sorted_by_score[0]
selected = raw_best
reason = "direct_rc2g_tri_boundary_equilibrium_score_win" if not
↳emergency_scoring_used else "rc2g_emergency_score_win"
no_repair_veto = False
hysteresis = False
floor_fallback = False
true_guard_rescue = False
class2_rescue = False
class4_rescue = False
class5_rescue = False
tri_rescue = False
zero_reject = False

guarded_candidates = [c for c in scored if
↳rc2c_is_guarded_context_candidate(c["variant"])]
best_guarded = max(guarded_candidates, key=lambda c: float(c.
↳get("selection_score", -1e9))) if guarded_candidates else None
true_guard_candidates = [c for c in scored if
↳rc2b_policy_family(c["variant"]) == "proto_true_global_guard"]
best_true_guard = max(true_guard_candidates, key=lambda c: float(c.
↳get("selection_score", -1e9))) if true_guard_candidates else None

if best_guarded is not None and int(after_task) >=
↳RC2C_NO_REPAIR_VETO_AFTER_TASK and rc2b_policy_family(raw_best["variant"])
↳== "proto_no_repair":
    margin = float(raw_best.get("selection_score", -1e9)) -
↳float(best_guarded.get("selection_score", -1e9))
    if margin < RC2C_NO_REPAIR_REQUIRED_MARGIN:
        selected = best_guarded
        reason = f"no_repair_veto_guarded_fallback_margin_{margin:+.4f}"
        no_repair_veto = True

if RC2G_ZERO_SEEN_CLASS_REJECT_ENABLED and
↳rc2f_candidate_zero_seen_class_count(selected) > 0:
    safe_candidates = [c for c in scored if
↳rc2f_candidate_zero_seen_class_count(c) == 0]
    if safe_candidates:
        selected = max(safe_candidates, key=lambda c: (float(c.
↳get("selection_score", -1e9)), rc2g_candidate_tri_min(c, after_task)))

```

```

        reason = "zero_seen_class_reject_rescue"
        zero_reject = True

    # Tri-min rescue: prefer a substantially more balanced 2/4/5 candidate if it
    # is not too far behind on validation score.
    tri_candidates = [c for c in scored if rc2g_tri_stats(c.get("proxy", {}),
    ↪after_task=after_task)["tri_values"]]
    if tri_candidates:
        best_tri = max(tri_candidates, key=lambda c: (rc2g_candidate_tri_min(c,
    ↪after_task), -rc2g_candidate_tri_gap(c, after_task), float(c.
    ↪get("selection_score", -1e9))))
        if (
            rc2g_candidate_tri_min(best_tri, after_task) >=
    ↪rc2g_candidate_tri_min(selected, after_task) + 0.08
            and rc2d_not_too_far(best_tri, selected, 0.10)
        ):
            selected = best_tri
            reason = "tri_boundary_min_rescue"
            tri_rescue = True

    # Class-specific rescue. RC2G gives class 2 and class 5 more rescue room
    # than RC2F, as long as tri-min does not collapse.
    for cls, floor, rescue_margin, flag_name in [
        (2, RC2G_CLASS2_FLOOR, 0.10, "class2"),
        (4, RC2G_CLASS4_FLOOR, 0.08, "class4"),
        (5, RC2G_CLASS5_FLOOR, 0.10, "class5"),
    ]:
        if not rc2f_class_applicable(cls, after_task):
            continue
        selected_val = rc2f_candidate_class(selected, cls)
        if rc2f_finite(selected_val) and selected_val < floor:
            cls_candidates = [c for c in scored if
    ↪rc2f_finite(rc2f_candidate_class(c, cls))]
            if cls_candidates:
                best_cls = max(cls_candidates, key=lambda c:
    ↪(rc2f_candidate_class(c, cls), rc2g_candidate_tri_min(c, after_task),
    ↪float(c.get("selection_score", -1e9))))
                tri_ok = rc2g_candidate_tri_min(best_cls, after_task) >=
    ↪rc2g_candidate_tri_min(selected, after_task) - 0.05
                not_far = rc2d_not_too_far(best_cls, selected, 0.12)
                if rc2f_candidate_class(best_cls, cls) >= selected_val +
    ↪rescue_margin and tri_ok and not_far:
                    selected = best_cls
                    reason = f"{flag_name}_equilibrium_rescue"
                    class2_rescue = class2_rescue or cls == 2
                    class4_rescue = class4_rescue or cls == 4

```

```

class5_rescue = class5_rescue or cls == 5

    if best_true_guard is not None and rc2d_not_too_far(best_true_guard,
↳selected, 0.08):
        tg_tri = rc2g_candidate_tri_min(best_true_guard, after_task)
        se_tri = rc2g_candidate_tri_min(selected, after_task)
        tg_acc = rc2d_candidate_proxy_acc(best_true_guard)
        se_acc = rc2d_candidate_proxy_acc(selected)
        if (tg_tri >= se_tri + 0.06 or tg_acc >= se_acc + 0.015) and
↳rc2c_candidate_meets_hard_floors(best_true_guard):
            selected = best_true_guard
            reason = "true_global_guard_rescue_for_tri_boundary_alignment"
            true_guard_rescue = True

    if RC2C_FLOOR_RISK_FALLBACK_ENABLED and not
↳rc2c_candidate_meets_hard_floors(selected):
        floor_safe = [c for c in scored if rc2c_candidate_meets_hard_floors(c)]
        if floor_safe:
            selected = max(floor_safe, key=lambda c: (float(c.
↳get("selection_score", -1e9)), rc2g_candidate_tri_min(c, after_task)))
            reason = "tri_boundary_hard_floor_fallback"
            floor_fallback = True

    if best_guarded is not None and not
↳rc2c_is_guarded_context_candidate(selected["variant"]) and
↳rc2d_not_too_far(best_guarded, selected, RC2D_GUARDED_HYSTERESIS_MARGIN):
        bg_tri = rc2g_candidate_tri_min(best_guarded, after_task)
        se_tri = rc2g_candidate_tri_min(selected, after_task)
        if bg_tri >= se_tri - 0.03 and
↳rc2c_candidate_meets_hard_floors(best_guarded):
            selected = best_guarded
            reason = "tri_boundary_guarded_context_hysteresis"
            hysteresis = True

    selected_tri = rc2g_tri_stats(selected.get("proxy", {}),
↳after_task=after_task)
    raw_tri = rc2g_tri_stats(raw_best.get("proxy", {}), after_task=after_task)
    return selected, {
        "raw_best_variant": raw_best["variant"],
        "raw_best_policy_family": raw_best["policy_family"],
        "raw_best_proxy_score": raw_best.get("selection_score", np.nan),
        "best_guarded_variant": best_guarded["variant"] if best_guarded is not
↳None else "",
        "best_guarded_proxy_score": best_guarded.get("selection_score", np.nan)
↳if best_guarded is not None else np.nan,

```

```

        "best_true_guard_variant": best_true_guard["variant"] if
↪best_true_guard is not None else "",
        "best_true_guard_proxy_score": best_true_guard.get("selection_score",
↪np.nan) if best_true_guard is not None else np.nan,
        "selection_reason": reason,
        "no_repair_veto_applied": no_repair_veto,
        "guarded_hysteresis_applied": hysteresis,
        "risk_floor_fallback_applied": floor_fallback,
        "true_guard_rescue_applied": true_guard_rescue,
        "class2_rescue_applied": class2_rescue,
        "class4_rescue_applied": class4_rescue,
        "class5_rescue_applied": class5_rescue,
        "zero_seen_class_reject_applied": zero_reject,
        "rc2g_tri_rescue_applied": tri_rescue,
        "rc2g_selector_version": RC2G_SELECTOR_VERSION,
        "rc2g_emergency_scoring_used": emergency_scoring_used,
        "rc2g_selected_tri_min": selected_tri["tri_min"],
        "rc2g_selected_tri_mean": selected_tri["tri_mean"],
        "rc2g_selected_tri_gap": selected_tri["tri_gap"],
        "rc2g_raw_best_tri_min": raw_tri["tri_min"],
        "rc2g_raw_best_tri_gap": raw_tri["tri_gap"],
        "rc2g_class2_applicable": rc2f_class_applicable(2, after_task),
        "rc2g_class4_applicable": rc2f_class_applicable(4, after_task),
        "rc2g_class5_applicable": rc2f_class_applicable(5, after_task),
    }

print("RC2G tri-boundary equilibrium overrides loaded:", RC2G_SELECTOR_VERSION)
print("RC2G_TRI_CLASSES:", RC2G_TRI_CLASSES)
print("RC2G_CLASS2/4/5_FLOORS:", RC2G_CLASS2_FLOOR, RC2G_CLASS4_FLOOR,
↪RC2G_CLASS5_FLOOR)

```

```

RC2G tri-boundary equilibrium overrides loaded: RC2g_tri_boundary_equilibrium_v1
RC2G_TRI_CLASSES: [2, 4, 5]
RC2G_CLASS2/4/5_FLOORS: 0.8 0.75 0.8

```

```

[19]: # -----
# v1.1-RC2h RC1b-anchored conservative selector overrides
# -----
# RC2G evidence showed that free validation selection can underperform the
# fixed RC1b context-gap candidate. RC2H therefore treats the RC1b candidate
# as a default deployable anchor and permits overrides only when a candidate
# clearly beats the anchor on validation memory and passes safety floors.
#
# No final-test examples are used by this selector.
# -----

RC2H_SELECTOR_VERSION = "RC2h_rc1b_anchored_conservative_v1"

```

```

RC2H_ANCHOR_VARIANTS = [
    "context_gap_selected",
    "context_temp_blend_selected_head_guard_035",
]
RC2H_PRIMARY_ANCHOR_VARIANT = "context_gap_selected"

# Override rules. These are intentionally conservative; they decide whether a
# candidate may replace RC1b, not whether it merely has the largest proxy score.
RC2H_OVERRIDE_SCORE_MARGIN = 0.025 if RUN_MODE in ["smoke"] else 0.018
RC2H_OVERRIDE_ACC_MARGIN = 0.010
RC2H_OVERRIDE_TRI_MIN_MARGIN = -0.010 # candidate may be slightly below
    ↪ anchor, but not collapse boundary safety
RC2H_CLASS2_FLOOR = 0.80
RC2H_CLASS4_FLOOR = 0.45
RC2H_CLASS5_FLOOR = 0.80
RC2H_MIN_TASK_FLOOR = 0.80
RC2H_MIN_CLASS_FLOOR = 0.75
RC2H_TRI_MIN_FLOOR = 0.45
RC2H_MAX_TRI_GAP = 0.45
RC2H_REJECT_ZERO_SEEN_CLASS = True
RC2H_ALLOW_NO_REPAIR_OVERRIDE = False
RC2H_NO_REPAIR_EXCEPTION_SCORE_MARGIN = 0.060
RC2H_ANCHOR_PRIOR_BONUS = 0.000 # selection is governed by override rules, not
    ↪ by hidden score inflation

def rc2h_finite(x):
    try:
        return np.isfinite(float(x))
    except Exception:
        return False

def rc2h_value(proxy, key, neutral=np.nan):
    try:
        v = proxy.get(key, neutral)
        return float(v) if rc2h_finite(v) else neutral
    except Exception:
        return neutral

def rc2h_class_applicable(cls, after_task):
    try:
        return bool(cls in seen_classes_until(int(after_task)))
    except Exception:
        # Conservative fallback: class metrics are considered applicable only
        # when they are present and finite in the proxy.

```

```

        return True

def rc2h_is_anchor_variant(variant: str) -> bool:
    return str(variant) in RC2H_ANCHOR_VARIANTS or
    ↪rc2b_policy_family(str(variant)) == "rc1b_guarded_context_gap"

def rc2h_anchor_rank(candidate):
    variant = str(candidate.get("variant", ""))
    if variant == RC2H_PRIMARY_ANCHOR_VARIANT:
        return 2
    if variant in RC2H_ANCHOR_VARIANTS:
        return 1
    if rc2b_policy_family(variant) == "rc1b_guarded_context_gap":
        return 0
    return -1

def rc2h_tri_stats_from_candidate(candidate, after_task):
    p = candidate.get("proxy", {})
    if "rc2g_tri_stats" in globals():
        try:
            return rc2g_tri_stats(p, after_task=after_task)
        except Exception:
            pass
    vals = []
    for cls, key in [(2, "proxy_class2"), (4, "proxy_class4"), (5,
    ↪"proxy_class5")]:
        if rc2h_class_applicable(cls, after_task):
            v = rc2h_value(p, key, neutral=np.nan)
            if rc2h_finite(v):
                vals.append(v)
    if not vals:
        return {"tri_min": 0.0, "tri_mean": 0.0, "tri_gap": 0.0}
    return {"tri_min": float(np.min(vals)), "tri_mean": float(np.mean(vals)),
    ↪"tri_gap": float(np.max(vals) - np.min(vals))}

def rc2h_candidate_acc(candidate):
    return rc2h_value(candidate.get("proxy", {}), "proxy_acc", neutral=-1e9)

def rc2h_candidate_min_task(candidate):
    p = candidate.get("proxy", {})
    return rc2h_value(p, "proxy_min_task", neutral=rc2h_value(p,
    ↪"proxy_min_class", neutral=rc2h_candidate_acc(candidate)))

```



```

def rc2h_candidate_min_class(candidate):
    p = candidate.get("proxy", {})
    return rc2h_value(p, "proxy_min_class", 1)
    ↪ neutral=rc2h_candidate_acc(candidate))

def rc2h_zero_seen_count(candidate):
    p = candidate.get("proxy", {})
    try:
        return int(p.get("proxy_zero_seen_class_count", 0) or 0)
    except Exception:
        return 0

def rc2h_candidate_passes_override_floors(candidate, anchor, after_task):
    """Return (ok, reasons) using validation-memory proxy values only."""
    p = candidate.get("proxy", {})
    reasons = []

    acc = rc2h_candidate_acc(candidate)
    anchor_acc = rc2h_candidate_acc(anchor)
    if not (rc2h_finite(acc) and rc2h_finite(anchor_acc)):
        reasons.append("non_finite_acc")
    elif acc < anchor_acc + RC2H_OVERRIDE_ACC_MARGIN:
        reasons.append(f"acc_margin<{RC2H_OVERRIDE_ACC_MARGIN:.3f}")

    score = float(candidate.get("selection_score", -1e9))
    anchor_score = float(anchor.get("selection_score", -1e9))
    if not (rc2h_finite(score) and rc2h_finite(anchor_score)):
        reasons.append("non_finite_score")
    elif score < anchor_score + RC2H_OVERRIDE_SCORE_MARGIN:
        reasons.append(f"score_margin<{RC2H_OVERRIDE_SCORE_MARGIN:.3f}")

    min_task = rc2h_candidate_min_task(candidate)
    if rc2h_finite(min_task) and min_task < RC2H_MIN_TASK_FLOOR:
        reasons.append(f"min_task<{RC2H_MIN_TASK_FLOOR:.2f}")

    min_class = rc2h_candidate_min_class(candidate)
    if rc2h_finite(min_class) and min_class < RC2H_MIN_CLASS_FLOOR:
        reasons.append(f"min_class<{RC2H_MIN_CLASS_FLOOR:.2f}")

    class_floor_specs = [
        (2, "proxy_class2", RC2H_CLASS2_FLOOR),
        (4, "proxy_class4", RC2H_CLASS4_FLOOR),
        (5, "proxy_class5", RC2H_CLASS5_FLOOR),

```

```

]
for cls, key, floor in class_floor_specs:
    if rc2h_class_applicable(cls, after_task):
        v = rc2h_value(p, key, neutral=np.nan)
        if rc2h_finite(v) and v < floor:
            reasons.append(f"class{cls}<{floor:.2f}")

    tri = rc2h_tri_stats_from_candidate(candidate, after_task)
    anchor_tri = rc2h_tri_stats_from_candidate(anchor, after_task)
    if rc2h_finite(tri.get("tri_min", np.nan)) and tri["tri_min"] <_
↳RC2H_TRI_MIN_FLOOR:
        reasons.append(f"tri_min<{RC2H_TRI_MIN_FLOOR:.2f}")
    if rc2h_finite(tri.get("tri_gap", np.nan)) and tri["tri_gap"] >_
↳RC2H_MAX_TRI_GAP:
        reasons.append(f"tri_gap>{RC2H_MAX_TRI_GAP:.2f}")
    if rc2h_finite(tri.get("tri_min", np.nan)) and rc2h_finite(anchor_tri.
↳get("tri_min", np.nan)):
        if tri["tri_min"] < anchor_tri["tri_min"] +_
↳RC2H_OVERRIDE_TRI_MIN_MARGIN:
            reasons.append("tri_min_below_anchor")

    if RC2H_REJECT_ZERO_SEEN_CLASS and rc2h_zero_seen_count(candidate) > 0:
        reasons.append("zero_seen_class")

    family = rc2b_policy_family(candidate.get("variant", ""))
    if family == "proto_no_repair" and not RC2H_ALLOW_NO_REPAIR_OVERRIDE:
        if score < anchor_score + RC2H_NO_REPAIR_EXCEPTION_SCORE_MARGIN:
            reasons.append("no_repair_veto")

    return len(reasons) == 0, reasons

def rc2h_select_anchor(scored):
    anchors = [c for c in scored if rc2h_is_anchor_variant(c.get("variant",_
↳""))]
    if not anchors:
        return None
    # Prefer the explicit primary anchor when it is finite and not absurdly_
↳weak;
    # otherwise use the best guarded context-gap anchor by score.
    primary = [c for c in anchors if c.get("variant") ==_
↳RC2H_PRIMARY_ANCHOR_VARIANT]
    if primary:
        return max(primary, key=lambda c: float(c.get("selection_score", -1e9)))
    return max(anchors, key=lambda c: (rc2h_anchor_rank(c), float(c.
↳get("selection_score", -1e9)), rc2h_candidate_acc(c)))

```

```

def rc2h_validation_score(proxy, baseline_proxy, variant: str, after_task=None):
    """Validation-memory score used for ordering; RC2H final choice still
    ↪applies anchor override gates."""
    try:
        base_score = rc2d_gate_aligned_score(proxy, baseline_proxy, variant,
        ↪after_task=after_task)
    except Exception:
        try:
            base_score = rc2f_nan_safe_validation_base(proxy, baseline_proxy)
        except Exception:
            base_score = validation_score(proxy, baseline_proxy)
    tri = rc2h_tri_stats_from_candidate({"proxy": proxy}, after_task)
    zero_penalty = 0.20 * int(proxy.get("proxy_zero_seen_class_count", 0) or 0)
    anchor_bonus = RC2H_ANCHOR_PRIOR_BONUS if rc2h_is_anchor_variant(variant)
    ↪else 0.0
    no_repair_penalty = 0.02 if rc2b_policy_family(variant) ==
    ↪"proto_no_repair" and int(after_task or 0) >= 2 else 0.0
    score = float(base_score) + anchor_bonus + 0.10 * tri.get("tri_min", 0.0) -
    ↪0.05 * tri.get("tri_gap", 0.0) - zero_penalty - no_repair_penalty
    return float(score) if rc2h_finite(score) else -1e9

# Override previous RC2G scoring only for RC2H. Other compatibility profiles
↪keep existing logic.
def rc2b_validation_score(proxy, baseline_proxy, variant: str, after_task=None):
    if globals().get("POLICY_BRANCH") == "RC2h":
        return rc2h_validation_score(proxy, baseline_proxy, variant,
        ↪after_task=after_task)
    if globals().get("POLICY_BRANCH") == "RC2b" or not
    ↪RC2C_RISK_AWARE_SELECTOR_ENABLED:
        try:
            return rc2f_nan_safe_validation_base(proxy, baseline_proxy)
        except Exception:
            return validation_score(proxy, baseline_proxy)
    try:
        return rc2d_gate_aligned_score(proxy, baseline_proxy, variant,
        ↪after_task=after_task)
    except Exception:
        return validation_score(proxy, baseline_proxy)

def rc2c_select_candidate(candidates, after_task):
    """RC2H selector: keep RC1b anchor unless an override is clearly safer and
    ↪better."""

```

```

    scored = [c for c in candidates if rc2h_finite(c.get("selection_score",
↪-1e9))]
    emergency_scoring_used = False
    if not scored and globals().get("RC2G_EMERGENCY_SCORE_IF_ALL_NAN", True):
        for c in candidates:
            try:
                c["selection_score"] = rc2f_emergency_candidate_score(c,
↪after_task=after_task)
            except Exception:
                c["selection_score"] = rc2h_candidate_acc(c)
                c["rc2h_emergency_score_used"] = True
            scored = [c for c in candidates if rc2h_finite(c.get("selection_score",
↪-1e9))]
            emergency_scoring_used = True

    if not scored:
        # Last-resort fallback: choose the best available anchor by proxy if
↪possible.
        scored = candidates

    raw_best = max(
        scored,
        key=lambda c: (
            float(c.get("selection_score", -1e9)) if rc2h_finite(c.
↪get("selection_score", -1e9)) else -1e9,
            rc2h_candidate_acc(c),
            -float(c.get("complexity_rank", rc2b_policy_complexity_rank(c.
↪get("variant", "")))),
        ),
    )
    anchor = rc2h_select_anchor(scored)
    if anchor is None:
        # If an anchor candidate is unavailable because of an unexpected
↪candidate pool,
        # fall back to the best guarded context candidate or the raw best;
↪report it.
        guarded = [c for c in scored if rc2b_policy_family(c.get("variant",
↪")) == "rc1b_guarded_context_gap"]
        anchor = max(guarded, key=lambda c: float(c.get("selection_score",
↪-1e9))) if guarded else raw_best

    selected = anchor
    override_applied = False
    override_rejected_reason = "raw_best_is_anchor"
    reason = "keep_rc1b_anchor"
    no_repair_veto = False

```

```

anchor_kept = True

if raw_best.get("variant") != anchor.get("variant"):
    ok, reject_reasons = rc2h_candidate_passes_override_floors(raw_best,
↳ anchor, after_task)
    if ok:
        selected = raw_best
        override_applied = True
        anchor_kept = False
        override_rejected_reason = ""
        reason = "override_rc1b_anchor_with_margin_and_floors"
    else:
        override_rejected_reason = ";".join(reject_reasons)
        reason = "keep_rc1b_anchor_reject_override"
        no_repair_veto = "no_repair_veto" in reject_reasons

    anchor_score = float(anchor.get("selection_score", np.nan)) if
↳ rc2h_finite(anchor.get("selection_score", np.nan)) else np.nan
    raw_best_score = float(raw_best.get("selection_score", np.nan)) if
↳ rc2h_finite(raw_best.get("selection_score", np.nan)) else np.nan
    override_score_margin = raw_best_score - anchor_score if
↳ rc2h_finite(raw_best_score) and rc2h_finite(anchor_score) else np.nan
    anchor_acc = rc2h_candidate_acc(anchor)
    raw_best_acc = rc2h_candidate_acc(raw_best)
    override_acc_margin = raw_best_acc - anchor_acc if
↳ rc2h_finite(raw_best_acc) and rc2h_finite(anchor_acc) else np.nan
    anchor_tri = rc2h_tri_stats_from_candidate(anchor, after_task)
    raw_best_tri = rc2h_tri_stats_from_candidate(raw_best, after_task)

    return selected, {
        "raw_best_variant": raw_best.get("variant", ""),
        "raw_best_policy_family": raw_best.get("policy_family",
↳ rc2b_policy_family(raw_best.get("variant", ""))),
        "raw_best_proxy_score": raw_best_score,
        "best_guarded_variant": anchor.get("variant", ""),
        "best_guarded_proxy_score": anchor_score,
        "selection_reason": reason,
        "no_repair_veto_applied": bool(no_repair_veto),
        "guarded_hysteresis_applied": False,
        "risk_floor_fallback_applied": bool(anchor_kept and raw_best.
↳ get("variant") != anchor.get("variant")),
        "true_guard_rescue_applied": False,
        "class2_rescue_applied": False,
        "class4_rescue_applied": False,
        "class5_rescue_applied": False,
        "tri_rescue_applied": False,

```

```

        "zero_seen_class_reject_applied": "zero_seen_class" in _
    ↪ override_rejected_reason,
        "rc2h_selector_version": RC2H_SELECTOR_VERSION,
        "rc2h_anchor_variant": anchor.get("variant", ""),
        "rc2h_anchor_method": anchor.get("method", ""),
        "rc2h_anchor_score": anchor_score,
        "rc2h_anchor_acc": anchor_acc,
        "rc2h_anchor_tri_min": anchor_tri.get("tri_min", np.nan),
        "rc2h_raw_best_score": raw_best_score,
        "rc2h_raw_best_acc": raw_best_acc,
        "rc2h_raw_best_tri_min": raw_best_tri.get("tri_min", np.nan),
        "rc2h_override_score_margin": override_score_margin,
        "rc2h_override_acc_margin": override_acc_margin,
        "rc2h_anchor_kept": bool(anchor_kept),
        "rc2h_override_applied": bool(override_applied),
        "rc2h_override_rejected_reason": override_rejected_reason,
        "rc2h_emergency_scoring_used": bool(emergency_scoring_used),
    }

print("RC2H anchored selector installed:", RC2H_SELECTOR_VERSION)
print("RC2H anchor variants:", RC2H_ANCHOR_VARIANTS)
print("RC2H override margins:", RC2H_OVERRIDE_SCORE_MARGIN, _
    ↪ RC2H_OVERRIDE_ACC_MARGIN)

```

```

RC2H anchored selector installed: RC2h_rc1b_anchored_conservative_v1
RC2H anchor variants: ['context_gap_selected',
'context_temp_blend_selected_head_guard_035']
RC2H override margins: 0.018 0.01

```

1.18 11. Final frozen representation probes

The probes remain the diagnostic ceiling. If the deployed candidate stays close to the proto-context zf probe while the oracle-context probe remains higher, the remaining frontier is context inference / context memory rather than readout micro-repair.

```

[20]: class LinearProbe(nn.Module):
    def __init__(self, input_dim, n_classes=N_CLASSES):
        super().__init__()
        self.linear = nn.Linear(input_dim, n_classes)
    def forward(self, x):
        return self.linear(x)

def freeze_model(model):
    model.eval()
    for p in model.parameters():
        p.requires_grad_(False)
    return model

```

```

@torch.no_grad()
def extract_mmals_features(model, dataset, task_id, context_mode, prototypes,
    ↪feature_kind="zf", max_items=None, batch_size=BATCH_SIZE):
    model.eval()
    config = probe_config_from_mode(context_mode)
    X, y = dataset.tensors
    if max_items is not None and len(X) > max_items:
        idx = torch.randperm(len(X))[:max_items]
        X = X[idx]
        y = y[idx]
    feats, labels, tasks = [], [], []
    loader = make_loader(TensorDataset(X, y), batch_size=batch_size,
    ↪shuffle=False)
    for xb, yb in loader:
        xb, yb = xb.to(DEVICE), yb.to(DEVICE)
        logits, cache = forward_by_method(model, xb, task_id, config,
    ↪prototypes=prototypes, train=False, y=yb, return_cache=True)
        if feature_kind == "zf":
            f = cache["zf"]
        elif feature_kind == "context_feat":
            f = cache["context_feat"]
        elif feature_kind.startswith("host"):
            hid = int(feature_kind.replace("host", ""))
            f = cache["host_z"][:, hid, :]
        elif feature_kind.startswith("transformed"):
            hid = int(feature_kind.replace("transformed", ""))
            f = cache["transformed"][:, hid, :]
        elif feature_kind == "routes":
            f = cache["routes"]
        else:
            raise ValueError(f"Unknown feature_kind: {feature_kind}")
        feats.append(f.detach().cpu())
        labels.append(yb.detach().cpu())
        tasks.append(torch.full((yb.shape[0],), task_id, dtype=torch.long))
    return torch.cat(feats, dim=0).float(), torch.cat(labels, dim=0).long(),
    ↪torch.cat(tasks, dim=0).long()

def build_probe_dataset(model, tasks, prototypes, context_mode="proto",
    ↪feature_kind="zf", train=True, max_per_task=PROBE_MAX_PER_TASK):
    Xs, ys, ts = [], [], []
    key = "train" if train else "test"
    for task_id, task in enumerate(tasks):
        X, y, t = extract_mmals_features(
            model, task[key], task_id, context_mode=context_mode,
            prototypes=prototypes, feature_kind=feature_kind,
            max_items=max_per_task if train else None,
        )

```

```

        Xs.append(X)
        ys.append(y)
        ts.append(t)
    return TensorDataset(torch.cat(Xs, dim=0), torch.cat(ys, dim=0), torch.
↪cat(ts, dim=0))

def train_linear_probe(train_ds, input_dim, seed=0, epochs=PROBE_EPOCHS):
    set_seed(seed + 70000)
    probe = LinearProbe(input_dim=input_dim).to(DEVICE)
    opt = torch.optim.Adam(probe.parameters(), lr=PROBE_LR)
    loader = make_loader(train_ds, batch_size=BATCH_SIZE, shuffle=True)
    rows = []
    for epoch in range(epochs):
        probe.train()
        total_loss, n_batches = 0.0, 0
        for xb, yb, _tb in loader:
            xb, yb = xb.to(DEVICE), yb.to(DEVICE)
            opt.zero_grad(set_to_none=True)
            logits = probe(xb)
            loss = F.cross_entropy(logits, yb)
            loss.backward()
            opt.step()
            total_loss += float(loss.detach().cpu())
            n_batches += 1
        rows.append({"epoch": epoch, "probe_loss": total_loss / max(n_batches, 1)})
↪1)})
    return probe, pd.DataFrame(rows)

@torch.no_grad()
def evaluate_probe(probe, test_ds, method_name, seed, backbone_source,
↪feature_kind, context_mode):
    probe.eval()
    X, y, task_ids = test_ds.tensors
    final_task = N_TASKS - 1
    seen = seen_classes_until(final_task)
    rows = []
    mat = torch.zeros(N_CLASSES, N_CLASSES, dtype=torch.long)
    for task_id in range(N_TASKS):
        mask = (task_ids == task_id)
        if not mask.any():
            continue
        ds = TensorDataset(X[mask], y[mask])
        loader = make_loader(ds, batch_size=BATCH_SIZE, shuffle=False)
        correct, total = 0, 0
        class2_margin_24, class5_margin_54 = [], []
        class2_logit_rank, class5_logit_rank = [], []
        for xb, yb in loader:

```



```

xb, yb = xb.to(DEVICE), yb.to(DEVICE)
logits = mask_logits_to_seen_classes(probe(xb), seen)
pred = logits.argmax(dim=-1)
correct += (pred == yb).sum().item()
total += yb.numel()
if (yb == 2).any():
    m = (yb == 2)
    class2_margin_24.append((logits[m, 2] - logits[m, 4]).detach().
↪cpu())

    ranks = (logits[m] > logits[m, 2].unsqueeze(-1)).sum(dim=-1) + 1
    class2_logit_rank.append(ranks.float().detach().cpu())
if (yb == 5).any():
    m = (yb == 5)
    class5_margin_54.append((logits[m, 5] - logits[m, 4]).detach().
↪cpu())

    ranks = (logits[m] > logits[m, 5].unsqueeze(-1)).sum(dim=-1) + 1
    class5_logit_rank.append(ranks.float().detach().cpu())
for yt, yp in zip(yb.detach().cpu(), pred.detach().cpu()):
    mat[int(yt), int(yp)] += 1
rows.append({
    "method": method_name,
    "seed": seed,
    "model_family": "probe",
    "after_task": final_task,
    "task_id": task_id,
    "task_name": f"probe_task_{task_id}",
    "seen_classes": str(seen),
    "trainable_params": count_parameters(probe),
    "total_params": total_parameters(probe),
    "model_size_mb_trainable": model_size_mb(probe,
↪trainable_only=True),
    "model_size_mb_total": model_size_mb(probe, trainable_only=False),
    "uses_balanced_replay": False,
    "uses_output_distill": False,
    "probe_row": True,
    "backbone_source": backbone_source,
    "probe_feature": feature_kind,
    "probe_context_mode": context_mode,
    "replay_memory_items": 0,
    "memory_per_class": MEMORY_PER_CLASS,
    "memory_per_task": MEMORY_PER_TASK,
    "acc": correct / max(total, 1),
    "context_entropy": np.nan,
    "context_margin": np.nan,
    "context_top1_acc": np.nan,
    "context_brier": np.nan,
    "context_ece": np.nan,

```

```

        "latent_frechet_margin": np.nan,
        "prototype_weight": np.nan,
        "class2_margin_vs4": float(torch.cat(class2_margin_24).mean()) if_
↪class2_margin_24 else np.nan,
        "class5_margin_vs4": float(torch.cat(class5_margin_54).mean()) if_
↪class5_margin_54 else np.nan,
        "class2_logit_rank": float(torch.cat(class2_logit_rank).mean()) if_
↪class2_logit_rank else np.nan,
        "class5_logit_rank": float(torch.cat(class5_logit_rank).mean()) if_
↪class5_logit_rank else np.nan,
    })
    cm_rows = []
    for true_class in range(N_CLASSES):
        denom = mat[true_class].sum().item()
        for pred_class in range(N_CLASSES):
            cm_rows.append({
                "method": method_name,
                "seed": seed,
                "after_task": final_task,
                "backbone_source": backbone_source,
                "true_class": true_class,
                "pred_class": pred_class,
                "count": int(mat[true_class, pred_class].item()),
                "rate": float(mat[true_class, pred_class].item() / max(denom,
↪1)),
            })
    return pd.DataFrame(rows), pd.DataFrame(cm_rows)

def short_backbone_name(method: str) -> str:
    mapping = {
        "mmals_uniform_balanced_replay": "uniform",
        "mmals_proto_first_balanced_replay": "proto",
        "mmals_oracle_context_balanced_replay": "oracle",
    }
    return mapping.get(method, method.replace("mmals_", "").
↪replace("_balanced_replay", ""))

def run_representation_audit(artifact: Dict[str, Any]):
    backbone_source = artifact["method"]
    seed = artifact["seed"]
    model = freeze_model(artifact["model"])
    tasks = artifact["tasks"]
    prototypes = artifact["context_prototypes"]
    all_h, all_c, probe_loss_rows = [], [], []
    # zf probes under uniform/proto/oracle contexts.
    for mode in PROBE_CONTEXT_MODES:

```

```

train_ds = build_probe_dataset(model, tasks, prototypes,
↪context_mode=mode, feature_kind="zf", train=True)
test_ds = build_probe_dataset(model, tasks, prototypes,
↪context_mode=mode, feature_kind="zf", train=False)
input_dim = train_ds.tensors[0].shape[1]
probe, loss_df = train_linear_probe(train_ds, input_dim=input_dim,
↪seed=seed)
method_name = f"probe_{short_backbone_name(backbone_source)}_zf_{mode}"
loss_df["method"] = method_name
loss_df["seed"] = seed
loss_df["backbone_source"] = backbone_source
loss_df["probe_feature"] = "zf"
loss_df["probe_context_mode"] = mode
h, c = evaluate_probe(probe, test_ds, method_name, seed,
↪backbone_source, "zf", mode)
all_h.append(h)
all_c.append(c)
probe_loss_rows.append(loss_df)
print(f"[{method_name} seed={seed}] probe final task accs: {[round(x,
↪3) for x in h.acc.tolist()]}")
# Host/transformed-host probes, limited to transformed host latents for
↪compactness.
for hid in range(N_HOSTS):
    feature_kind = f"transformed{hid}"
    mode = HOST_PROBE_MODE
    train_ds = build_probe_dataset(model, tasks, prototypes,
↪context_mode=mode, feature_kind=feature_kind, train=True)
    test_ds = build_probe_dataset(model, tasks, prototypes,
↪context_mode=mode, feature_kind=feature_kind, train=False)
    input_dim = train_ds.tensors[0].shape[1]
    probe, loss_df = train_linear_probe(train_ds, input_dim=input_dim,
↪seed=seed + hid + 10)
    method_name =
↪f"probe_{short_backbone_name(backbone_source)}_{feature_kind}_{mode}"
    loss_df["method"] = method_name
    loss_df["seed"] = seed
    loss_df["backbone_source"] = backbone_source
    loss_df["probe_feature"] = feature_kind
    loss_df["probe_context_mode"] = mode
    h, c = evaluate_probe(probe, test_ds, method_name, seed,
↪backbone_source, feature_kind, mode)
    all_h.append(h)
    all_c.append(c)
    probe_loss_rows.append(loss_df)
    print(f"[{method_name} seed={seed}] host probe accs: {[round(x, 3) for
↪x in h.acc.tolist()]}")

```

```

    return pd.concat(all_h, ignore_index=True), pd.concat(all_c,
↪ignore_index=True), pd.concat(probe_loss_rows, ignore_index=True)

```

1.19 12. Run paired v1.0-RC1b validation audit

This cell performs:

one base MMALS backbone → cloned controls and RC1b candidate variants

The primary controls are:

- paired_proto_global_no_repair: proto-first context, copied global head.
- paired_proto_kl_only_true_noop: deliberately identical no-op. It should match no-repair.
- paired_proto_global_head_ce_kl_guard_035: true corrected global CE+KL guarded-head control.

The deployable context rows are:

- paired_context_blend_selected_global: context blend selection only.
- paired_context_temp_blend_selected_global: context temp+blend selection only.
- paired_context_blend_selected_head_guard_035: rejected original RC1 diagnostic.
- paired_context_temp_blend_selected_head_guard_035: RC1b equivalent diagnostic.
- paired_context_gap_selected: **primary v1.0-RC1b candidate row**.

Oracle rows are references only. They measure the ceiling when the task/context is known; they are not deployable task-free methods.

1.19.1 12.1 Control sanity audit

Before running the expensive experiment, this cell verifies that guarded-head activation is explicit and correct: true global guard and context+guard must activate guard; no-repair, no-op and context-only must not.

```

[21]: # Control-fix sanity audit: verify that guarded-head activation is explicit and
↪correct.
guard_map = {variant: variant_uses_guarded_head(variant) for variant in
↪SELECTED_CONTEXT_HEAD_VARIANTS}
print("Guard activation map:")
for k, v in guard_map.items():
    print(f" - {k}: {v}")

assert guard_map["proto_global_head_ce_kl_guard_035"] is True, "Control-fix
↪failed: true global guard is inactive."
assert guard_map["proto_global_no_repair"] is False, "No-repair must not
↪activate guard."
assert guard_map["proto_kl_only_true_noop"] is False, "No-op must not activate
↪guard."
assert guard_map["context_temp_blend_selected_head_guard_035"] is True,
↪"Context+guard candidate must activate guard."

```

```

assert guard_map["context_temp_blend_selected_global"] is False, "Context-only_
↪candidate must not activate guard."
print("Control-fix sanity checks passed.")

```

Guard activation map:

- proto_global_no_repair: False
- proto_kl_only_true_noop: False
- proto_global_head_ce_kl_guard_035: True
- oracle_global_no_repair_reference: False
- oracle_global_head_ce_kl_guard_035_reference: True
- context_blend_selected_global: False
- context_temp_blend_selected_global: False
- context_blend_selected_head_guard_035: True
- context_temp_blend_selected_head_guard_035: True
- context_gap_selected: True

Control-fix sanity checks passed.

1.20 12.2 Comparative baseline suite

This section adds standard continual-learning baselines to the same overlapping class-incremental stress protocol.

The MMALS paired audit remains unchanged. Baselines are trained sequentially on the same tasks and evaluated with the same seen-class mask. The joint upper bound is trained on all tasks jointly and is diagnostic, not deployable in a continual-learning setting.

Diagnostic patch. Standard baseline confusion matrices now export normalized **rate** values, matching the MMALS confusion tables. The interpretation section also separates deployable baselines, deployable MMALS rows, oracle references, probes, and the joint upper bound.

```

[22]: class SparseMoEClassifier(nn.Module):
    """Small sparse mixture-of-experts baseline with a learned soft gate."""
    def __init__(self, input_dim=INPUT_DIM, hidden_dim=HIDDEN_DIM,
↪latent_dim=LATENT_DIM, n_experts=N_HOSTS, n_classes=N_CLASSES):
        super().__init__()
        self.experts = nn.ModuleList([MLPBlock(input_dim, hidden_dim,
↪latent_dim) for _ in range(n_experts)])
        self.gate = nn.Sequential(
            nn.Linear(input_dim, ROUTE_HIDDEN_DIM),
            nn.ReLU(),
            nn.Linear(ROUTE_HIDDEN_DIM, n_experts),
        )
        self.head = nn.Linear(latent_dim, n_classes)

    def forward(self, x, return_gate=False):
        weights = F.softmax(self.gate(x) / TEMPERATURE, dim=-1)
        z_stack = torch.stack([expert(x) for expert in self.experts], dim=1)
        z = (weights.unsqueeze(-1) * z_stack).sum(dim=1)

```

```

        logits = self.head(z)
        if return_gate:
            return logits, weights
        return logits

def set_requires_grad(module_or_param, requires_grad: bool):
    """
    Enable or disable gradient updates for a module or a single parameter.
    Used by the PNN baseline to freeze old columns and train only the current
    ↪column.
    """
    if isinstance(module_or_param, nn.Module):
        for p in module_or_param.parameters():
            p.requires_grad_(requires_grad)
    elif isinstance(module_or_param, torch.nn.Parameter):
        module_or_param.requires_grad_(requires_grad)
    else:
        raise TypeError(f"Unsupported type for set_requires_grad:
    ↪{type(module_or_param)}")

class PNNSimpleClassifier(nn.Module):
    """Fixed-column PNN-style modular baseline.

    One column is activated for each task; old columns are frozen after their
    ↪task.

    The global head reads the concatenation of all columns. This is a compact
    ↪Colab-safe
    approximation of a progressive neural network control.
    """
    def __init__(self, input_dim=INPUT_DIM, hidden_dim=HIDDEN_DIM,
    ↪latent_dim=LATENT_DIM, n_tasks=N_TASKS, n_classes=N_CLASSES):
        super().__init__()
        self.columns = nn.ModuleList([MLPBlock(input_dim, hidden_dim,
    ↪latent_dim) for _ in range(n_tasks)])
        self.head = nn.Linear(latent_dim * n_tasks, n_classes)

    def set_trainable_column(self, task_id: int):
        for i, col in enumerate(self.columns):
            set_requires_grad(col, i == task_id)
        set_requires_grad(self.head, True)

    def forward(self, x):
        zs = [col(x) for col in self.columns]
        return self.head(torch.cat(zs, dim=-1))

```

```

def baseline_forward(model, xb, method=None):
    if isinstance(model, SparseMoEClassifier):
        logits, gate = model(xb, return_gate=True)
        return logits, gate
    return model(xb), None

@torch.no_grad()
def evaluate_standard_model(model, tasks, upto_task, method, seed,
    ↪model_family, replay_memory_items=0):
    model.eval()
    rows = []
    seen = seen_classes_until(upto_task)
    trainable = count_parameters(model)
    total = total_parameters(model)
    for task_id in range(upto_task + 1):
        loader = make_loader(tasks[task_id]["test"], batch_size=BATCH_SIZE,
    ↪shuffle=False)
        correct, total_items = 0, 0
        gate_entropies = []
        class2_margin_24, class5_margin_54 = [], []
        class2_rank, class5_rank = [], []
        for xb, yb in loader:
            xb, yb = xb.to(DEVICE), yb.to(DEVICE)
            logits, gate = baseline_forward(model, xb, method=method)
            logits_eval = mask_logits_to_seen_classes(logits, seen)
            pred = logits_eval.argmax(dim=-1)
            correct += (pred == yb).sum().item()
            total_items += yb.numel()
            if gate is not None:
                gate_entropies.append(entropy_from_probs(gate).detach().cpu())
            if (yb == 2).any():
                m = (yb == 2)
                class2_margin_24.append((logits_eval[m, 2] - logits_eval[m, 4]).
    ↪detach().cpu())
                class2_rank.append(((logits_eval[m] > logits_eval[m, 2].
    ↪unsqueeze(-1)).sum(dim=-1) + 1).float().detach().cpu())
            if (yb == 5).any():
                m = (yb == 5)
                class5_margin_54.append((logits_eval[m, 5] - logits_eval[m, 4]).
    ↪detach().cpu())
                class5_rank.append(((logits_eval[m] > logits_eval[m, 5].
    ↪unsqueeze(-1)).sum(dim=-1) + 1).float().detach().cpu())
            rows.append({
                "method": method,
                "seed": seed,
                "model_family": model_family,

```

```

        "after_task": upto_task,
        "task_id": task_id,
        "task_name": tasks[task_id]["name"],
        "seen_classes": str(seen),
        "acc": correct / max(total_items, 1),
        "context_entropy": float(torch.cat(gate_entropies).mean()) if
↪gate_entropies else np.nan,
        "context_margin": np.nan,
        "context_top1_acc": np.nan,
        "context_brier": np.nan,
        "context_ece": np.nan,
        "latent_frechet_margin": np.nan,
        "prototype_weight": 0.0,
        "class2_margin_vs4": float(torch.cat(class2_margin_24).mean()) if
↪class2_margin_24 else np.nan,
        "class5_margin_vs4": float(torch.cat(class5_margin_54).mean()) if
↪class5_margin_54 else np.nan,
        "class2_logit_rank": float(torch.cat(class2_rank).mean()) if
↪class2_rank else np.nan,
        "class5_logit_rank": float(torch.cat(class5_rank).mean()) if
↪class5_rank else np.nan,
        "trainable_params": trainable,
        "total_params": total,
        "backbone_params": total,
        "readout_trainable_params": np.nan,
        "readout_total_params": np.nan,
        "model_size_mb_total": total * 4 / (1024 ** 2),
        "uses_balanced_replay": method == "baseline_balanced_replay",
        "uses_output_distill": method == "baseline_lwf",
        "uses_local_mycorrhizal_head": False,
        "uses_global_head_refresh": False,
        "uses_context_selection": False,
        "uses_oracle_context_reference": False,
        "probe_row": False,
        "backbone_source": "baseline_sequential_or_joint",
        "probe_feature": "baseline_logits",
        "probe_context_mode": "none",
        "replay_memory_items": replay_memory_items,
        "memory_per_class": MEMORY_PER_CLASS,
        "memory_per_task": MEMORY_PER_TASK,
        "paired_same_backbone": False,
        "readout_variant": "baseline",
        "context_variant": "none",
    })
    return pd.DataFrame(rows)

```

```
@torch.no_grad()
```



```

def class_confusion_matrix_standard(model, tasks, upto_task, method, seed):
    """Class-confusion table for standard baselines, normalized like the MMALS
    ↪ confusion table.

    Diagnostic patch:
    the original benchmark-harness smoke run wrote raw counts but did not write
    ↪ the
    per-true-class `rate` column. `class_diagnostics_from_confusion` expects
    ↪ this
    normalized field; without it, baseline class diagnostics become NaN/flat and
    cannot be compared fairly to MMALS rows.
    """
    model.eval()
    seen = seen_classes_until(upto_task)
    cm = torch.zeros((N_CLASSES, N_CLASSES), dtype=torch.long)
    for task_id in range(upto_task + 1):
        loader = make_loader(tasks[task_id]["test"], batch_size=BATCH_SIZE,
        ↪ shuffle=False)
        for xb, yb in loader:
            xb, yb = xb.to(DEVICE), yb.to(DEVICE)
            logits, _ = baseline_forward(model, xb, method=method)
            pred = mask_logits_to_seen_classes(logits, seen).argmax(dim=-1)
            for t, p in zip(yb.detach().cpu(), pred.detach().cpu()):
                cm[int(t), int(p)] += 1

    rows = []
    for true_cls in range(N_CLASSES):
        row_total = int(cm[true_cls].sum().item())
        for pred_cls in range(N_CLASSES):
            count = int(cm[true_cls, pred_cls].item())
            rows.append({
                "true_class": true_cls,
                "pred_class": pred_cls,
                "count": count,
                "rate": float(count / max(row_total, 1)),
                "method": method,
                "seed": seed,
                "after_task": upto_task,
                "backbone_source": "baseline_sequential_or_joint",
                "readout_variant": "baseline",
                "context_variant": "none",
                "uses_oracle_context_reference": False,
            })
    return pd.DataFrame(rows)

def ewc_compute_fisher(model, dataset, max_items=512):
    model.eval()

```

```

    params = {n: p.detach().clone() for n, p in model.named_parameters() if p.
↪requires_grad}
    fisher = {n: torch.zeros_like(p, device=DEVICE) for n, p in model.
↪named_parameters() if p.requires_grad}
    X, y = dataset.tensors
    if len(X) > max_items:
        idx = torch.randperm(len(X))[:max_items]
        dataset = TensorDataset(X[idx], y[idx])
    loader = make_loader(dataset, batch_size=BATCH_SIZE, shuffle=False)
    n_batches = 0
    for xb, yb in loader:
        xb, yb = xb.to(DEVICE), yb.to(DEVICE)
        model.zero_grad(set_to_none=True)
        logits, _ = baseline_forward(model, xb)
        loss = F.cross_entropy(logits, yb)
        loss.backward()
        for name, p in model.named_parameters():
            if p.grad is not None and name in fisher:
                fisher[name] += p.grad.detach().pow(2)
        n_batches += 1
    for name in fisher:
        fisher[name] = fisher[name] / max(n_batches, 1)
    return {"params": params, "fisher": fisher}

def ewc_penalty(model, ewc_snapshots):
    if not ewc_snapshots:
        return torch.zeros((), device=DEVICE)
    penalty = torch.zeros((), device=DEVICE)
    current = dict(model.named_parameters())
    for snap in ewc_snapshots:
        for name, old_param in snap["params"].items():
            if name in current:
                penalty = penalty + (snap["fisher"][name] * (current[name] -
↪old_param.to(DEVICE)).pow(2)).sum()
    return penalty

def train_sequential_baseline(method: str, seed: int):
    set_seed(seed)
    clear_memory()
    tasks = build_stress_tasks(DATASET_NAME, seed=seed)
    if method == "baseline_sparse_moe":
        model = SparseMoEClassifier().to(DEVICE)
        model_family = "sparse_moe"
    elif method == "baseline_pnn":
        model = PNNSimpleClassifier().to(DEVICE)
        model_family = "pnn_fixed_columns"
    else:

```

```

        model = MLPBaseline().to(DEVICE)
        model_family = "mlp_baseline"
        class_memory: Dict[int, Dict[str, torch.Tensor]] = {}
        memory_buffers: Dict[int, TensorDataset] = {}
        teacher = None
        ewc_snapshots = []
        history_rows, loss_rows, confusion_rows = [], [], []
        for task_id, task in enumerate(tasks):
            if method == "baseline_pnn":
                model.set_trainable_column(task_id)
                opt = torch.optim.Adam([p for p in model.parameters() if p.
↪requires_grad], lr=LR)
                loader = make_loader(task["train"], batch_size=BATCH_SIZE, shuffle=True)
                for epoch in range(EPOCHS_PER_TASK):
                    model.train()
                    sums = {"loss": 0.0, "task": 0.0, "replay": 0.0, "distill": 0.0,
↪"ewc": 0.0, "sparse": 0.0}
                    n_batches = 0
                    for xb, yb in loader:
                        xb, yb = xb.to(DEVICE), yb.to(DEVICE)
                        opt.zero_grad(set_to_none=True)
                        logits, gate = baseline_forward(model, xb, method=method)
                        task_loss = F.cross_entropy(logits, yb)
                        replay_loss = torch.zeros(), device=DEVICE
                        if method in ["baseline_experience_replay",
↪"baseline_balanced_replay"] and class_memory:
                            xb_r, yb_r, _tb =
↪sample_balanced_class_memory(class_memory, batch_size=BALANCED_REPLAY_BATCH)
                            if xb_r is not None:
                                logits_r, _ = baseline_forward(model, xb_r,
↪method=method)
                                replay_loss = F.cross_entropy(logits_r, yb_r) *
↪BASELINE_REPLAY_WEIGHT
                                distill_loss = torch.zeros(), device=DEVICE
                                if method == "baseline_lwf" and teacher is not None:
                                    with torch.no_grad():
                                        teacher_logits, _ = baseline_forward(teacher, xb,
↪method=method)
                                        distill_loss = LWF_LAMBDA * kl_distill_loss(logits,
↪teacher_logits)
                                ewc_loss = torch.zeros(), device=DEVICE
                                if method == "baseline_ewc" and ewc_snapshots:
                                    ewc_loss = EWC_LAMBDA * ewc_penalty(model, ewc_snapshots)
                                sparse_loss = torch.zeros(), device=DEVICE
                                if method == "baseline_sparse_moe" and gate is not None:

```

```

        sparse_loss = SPARSE_MOE_ENTROPY_WEIGHT *
entropy_from_probs(gate).mean()
        loss = task_loss + replay_loss + distill_loss + ewc_loss +
sparse_loss
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=5.0)
        opt.step()
        sums["loss"] += float(loss.detach().cpu())
        sums["task"] += float(task_loss.detach().cpu())
        sums["replay"] += float(replay_loss.detach().cpu())
        sums["distill"] += float(distill_loss.detach().cpu())
        sums["ewc"] += float(ewc_loss.detach().cpu())
        sums["sparse"] += float(sparse_loss.detach().cpu())
        n_batches += 1
    loss_rows.append({
        "method": method,
        "seed": seed,
        "task_id": task_id,
        "epoch": epoch,
        "loss": sums["loss"] / max(n_batches, 1),
        "task_loss": sums["task"] / max(n_batches, 1),
        "memory_loss": sums["distill"] / max(n_batches, 1),
        "context_loss": 0.0,
        "context_replay_loss": 0.0,
        "balanced_replay_loss": sums["replay"] / max(n_batches, 1),
        "fungal_loss": sums["sparse"] / max(n_batches, 1),
        "ewc_loss": sums["ewc"] / max(n_batches, 1),
    })
    eval_rows = evaluate_standard_model(model, tasks, task_id, method,
seed, model_family, replay_memory_items=int(sum(len(v["y"]) for v in
class_memory.values()))
    history_rows.append(eval_rows)
    print(f"[{method}] seed={seed}] after task {task_id}: {[round(a, 3) for
a in eval_rows['acc'].tolist()]}")
    class_memory = update_class_memory(class_memory, task["train"],
task_id, MEMORY_PER_CLASS, seed=seed + 3000 + task_id)
    memory_buffers[task_id] = build_memory_subset(task["train"],
MEMORY_PER_TASK, seed=seed + 4000 + task_id)
    if method == "baseline_lwf":
        teacher = clone_teacher(model)
    if method == "baseline_ewc":
        ewc_snapshots.append(ewc_compute_fisher(model, task["train"],
max_items=min(512, len(task["train"]))))
    clear_memory()
    confusion_rows.append(class_confusion_matrix_standard(model, tasks, N_TASKS,
- 1, method, seed))

```

```

        return pd.concat(history_rows, ignore_index=True), pd.DataFrame(loss_rows),
        pd.concat(confusion_rows, ignore_index=True)

def train_joint_upper_bound_baseline(seed: int):
    set_seed(seed)
    clear_memory()
    tasks = build_stress_tasks(DATASET_NAME, seed=seed)
    model = MLPBaseline().to(DEVICE)
    X_train = torch.cat([t["train"].tensors[0] for t in tasks], dim=0)
    y_train = torch.cat([t["train"].tensors[1] for t in tasks], dim=0)
    joint_ds = TensorDataset(X_train, y_train)
    opt = torch.optim.Adam(model.parameters(), lr=LR)
    # Same total training budget as one pass over all sequential tasks.
    for epoch in range(EPOCHS_PER_TASK * N_TASKS):
        model.train()
        for xb, yb in make_loader(joint_ds, batch_size=BATCH_SIZE,
        shuffle=True):
            xb, yb = xb.to(DEVICE), yb.to(DEVICE)
            opt.zero_grad(set_to_none=True)
            logits, _ = baseline_forward(model, xb)
            loss = F.cross_entropy(logits, yb)
            loss.backward()
            opt.step()
        method = "baseline_joint_upper_bound"
        h = evaluate_standard_model(model, tasks, N_TASKS - 1, method, seed,
        "joint_upper_bound", replay_memory_items=len(joint_ds))
        # Fill a final-only loss row; forgetting is not meaningful for joint upper
        bound.
        l = pd.DataFrame([{"method": method, "seed": seed, "task_id": N_TASKS - 1,
        "epoch": EPOCHS_PER_TASK * N_TASKS, "loss": np.nan, "task_loss": np.nan}])
        c = class_confusion_matrix_standard(model, tasks, N_TASKS - 1, method, seed)
        print(f"[{method}] seed={seed} final joint eval: {[round(a, 3) for a in
        h['acc'].tolist()]}")
        return h, l, c

def run_benchmark_baselines_for_seed(seed: int):
    method_map = {
        "finetune": "baseline_finetune",
        "ewc": "baseline_ewc",
        "lwf": "baseline_lwf",
        "experience_replay": "baseline_experience_replay",
        "balanced_replay": "baseline_balanced_replay",
        "sparse_moe": "baseline_sparse_moe",
        "pnn": "baseline_pnn",
        "joint_upper_bound": "baseline_joint_upper_bound",
    }

```

```

histories, losses, confusions = [], [], []
for short_name in SELECTED_BASELINE_METHODS:
    method = method_map[short_name]
    if method == "baseline_joint_upper_bound":
        h, l, c = train_joint_upper_bound_baseline(seed)
    else:
        h, l, c = train_sequential_baseline(method, seed)
    histories.append(h)
    losses.append(l)
    confusions.append(c)
return histories, losses, confusions

```

```

[23]: start_persistent_logging()
log_event("experiment_started", payload={
    "rc2c_candidate_variants": RC2B_CANDIDATE_VARIANTS,
    "baseline_methods": SELECTED_BASELINE_METHODS,
    "strict_no_final_test_selection": RC2B_STRICT_NO_FINAL_TEST_SELECTION,
    "experiment_profile": EXPERIMENT_PROFILE,
    "policy_branch": POLICY_BRANCH,
    "hydro_audit_enabled": RC3_HYDRO_AUDIT_ENABLED,
    "hydro_regularizer_enabled": RC3_HYDRO_REGULARIZER_ENABLED,
    "hydro_regularizer_mode": RC3_HYDRO_REGULARIZER_MODE,
    "hydro_selection_enabled": RC3_HYDRO_SELECTION_ENABLED,
    "hydro_policy_mode": RC3_HYDRO_POLICY_MODE,
})
experiment_t0 = time.time()

all_history, all_losses, all_confusions, all_probe_losses, all_repair_logs,
↳ all_selection_logs = [], [], [], [], [], []

for seed in SEEDS:
    seed_t0 = time.time()
    log_event("seed_started", seed=seed,
↳ phase="rc2c_risk_aware_base_proto_and_policy_selection")
    base_run = train_base_proto_first_checkpoints(seed)
    all_losses.append(base_run["loss_df"])

    for checkpoint in base_run["checkpoints"]:
        selected_h, selected_c, candidate_h, candidate_c, rlog, slog =
↳ apply_rc2b_strict_policy_to_checkpoint(base_run, checkpoint)
        all_history.append(selected_h)
        if not candidate_h.empty:
            all_history.append(candidate_h)
        if not selected_c.empty:
            all_confusions.append(selected_c)
        if not candidate_c.empty:
            all_confusions.append(candidate_c)

```

```

        if not rlog.empty:
            all_repair_logs.append(rlog)
        if not slog.empty:
            all_selection_logs.append(slog)
        log_event(
            "rc2b_policy_selected",
            seed=int(checkpoint["seed"]),
            task_id=int(checkpoint["after_task"]),
            method=RC2B_METHOD,
            phase="policy_selection_before_final_test",
            payload=slog[slog.get("is_selected_candidate", False) == True].
↳to_dict("records")[:1] if not slog.empty else {},
        )

    if RUN_FINAL_PROBES:
        artifact = final_probe_artifact_from_base_run(base_run)
        ph, pc, pl = run_representation_audit(artifact)
        all_history.append(ph)
        all_confusions.append(pc)
        all_probe_losses.append(pl)
        del artifact
    del base_run
    clear_memory()

    # Comparative baseline suite on the same seed and task construction.
    base_histories, base_losses, base_confusions = ◻
    ↳run_benchmark_baselines_for_seed(seed)
    all_history.extend(base_histories)
    all_losses.extend(base_losses)
    all_confusions.extend(base_confusions)
    clear_memory()
    log_event("seed_completed", seed=seed, ◻
    ↳phase="rc2c_risk_aware_base_proto_and_policy_selection", ◻
    ↳payload={"duration_sec": time.time() - seed_t0})

history_df = pd.concat(all_history, ignore_index=True) if all_history else pd.
↳DataFrame()
loss_df = pd.concat(all_losses, ignore_index=True) if all_losses else pd.
↳DataFrame()
repair_log_df = pd.concat(all_repair_logs, ignore_index=True) if ◻
↳all_repair_logs else pd.DataFrame()
selection_log_df = pd.concat(all_selection_logs, ignore_index=True) if ◻
↳all_selection_logs else pd.DataFrame()
confusion_df = pd.concat(all_confusions, ignore_index=True) if all_confusions ◻
↳else pd.DataFrame()

```

```

probe_loss_df = pd.concat(all_probe_losses, ignore_index=True) if
    ↪all_probe_losses else pd.DataFrame()

deployed_history = history_df[(history_df.probe_row == False)] if not
    ↪history_df.empty and "probe_row" in history_df.columns else history_df
forgetting_df = forgetting_from_history(deployed_history) if not
    ↪deployed_history.empty else pd.DataFrame()
class_diag_df = class_diagnostics_from_confusion(confusion_df) if not
    ↪confusion_df.empty else pd.DataFrame()

history_path = RESULTS_DIR / f"history_{RUN_MODE}.csv"
loss_path = RESULTS_DIR / f"losses_{RUN_MODE}.csv"
repair_log_path = RESULTS_DIR / f"context_gap_repair_logs_{RUN_MODE}.csv"
selection_log_path = RESULTS_DIR / f"rc2b_policy_selection_trace_{RUN_MODE}.csv"
probe_loss_path = RESULTS_DIR / f"probe_losses_{RUN_MODE}.csv"
forgetting_path = RESULTS_DIR / f"forgetting_{RUN_MODE}.csv"
confusion_path = RESULTS_DIR / f"class_confusion_{RUN_MODE}.csv"
class_diag_path = RESULTS_DIR / f"class_diagnostics_{RUN_MODE}.csv"

history_df.to_csv(history_path, index=False)
loss_df.to_csv(loss_path, index=False)
repair_log_df.to_csv(repair_log_path, index=False)
selection_log_df.to_csv(selection_log_path, index=False)
probe_loss_df.to_csv(probe_loss_path, index=False)
forgetting_df.to_csv(forgetting_path, index=False)
confusion_df.to_csv(confusion_path, index=False)
class_diag_df.to_csv(class_diag_path, index=False)

print("Saved:")
_saved_core_paths = [history_path, loss_path, repair_log_path,
    ↪selection_log_path, probe_loss_path, forgetting_path, confusion_path,
    ↪class_diag_path]
for p in _saved_core_paths:
    print(" -", p)
if not selection_log_df.empty:
    print("RC2b selected policy counts:")
    display(selection_log_df[selection_log_df["is_selected_candidate"] == True].
    ↪groupby(["selected_candidate_variant", "selected_policy_family"],
    ↪as_index=False).size())
log_event("experiment_core_results_saved", phase="export",
    ↪payload={"duration_sec": time.time() - experiment_t0, "files": [str(p) for p
    ↪in _saved_core_paths]})
# Keep logging active for interpretation/audit cells; it will be stopped in the
↪final packaging cell.

```

[base_proto seed=0] task=0 epoch=1/8 loss=1.9982 task=0.9697 memory=0.0000
 balanced=0.0000


```

[base_proto seed=0] task=0 epoch=2/8 loss=0.6132 task=0.3132 memory=0.0000
balanced=0.0000
[base_proto seed=0] task=0 epoch=3/8 loss=0.5025 task=0.2561 memory=0.0000
balanced=0.0000
[base_proto seed=0] task=0 epoch=4/8 loss=0.4533 task=0.2236 memory=0.0000
balanced=0.0000
[base_proto seed=0] task=0 epoch=5/8 loss=0.4301 task=0.2097 memory=0.0000
balanced=0.0000
[base_proto seed=0] task=0 epoch=6/8 loss=0.4155 task=0.2039 memory=0.0000
balanced=0.0000
[base_proto seed=0] task=0 epoch=7/8 loss=0.4028 task=0.2001 memory=0.0000
balanced=0.0000
[base_proto seed=0] task=0 epoch=8/8 loss=0.3904 task=0.1956 memory=0.0000
balanced=0.0000
[base_proto seed=0] calibrated context temperature after task 0: 0.25
[base_proto seed=0] task=1 epoch=1/8 loss=5.9971 task=1.6252 memory=0.6382
balanced=0.2744
[base_proto seed=0] task=1 epoch=2/8 loss=2.9186 task=0.5433 memory=0.2923
balanced=0.2405
[base_proto seed=0] task=1 epoch=3/8 loss=2.2425 task=0.3951 memory=0.1251
balanced=0.2533
[base_proto seed=0] task=1 epoch=4/8 loss=2.0118 task=0.3610 memory=0.0787
balanced=0.2713
[base_proto seed=0] task=1 epoch=5/8 loss=1.8399 task=0.3360 memory=0.0640
balanced=0.2382
[base_proto seed=0] task=1 epoch=6/8 loss=1.7259 task=0.3140 memory=0.0539
balanced=0.2068
[base_proto seed=0] task=1 epoch=7/8 loss=1.7102 task=0.3002 memory=0.0507
balanced=0.2410
[base_proto seed=0] task=1 epoch=8/8 loss=1.6178 task=0.2954 memory=0.0513
balanced=0.2003
[base_proto seed=0] calibrated context temperature after task 1: 0.80
[base_proto seed=0] task=2 epoch=1/8 loss=5.3350 task=1.6519 memory=0.4755
balanced=0.3255
[base_proto seed=0] task=2 epoch=2/8 loss=3.2955 task=0.7741 memory=0.3994
balanced=0.3150
[base_proto seed=0] task=2 epoch=3/8 loss=2.5978 task=0.5426 memory=0.2422
balanced=0.3121
[base_proto seed=0] task=2 epoch=4/8 loss=2.2258 task=0.4486 memory=0.1834
balanced=0.2762
[base_proto seed=0] task=2 epoch=5/8 loss=1.9632 task=0.3754 memory=0.1405
balanced=0.2486
[base_proto seed=0] task=2 epoch=6/8 loss=1.7562 task=0.3247 memory=0.1009
balanced=0.1893
[base_proto seed=0] task=2 epoch=7/8 loss=1.6682 task=0.2943 memory=0.0726
balanced=0.1909
[base_proto seed=0] task=2 epoch=8/8 loss=1.5694 task=0.2694 memory=0.0662
balanced=0.1915

```

```

[base_proto seed=0] calibrated context temperature after task 2: 0.60
[base_proto seed=0] task=3 epoch=1/8 loss=5.9942 task=2.2293 memory=0.9879
balanced=0.3108
[base_proto seed=0] task=3 epoch=2/8 loss=3.7559 task=1.1203 memory=0.7505
balanced=0.2424
[base_proto seed=0] task=3 epoch=3/8 loss=3.0703 task=0.7419 memory=0.6397
balanced=0.2663
[base_proto seed=0] task=3 epoch=4/8 loss=2.7647 task=0.6134 memory=0.6053
balanced=0.2316
[base_proto seed=0] task=3 epoch=5/8 loss=2.5127 task=0.5351 memory=0.5091
balanced=0.2188
[base_proto seed=0] task=3 epoch=6/8 loss=2.3124 task=0.4665 memory=0.4255
balanced=0.2215
[base_proto seed=0] task=3 epoch=7/8 loss=2.1253 task=0.4222 memory=0.3339
balanced=0.2148
[base_proto seed=0] task=3 epoch=8/8 loss=1.8526 task=0.3368 memory=0.1974
balanced=0.1838
[base_proto seed=0] calibrated context temperature after task 3: 0.60
[base_proto seed=0] task=4 epoch=1/8 loss=5.0498 task=1.4399 memory=0.9929
balanced=0.8842
[base_proto seed=0] task=4 epoch=2/8 loss=2.8231 task=0.3599 memory=0.4674
balanced=0.6119
[base_proto seed=0] task=4 epoch=3/8 loss=2.3274 task=0.2916 memory=0.2974
balanced=0.5459
[base_proto seed=0] task=4 epoch=4/8 loss=2.1113 task=0.2447 memory=0.2426
balanced=0.5349
[base_proto seed=0] task=4 epoch=5/8 loss=1.9702 task=0.2078 memory=0.2712
balanced=0.4708
[base_proto seed=0] task=4 epoch=6/8 loss=1.8782 task=0.1803 memory=0.2914
balanced=0.4339
[base_proto seed=0] task=4 epoch=7/8 loss=1.8058 task=0.1713 memory=0.2846
balanced=0.3966
[base_proto seed=0] task=4 epoch=8/8 loss=1.8883 task=0.1852 memory=0.3374
balanced=0.4462
[base_proto seed=0] calibrated context temperature after task 4: 0.60
[base_proto seed=0] completed in 112.0s
[RC2h seed=0] after task 0: selected context_gap_selected proxy_acc=0.9070
score=1.2123
[RC2h seed=0] after task 1: selected context_gap_selected proxy_acc=0.9271
score=1.8644
[RC2h seed=0] after task 2: selected context_gap_selected proxy_acc=0.9227
score=1.8081
[RC2h seed=0] after task 3: selected context_blend_selected_global
proxy_acc=0.9091 score=1.6905
[RC2h seed=0] after task 4: selected context_gap_selected proxy_acc=0.9182
score=0.4770
[probe_proto__zf_proto seed=0] probe final task accs: [0.95, 0.925, 0.89, 0.838,
0.887]

```

[probe_proto__zf_oracle seed=0] probe final task accs: [0.953, 0.939, 0.938, 0.891, 0.948]
 [probe_proto__transformed0_oracle seed=0] host probe accs: [0.43, 0.429, 0.104, 0.58, 0.905]
 [probe_proto__transformed1_oracle seed=0] host probe accs: [0.439, 0.615, 0.879, 0.417, 0.366]
 [probe_proto__transformed2_oracle seed=0] host probe accs: [0.764, 0.964, 0.494, 0.019, 0.347]
 [probe_proto__transformed3_oracle seed=0] host probe accs: [0.904, 0.86, 0.777, 0.85, 0.887]
 [probe_proto__transformed4_oracle seed=0] host probe accs: [0.854, 0.777, 0.47, 0.376, 0.723]
 [baseline_finetune seed=0] after task 0: [0.981]
 [baseline_finetune seed=0] after task 1: [0.486, 0.99]
 [baseline_finetune seed=0] after task 2: [0.0, 0.487, 0.961]
 [baseline_finetune seed=0] after task 3: [0.0, 0.0, 0.477, 0.941]
 [baseline_finetune seed=0] after task 4: [0.0, 0.0, 0.0, 0.5, 1.0]
 [baseline_ewc seed=0] after task 0: [0.981]
 [baseline_ewc seed=0] after task 1: [0.489, 0.989]
 [baseline_ewc seed=0] after task 2: [0.0, 0.487, 0.964]
 [baseline_ewc seed=0] after task 3: [0.0, 0.0, 0.484, 0.944]
 [baseline_ewc seed=0] after task 4: [0.0, 0.0, 0.0, 0.5, 1.0]
 [baseline_lwf seed=0] after task 0: [0.981]
 [baseline_lwf seed=0] after task 1: [0.96, 0.515]
 [baseline_lwf seed=0] after task 2: [0.907, 0.974, 0.502]
 [baseline_lwf seed=0] after task 3: [0.264, 0.706, 0.92, 0.554]
 [baseline_lwf seed=0] after task 4: [0.059, 0.059, 0.39, 0.881, 0.985]
 [baseline_experience_replay seed=0] after task 0: [0.981]
 [baseline_experience_replay seed=0] after task 1: [0.961, 0.973]
 [baseline_experience_replay seed=0] after task 2: [0.884, 0.955, 0.922]
 [baseline_experience_replay seed=0] after task 3: [0.876, 0.816, 0.796, 0.897]
 [baseline_experience_replay seed=0] after task 4: [0.915, 0.776, 0.681, 0.841, 0.968]
 [baseline_balanced_replay seed=0] after task 0: [0.981]
 [baseline_balanced_replay seed=0] after task 1: [0.961, 0.973]
 [baseline_balanced_replay seed=0] after task 2: [0.884, 0.955, 0.922]
 [baseline_balanced_replay seed=0] after task 3: [0.876, 0.816, 0.796, 0.897]
 [baseline_balanced_replay seed=0] after task 4: [0.915, 0.776, 0.681, 0.841, 0.968]
 [baseline_sparse_moe seed=0] after task 0: [0.978]
 [baseline_sparse_moe seed=0] after task 1: [0.485, 0.99]
 [baseline_sparse_moe seed=0] after task 2: [0.0, 0.487, 0.964]
 [baseline_sparse_moe seed=0] after task 3: [0.0, 0.0, 0.458, 0.945]
 [baseline_sparse_moe seed=0] after task 4: [0.0, 0.0, 0.0, 0.5, 0.999]
 [baseline_pnn seed=0] after task 0: [0.988]
 [baseline_pnn seed=0] after task 1: [0.481, 0.984]
 [baseline_pnn seed=0] after task 2: [0.0, 0.486, 0.96]
 [baseline_pnn seed=0] after task 3: [0.0, 0.0, 0.492, 0.924]

```

[baseline_pnn seed=0] after task 4: [0.0, 0.0, 0.0, 0.496, 0.998]
[baseline_joint_upper_bound seed=0] final joint eval: [0.909, 0.81, 0.8, 0.89, 0.925]
[base_proto seed=1] task=0 epoch=1/8 loss=2.2393 task=0.9040 memory=0.0000
balanced=0.0000
[base_proto seed=1] task=0 epoch=2/8 loss=0.6105 task=0.2822 memory=0.0000
balanced=0.0000
[base_proto seed=1] task=0 epoch=3/8 loss=0.4889 task=0.2299 memory=0.0000
balanced=0.0000
[base_proto seed=1] task=0 epoch=4/8 loss=0.4348 task=0.1996 memory=0.0000
balanced=0.0000
[base_proto seed=1] task=0 epoch=5/8 loss=0.4193 task=0.1959 memory=0.0000
balanced=0.0000
[base_proto seed=1] task=0 epoch=6/8 loss=0.3992 task=0.1845 memory=0.0000
balanced=0.0000
[base_proto seed=1] task=0 epoch=7/8 loss=0.3903 task=0.1841 memory=0.0000
balanced=0.0000
[base_proto seed=1] task=0 epoch=8/8 loss=0.3758 task=0.1762 memory=0.0000
balanced=0.0000
[base_proto seed=1] calibrated context temperature after task 0: 0.25
[base_proto seed=1] task=1 epoch=1/8 loss=6.4131 task=1.4879 memory=0.5950
balanced=0.2143
[base_proto seed=1] task=1 epoch=2/8 loss=3.4551 task=0.5879 memory=0.2649
balanced=0.2296
[base_proto seed=1] task=1 epoch=3/8 loss=2.7260 task=0.4014 memory=0.1468
balanced=0.1950
[base_proto seed=1] task=1 epoch=4/8 loss=2.3926 task=0.3516 memory=0.0928
balanced=0.1764
[base_proto seed=1] task=1 epoch=5/8 loss=2.1680 task=0.3274 memory=0.0587
balanced=0.1632
[base_proto seed=1] task=1 epoch=6/8 loss=2.0638 task=0.3085 memory=0.0498
balanced=0.1660
[base_proto seed=1] task=1 epoch=7/8 loss=1.9539 task=0.2911 memory=0.0381
balanced=0.1495
[base_proto seed=1] task=1 epoch=8/8 loss=1.9021 task=0.2789 memory=0.0366
balanced=0.1377
[base_proto seed=1] calibrated context temperature after task 1: 0.80
[base_proto seed=1] task=2 epoch=1/8 loss=5.4684 task=1.3871 memory=0.4438
balanced=0.2626
[base_proto seed=1] task=2 epoch=2/8 loss=3.2251 task=0.5001 memory=0.2614
balanced=0.2634
[base_proto seed=1] task=2 epoch=3/8 loss=2.4979 task=0.3902 memory=0.1352
balanced=0.2168
[base_proto seed=1] task=2 epoch=4/8 loss=2.1944 task=0.3487 memory=0.1107
balanced=0.2006
[base_proto seed=1] task=2 epoch=5/8 loss=2.0026 task=0.3218 memory=0.0961
balanced=0.1853
[base_proto seed=1] task=2 epoch=6/8 loss=1.8655 task=0.2987 memory=0.0897

```

```

balanced=0.1690
[base_proto seed=1] task=2 epoch=7/8 loss=1.7479 task=0.2887 memory=0.0800
balanced=0.1575
[base_proto seed=1] task=2 epoch=8/8 loss=1.6393 task=0.2727 memory=0.0659
balanced=0.1362
[base_proto seed=1] calibrated context temperature after task 2: 0.60
[base_proto seed=1] task=3 epoch=1/8 loss=5.8703 task=2.0881 memory=0.8630
balanced=0.3139
[base_proto seed=1] task=3 epoch=2/8 loss=3.8781 task=0.9884 memory=0.7426
balanced=0.2900
[base_proto seed=1] task=3 epoch=3/8 loss=3.1271 task=0.7831 memory=0.5688
balanced=0.2796
[base_proto seed=1] task=3 epoch=4/8 loss=2.6403 task=0.6407 memory=0.3932
balanced=0.2558
[base_proto seed=1] task=3 epoch=5/8 loss=2.3372 task=0.5388 memory=0.3068
balanced=0.2319
[base_proto seed=1] task=3 epoch=6/8 loss=2.0647 task=0.4351 memory=0.2312
balanced=0.1936
[base_proto seed=1] task=3 epoch=7/8 loss=1.8675 task=0.3628 memory=0.1876
balanced=0.1611
[base_proto seed=1] task=3 epoch=8/8 loss=1.8380 task=0.3501 memory=0.1889
balanced=0.1864
[base_proto seed=1] calibrated context temperature after task 3: 0.60
[base_proto seed=1] task=4 epoch=1/8 loss=6.1489 task=2.0596 memory=1.3276
balanced=0.7026
[base_proto seed=1] task=4 epoch=2/8 loss=3.1651 task=0.8911 memory=0.3479
balanced=0.3344
[base_proto seed=1] task=4 epoch=3/8 loss=2.3032 task=0.3683 memory=0.2112
balanced=0.3196
[base_proto seed=1] task=4 epoch=4/8 loss=2.0139 task=0.2919 memory=0.1234
balanced=0.2997
[base_proto seed=1] task=4 epoch=5/8 loss=1.9054 task=0.2510 memory=0.1493
balanced=0.2967
[base_proto seed=1] task=4 epoch=6/8 loss=1.7252 task=0.2287 memory=0.1162
balanced=0.2437
[base_proto seed=1] task=4 epoch=7/8 loss=1.6460 task=0.2141 memory=0.1287
balanced=0.2256
[base_proto seed=1] task=4 epoch=8/8 loss=1.5620 task=0.2102 memory=0.1006
balanced=0.2204
[base_proto seed=1] calibrated context temperature after task 4: 0.80
[base_proto seed=1] completed in 108.4s
[RC2h seed=1] after task 0: selected context_gap_selected proxy_acc=0.9301
score=1.2461
[RC2h seed=1] after task 1: selected context_gap_selected proxy_acc=0.9263
score=1.8712
[RC2h seed=1] after task 2: selected context_gap_selected proxy_acc=0.9227
score=1.7687
[RC2h seed=1] after task 3: selected context_gap_selected proxy_acc=0.9353

```

score=1.7884
 [RC2h seed=1] after task 4: selected context_gap_selected proxy_acc=0.9582
 score=1.6687
 [probe_proto__zf_proto seed=1] probe final task accs: [0.973, 0.926, 0.882, 0.927, 0.927]
 [probe_proto__zf_oracle seed=1] probe final task accs: [0.97, 0.943, 0.948, 0.931, 0.926]
 [probe_proto__transformed0_oracle seed=1] host probe accs: [0.667, 0.682, 0.851, 0.421, 0.501]
 [probe_proto__transformed1_oracle seed=1] host probe accs: [0.631, 0.843, 0.705, 0.399, 0.608]
 [probe_proto__transformed2_oracle seed=1] host probe accs: [0.754, 0.566, 0.372, 0.704, 0.944]
 [probe_proto__transformed3_oracle seed=1] host probe accs: [0.824, 0.843, 0.838, 0.833, 0.909]
 [probe_proto__transformed4_oracle seed=1] host probe accs: [0.757, 0.805, 0.341, 0.029, 0.522]
 [baseline_finetune seed=1] after task 0: [0.98]
 [baseline_finetune seed=1] after task 1: [0.487, 0.985]
 [baseline_finetune seed=1] after task 2: [0.0, 0.477, 0.961]
 [baseline_finetune seed=1] after task 3: [0.0, 0.0, 0.464, 0.943]
 [baseline_finetune seed=1] after task 4: [0.0, 0.0, 0.0, 0.5, 0.999]
 [baseline_ewc seed=1] after task 0: [0.98]
 [baseline_ewc seed=1] after task 1: [0.492, 0.978]
 [baseline_ewc seed=1] after task 2: [0.0, 0.484, 0.956]
 [baseline_ewc seed=1] after task 3: [0.0, 0.0, 0.463, 0.95]
 [baseline_ewc seed=1] after task 4: [0.0, 0.0, 0.0, 0.5, 0.999]
 [baseline_lwf seed=1] after task 0: [0.98]
 [baseline_lwf seed=1] after task 1: [0.97, 0.505]
 [baseline_lwf seed=1] after task 2: [0.925, 0.954, 0.52]
 [baseline_lwf seed=1] after task 3: [0.245, 0.62, 0.9, 0.614]
 [baseline_lwf seed=1] after task 4: [0.0, 0.0, 0.315, 0.814, 0.985]
 [baseline_experience_replay seed=1] after task 0: [0.98]
 [baseline_experience_replay seed=1] after task 1: [0.945, 0.966]
 [baseline_experience_replay seed=1] after task 2: [0.894, 0.927, 0.925]
 [baseline_experience_replay seed=1] after task 3: [0.87, 0.779, 0.802, 0.891]
 [baseline_experience_replay seed=1] after task 4: [0.919, 0.767, 0.65, 0.821, 0.969]
 [baseline_balanced_replay seed=1] after task 0: [0.98]
 [baseline_balanced_replay seed=1] after task 1: [0.945, 0.966]
 [baseline_balanced_replay seed=1] after task 2: [0.894, 0.927, 0.925]
 [baseline_balanced_replay seed=1] after task 3: [0.87, 0.779, 0.802, 0.891]
 [baseline_balanced_replay seed=1] after task 4: [0.919, 0.767, 0.65, 0.821, 0.969]
 [baseline_sparse_moe seed=1] after task 0: [0.975]
 [baseline_sparse_moe seed=1] after task 1: [0.492, 0.984]
 [baseline_sparse_moe seed=1] after task 2: [0.0, 0.486, 0.956]
 [baseline_sparse_moe seed=1] after task 3: [0.0, 0.0, 0.446, 0.94]

```

[baseline_sparse_moe seed=1] after task 4: [0.0, 0.0, 0.0, 0.5, 1.0]
[baseline_pnn seed=1] after task 0: [0.976]
[baseline_pnn seed=1] after task 1: [0.487, 0.988]
[baseline_pnn seed=1] after task 2: [0.0, 0.486, 0.956]
[baseline_pnn seed=1] after task 3: [0.0, 0.0, 0.469, 0.948]
[baseline_pnn seed=1] after task 4: [0.0, 0.0, 0.0, 0.5, 1.0]
[baseline_joint_upper_bound seed=1] final joint eval: [0.915, 0.886, 0.863, 0.871, 0.881]
[base_proto seed=2] task=0 epoch=1/8 loss=2.0397 task=0.9444 memory=0.0000
balanced=0.0000
[base_proto seed=2] task=0 epoch=2/8 loss=0.6053 task=0.3010 memory=0.0000
balanced=0.0000
[base_proto seed=2] task=0 epoch=3/8 loss=0.4877 task=0.2358 memory=0.0000
balanced=0.0000
[base_proto seed=2] task=0 epoch=4/8 loss=0.4408 task=0.2075 memory=0.0000
balanced=0.0000
[base_proto seed=2] task=0 epoch=5/8 loss=0.4283 task=0.2043 memory=0.0000
balanced=0.0000
[base_proto seed=2] task=0 epoch=6/8 loss=0.4158 task=0.2018 memory=0.0000
balanced=0.0000
[base_proto seed=2] task=0 epoch=7/8 loss=0.3971 task=0.1905 memory=0.0000
balanced=0.0000
[base_proto seed=2] task=0 epoch=8/8 loss=0.3865 task=0.1881 memory=0.0000
balanced=0.0000
[base_proto seed=2] calibrated context temperature after task 0: 0.25
[base_proto seed=2] task=1 epoch=1/8 loss=6.5111 task=1.5935 memory=0.6193
balanced=0.2447
[base_proto seed=2] task=1 epoch=2/8 loss=3.2524 task=0.4515 memory=0.2993
balanced=0.2131
[base_proto seed=2] task=1 epoch=3/8 loss=2.3255 task=0.3479 memory=0.1581
balanced=0.2302
[base_proto seed=2] task=1 epoch=4/8 loss=1.9621 task=0.3314 memory=0.1143
balanced=0.2331
[base_proto seed=2] task=1 epoch=5/8 loss=1.8229 task=0.3097 memory=0.0755
balanced=0.2259
[base_proto seed=2] task=1 epoch=6/8 loss=1.7437 task=0.2865 memory=0.0673
balanced=0.2095
[base_proto seed=2] task=1 epoch=7/8 loss=1.6430 task=0.2665 memory=0.0567
balanced=0.2045
[base_proto seed=2] task=1 epoch=8/8 loss=1.5740 task=0.2520 memory=0.0541
balanced=0.1922
[base_proto seed=2] calibrated context temperature after task 1: 0.80
[base_proto seed=2] task=2 epoch=1/8 loss=5.1648 task=1.5308 memory=0.3621
balanced=0.2744
[base_proto seed=2] task=2 epoch=2/8 loss=3.2792 task=0.7341 memory=0.3468
balanced=0.2518
[base_proto seed=2] task=2 epoch=3/8 loss=2.5938 task=0.4574 memory=0.2248
balanced=0.2616

```

```

[base_proto seed=2] task=2 epoch=4/8 loss=2.2123 task=0.3739 memory=0.1486
balanced=0.2324
[base_proto seed=2] task=2 epoch=5/8 loss=1.9369 task=0.3027 memory=0.0987
balanced=0.2083
[base_proto seed=2] task=2 epoch=6/8 loss=1.7707 task=0.2716 memory=0.0746
balanced=0.1881
[base_proto seed=2] task=2 epoch=7/8 loss=1.6927 task=0.2569 memory=0.0581
balanced=0.1891
[base_proto seed=2] task=2 epoch=8/8 loss=1.6186 task=0.2475 memory=0.0530
balanced=0.1858
[base_proto seed=2] calibrated context temperature after task 2: 0.80
[base_proto seed=2] task=3 epoch=1/8 loss=6.5400 task=2.3503 memory=1.2019
balanced=0.3230
[base_proto seed=2] task=3 epoch=2/8 loss=3.8151 task=1.1209 memory=0.7011
balanced=0.2658
[base_proto seed=2] task=3 epoch=3/8 loss=3.3978 task=0.7340 memory=0.9132
balanced=0.2637
[base_proto seed=2] task=3 epoch=4/8 loss=2.7345 task=0.5842 memory=0.5305
balanced=0.2411
[base_proto seed=2] task=3 epoch=5/8 loss=2.3220 task=0.4436 memory=0.3440
balanced=0.2225
[base_proto seed=2] task=3 epoch=6/8 loss=1.9955 task=0.3611 memory=0.1516
balanced=0.2015
[base_proto seed=2] task=3 epoch=7/8 loss=1.8150 task=0.3092 memory=0.0994
balanced=0.1767
[base_proto seed=2] task=3 epoch=8/8 loss=1.7100 task=0.2846 memory=0.0802
balanced=0.1586
[base_proto seed=2] calibrated context temperature after task 3: 0.60
[base_proto seed=2] task=4 epoch=1/8 loss=5.7083 task=1.8077 memory=1.1514
balanced=0.8231
[base_proto seed=2] task=4 epoch=2/8 loss=3.1037 task=0.5079 memory=0.3940
balanced=0.6816
[base_proto seed=2] task=4 epoch=3/8 loss=2.4252 task=0.3423 memory=0.2912
balanced=0.5081
[base_proto seed=2] task=4 epoch=4/8 loss=2.4543 task=0.3131 memory=0.4462
balanced=0.5333
[base_proto seed=2] task=4 epoch=5/8 loss=2.6927 task=0.3115 memory=0.7129
balanced=0.5811
[base_proto seed=2] task=4 epoch=6/8 loss=2.1035 task=0.2729 memory=0.3112
balanced=0.4826
[base_proto seed=2] task=4 epoch=7/8 loss=1.9708 task=0.2588 memory=0.2409
balanced=0.4770
[base_proto seed=2] task=4 epoch=8/8 loss=1.9212 task=0.2367 memory=0.3009
balanced=0.4324
[base_proto seed=2] calibrated context temperature after task 4: 0.80
[base_proto seed=2] completed in 108.6s
[RC2h seed=2] after task 0: selected context_gap_selected proxy_acc=0.9574
score=1.2822

```


[RC2h seed=2] after task 1: selected context_gap_selected proxy_acc=0.9401
 score=1.8789
 [RC2h seed=2] after task 2: selected context_gap_selected proxy_acc=0.9270
 score=1.8237
 [RC2h seed=2] after task 3: selected context_gap_selected proxy_acc=0.8641
 score=1.3651
 [RC2h seed=2] after task 4: selected context_temp_blend_selected_global
 proxy_acc=0.9553 score=1.8036
 [probe_proto__zf_proto seed=2] probe final task accs: [0.974, 0.971, 0.889,
 0.909, 0.95]
 [probe_proto__zf_oracle seed=2] probe final task accs: [0.976, 0.932, 0.958,
 0.946, 0.961]
 [probe_proto__transformed0_oracle seed=2] host probe accs: [0.615, 0.721, 0.766,
 0.654, 0.74]
 [probe_proto__transformed1_oracle seed=2] host probe accs: [0.818, 0.802, 0.745,
 0.64, 0.699]
 [probe_proto__transformed2_oracle seed=2] host probe accs: [0.873, 0.891, 0.816,
 0.776, 0.89]
 [probe_proto__transformed3_oracle seed=2] host probe accs: [0.825, 0.632, 0.421,
 0.584, 0.89]
 [probe_proto__transformed4_oracle seed=2] host probe accs: [0.549, 0.723, 0.874,
 0.544, 0.55]
 [baseline_finetune seed=2] after task 0: [0.98]
 [baseline_finetune seed=2] after task 1: [0.497, 0.983]
 [baseline_finetune seed=2] after task 2: [0.0, 0.492, 0.97]
 [baseline_finetune seed=2] after task 3: [0.0, 0.0, 0.468, 0.931]
 [baseline_finetune seed=2] after task 4: [0.0, 0.0, 0.0, 0.5, 0.999]
 [baseline_ewc seed=2] after task 0: [0.98]
 [baseline_ewc seed=2] after task 1: [0.495, 0.984]
 [baseline_ewc seed=2] after task 2: [0.0, 0.489, 0.974]
 [baseline_ewc seed=2] after task 3: [0.0, 0.0, 0.474, 0.948]
 [baseline_ewc seed=2] after task 4: [0.0, 0.0, 0.0, 0.5, 0.998]
 [baseline_lwf seed=2] after task 0: [0.98]
 [baseline_lwf seed=2] after task 1: [0.963, 0.516]
 [baseline_lwf seed=2] after task 2: [0.89, 0.974, 0.522]
 [baseline_lwf seed=2] after task 3: [0.305, 0.634, 0.922, 0.672]
 [baseline_lwf seed=2] after task 4: [0.022, 0.0, 0.324, 0.824, 0.991]
 [baseline_experience_replay seed=2] after task 0: [0.98]
 [baseline_experience_replay seed=2] after task 1: [0.951, 0.978]
 [baseline_experience_replay seed=2] after task 2: [0.891, 0.956, 0.939]
 [baseline_experience_replay seed=2] after task 3: [0.895, 0.831, 0.839, 0.853]
 [baseline_experience_replay seed=2] after task 4: [0.896, 0.748, 0.72, 0.901,
 0.969]
 [baseline_balanced_replay seed=2] after task 0: [0.98]
 [baseline_balanced_replay seed=2] after task 1: [0.951, 0.978]
 [baseline_balanced_replay seed=2] after task 2: [0.891, 0.956, 0.939]
 [baseline_balanced_replay seed=2] after task 3: [0.895, 0.831, 0.839, 0.853]
 [baseline_balanced_replay seed=2] after task 4: [0.896, 0.748, 0.72, 0.901,

0.969]

[baseline_sparse_moe seed=2] after task 0: [0.984]

[baseline_sparse_moe seed=2] after task 1: [0.495, 0.983]

[baseline_sparse_moe seed=2] after task 2: [0.0, 0.482, 0.968]

[baseline_sparse_moe seed=2] after task 3: [0.0, 0.0, 0.479, 0.943]

[baseline_sparse_moe seed=2] after task 4: [0.0, 0.0, 0.0, 0.5, 0.999]

[baseline_pnn seed=2] after task 0: [0.98]

[baseline_pnn seed=2] after task 1: [0.497, 0.984]

[baseline_pnn seed=2] after task 2: [0.0, 0.491, 0.969]

[baseline_pnn seed=2] after task 3: [0.0, 0.0, 0.499, 0.881]

[baseline_pnn seed=2] after task 4: [0.0, 0.0, 0.0, 0.5, 0.99]

[baseline_joint_upper_bound seed=2] final joint eval: [0.938, 0.911, 0.845, 0.811, 0.885]

Saved:

- /content/drive/MyDrive/MMALS/v1_1_RC2h_rc1b_anchored_conservative_selector/RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT/MMALS_v11_RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT_FashionMNIST_evidence_20260526T183427Z_seeds3/history_evidence.csv
- /content/drive/MyDrive/MMALS/v1_1_RC2h_rc1b_anchored_conservative_selector/RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT/MMALS_v11_RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT_FashionMNIST_evidence_20260526T183427Z_seeds3/losses_evidence.csv
- /content/drive/MyDrive/MMALS/v1_1_RC2h_rc1b_anchored_conservative_selector/RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT/MMALS_v11_RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT_FashionMNIST_evidence_20260526T183427Z_seeds3/context_gap_repair_logs_evidence.csv
- /content/drive/MyDrive/MMALS/v1_1_RC2h_rc1b_anchored_conservative_selector/RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT/MMALS_v11_RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT_FashionMNIST_evidence_20260526T183427Z_seeds3/rc2b_policy_selection_trace_evidence.csv
- /content/drive/MyDrive/MMALS/v1_1_RC2h_rc1b_anchored_conservative_selector/RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT/MMALS_v11_RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT_FashionMNIST_evidence_20260526T183427Z_seeds3/probe_losses_evidence.csv
- /content/drive/MyDrive/MMALS/v1_1_RC2h_rc1b_anchored_conservative_selector/RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT/MMALS_v11_RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT_FashionMNIST_evidence_20260526T183427Z_seeds3/forgetting_evidence.csv
- /content/drive/MyDrive/MMALS/v1_1_RC2h_rc1b_anchored_conservative_selector/RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT/MMALS_v11_RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT_FashionMNIST_evidence_20260526T183427Z_seeds3/class_confusion_evidence.csv
- /content/drive/MyDrive/MMALS/v1_1_RC2h_rc1b_anchored_conservative_selector/RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT/MMALS_v11_RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT_FashionMNIST_evidence_20260526T183427Z_seeds3/class_diagnostics_evidence.csv

RC2b selected policy counts:

	selected_candidate_variant	selected_policy_family	size
0	context_blend_selected_global	context_only_global	1
1	context_gap_selected	rc1b_guarded_context_gap	13
2	context_temp_blend_selected_global	context_only_global	1

```
[23]: {'trace_id': '49d3678f2bbf0046',
      'run_id': 'MMALS_v11_RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT_FashionMNIST_evidence_20260526T183427Z_seeds3',
      'timestamp_utc': '2026-05-26T18:51:53.386642+00:00',
      'level': 'INFO',
      'event_type': 'experiment_core_results_saved',
      'dataset': 'FashionMNIST',
      'run_mode': 'evidence',
      'seed': None,
      'task_id': None,
      'method': None,
      'variant': None,
      'phase': 'export',
      'config_hash':
'7e545c393f83086ae0b9e2a6b04aca0d5048aee8de5aa098bcf789226705a2a5',
      'payload': {'duration_sec': 998.327232837677,
      'files': ['/content/drive/MyDrive/MMALS/v1_1_RC2h_rc1b_anchored_conservative_selector/RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT/MMALS_v11_RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT_FashionMNIST_evidence_20260526T183427Z_seeds3/history_evidence.csv',
      '/content/drive/MyDrive/MMALS/v1_1_RC2h_rc1b_anchored_conservative_selector/RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT/MMALS_v11_RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT_FashionMNIST_evidence_20260526T183427Z_seeds3/losses_evidence.csv',
      '/content/drive/MyDrive/MMALS/v1_1_RC2h_rc1b_anchored_conservative_selector/RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT/MMALS_v11_RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT_FashionMNIST_evidence_20260526T183427Z_seeds3/context_gap_repair_logs_evidence.csv',
      '/content/drive/MyDrive/MMALS/v1_1_RC2h_rc1b_anchored_conservative_selector/RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT/MMALS_v11_RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT_FashionMNIST_evidence_20260526T183427Z_seeds3/rc2b_policy_selection_trace_evidence.csv',
      '/content/drive/MyDrive/MMALS/v1_1_RC2h_rc1b_anchored_conservative_selector/RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT/MMALS_v11_RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT_FashionMNIST_evidence_20260526T183427Z_seeds3/probe_losses_evidence.csv',
      '/content/drive/MyDrive/MMALS/v1_1_RC2h_rc1b_anchored_conservative_selector/RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT/MMALS_v11_RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT_FashionMNIST_evidence_20260526T183427Z_seeds3/forgetting_evidence.csv',
      '/content/drive/MyDrive/MMALS/v1_1_RC2h_rc1b_anchored_conservative_selector/RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT/MMALS_v11_RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT_FashionMNIST_evidence_20260526T183427Z_seeds3/forgetting_evidence.csv']
    }
```

```
TIVE_HYDRO_AUDIT_FashionMNIST_evidence_20260526T183427Z_seeds3/class_confusion_evidence.csv',
    '/content/drive/MyDrive/MMALS/v1_1_RC2h_rc1b_anchored_conservative_selector/R_C2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT/MMALS_v11_RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT_FashionMNIST_evidence_20260526T183427Z_seeds3/class_diagnostics_evidence.csv']}]}
```

1.21 13. Scorecards and diagnostic tables

The central comparison is paired delta versus paired_proto_global_no_repair and versus the corrected paired_proto_global_head_ce_kl_guard_035, not independent model-vs-model training.

```
[24]: def final_scorecard(history_df, forgetting_df, confusion_df=None):
    final_task = int(history_df.after_task.max())
    final_hist = history_df[history_df.after_task == final_task]
    final_forgetting = forgetting_df[forgetting_df.after_task == final_task] if
    ↪not forgetting_df.empty else pd.DataFrame(columns=["method", "seed",
    ↪"avg_forgetting"])
    # Ensure optional columns exist.
    for col in ["backbone_params", "readout_trainable_params",
    ↪"readout_total_params"]:
        if col not in final_hist.columns:
            final_hist[col] = np.nan
    acc_summary = final_hist.groupby(["method", "seed"], as_index=False).agg(
        model_family=("model_family", "first"),
        final_avg_acc=("acc", "mean"),
        min_task_acc=("acc", "min"),
        context_entropy_mean=("context_entropy", "mean"),
        context_margin_mean=("context_margin", "mean"),
        context_top1_acc_mean=("context_top1_acc", "mean"),
        latent_frechet_margin_mean=("latent_frechet_margin", "mean"),
        prototype_weight_mean=("prototype_weight", "mean"),
        class2_margin_vs4_mean=("class2_margin_vs4", "mean"),
        class5_margin_vs4_mean=("class5_margin_vs4", "mean"),
        class2_logit_rank_mean=("class2_logit_rank", "mean"),
        class5_logit_rank_mean=("class5_logit_rank", "mean"),
        trainable_params=("trainable_params", "max"),
        total_params=("total_params", "max"),
        backbone_params=("backbone_params", "max"),
        readout_trainable_params=("readout_trainable_params", "max"),
        readout_total_params=("readout_total_params", "max"),
        model_size_mb_total=("model_size_mb_total", "max"),
        probe_row=("probe_row", "max"),
        backbone_source=("backbone_source", "first"),
        probe_feature=("probe_feature", "first"),
        probe_context_mode=("probe_context_mode", "first"),
```

```

)
mid = final_hist[final_hist.task_id.isin([2, 3])].groupby(["method",
↪ "seed"], as_index=False).agg(
    middle_avg_acc=("acc", "mean"), middle_min_acc=("acc", "min")
)
merged = acc_summary.merge(mid, on=["method", "seed"], how="left")
merged = merged.merge(final_forgetting[["method", "seed",
↪ "avg_forgetting"]], on=["method", "seed"], how="left")
merged.loc[merged["probe_row"].astype(bool), "avg_forgetting"] = np.nan
class_diag = class_diagnostics_from_confusion(confusion_df) if confusion_df
↪ is not None and not confusion_df.empty else pd.DataFrame()
if not class_diag.empty:
    merged = merged.merge(class_diag, on=["method", "seed"], how="left")
else:
    for col in ["old_class_acc", "recent_class_acc", "class0_acc",
↪ "class1_acc", "class2_acc", "class3_acc", "class4_acc", "class5_acc",
↪ "min_class_acc", "pred4_bias"]:
        merged[col] = np.nan
summary = merged.groupby("method", as_index=False).agg(
    model_family=("model_family", "first"),
    final_avg_acc_mean=("final_avg_acc", "mean"),
    final_avg_acc_std=("final_avg_acc", "std"),
    min_task_acc_mean=("min_task_acc", "mean"),
    avg_forgetting_mean=("avg_forgetting", "mean"),
    middle_avg_acc_mean=("middle_avg_acc", "mean"),
    middle_min_acc_mean=("middle_min_acc", "mean"),
    old_class_acc_mean=("old_class_acc", "mean"),
    recent_class_acc_mean=("recent_class_acc", "mean"),
    class0_acc_mean=("class0_acc", "mean"),
    class1_acc_mean=("class1_acc", "mean"),
    class2_acc_mean=("class2_acc", "mean"),
    class3_acc_mean=("class3_acc", "mean"),
    class4_acc_mean=("class4_acc", "mean"),
    class5_acc_mean=("class5_acc", "mean"),
    min_class_acc_mean=("min_class_acc", "mean"),
    pred4_bias_mean=("pred4_bias", "mean"),
    class2_margin_vs4_mean=("class2_margin_vs4_mean", "mean"),
    class5_margin_vs4_mean=("class5_margin_vs4_mean", "mean"),
    class2_logit_rank_mean=("class2_logit_rank_mean", "mean"),
    class5_logit_rank_mean=("class5_logit_rank_mean", "mean"),
    trainable_params_mean=("trainable_params", "mean"),
    total_params_mean=("total_params", "mean"),
    backbone_params_mean=("backbone_params", "mean"),
    readout_total_params_mean=("readout_total_params", "mean"),
    model_size_mb_total_mean=("model_size_mb_total", "mean"),
    probe_row=("probe_row", "max"),
    backbone_source=("backbone_source", "first"),

```

```

        probe_feature=("probe_feature", "first"),
        probe_context_mode=("probe_context_mode", "first"),
    )
    return summary.sort_values("final_avg_acc_mean", ascending=False), merged

scorecard_df, per_seed_df = final_scorecard(history_df, forgetting_df,
    ↪confusion_df)
scorecard_path = RESULTS_DIR / f"scorecard_{RUN_MODE}.csv"
per_seed_path = RESULTS_DIR / f"per_seed_scorecard_{RUN_MODE}.csv"
scorecard_df.to_csv(scorecard_path, index=False)
per_seed_df.to_csv(per_seed_path, index=False)
print("Saved scorecards:")
print(" -", scorecard_path)
print(" -", per_seed_path)
display(scorecard_df)

# -----
# Diagnostic patch: separate deployable methods from diagnostic ceilings/probes.
# -----
def method_role(method: str, probe_row: bool = False):
    m = str(method)
    if bool(probe_row):
        return "diagnostic_probe"
    if "joint_upper_bound" in m:
        return "diagnostic_upper_bound"
    if "oracle" in m:
        return "diagnostic_oracle_reference"
    if m.startswith("paired_"):
        return "deployable_mmals_paired"
    if m.startswith("baseline_"):
        return "deployable_continual_baseline"
    return "other"

scorecard_df["method_role"] = [
    method_role(row.method, row.probe_row)
    for row in scorecard_df.itertuples(index=False)
]
per_seed_df["method_role"] = [
    method_role(row.method, row.probe_row)
    for row in per_seed_df.itertuples(index=False)
]

scorecard_path = RESULTS_DIR / f"scorecard_{RUN_MODE}.csv"
per_seed_path = RESULTS_DIR / f"per_seed_scorecard_{RUN_MODE}.csv"
scorecard_df.to_csv(scorecard_path, index=False)
per_seed_df.to_csv(per_seed_path, index=False)

```

```
compact_cols = [
    "method_role", "method", "model_family", "final_avg_acc_mean",
    "min_task_acc_mean", "avg_forgetting_mean", "class2_acc_mean",
    "class5_acc_mean", "pred4_bias_mean", "total_params_mean",
    "probe_row"
]
compact_cols = [c for c in compact_cols if c in scorecard_df.columns]
benchmark_groups_df = scorecard_df[compact_cols].sort_values(
    ["method_role", "final_avg_acc_mean"], ascending=[True, False]
)
benchmark_groups_path = RESULTS_DIR / f"benchmark_method_groups_{RUN_MODE}.csv"
benchmark_groups_df.to_csv(benchmark_groups_path, index=False)
print("Saved benchmark method groups:", benchmark_groups_path)
display(benchmark_groups_df)
```

	method	model_family \
20	probe_proto_zf_oracle	probe
21	probe_proto_zf_proto	probe
12	paired_proto_global_head_ce_kl_guard_035	mmals_rc2b_candidate
9	paired_context_gap_selected	mmals_rc2b_candidate
11	paired_context_temp_blend_selected_head_guard_035	mmals_rc2b_candidate
14	paired_rc2h_rc1b_anchored_conservative_policy	mmals_rc2b_policy
4	baseline_joint_upper_bound	joint_upper_bound
13	paired_proto_global_no_repair	mmals_rc2b_candidate
10	paired_context_temp_blend_selected_global	mmals_rc2b_candidate
8	paired_context_blend_selected_global	mmals_rc2b_candidate
0	baseline_balanced_replay	mlp_baseline
2	baseline_experience_replay	mlp_baseline
18	probe_proto_transformed3_oracle	probe
17	probe_proto_transformed2_oracle	probe
16	probe_proto_transformed1_oracle	probe
15	probe_proto_transformed0_oracle	probe
19	probe_proto_transformed4_oracle	probe
5	baseline_lwf	mlp_baseline
7	baseline_sparse_moe	sparse_moe
3	baseline_finetune	mlp_baseline
1	baseline_ewc	mlp_baseline
6	baseline_pnn	pnn_fixed_columns

	final_avg_acc_mean	final_avg_acc_std	min_task_acc_mean \
20	0.943917	0.010631	0.916667
21	0.921250	0.020906	0.869583
12	0.895667	0.021518	0.805833
9	0.885750	0.038697	0.790833
11	0.885750	0.038697	0.790833
14	0.878833	0.035382	0.735833
4	0.876000	0.008430	0.824583

13	0.870917	0.030509	0.663333
10	0.863833	0.049479	0.662917
8	0.852000	0.070617	0.603333
0	0.836083	0.010751	0.683750
2	0.836083	0.010751	0.683750
18	0.791750	0.105060	0.674167
17	0.678250	0.166112	0.389167
16	0.640333	0.098792	0.468333
15	0.604500	0.106331	0.380000
19	0.592917	0.088347	0.316250
5	0.443250	0.027690	0.019583
7	0.299833	0.000144	0.000000
3	0.299833	0.000144	0.000000
1	0.299750	0.000250	0.000000
6	0.298917	0.001010	0.000000

	avg_forgetting_mean	middle_avg_acc_mean	middle_min_acc_mean	\
20	NaN	0.935208	0.922917	
21	NaN	0.889167	0.869583	
12	0.062292	0.852708	0.811667	
9	0.077188	0.822292	0.790833	
11	0.077188	0.822292	0.790833	
14	0.095833	0.803125	0.735833	
4	0.000000	0.846667	0.824583	
13	0.110417	0.780417	0.670417	
10	0.119063	0.763542	0.670833	
8	0.133542	0.737083	0.616667	
0	0.141458	0.769167	0.683750	
2	0.141458	0.769167	0.683750	
18	NaN	0.717083	0.677083	
17	NaN	0.530208	0.389167	
16	NaN	0.630833	0.485417	
15	NaN	0.562708	0.392917	
19	NaN	0.438958	0.316250	
5	0.618021	0.591250	0.342917	
7	0.842292	0.250000	0.000000	
3	0.842187	0.250000	0.000000	
1	0.843854	0.250000	0.000000	
6	0.836667	0.249375	0.000000	

	old_class_acc_mean	recent_class_acc_mean	...	class5_logit_rank_mean	\
20	0.954028	0.941667	...	1.032500	
21	0.937083	0.924444	...	1.011667	
12	0.918750	0.900556	...	1.005833	
9	0.907639	0.894583	...	1.005833	
11	0.907639	0.894583	...	1.005833	
14	0.887500	0.901667	...	1.007500	
4	0.881944	0.885972	...	1.063333	

13	0.863750	0.914306	...	1.008333
10	0.850556	0.913194	...	1.008333
8	0.839583	0.904306	...	1.008333
0	0.805972	0.900833	...	1.006667
2	0.805972	0.900833	...	1.006667
18	0.798333	0.820972	...	1.070833
17	0.756667	0.632083	...	1.145000
16	0.700000	0.609444	...	1.291667
15	0.554722	0.682778	...	1.135833
19	0.722222	0.526806	...	1.122500
5	0.018194	0.885556	...	1.034167
7	0.000000	0.666250	...	1.000833
3	0.000000	0.666250	...	1.000833
1	0.000000	0.665972	...	1.001667
6	0.000000	0.663750	...	1.006667

	trainable_params_mean	total_params_mean	backbone_params_mean	\
20	3.900000e+02	390.0	NaN	
21	3.900000e+02	390.0	NaN	
12	3.900000e+02	1337606.0	1337216.0	
9	3.900000e+02	1337606.0	1337216.0	
11	3.900000e+02	1337606.0	1337216.0	
14	4.461287e+05	1337606.0	1337216.0	
4	2.177980e+05	217798.0	217798.0	
13	1.337606e+06	1337606.0	1337216.0	
10	1.337606e+06	1337606.0	1337216.0	
8	1.337606e+06	1337606.0	1337216.0	
0	2.177980e+05	217798.0	217798.0	
2	2.177980e+05	217798.0	217798.0	
18	3.900000e+02	390.0	NaN	
17	3.900000e+02	390.0	NaN	
16	3.900000e+02	390.0	NaN	
15	3.900000e+02	390.0	NaN	
19	3.900000e+02	390.0	NaN	
5	2.177980e+05	217798.0	217798.0	
7	1.188555e+06	1188555.0	1188555.0	
3	2.177980e+05	217798.0	217798.0	
1	2.177980e+05	217798.0	217798.0	
6	2.193340e+05	1088966.0	1088966.0	

	readout_total_params_mean	model_size_mb_total_mean	probe_row	\
20	NaN	0.001488	True	
21	NaN	0.001488	True	
12	390.0	5.102562	False	
9	390.0	5.102562	False	
11	390.0	5.102562	False	
14	390.0	5.102562	False	
4	NaN	0.830833	False	

13	390.0	5.102562	False
10	390.0	5.102562	False
8	390.0	5.102562	False
0	NaN	0.830833	False
2	NaN	0.830833	False
18	NaN	0.001488	True
17	NaN	0.001488	True
16	NaN	0.001488	True
15	NaN	0.001488	True
19	NaN	0.001488	True
5	NaN	0.830833	False
7	NaN	4.533978	False
3	NaN	0.830833	False
1	NaN	0.830833	False
6	NaN	4.154076	False

	backbone_source	probe_feature \
20	mmals_proto_first_balanced_replay	zf
21	mmals_proto_first_balanced_replay	zf
12	same_proto_first_checkpoint	original_backbone_zf_rc2b_policy
9	same_proto_first_checkpoint	original_backbone_zf_rc2b_policy
11	same_proto_first_checkpoint	original_backbone_zf_rc2b_policy
14	same_proto_first_checkpoint	original_backbone_zf_rc2b_policy
4	baseline_sequential_or_joint	baseline_logits
13	same_proto_first_checkpoint	original_backbone_zf_rc2b_policy
10	same_proto_first_checkpoint	original_backbone_zf_rc2b_policy
8	same_proto_first_checkpoint	original_backbone_zf_rc2b_policy
0	baseline_sequential_or_joint	baseline_logits
2	baseline_sequential_or_joint	baseline_logits
18	mmals_proto_first_balanced_replay	transformed3
17	mmals_proto_first_balanced_replay	transformed2
16	mmals_proto_first_balanced_replay	transformed1
15	mmals_proto_first_balanced_replay	transformed0
19	mmals_proto_first_balanced_replay	transformed4
5	baseline_sequential_or_joint	baseline_logits
7	baseline_sequential_or_joint	baseline_logits
3	baseline_sequential_or_joint	baseline_logits
1	baseline_sequential_or_joint	baseline_logits
6	baseline_sequential_or_joint	baseline_logits

	probe_context_mode
20	oracle
21	proto
12	proto_first_rc2b_strict
9	proto_first_rc2b_strict
11	proto_first_rc2b_strict
14	proto_first_rc2b_strict
4	none

```

13 proto_first_rc2b_strict
10 proto_first_rc2b_strict
8  proto_first_rc2b_strict
0
2
18
17
16
15
19
5
7
3
1
6

```

[22 rows x 31 columns]

```

method_role \
0  deployable_continual_baseline
2  deployable_continual_baseline
5  deployable_continual_baseline
7  deployable_continual_baseline
3  deployable_continual_baseline
1  deployable_continual_baseline
6  deployable_continual_baseline
12 deployable_mmals_paired
9  deployable_mmals_paired
11 deployable_mmals_paired
14 deployable_mmals_paired
13 deployable_mmals_paired
10 deployable_mmals_paired
8  deployable_mmals_paired
20 diagnostic_probe
21 diagnostic_probe
18 diagnostic_probe
17 diagnostic_probe
16 diagnostic_probe
15 diagnostic_probe
19 diagnostic_probe
4  diagnostic_upper_bound

```

```

method          model_family \
0  baseline_balanced_replay  mlp_baseline
2  baseline_experience_replay mlp_baseline
5  baseline_lwf              mlp_baseline
7  baseline_sparse_moe       sparse_moe
3  baseline_finetune         mlp_baseline

```

1	baseline_ewc	mlp_baseline
6	baseline_pnn	pnn_fixed_columns
12	paired_proto_global_head_ce_kl_guard_035	mmals_rc2b_candidate
9	paired_context_gap_selected	mmals_rc2b_candidate
11	paired_context_temp_blend_selected_head_guard_035	mmals_rc2b_candidate
14	paired_rc2h_rc1b_anchored_conservative_policy	mmals_rc2b_policy
13	paired_proto_global_no_repair	mmals_rc2b_candidate
10	paired_context_temp_blend_selected_global	mmals_rc2b_candidate
8	paired_context_blend_selected_global	mmals_rc2b_candidate
20	probe_proto__zf_oracle	probe
21	probe_proto__zf_proto	probe
18	probe_proto__transformed3_oracle	probe
17	probe_proto__transformed2_oracle	probe
16	probe_proto__transformed1_oracle	probe
15	probe_proto__transformed0_oracle	probe
19	probe_proto__transformed4_oracle	probe
4	baseline_joint_upper_bound	joint_upper_bound

	final_avg_acc_mean	min_task_acc_mean	avg_forgetting_mean \
0	0.836083	0.683750	0.141458
2	0.836083	0.683750	0.141458
5	0.443250	0.019583	0.618021
7	0.299833	0.000000	0.842292
3	0.299833	0.000000	0.842187
1	0.299750	0.000000	0.843854
6	0.298917	0.000000	0.836667
12	0.895667	0.805833	0.062292
9	0.885750	0.790833	0.077188
11	0.885750	0.790833	0.077188
14	0.878833	0.735833	0.095833
13	0.870917	0.663333	0.110417
10	0.863833	0.662917	0.119063
8	0.852000	0.603333	0.133542
20	0.943917	0.916667	NaN
21	0.921250	0.869583	NaN
18	0.791750	0.674167	NaN
17	0.678250	0.389167	NaN
16	0.640333	0.468333	NaN
15	0.604500	0.380000	NaN
19	0.592917	0.316250	NaN
4	0.876000	0.824583	0.000000

	class2_acc_mean	class5_acc_mean	pred4_bias_mean	total_params_mean \
0	0.605417	0.997500	0.241944	217798.0
2	0.605417	0.997500	0.241944	217798.0
5	0.000417	0.975000	0.476319	217798.0
7	0.000000	0.999167	0.829097	1188555.0
3	0.000000	0.999167	0.829514	217798.0

1	0.000000	0.998333	0.829792	217798.0
6	0.000000	0.993333	0.823194	1088966.0
12	0.886250	0.995833	0.158472	1337606.0
9	0.842917	0.995833	0.168681	1337606.0
11	0.842917	0.995833	0.168681	1337606.0
14	0.769583	0.994167	0.187778	1337606.0
13	0.702500	0.993333	0.197778	1337606.0
10	0.660417	0.993333	0.210139	1337606.0
8	0.630417	0.993333	0.219444	1337606.0
20	0.927083	0.972500	0.168056	390.0
21	0.874583	0.990833	0.164861	390.0
18	0.695000	0.965833	0.201875	390.0
17	0.674583	0.890000	0.174444	390.0
16	0.844167	0.857500	0.090556	390.0
15	0.536667	0.948333	0.237014	390.0
19	0.731250	0.945000	0.076736	390.0
4	0.807500	0.945000	0.188472	217798.0

	probe_row
0	False
2	False
5	False
7	False
3	False
1	False
6	False
12	False
9	False
11	False
14	False
13	False
10	False
8	False
20	True
21	True
18	True
17	True
16	True
15	True
19	True
4	False

1.22 14. Plots

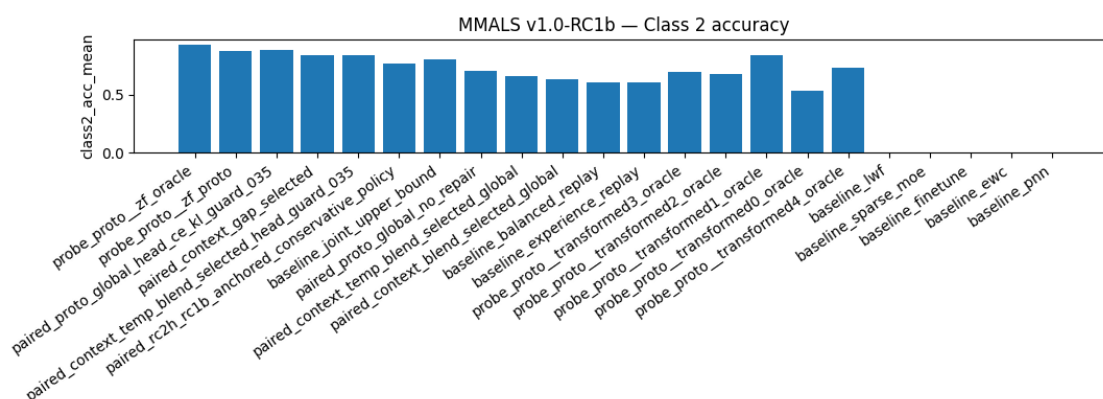
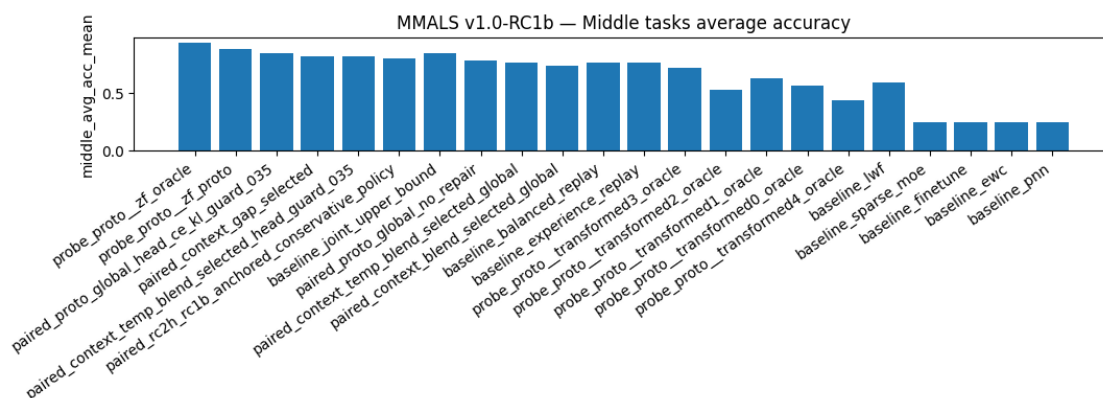
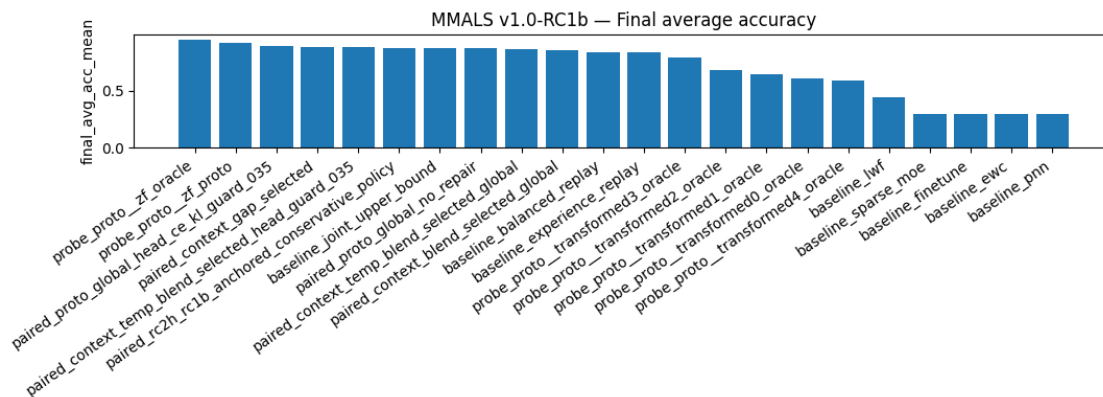
Quick visual debugging plots for the paired RC1b candidate-freeze audit. These are not publication figures.

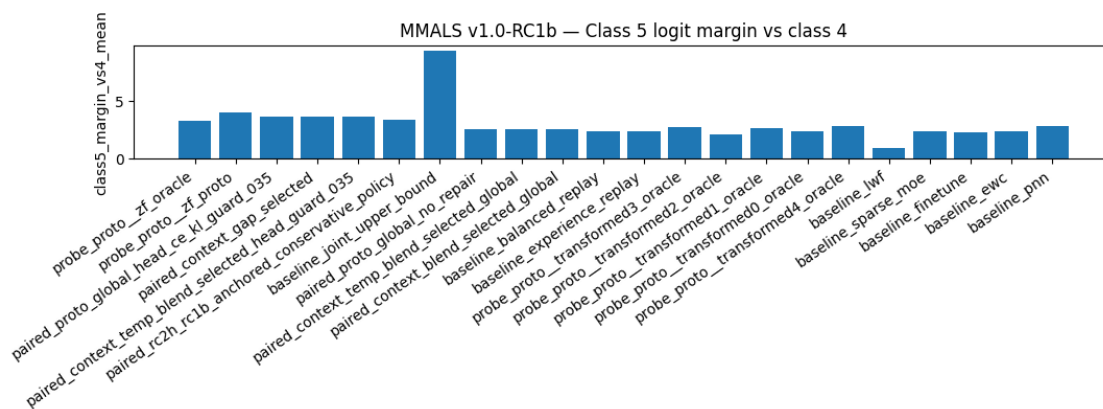
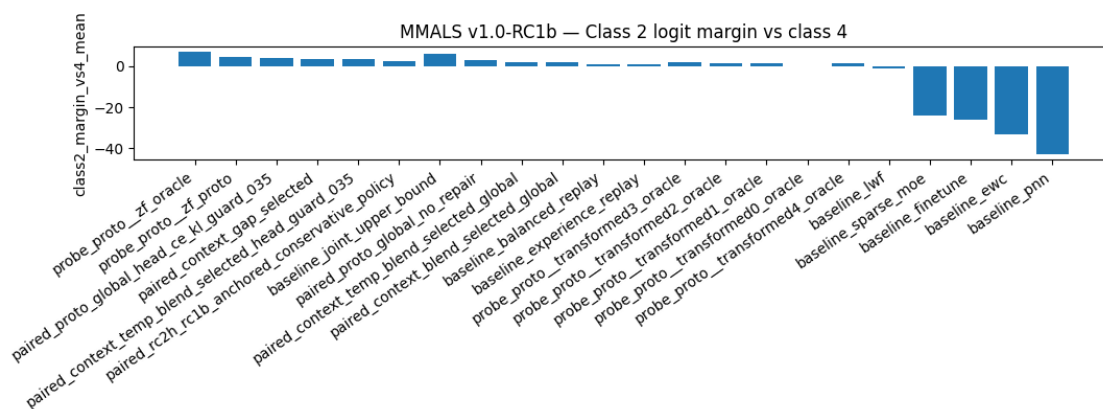
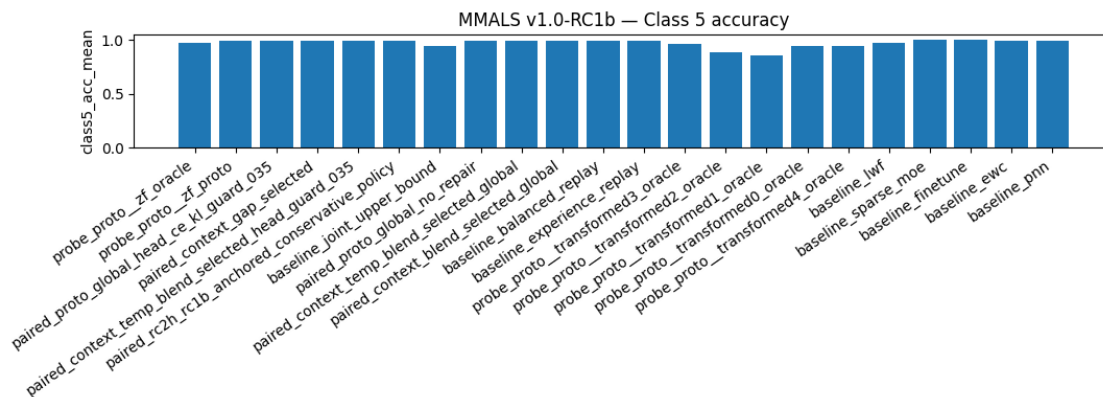
```

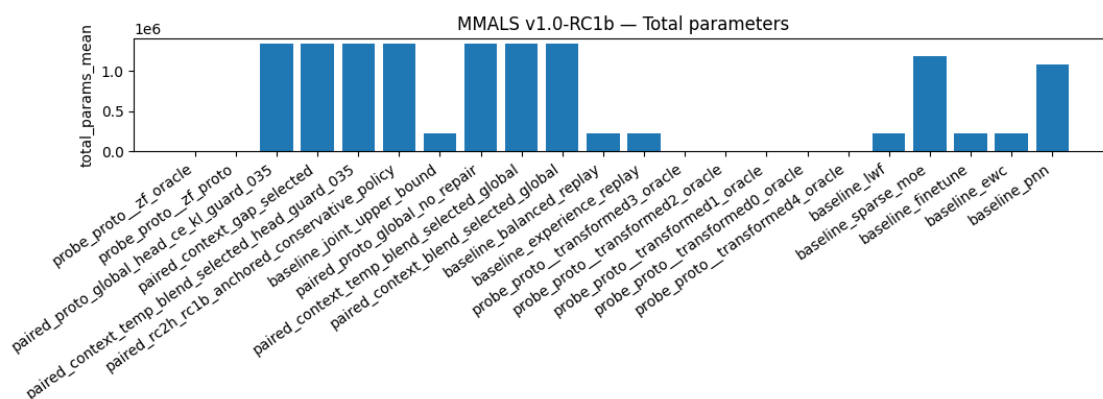
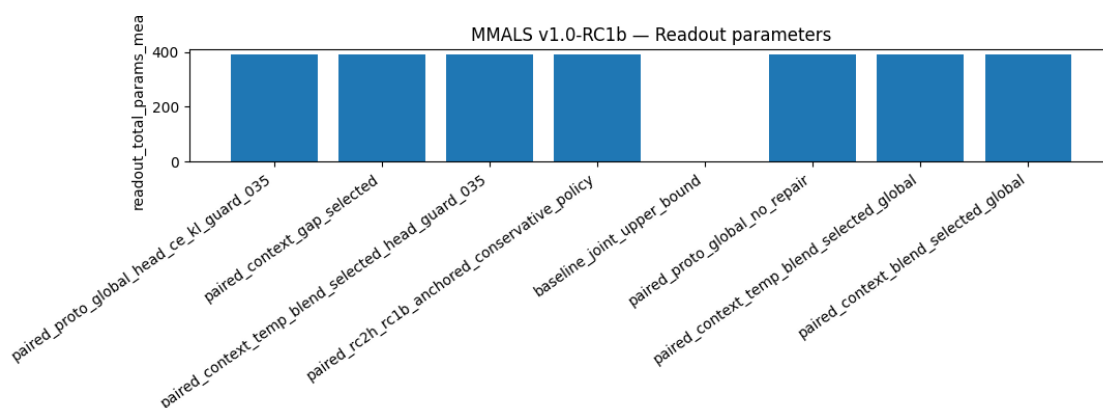
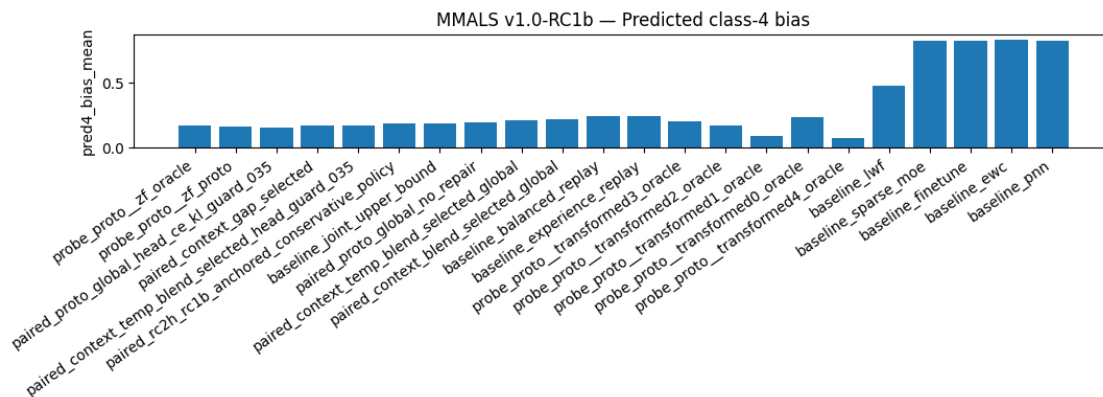
[25]: if not scorecard_df.empty:
    for metric, title in [
        ("final_avg_acc_mean", "Final average accuracy"),
        ("middle_avg_acc_mean", "Middle tasks average accuracy"),
        ("class2_acc_mean", "Class 2 accuracy"),
        ("class5_acc_mean", "Class 5 accuracy"),
        ("class2_margin_vs4_mean", "Class 2 logit margin vs class 4"),
        ("class5_margin_vs4_mean", "Class 5 logit margin vs class 4"),
        ("pred4_bias_mean", "Predicted class-4 bias"),
        ("readout_total_params_mean", "Readout parameters"),
        ("total_params_mean", "Total parameters"),
    ]:
        if metric not in scorecard_df.columns:
            continue
        plt.figure(figsize=(11, 4))
        plt.bar(scorecard_df["method"], scorecard_df[metric])
        plt.xticks(rotation=35, ha="right")
        plt.ylabel(metric)
        plt.title(f"MMALS v1.0-RC1b - {title}")
        plt.tight_layout()
        plt.show()

if 'confusion_df' in globals() and not confusion_df.empty:
    for method in confusion_df.method.unique():
        cm = confusion_df[confusion_df.method == method].pivot_table(
            index="true_class", columns="pred_class", values="rate",
            fill_value=0.0
        ).reindex(index=range(N_CLASSES), columns=range(N_CLASSES),
            fill_value=0.0)
        plt.figure(figsize=(5, 4))
        plt.imshow(cm.values, aspect="auto")
        plt.xticks(range(N_CLASSES), range(N_CLASSES))
        plt.yticks(range(N_CLASSES), range(N_CLASSES))
        plt.xlabel("Predicted class")
        plt.ylabel("True class")
        plt.title(f"Class confusion - {method}")
        plt.colorbar(label="rate")
        plt.tight_layout()
        plt.show()

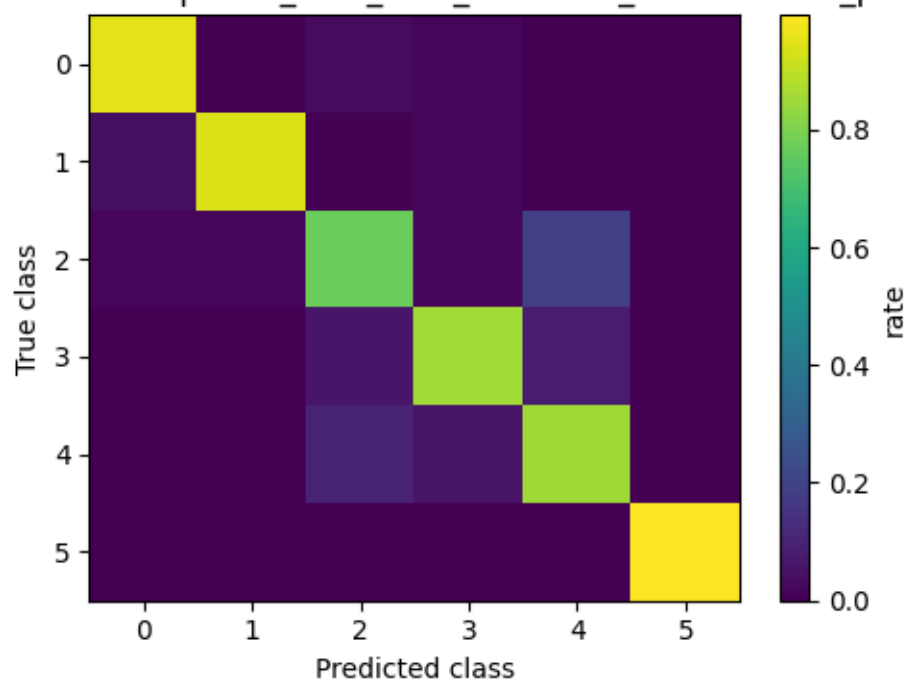
```



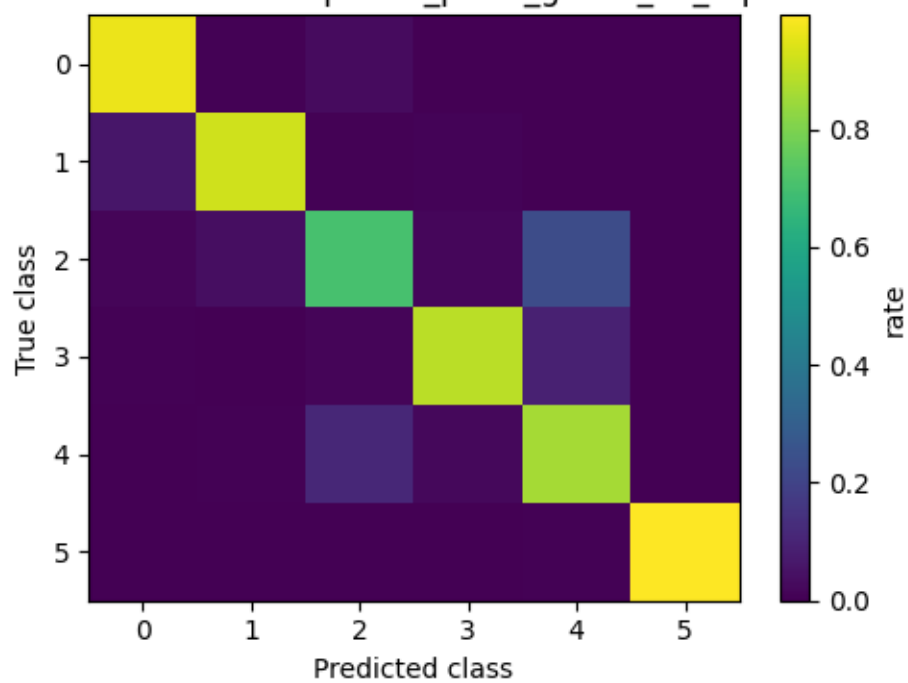




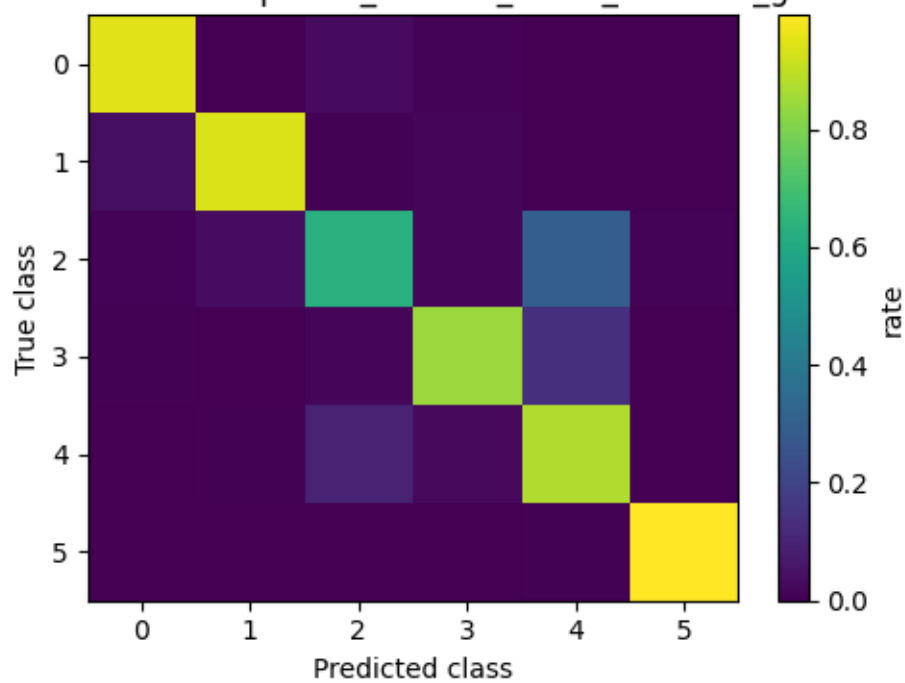
Class confusion — paired_rc2h_rc1b_anchored_conservative_policy



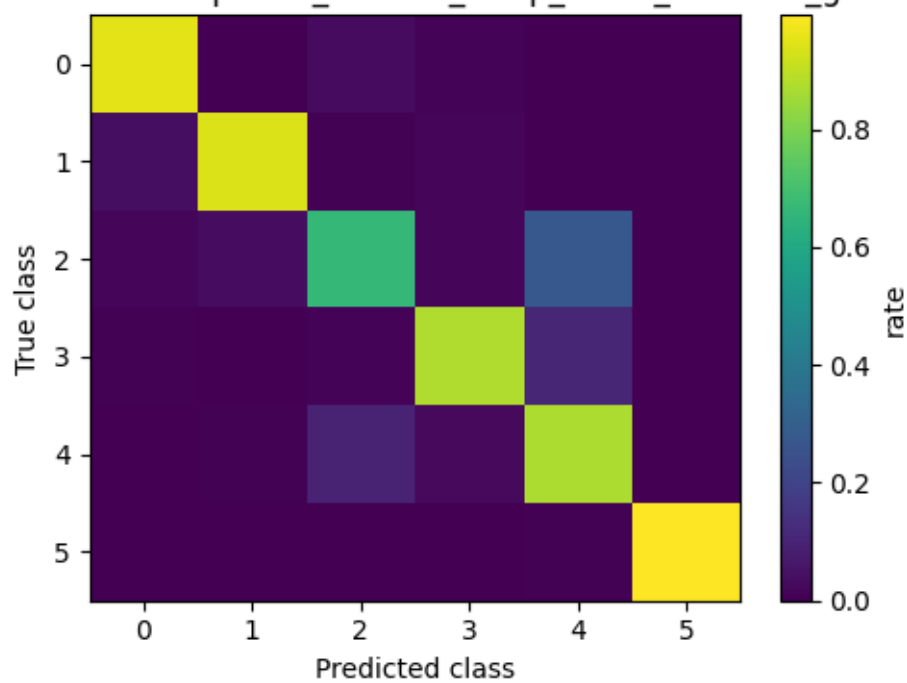
Class confusion — paired_proto_global_no_repair



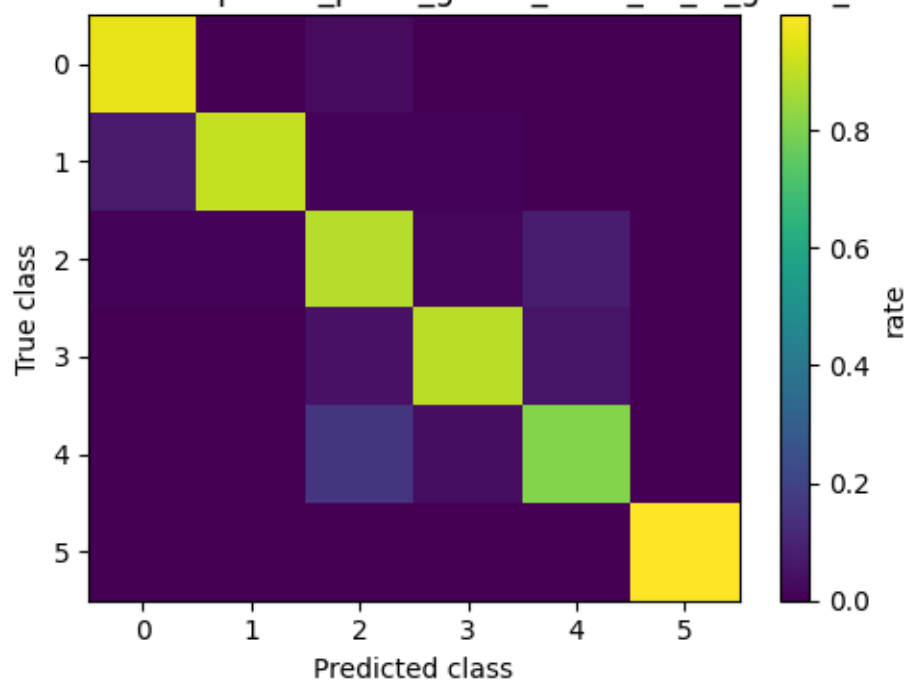
Class confusion — paired_context_blend_selected_global



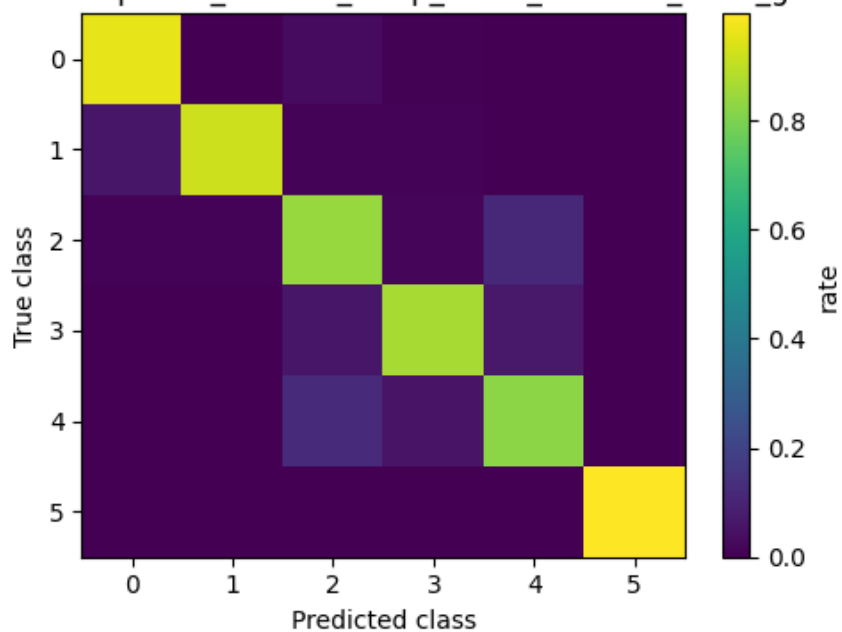
Class confusion — paired_context_temp_blend_selected_global

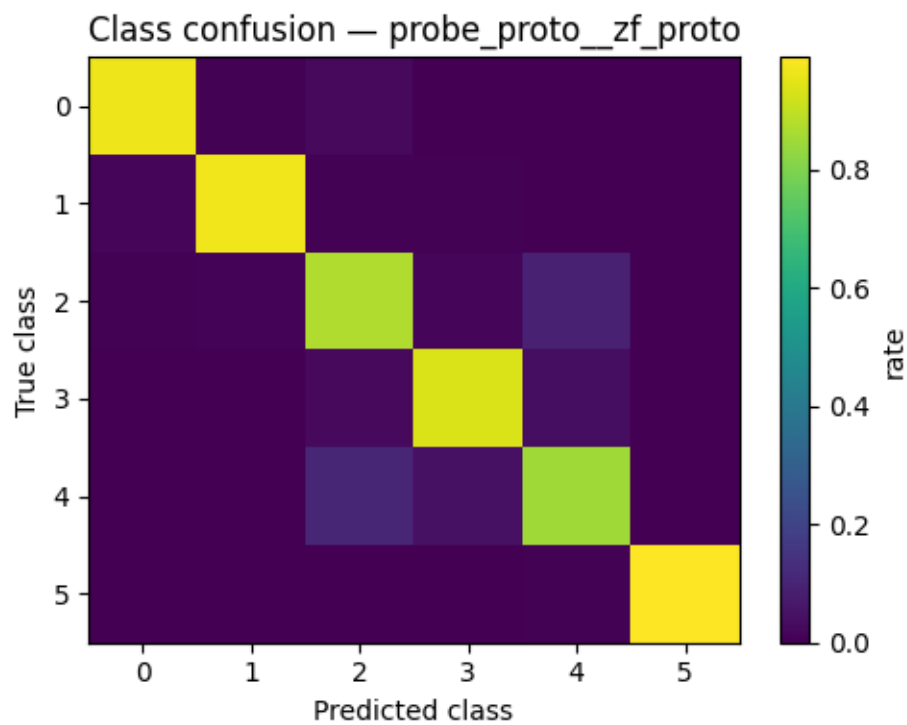
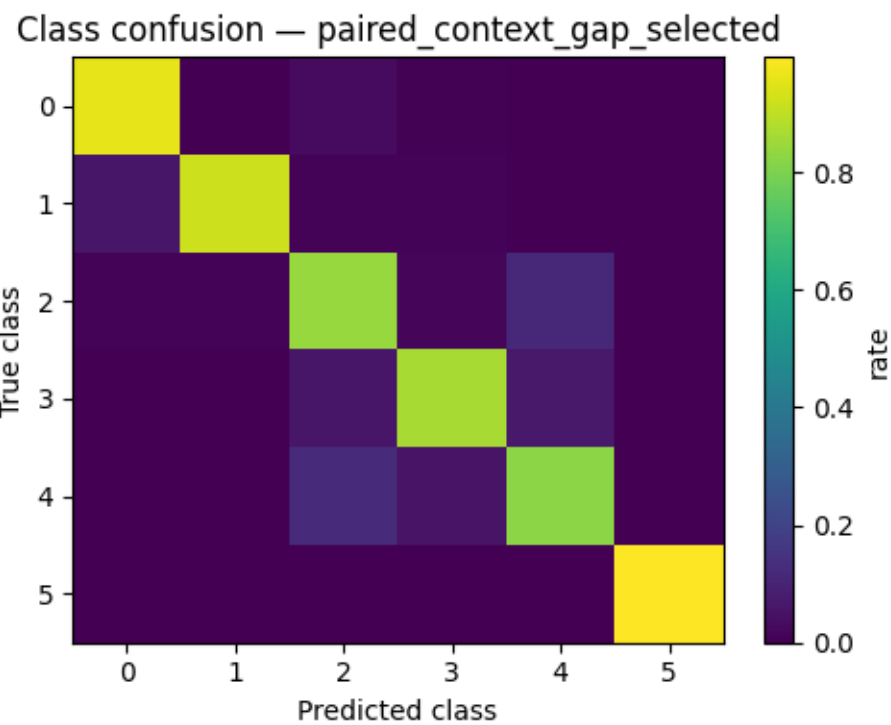


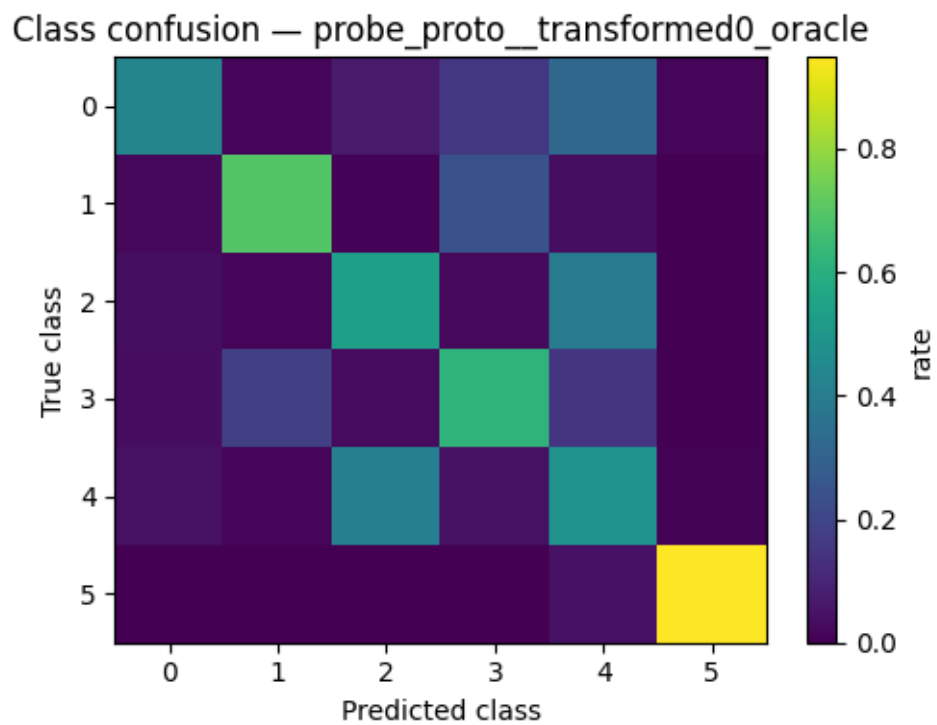
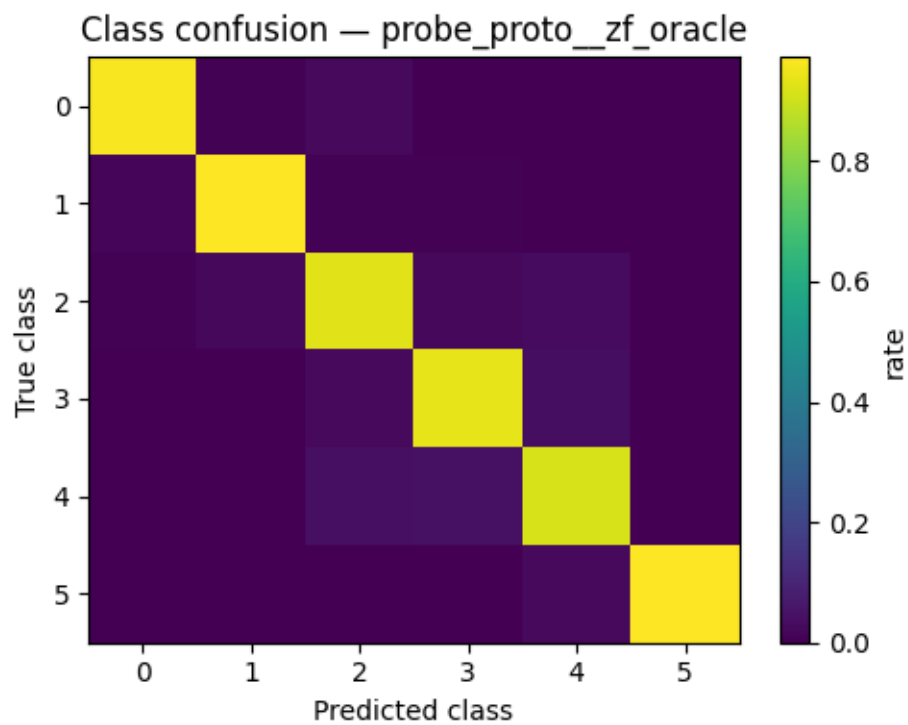
Class confusion — paired_proto_global_head_ce_kl_guard_035



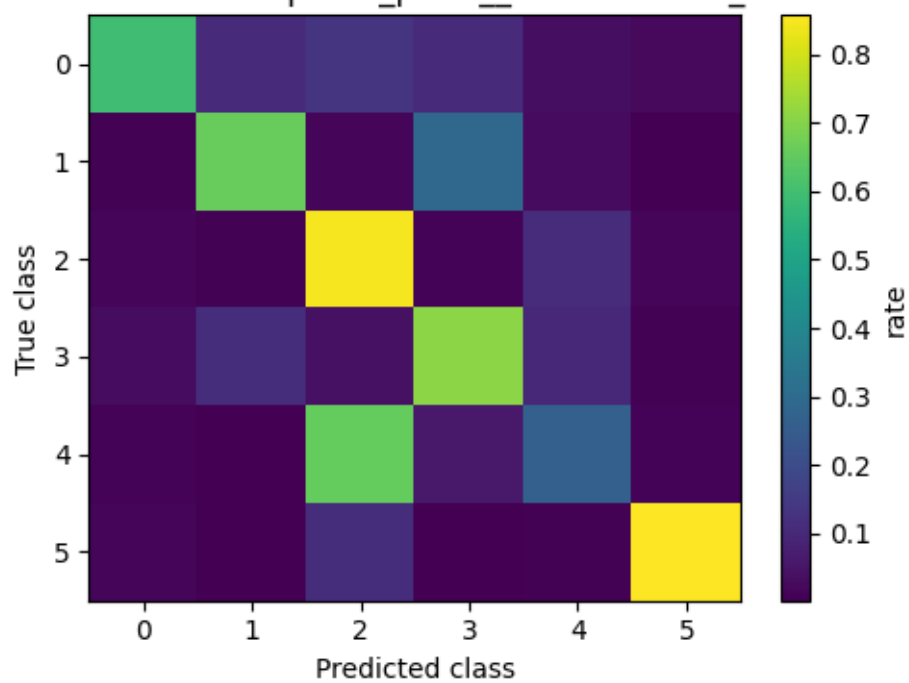
Class confusion — paired_context_temp_blend_selected_head_guard_035



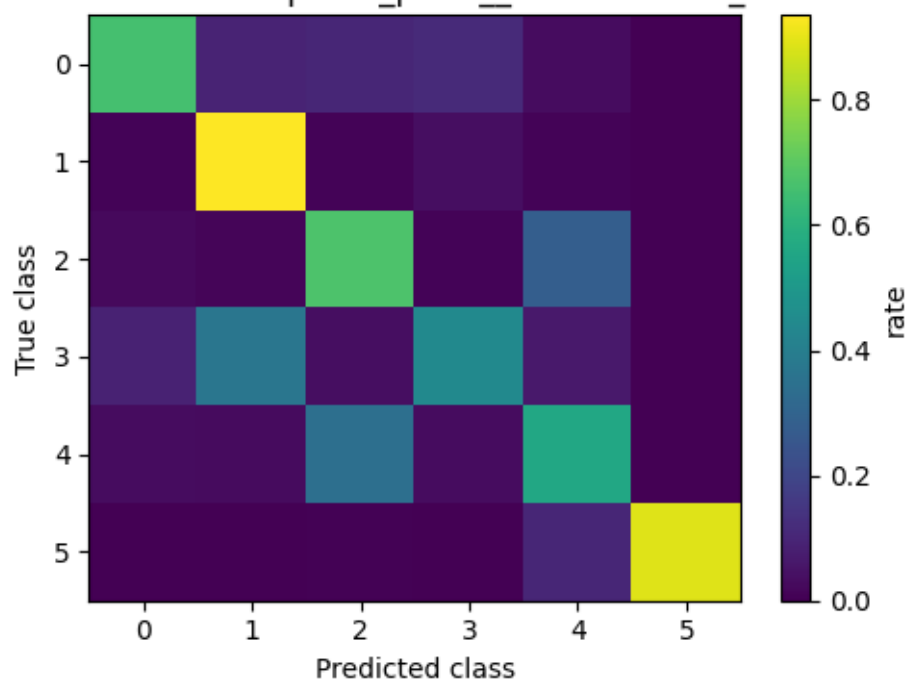




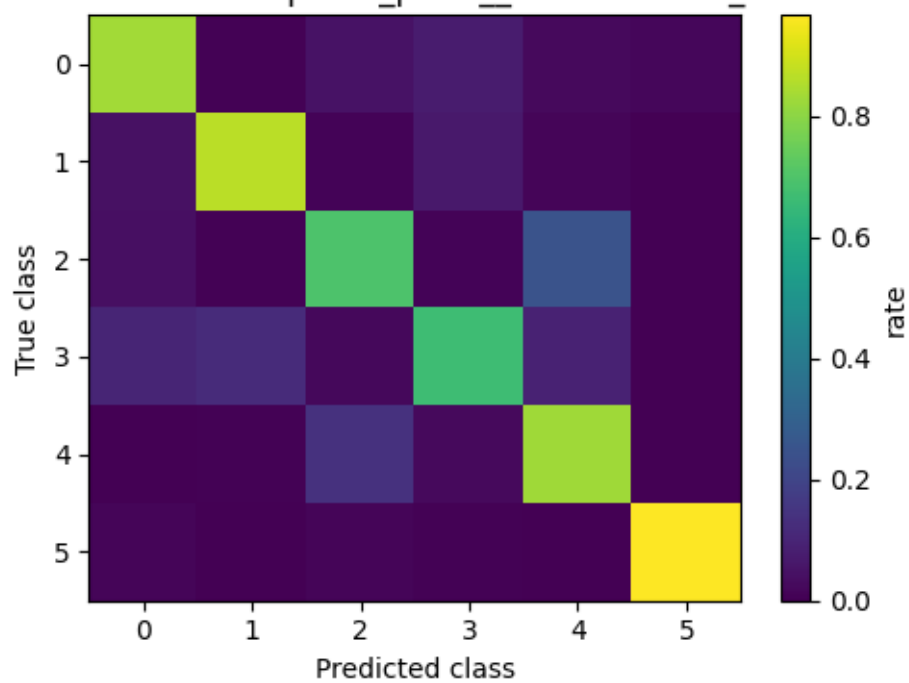
Class confusion — probe_proto_transformed1_oracle



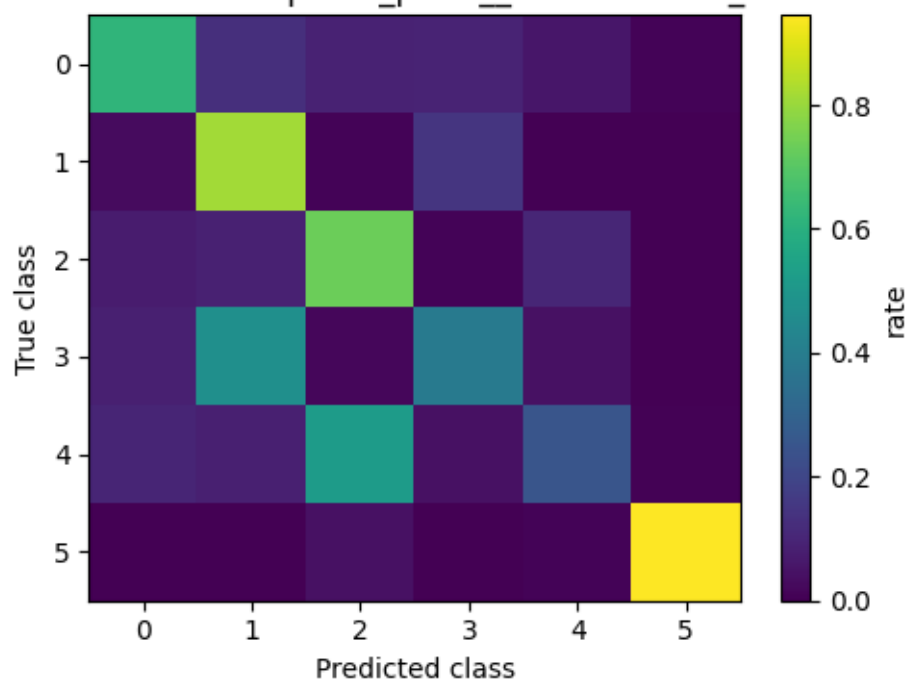
Class confusion — probe_proto_transformed2_oracle

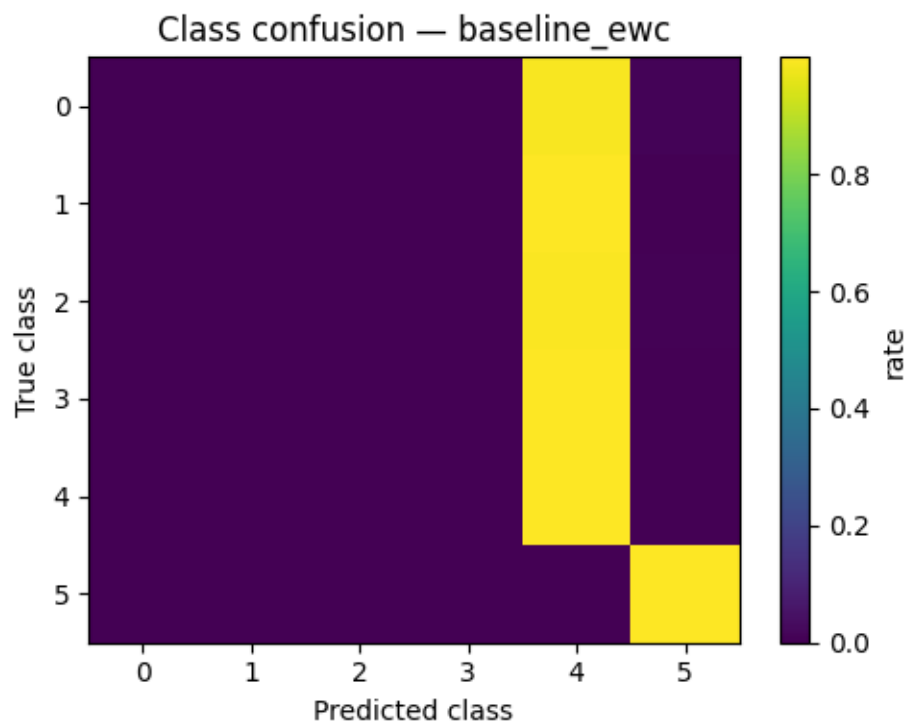
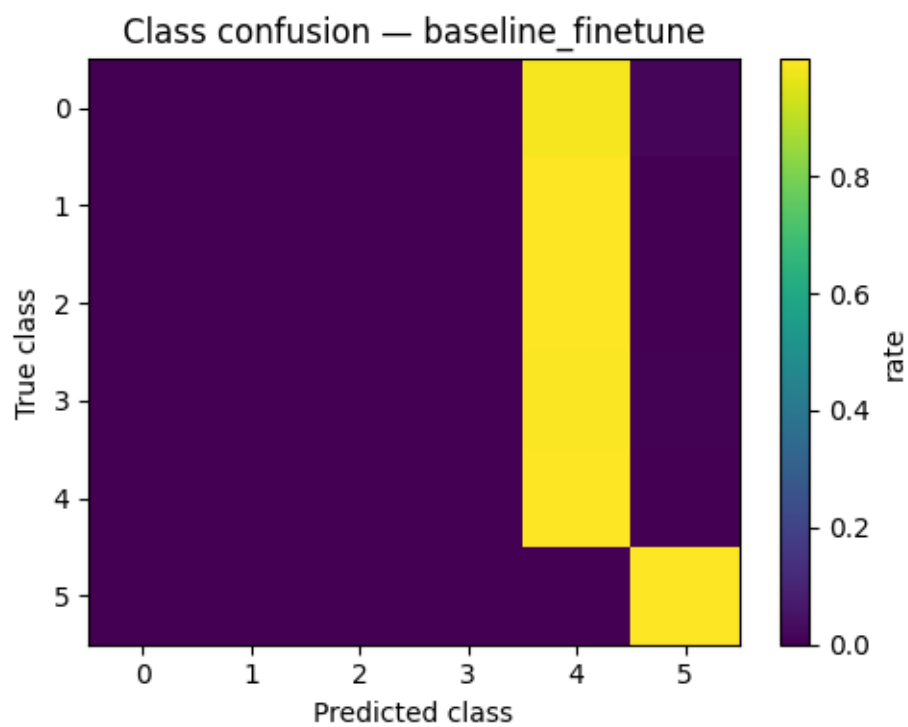


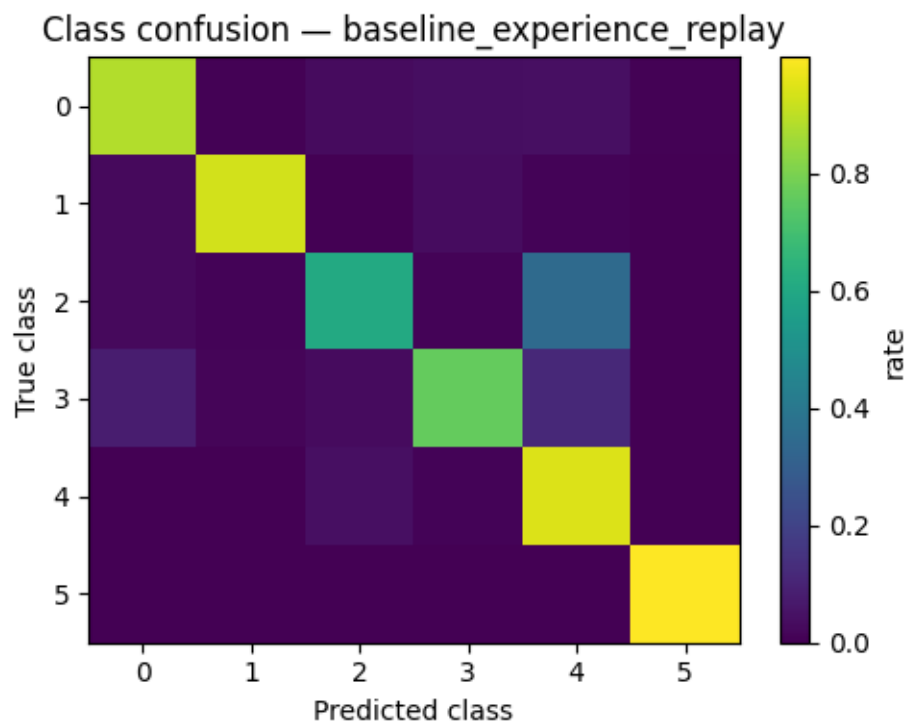
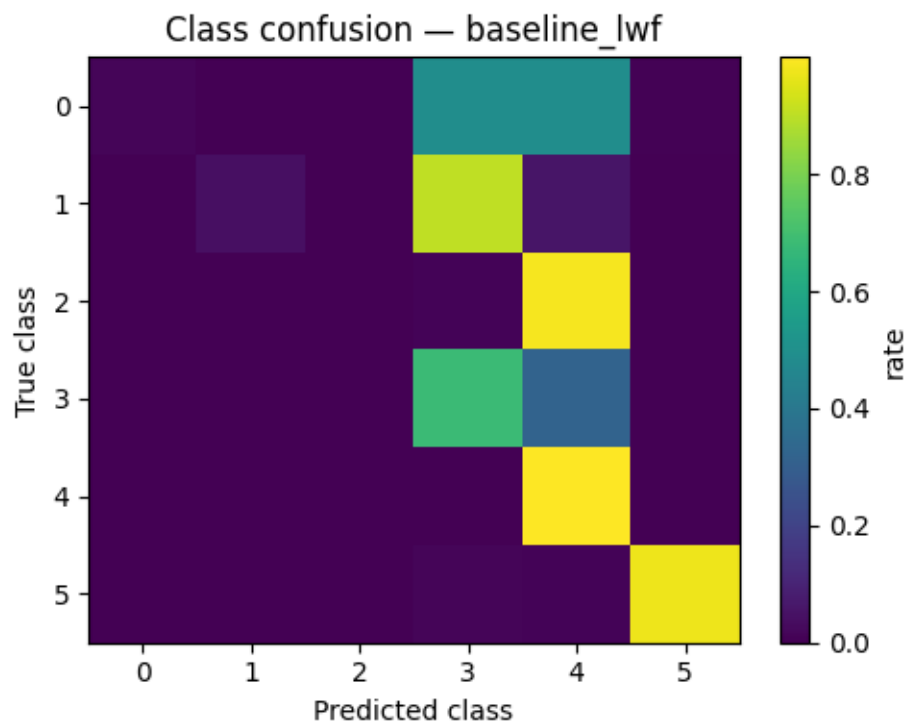
Class confusion — probe_proto_transformed3_oracle

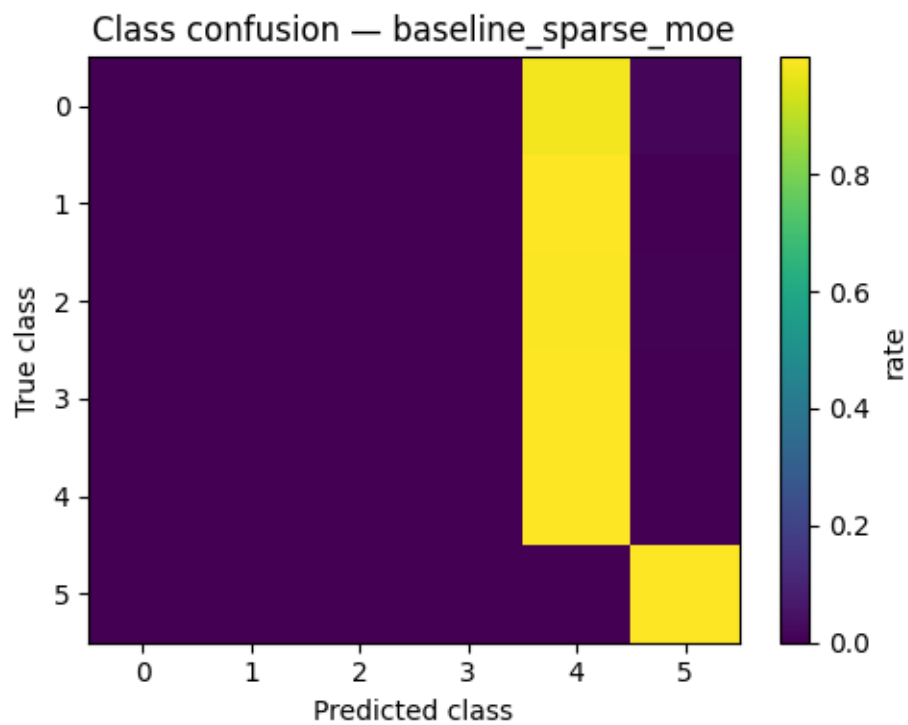
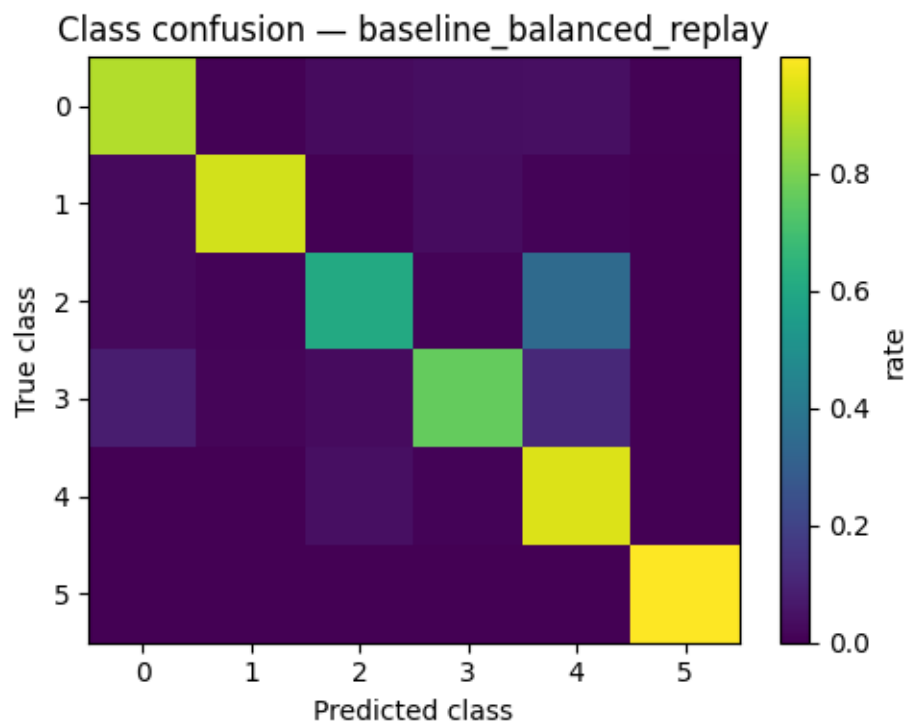


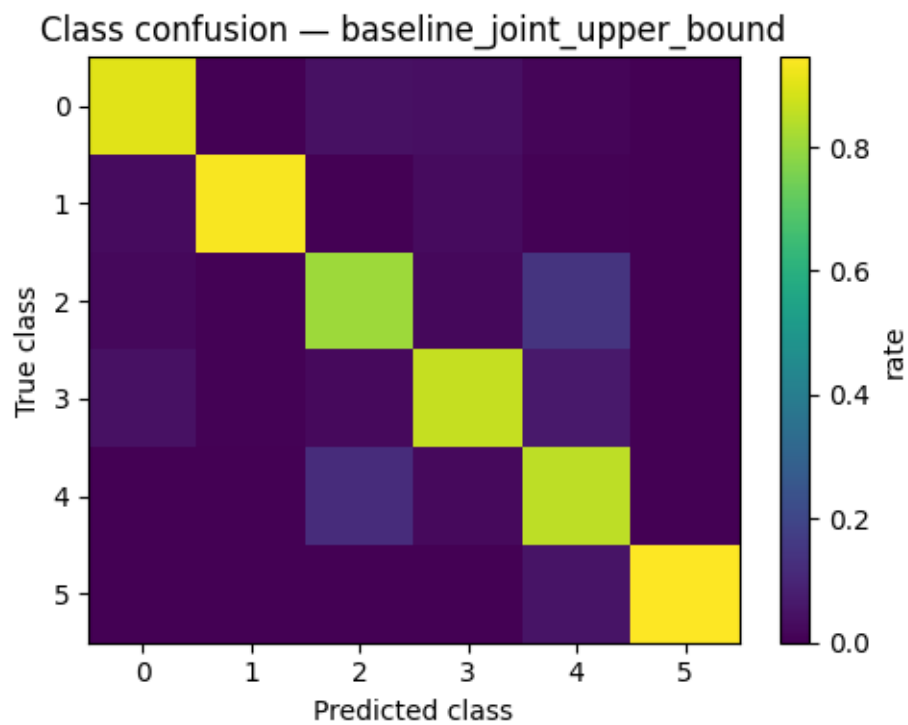
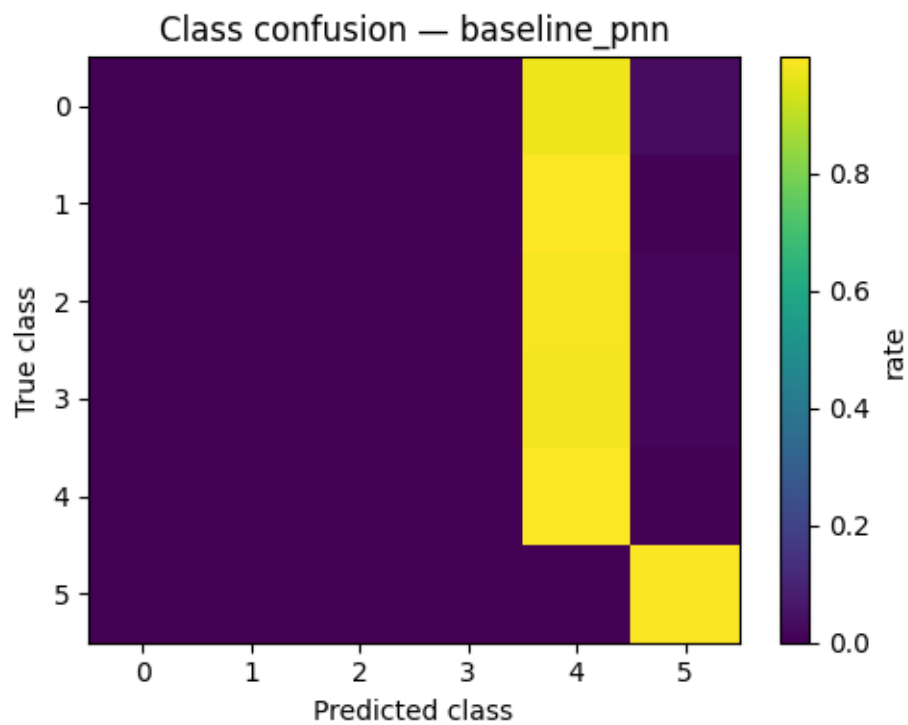
Class confusion — probe_proto_transformed4_oracle











1.23 15. Automatic interpretation and RC1b validation gate

This interpretation is deliberately conservative. It separates deployable inferred-context variants from oracle references, compares the RC1b candidate against the true global guard, computes probe gaps, writes a pass/fail validation-gate table, and keeps the original RC1 row only as a rejected diagnostic.

```
[26]: def get_row(scorecard_df, method):
    sub = scorecard_df[scorecard_df.method == method]
    return sub.iloc[0] if not sub.empty else None

def metric(row, name):
    if row is None or name not in row:
        return np.nan
    return float(row[name])

def method_role(method: str, probe_row: bool = False):
    """Classify methods so diagnostics are not mistaken for deployable models.
    ↪ """
    m = str(method)
    if bool(probe_row):
        return "diagnostic_probe"
    if "joint_upper_bound" in m:
        return "diagnostic_upper_bound"
    if "oracle" in m:
        return "diagnostic_oracle_reference"
    if m.startswith("paired_"):
        return "deployable_mmals_paired"
    if m.startswith("baseline_"):
        return "deployable_continual_baseline"
    return "other"

def non_diagnostic_scorecard(scorecard_df):
    if scorecard_df.empty:
        return scorecard_df
    if "method_role" not in scorecard_df.columns:
        tmp = scorecard_df.copy()
        tmp["method_role"] = [method_role(r.method, r.probe_row) for r in tmp.
        ↪ itertuples(index=False)]
    else:
        tmp = scorecard_df
        return tmp[tmp["method_role"].isin(["deployable_mmals_paired",
        ↪ "deployable_continual_baseline"])]

def paired_context_delta_table(scorecard_df):
    base = get_row(scorecard_df, "paired_proto_global_no_repair")
    global_guard = get_row(scorecard_df,
    ↪ "paired_proto_global_head_ce_kl_guard_035")
```

```

rows = []
for variant in SELECTED_CONTEXT_HEAD_VARIANTS:
    method = method_name_for_context_variant(variant)
    row = get_row(scorecard_df, method)
    if row is None or base is None:
        continue
    rows.append({
        "method": method,
        "is_oracle_reference": "oracle" in method,
        "delta_acc_vs_proto_no_repair": metric(row, "final_avg_acc_mean") -
↪metric(base, "final_avg_acc_mean"),
        "delta_acc_vs_proto_global_guard": metric(row,
↪"final_avg_acc_mean") - metric(global_guard, "final_avg_acc_mean") if
↪global_guard is not None else np.nan,
        "delta_min_task_vs_no_repair": metric(row, "min_task_acc_mean") -
↪metric(base, "min_task_acc_mean"),
        "delta_forgetting_vs_no_repair": metric(base,
↪"avg_forgetting_mean") - metric(row, "avg_forgetting_mean"),
        "delta_class2_vs_no_repair": metric(row, "class2_acc_mean") -
↪metric(base, "class2_acc_mean"),
        "delta_class5_vs_no_repair": metric(row, "class5_acc_mean") -
↪metric(base, "class5_acc_mean"),
        "context_temperature": metric(row, "context_entropy_mean"),
        "readout_params": metric(row, "readout_total_params_mean"),
    })
    return pd.DataFrame(rows).sort_values("delta_acc_vs_proto_no_repair",
↪ascending=False)

delta_df = paired_context_delta_table(scorecard_df)
delta_path = RESULTS_DIR / f"paired_context_deltas_{RUN_MODE}.csv"
delta_df.to_csv(delta_path, index=False)
print("Saved paired context deltas:", delta_path)
display(delta_df)

base = get_row(scorecard_df, "paired_proto_global_no_repair")
noop = get_row(scorecard_df, "paired_proto_kl_only_true_noop")
global_guard = get_row(scorecard_df, "paired_proto_global_head_ce_kl_guard_035")
oracle_ref = get_row(scorecard_df, "paired_oracle_global_no_repair_reference")
oracle_guard = get_row(scorecard_df,
↪"paired_oracle_global_head_ce_kl_guard_035_reference")

non_probe = scorecard_df[scorecard_df.probe_row == False] if not scorecard_df.
↪empty else pd.DataFrame()
deployable = non_diagnostic_scorecard(scorecard_df)
deployable_mmals = deployable[deployable.method.str.startswith("paired_",
↪na=False)] if not deployable.empty else pd.DataFrame()

```

```

deployable_baselines = deployable[deployable.method.str.startswith("baseline_",
↪na=False)] if not deployable.empty else pd.DataFrame()
context_selected = deployable_mmals[deployable_mmals.method.str.
↪contains("context_", case=False, na=False)] if not deployable_mmals.empty
↪else pd.DataFrame()

best_deployable = deployable.sort_values("final_avg_acc_mean", ascending=False).
↪iloc[0] if not deployable.empty else None
best_deployable_mmals = deployable_mmals.sort_values("final_avg_acc_mean",
↪ascending=False).iloc[0] if not deployable_mmals.empty else None
best_continual_baseline = deployable_baselines.
↪sort_values("final_avg_acc_mean", ascending=False).iloc[0] if not
↪deployable_baselines.empty else None
best_context = context_selected.sort_values("final_avg_acc_mean",
↪ascending=False).iloc[0] if not context_selected.empty else None

print("\n=== MMALS v1.0-RC1b automatic interpretation ===")
if base is not None:
    print(f"paired_proto_global_no_repair final_avg_acc = {metric(base,
↪'final_avg_acc_mean'):.4f}")
if noop is not None and base is not None:
    noop_delta = metric(noop, "final_avg_acc_mean") - metric(base,
↪"final_avg_acc_mean")
    print(f"proto_kl_only_true_noop delta vs proto_no_repair = {noop_delta:+.
↪6f}")
    if abs(noop_delta) < 1e-6:
        print("No-op control is clean: proto_kl_only_true_noop matches
↪proto_global_no_repair.")
    else:
        print("Warning: no-op differs from no_repair. Check deterministic
↪evaluation or audit implementation.")
if global_guard is not None:
    print(f"TRUE proto global CE+KL guard: {global_guard.method}
↪acc={global_guard.final_avg_acc_mean:.4f}")
    if base is not None:
        true_guard_delta = global_guard.final_avg_acc_mean - metric(base,
↪"final_avg_acc_mean")
        print(f"TRUE guard delta vs no-repair: {true_guard_delta:+.4f}")
        if abs(true_guard_delta) < 1e-6:
            print("Warning: TRUE guard still equals no-repair. Inspect the
↪guarded-head training logs.")
        else:
            print("Control-fix signal: TRUE guard differs from no-repair.")
if best_context is not None:
    print(f"Best deployable inferred-context variant: {best_context.method}
↪acc={best_context.final_avg_acc_mean:.4f}")

```

```

    if base is not None:
        print(f"Delta vs proto no-repair: {best_context.final_avg_acc_mean -
↪metric(base, 'final_avg_acc_mean'):+.4f}")
    if global_guard is not None:
        print(f"Delta vs TRUE proto global guarded head: {best_context.
↪final_avg_acc_mean - global_guard.final_avg_acc_mean:+.4f}")
        print(f"Class-2 delta vs no-repair: {best_context.class2_acc_mean -
↪metric(base, 'class2_acc_mean'):+.4f}")
        print(f"Min-task delta vs no-repair: {best_context.min_task_acc_mean -
↪metric(base, 'min_task_acc_mean'):+.4f}")
        print(f"Forgetting reduction vs no-repair: {metric(base,
↪'avg_forgetting_mean') - best_context.avg_forgetting_mean:+.4f}")
if best_deployable is not None:
    print(f"Best deployable method overall, excluding probes/oracle/joint upper
↪bound: {best_deployable.method} acc={best_deployable.final_avg_acc_mean:.
↪4f}")
if best_deployable_mmals is not None:
    print(f"Best deployable MMALS paired row: {best_deployable_mmals.method}
↪acc={best_deployable_mmals.final_avg_acc_mean:.4f}")
if best_continual_baseline is not None:
    print(f"Best deployable continual-learning baseline:
↪{best_continual_baseline.method} acc={best_continual_baseline.
↪final_avg_acc_mean:.4f}")
if oracle_ref is not None:
    print(f"Oracle context reference: {oracle_ref.method} acc={oracle_ref.
↪final_avg_acc_mean:.4f}")
if oracle_guard is not None:
    print(f"Oracle context + guarded head reference: {oracle_guard.method}
↪acc={oracle_guard.final_avg_acc_mean:.4f}")

probe_rows = scorecard_df[(scorecard_df.probe_row == True) & (scorecard_df.
↪probe_feature == "zf")] if not scorecard_df.empty else pd.DataFrame()
if not probe_rows.empty and best_deployable is not None:
    proto_probe = probe_rows[probe_rows.probe_context_mode == "proto"].
↪sort_values("final_avg_acc_mean", ascending=False).iloc[0]
    oracle_probe = probe_rows[probe_rows.probe_context_mode == "oracle"].
↪sort_values("final_avg_acc_mean", ascending=False).iloc[0]
    head_gap = proto_probe.final_avg_acc_mean - best_deployable.
↪final_avg_acc_mean
    context_gap = oracle_probe.final_avg_acc_mean - proto_probe.
↪final_avg_acc_mean
    total_gap = oracle_probe.final_avg_acc_mean - best_deployable.
↪final_avg_acc_mean
    print(f"Best proto-context z_f probe: {proto_probe.method} acc={proto_probe.
↪final_avg_acc_mean:.4f}")

```



```

    print(f"Best oracle-context z_f probe: {oracle_probe.method}\n")
    acc={oracle_probe.final_avg_acc_mean:.4f}")
    print(f"Remaining deployed-to-proto-probe head gap: {head_gap:+.4f}")
    print(f"Remaining proto-to-oracle context gap: {context_gap:+.4f}")
    print(f"Total deployed-to-oracle-probe gap: {total_gap:+.4f}")

# Explicit context-effect isolation: context-only vs context+guard.
context_only = get_row(scorecard_df,
    ↪"paired_context_temp_blend_selected_global")
context_guard = get_row(scorecard_df,
    ↪"paired_context_temp_blend_selected_head_guard_035")
if context_only is not None and context_guard is not None:
    print("\nContext effect isolation:")
    print(f"Context-only temp+blend: {context_only.final_avg_acc_mean:.4f}")
    print(f"Context+guard temp+blend: {context_guard.final_avg_acc_mean:.4f}")
    print(f"Added value of guarded readout after context selection:\n")
    ↪{context_guard.final_avg_acc_mean - context_only.final_avg_acc_mean:+.4f}")
    if global_guard is not None:
        print(f"Context+guard vs TRUE global guard: {context_guard.
    ↪final_avg_acc_mean - global_guard.final_avg_acc_mean:+.4f}")

if RUN_MODE == "smoke":
    print("\nSmoke-mode note:")
    print("- Smoke is an execution and diagnostics check only.")
    print("- Do not freeze or reject RC1b from smoke-mode benchmark numbers.")
    print("- Evidence/robust/benchmark modes are required before scientific
    ↪ranking versus baselines.")

print("\nDecision rule:")
print("- If deployable context-selected variants beat the TRUE proto global
    ↪guard and reduce total gap, v1.0-RC1b supports freezing temp/blend
    ↪context-gap selected guarded MMALS.")
print("- If they only match the TRUE global guard, context selection is
    ↪diagnostic but not yet a sufficient architecture change.")
print("- If oracle reference remains much higher, the next branch should target
    ↪learned context posterior / context memory, not more readout-only variants.")
print("- If proto and oracle probes converge, remaining work returns to readout/
    ↪probe-gap closure.")
print("- Do not claim oracle rows as deployable task-free MMALS.")

# Compact benchmark separation table for interpretation.
if "method_role" in scorecard_df.columns:
    role_summary = scorecard_df.groupby("method_role", as_index=False).agg(
        best_acc=("final_avg_acc_mean", "max"),

```

```

        n_methods=("method", "count"),
    ).sort_values("best_acc", ascending=False)
    role_summary_path = RESULTS_DIR / f"benchmark_role_summary_{RUN_MODE}.csv"
    role_summary.to_csv(role_summary_path, index=False)
    print("\nSaved benchmark role summary:", role_summary_path)
    display(role_summary)

# -----
# v1.0-RC1b robust validation gate
# -----
PRIMARY_RC1 = "paired_context_gap_selected"
SECONDARY_RC1 = "paired_context_temp_blend_selected_head_guard_035"
REJECTED_ORIGINAL_RC1 = "paired_context_blend_selected_head_guard_035"
TRUE_GUARD = "paired_proto_global_head_ce_kl_guard_035"
BASELINE = "paired_proto_global_no_repair"
NOOP = "paired_proto_kl_only_true_noop"

def row_or_nan(method):
    return get_row(scorecard_df, method)

rc1 = row_or_nan(PRIMARY_RC1)
rc1_secondary = row_or_nan(SECONDARY_RC1)
true_guard = row_or_nan(TRUE_GUARD)
base = row_or_nan(BASELINE)
noop = row_or_nan(NOOP)

# Per-seed win-rate against the corrected true global guard.
per_seed_wins = np.nan
per_seed_ties_or_wins = np.nan
if rc1 is not None and true_guard is not None and not per_seed_df.empty:
    p = per_seed_df[per_seed_df.method == PRIMARY_RC1][["seed",
    ↪ "final_avg_acc", "min_task_acc", "avg_forgetting", "class2_acc",
    ↪ "class5_acc"]]
    g = per_seed_df[per_seed_df.method == TRUE_GUARD][["seed", "final_avg_acc",
    ↪ "min_task_acc", "avg_forgetting", "class2_acc", "class5_acc"]]
    pg = p.merge(g, on="seed", suffixes=("_rc1", "_guard"))
    if not pg.empty:
        pg["acc_delta_rc1_vs_guard"] = pg["final_avg_acc_rc1"] -
    ↪ pg["final_avg_acc_guard"]
        pg["min_task_delta_rc1_vs_guard"] = pg["min_task_acc_rc1"] -
    ↪ pg["min_task_acc_guard"]
        pg["forgetting_reduction_rc1_vs_guard"] = pg["avg_forgetting_guard"] -
    ↪ pg["avg_forgetting_rc1"]
        pg["class2_delta_rc1_vs_guard"] = pg["class2_acc_rc1"] -
    ↪ pg["class2_acc_guard"]
        per_seed_wins = float((pg["acc_delta_rc1_vs_guard"] > 0.0).mean())

```

```

        per_seed_ties_or_wins = float((pg["acc_delta_rc1_vs_guard"] >= -0.0025).
↳mean())
        per_seed_cmp_path = RESULTS_DIR /
↳f"rc1b_per_seed_vs_true_guard_{RUN_MODE}.csv"
        pg.to_csv(per_seed_cmp_path, index=False)
        print("Saved RC1b per-seed comparison:", per_seed_cmp_path)
        display(pg)

# Probe gaps.
probe_rows = scorecard_df[(scorecard_df.probe_row == True) & (scorecard_df.
↳probe_feature == "zf")] if not scorecard_df.empty else pd.DataFrame()
proto_probe = None
oracle_probe = None
if not probe_rows.empty:
    proto_probe = probe_rows[probe_rows.probe_context_mode == "proto"].
↳sort_values("final_avg_acc_mean", ascending=False).iloc[0]
    oracle_probe = probe_rows[probe_rows.probe_context_mode == "oracle"].
↳sort_values("final_avg_acc_mean", ascending=False).iloc[0]

checks = []

def add_check(name, value, threshold_text, passed, details=""):
    checks.append({
        "check": name,
        "value": value,
        "threshold": threshold_text,
        "passed": bool(passed),
        "details": details,
    })

if base is not None and noop is not None:
    noop_delta = metric(noop, "final_avg_acc_mean") - metric(base,
↳"final_avg_acc_mean")
    add_check("no_op_equals_no_repair", noop_delta, "abs(delta) <= 1e-6",
↳abs(noop_delta) <= 1e-6,
        "No-op must be an evaluation no-op, not a hidden training variant.
↳")

if base is not None and true_guard is not None:
    guard_delta = metric(true_guard, "final_avg_acc_mean") - metric(base,
↳"final_avg_acc_mean")
    add_check("true_guard_active", guard_delta, "delta != 0 and preferably > +0.
↳005", abs(guard_delta) > 1e-6 and guard_delta > 0.005,
        "Corrected global CE+KL guard should not behave like no-repair.")

if rc1 is not None and base is not None:

```

```

    rc1_delta_base = metric(rc1, "final_avg_acc_mean") - metric(base,
↪ "final_avg_acc_mean")
    add_check("rc1_beats_no_repair_accuracy", rc1_delta_base, ">= +0.020",
↪ rc1_delta_base >= 0.020,
        "Primary RC1b candidate must materially beat no-repair.")
    rc1_min_delta_base = metric(rc1, "min_task_acc_mean") - metric(base,
↪ "min_task_acc_mean")
    add_check("rc1_improves_min_task_vs_no_repair", rc1_min_delta_base, ">= +0.
↪ 050", rc1_min_delta_base >= 0.050,
        "RC1b candidate must improve weakest-task robustness.")
    rc1_forget_red_base = metric(base, "avg_forgetting_mean") - metric(rc1,
↪ "avg_forgetting_mean")
    add_check("rc1_reduces_forgetting_vs_no_repair", rc1_forget_red_base, ">=
↪ +0.020", rc1_forget_red_base >= 0.020,
        "RC1b candidate should reduce forgetting, not only improve the
↪ last task.")

if rc1 is not None and true_guard is not None:
    rc1_delta_guard = metric(rc1, "final_avg_acc_mean") - metric(true_guard,
↪ "final_avg_acc_mean")
    rc1_min_delta_guard = metric(rc1, "min_task_acc_mean") - metric(true_guard,
↪ "min_task_acc_mean")
    rc1_forget_red_guard = metric(true_guard, "avg_forgetting_mean") -
↪ metric(rc1, "avg_forgetting_mean")
    rc1_class2_delta_guard = metric(rc1, "class2_acc_mean") -
↪ metric(true_guard, "class2_acc_mean")
    rc1_class5_delta_guard = metric(rc1, "class5_acc_mean") -
↪ metric(true_guard, "class5_acc_mean")
    robustness_preferable = (rc1_min_delta_guard >= 0.010 and
↪ rc1_forget_red_guard >= 0.005 and rc1_class2_delta_guard >= -0.005)
    add_check("rc1_vs_true_guard_accuracy_or_robustness", rc1_delta_guard,
        "accuracy >= +0.005 OR robustness-preferable", rc1_delta_guard >=
↪ 0.005 or robustness_preferable,
        f"min_task_delta={rc1_min_delta_guard:+.4f},
↪ forgetting_reduction={rc1_forget_red_guard:+.4f},
↪ class2_delta={rc1_class2_delta_guard:+.4f}")
    add_check("rc1_preserves_class5_vs_true_guard", rc1_class5_delta_guard, ">=
↪ -0.010", rc1_class5_delta_guard >= -0.010,
        "Class 5 preservation must not be sacrificed.")

if not np.isnan(per_seed_ties_or_wins):
    add_check("rc1_seed_stability_vs_true_guard", per_seed_ties_or_wins, ">= 0.
↪ 60 ties-or-wins", per_seed_ties_or_wins >= 0.60,
        "RC1b candidate should not be a one-seed artifact.")

if proto_probe is not None and oracle_probe is not None and rc1 is not None:

```

```

    head_gap = proto_probe.final_avg_acc_mean - metric(rc1,
↪ "final_avg_acc_mean")
    context_gap = oracle_probe.final_avg_acc_mean - proto_probe.
↪ final_avg_acc_mean
    add_check("probe_gap_interpretable", head_gap, "head_gap <= 0.025
↪ preferred", head_gap <= 0.025,
            f"proto_to_oracle_context_gap={context_gap:+.4f}; larger context
↪ gap suggests context frontier.")

rejected_original = row_or_nan(REJECTED_ORIGINAL_RC1)
secondary = row_or_nan(SECONDARY_RC1)

if rc1 is not None and rejected_original is not None:
    rc1b_delta_original = metric(rc1, "final_avg_acc_mean") -
↪ metric(rejected_original, "final_avg_acc_mean")
    add_check("rc1b_beats_rejected_original_rc1", rc1b_delta_original, ">= +0.
↪ 005 preferred", rc1b_delta_original >= 0.005,
            "RC1b should improve over the original blend-only guarded row
↪ that failed robust freeze.")

validation_gate_df = pd.DataFrame(checks)
validation_gate_path = RESULTS_DIR / f"rc1b_validation_gate_{RUN_MODE}.csv"
validation_gate_df.to_csv(validation_gate_path, index=False)
print("\n=== v1.0-RC1b validation gate ===")
print("Saved validation gate:", validation_gate_path)
display(validation_gate_df)

if not validation_gate_df.empty:
    passed_count = int(validation_gate_df["passed"].sum())
    total_count = int(len(validation_gate_df))
    print(f"Validation checks passed: {passed_count}/{total_count}")
    if RUN_MODE == "smoke":
        print("Decision: smoke passed only if the harness executed and
↪ diagnostics are coherent. Do not freeze/reject RC1b from smoke.")
        elif passed_count == total_count:
            print("Decision: RC1b passes this validation mode. Next step: benchmark
↪ campaign, not a new architecture branch.")
        elif passed_count >= max(1, total_count - 1):
            print("Decision: RC1b is nearly validated. Inspect the failed check
↪ before freezing.")
        else:
            print("Decision: do not freeze RC1b yet. Diagnose failed checks before
↪ benchmark campaign.")

```

	method	is_oracle_reference	\
1	paired_proto_global_head_ce_kl_guard_035	False	

5	paired_context_gap_selected	False
4	paired_context_temp_blend_selected_head_guard_035	False
0	paired_proto_global_no_repair	False
3	paired_context_temp_blend_selected_global	False
2	paired_context_blend_selected_global	False

	delta_acc_vs_proto_no_repair	delta_acc_vs_proto_global_guard \
1	0.024750	0.000000
5	0.014833	-0.009917
4	0.014833	-0.009917
0	0.000000	-0.024750
3	-0.007083	-0.031833
2	-0.018917	-0.043667

	delta_min_task_vs_no_repair	delta_forgetting_vs_no_repair \
1	0.142500	0.048125
5	0.127500	0.033229
4	0.127500	0.033229
0	0.000000	0.000000
3	-0.000417	-0.008646
2	-0.060000	-0.023125

	delta_class2_vs_no_repair	delta_class5_vs_no_repair	context_temperature \
1	0.183750	0.0025	NaN
5	0.140417	0.0025	NaN
4	0.140417	0.0025	NaN
0	0.000000	0.0000	NaN
3	-0.042083	0.0000	NaN
2	-0.072083	0.0000	NaN

	readout_params
1	390.0
5	390.0
4	390.0
0	390.0
3	390.0
2	390.0

	method_role	best_acc	n_methods
2	diagnostic_probe	0.943917	7
1	deployable_mmals_paired	0.895667	7
3	diagnostic_upper_bound	0.876000	1
0	deployable_continual_baseline	0.836083	7

	seed	final_avg_acc_rc1	min_task_acc_rc1	avg_forgetting_rc1 \
0	0	0.84125	0.75000	0.111563
1	1	0.91150	0.81875	0.052187
2	2	0.90450	0.80375	0.067813

	class2_acc_rc1	class5_acc_rc1	final_avg_acc_guard	min_task_acc_guard	\
0	0.82625	0.9925	0.87150	0.7500	
1	0.86125	0.9975	0.91275	0.8300	
2	0.84125	0.9975	0.90275	0.8375	

	avg_forgetting_guard	class2_acc_guard	class5_acc_guard	\
0	0.080000	0.88375	0.9925	
1	0.043125	0.88250	0.9975	
2	0.063750	0.89250	0.9975	

	acc_delta_rc1_vs_guard	min_task_delta_rc1_vs_guard	\
0	-0.03025	0.00000	
1	-0.00125	-0.01125	
2	0.00175	-0.03375	

	forgetting_reduction_rc1_vs_guard	class2_delta_rc1_vs_guard	\
0	-0.031562	-0.05750	
1	-0.009062	-0.02125	
2	-0.004063	-0.05125	

	check	value	\
0	true_guard_active	0.024750	
1	rc1_beats_no_repair_accuracy	0.014833	
2	rc1_improves_min_task_vs_no_repair	0.127500	
3	rc1_reduces_forgetting_vs_no_repair	0.033229	
4	rc1_vs_true_guard_accuracy_or_robustness	-0.009917	
5	rc1_preserves_class5_vs_true_guard	0.000000	
6	rc1_seed_stability_vs_true_guard	0.666667	
7	probe_gap_interpretable	0.035500	

	threshold	passed	\
0	delta != 0 and preferably > +0.005	True	
1	>= +0.020	False	
2	>= +0.050	True	
3	>= +0.020	True	
4	accuracy >= +0.005 OR robustness-preferable	False	
5	>= -0.010	True	
6	>= 0.60 ties-or-wins	True	
7	head_gap <= 0.025 preferred	False	

details

0	Corrected global CE+KL guard should not behave...
1	Primary RC1b candidate must materially beat no...
2	RC1b candidate must improve weakest-task robus...
3	RC1b candidate should reduce forgetting, not o...
4	min_task_delta=-0.0150, forgetting_reduction=-...
5	Class 5 preservation must not be sacrificed.
6	RC1b candidate should not be a one-seed artifact.

7 proto_to_oracle_context_gap=+0.0227; larger co...

1.24 15.2 Benchmark leaderboard and comparative gate

This cell compares the frozen RC1b candidate against the strongest non-oracle baseline rows. The decision rule is stricter than the internal paired audit:

- RC1b should beat fine-tuning, EWC, LwF and sparse MoE on average accuracy and robustness.
- Replay/PNN may remain strong controls; RC1b is benchmark-interesting if it is competitive while preserving the MMALS interpretability/probe-gap story.
- Joint upper bound is diagnostic, not a deployable continual-learning baseline.

```
[27]: # Benchmark-level leaderboard: exclude probes and oracle references.
leaderboard_df = scorecard_df.copy()
if not leaderboard_df.empty:
    leaderboard_df["is_probe"] = leaderboard_df["probe_row"].astype(bool)
    leaderboard_df["is_oracle_reference"] = leaderboard_df["method"].str.
    ↪contains("oracle", case=False, na=False)
    benchmark_leaderboard = leaderboard_df[(~leaderboard_df["is_probe"]) &
    ↪(~leaderboard_df["is_oracle_reference"])]].copy()
    benchmark_leaderboard = benchmark_leaderboard.
    ↪sort_values("final_avg_acc_mean", ascending=False)
    bench_path = RESULTS_DIR / f"benchmark_leaderboard_{RUN_MODE}.csv"
    benchmark_leaderboard.to_csv(bench_path, index=False)
    print("Saved benchmark leaderboard:", bench_path)
    display_cols = [
        "method", "final_avg_acc_mean", "final_avg_acc_std",
    ↪"min_task_acc_mean", "avg_forgetting_mean",
        "class2_acc_mean", "class5_acc_mean", "pred4_bias_mean",
    ↪"total_params_mean", "replay_memory_items_mean"
    ]
    display(benchmark_leaderboard[[c for c in display_cols if c in
    ↪benchmark_leaderboard.columns]])

    rc1b = get_row(scorecard_df, "paired_context_gap_selected")
    rc1b_equiv = get_row(scorecard_df,
    ↪"paired_context_temp_blend_selected_head_guard_035")
    true_guard = get_row(scorecard_df,
    ↪"paired_proto_global_head_ce_kl_guard_035")
    joint = get_row(scorecard_df, "baseline_joint_upper_bound")
    deployable_competitors = benchmark_leaderboard[~benchmark_leaderboard.
    ↪method.isin([
        "paired_context_gap_selected",
        "paired_context_temp_blend_selected_head_guard_035",
        "baseline_joint_upper_bound",
    ])]
```



```

    best_competitor = deployable_competitors.iloc[0] if not
↳deployable_competitors.empty else None

    print("\n=== Benchmark comparative interpretation ===")
    if rc1b is not None:
        print(f"RC1b primary candidate: {rc1b.method} acc={rc1b.
↳final_avg_acc_mean:.4f}, min_task={rc1b.min_task_acc_mean:.4f},
↳forgetting={rc1b.avg_forgetting_mean:.4f}")
        if rc1b_equiv is not None:
            print(f"RC1b equivalent diagnostic: {rc1b_equiv.method} acc={rc1b_equiv.
↳final_avg_acc_mean:.4f}")
        if true_guard is not None and rc1b is not None:
            print(f"Delta RC1b vs true global guard: {rc1b.final_avg_acc_mean -
↳true_guard.final_avg_acc_mean:+.4f} acc, {rc1b.min_task_acc_mean -
↳true_guard.min_task_acc_mean:+.4f} min-task")
            if best_competitor is not None and rc1b is not None:
                print(f"Best non-oracle, non-joint competitor: {best_competitor.method},
↳acc={best_competitor.final_avg_acc_mean:.4f}")
                print(f"Delta RC1b vs best competitor: {rc1b.final_avg_acc_mean -
↳best_competitor.final_avg_acc_mean:+.4f}")
            if joint is not None and rc1b is not None:
                print(f"Joint upper bound: {joint.final_avg_acc_mean:.4f}; RC1b gap to
↳joint={joint.final_avg_acc_mean - rc1b.final_avg_acc_mean:+.4f}")

    benchmark_gate_rows = []
    if rc1b is not None:
        def add_gate(name, passed, detail):
            benchmark_gate_rows.append({"check": name, "passed": bool(passed),
↳"detail": detail})
        add_gate("rc1b_above_true_guard", true_guard is not None and rc1b.
↳final_avg_acc_mean >= true_guard.final_avg_acc_mean - 0.002,
            f"RC1b={rc1b.final_avg_acc_mean:.4f}; true_guard={true_guard.
↳final_avg_acc_mean if true_guard is not None else np.nan:.4f}")
        add_gate("rc1b_above_finetune", get_row(scorecard_df,
↳"baseline_finetune") is not None and rc1b.final_avg_acc_mean >
↳get_row(scorecard_df, "baseline_finetune").final_avg_acc_mean,
            "RC1b should beat naive fine-tuning.")
        add_gate("rc1b_min_task_strong", rc1b.min_task_acc_mean >= 0.80 if not
↳np.isnan(rc1b.min_task_acc_mean) else False,
            f"min_task={rc1b.min_task_acc_mean:.4f}")
        add_gate("rc1b_forgetting_controlled", rc1b.avg_forgetting_mean <= 0.08,
↳if not np.isnan(rc1b.avg_forgetting_mean) else False,
            f"forgetting={rc1b.avg_forgetting_mean:.4f}")
        add_gate("rc1b_class2_not_collapsed", rc1b.class2_acc_mean >= 0.80 if
↳not np.isnan(rc1b.class2_acc_mean) else False,
            f"class2={rc1b.class2_acc_mean:.4f}")

```

```

        add_gate("rc1b_class5_preserved", rc1b.class5_acc_mean >= 0.95 if not_
↪np.isnan(rc1b.class5_acc_mean) else False,
                f"class5={rc1b.class5_acc_mean:.4f}")
        if best_competitor is not None:
            add_gate("rc1b_competitive_with_best_baseline", rc1b.
↪final_avg_acc_mean >= best_competitor.final_avg_acc_mean - 0.010,
                    f"RC1b={rc1b.final_avg_acc_mean:.4f}");
↪best_competitor={best_competitor.final_avg_acc_mean:.4f}")
        benchmark_gate_df = pd.DataFrame(benchmark_gate_rows)
        gate_path = RESULTS_DIR / f"benchmark_gate_{RUN_MODE}.csv"
        benchmark_gate_df.to_csv(gate_path, index=False)
        print("Saved benchmark gate:", gate_path)
        display(benchmark_gate_df)
    else:
        print("Scorecard is empty; run the experiment cells first.")

```

	method	final_avg_acc_mean \
12	paired_proto_global_head_ce_kl_guard_035	0.895667
9	paired_context_gap_selected	0.885750
11	paired_context_temp_blend_selected_head_guard_035	0.885750
14	paired_rc2h_rc1b_anchored_conservative_policy	0.878833
4	baseline_joint_upper_bound	0.876000
13	paired_proto_global_no_repair	0.870917
10	paired_context_temp_blend_selected_global	0.863833
8	paired_context_blend_selected_global	0.852000
0	baseline_balanced_replay	0.836083
2	baseline_experience_replay	0.836083
5	baseline_lwf	0.443250
7	baseline_sparse_moe	0.299833
3	baseline_finetune	0.299833
1	baseline_ewc	0.299750
6	baseline_pnn	0.298917

	final_avg_acc_std	min_task_acc_mean	avg_forgetting_mean \
12	0.021518	0.805833	0.062292
9	0.038697	0.790833	0.077188
11	0.038697	0.790833	0.077188
14	0.035382	0.735833	0.095833
4	0.008430	0.824583	0.000000
13	0.030509	0.663333	0.110417
10	0.049479	0.662917	0.119063
8	0.070617	0.603333	0.133542
0	0.010751	0.683750	0.141458
2	0.010751	0.683750	0.141458
5	0.027690	0.019583	0.618021
7	0.000144	0.000000	0.842292
3	0.000144	0.000000	0.842187

1	0.000250	0.000000	0.843854
6	0.001010	0.000000	0.836667

	class2_acc_mean	class5_acc_mean	pred4_bias_mean	total_params_mean
12	0.886250	0.995833	0.158472	1337606.0
9	0.842917	0.995833	0.168681	1337606.0
11	0.842917	0.995833	0.168681	1337606.0
14	0.769583	0.994167	0.187778	1337606.0
4	0.807500	0.945000	0.188472	217798.0
13	0.702500	0.993333	0.197778	1337606.0
10	0.660417	0.993333	0.210139	1337606.0
8	0.630417	0.993333	0.219444	1337606.0
0	0.605417	0.997500	0.241944	217798.0
2	0.605417	0.997500	0.241944	217798.0
5	0.000417	0.975000	0.476319	217798.0
7	0.000000	0.999167	0.829097	1188555.0
3	0.000000	0.999167	0.829514	217798.0
1	0.000000	0.998333	0.829792	217798.0
6	0.000000	0.993333	0.823194	1088966.0

	check	passed	\
0	rc1b_above_true_guard	False	
1	rc1b_above_finetune	True	
2	rc1b_min_task_strong	False	
3	rc1b_forgetting_controlled	True	
4	rc1b_class2_not_collapsed	True	
5	rc1b_class5_preserved	True	
6	rc1b_competitive_with_best_baseline	True	

```

detail
0 RC1b=0.8857; true_guard=0.8957
1 RC1b should beat naive fine-tuning.
2 min_task=0.7908
3 forgetting=0.0772
4 class2=0.8429
5 class5=0.9958
6 RC1b=0.8857; best_competitor=0.8957

```

1.25 16. Observability audit package and weak-seed diagnosis

This cell converts the benchmark results into auditable artifacts. It does not change scores. It produces seed-level, context-level, head-level, memory-level, and OTEL-style trace tables so that weak runs can be inspected instead of treated as black-box noise.

```

[28]: # -----
# Observability post-processing: seed, context, head, memory, and trace audits.
# -----
def _first_available(df, cols):

```

```

        return [c for c in cols if c in df.columns]

def build_seed_diagnostics(per_seed_df, class_diag_df):
    if per_seed_df.empty:
        return pd.DataFrame()
    keep = _first_available(per_seed_df, [
        "method", "seed", "method_role", "model_family", "final_avg_acc",
        ↪ "min_task_acc",
        "avg_forgetting", "class2_acc", "class5_acc", "pred4_bias",
        ↪ "total_params", "replay_memory_items"
    ])
    out = per_seed_df[keep].copy()
    out["is_rc1b_primary"] = out["method"].eq("paired_context_gap_selected")
    out["is_true_guard"] = out["method"].
    ↪ eq("paired_proto_global_head_ce_kl_guard_035")
    return out.sort_values(["seed", "final_avg_acc"], ascending=[True, False])

def build_rc1b_vs_guard_audit(per_seed_df):
    if per_seed_df.empty:
        return pd.DataFrame()
    rc = per_seed_df[per_seed_df.method == "paired_context_gap_selected"].copy()
    gd = per_seed_df[per_seed_df.method ==
    ↪ "paired_proto_global_head_ce_kl_guard_035"].copy()
    if rc.empty or gd.empty:
        return pd.DataFrame()
    cols = ["seed", "final_avg_acc", "min_task_acc", "avg_forgetting",
    ↪ "class2_acc", "class5_acc", "pred4_bias"]
    rc = rc[[c for c in cols if c in rc.columns]].rename(columns={c:
    ↪ f"rc1b_{c}" for c in cols if c != "seed"})
    gd = gd[[c for c in cols if c in gd.columns]].rename(columns={c:
    ↪ f"guard_{c}" for c in cols if c != "seed"})
    out = rc.merge(gd, on="seed", how="outer")
    for m in ["final_avg_acc", "min_task_acc", "class2_acc", "class5_acc"]:
        if f"rc1b_{m}" in out and f"guard_{m}" in out:
            out[f"delta_{m}_rc1b_minus_guard"] = out[f"rc1b_{m}"] -
    ↪ out[f"guard_{m}"]
        if "rc1b_avg_forgetting" in out and "guard_avg_forgetting" in out:
            out["delta_forgetting_reduction_rc1b_vs_guard"] =
    ↪ out["guard_avg_forgetting"] - out["rc1b_avg_forgetting"]
        out["weak_seed_flag"] = False
        if "delta_final_avg_acc_rc1b_minus_guard" in out:
            out["weak_seed_flag"] = out["weak_seed_flag"] |
    ↪ (out["delta_final_avg_acc_rc1b_minus_guard"] < 0)
        if "rc1b_class2_acc" in out:
            out["weak_seed_flag"] = out["weak_seed_flag"] | (out["rc1b_class2_acc"]
    ↪ < 0.80)

```

```

        if "rc1b_min_task_acc" in out:
            out["weak_seed_flag"] = out["weak_seed_flag"] | 1
    ↪(out["rc1b_min_task_acc"] < 0.80)
        return out.sort_values("seed")

def build_context_selection_audit(repair_log_df):
    if repair_log_df.empty:
        return pd.DataFrame()
    mask = repair_log_df.get("repair_type", pd.Series(dtype=str)).astype(str).
    ↪str.contains("context_config", na=False)
    cols = _first_available(repair_log_df, [
        "method", "seed", "after_task", "task_id", "variant", "repair_type", 1
    ↪"selection_mode",
        "candidate", "selected_candidate", "candidate_temp", "selected_temp", 1
    ↪"candidate_blend_name",
        "selected_blend_name", "candidate_blend_single", 1
    ↪"candidate_blend_latent", "candidate_blend_feature",
        "selected_blend_single", "selected_blend_latent", 1
    ↪"selected_blend_feature", "proxy_acc",
        "proxy_context_top1", "proxy_context_entropy", "proxy_context_margin", 1
    ↪"proxy_class2",
        "proxy_class5", "proxy_min_class", "proxy_score", 1
    ↪"selected_proxy_score", "selected_proxy_acc",
        "selected_proxy_context_top1", "selected_proxy_class2", 1
    ↪"selected_proxy_min_class"
    ])
    return repair_log_df.loc[mask, cols].copy()

def build_head_repair_audit(repair_log_df):
    if repair_log_df.empty:
        return pd.DataFrame()
    rt = repair_log_df.get("repair_type", pd.Series(dtype=str)).astype(str)
    mask = rt.str.contains("guard|head|readout|ce_kl", case=False, na=False) | 1
    ↪repair_log_df.get("uses_guarded_global_head", pd.Series(False, 1
    ↪index=repair_log_df.index)).fillna(False).astype(bool)
    cols = _first_available(repair_log_df, [
        "method", "seed", "after_task", "task_id", "variant", "repair_type", 1
    ↪"readout_train_loss",
        "readout_train_ce", "readout_train_kl", "readout_train_margin", 1
    ↪"kl_weight", "margin_weight",
        "uses_guarded_global_head", "true_guard_control_fixed", 1
    ↪"paired_same_backbone"
    ])
    return repair_log_df.loc[mask, cols].copy()

def build_memory_audit(history_df):

```

```

    if history_df.empty:
        return pd.DataFrame()
    final_task = int(history_df.after_task.max()) if "after_task" in history_df
    ↪else N_TASKS - 1
    sub = history_df[(history_df.after_task == final_task) & (history_df.
    ↪task_id == final_task)].copy()
    cols = _first_available(sub, [
        "method", "seed", "model_family", "replay_memory_items",
    ↪"memory_per_class", "memory_per_task",
        "total_params", "readout_total_params", "model_size_mb_total",
    ↪"uses_balanced_replay",
        "uses_context_selection", "uses_global_head_refresh",
    ↪"context_temperature_used",
        "batch_blend_single_used", "batch_blend_latent_used",
    ↪"batch_blend_feature_used"
    ])
    return sub[cols].sort_values(["seed", "method"]).copy()

def build_probe_gap_audit(scorecard_df):
    if scorecard_df.empty:
        return pd.DataFrame()
    def row(method):
        s = scorecard_df[scorecard_df.method == method]
        return s.iloc[0] if not s.empty else None
    rc = row("paired_context_gap_selected")
    proto = row("probe_proto__zf_proto")
    oracle = row("probe_proto__zf_oracle")
    rows = []
    if rc is not None and proto is not None:
        rows.append({
            "gap_name": "deployed_to_proto_zf_probe",
            "from_method": "paired_context_gap_selected",
            "to_method": "probe_proto__zf_proto",
            "gap": float(proto.final_avg_acc_mean - rc.final_avg_acc_mean),
        })
    if proto is not None and oracle is not None:
        rows.append({
            "gap_name": "proto_to_oracle_context_gap",
            "from_method": "probe_proto__zf_proto",
            "to_method": "probe_proto__zf_oracle",
            "gap": float(oracle.final_avg_acc_mean - proto.final_avg_acc_mean),
        })
    if rc is not None and oracle is not None:
        rows.append({
            "gap_name": "deployed_to_oracle_zf_probe",
            "from_method": "paired_context_gap_selected",

```

```

        "to_method": "probe_proto_zf_oracle",
        "gap": float(oracle.final_avg_acc_mean - rc.final_avg_acc_mean),
    })
    return pd.DataFrame(rows)

seed_diag_df = build_seed_diagnostics(per_seed_df, class_diag_df)
rc1b_guard_audit_df = build_rc1b_vs_guard_audit(per_seed_df)
context_selection_audit_df = build_context_selection_audit(repair_log_df)
head_repair_audit_df = build_head_repair_audit(repair_log_df)
memory_audit_df = build_memory_audit(history_df)
probe_gap_audit_df = build_probe_gap_audit(scorecard_df)

exports = {
    "seed_diagnostics": seed_diag_df,
    "rc1b_vs_true_guard_seed_audit": rc1b_guard_audit_df,
    "context_selection_trace": context_selection_audit_df,
    "head_repair_trace": head_repair_audit_df,
    "memory_audit": memory_audit_df,
    "probe_gap_audit": probe_gap_audit_df,
}
for name, df in exports.items():
    path = AUDIT_DIR / f"{name}_{RUN_MODE}.csv"
    df.to_csv(path, index=False)
    print("Saved audit artifact:", path)

# Create OTEL-style span rows: one span per seed/method final summary and per_
↳ weak-seed diagnosis.
span_rows = []
for _, r in seed_diag_df.iterrows():
    span_rows.append({
        "trace_id": TRACE_ID,
        "span_id": hashlib.sha256(f"{RUN_ID}:{r.get('seed')}:{r.get('method')}").
↳ encode()).hexdigest()[:16],
        "parent_span_id": TRACE_ID,
        "span_name": "method_seed_final_score",
        "timestamp_utc": datetime.now(timezone.utc).isoformat(),
        "seed": safe_jsonable(r.get("seed")),
        "method": safe_jsonable(r.get("method")),
        "final_avg_acc": safe_jsonable(r.get("final_avg_acc")),
        "min_task_acc": safe_jsonable(r.get("min_task_acc")),
        "avg_forgetting": safe_jsonable(r.get("avg_forgetting")),
        "class2_acc": safe_jsonable(r.get("class2_acc")),
        "class5_acc": safe_jsonable(r.get("class5_acc")),
        "pred4_bias": safe_jsonable(r.get("pred4_bias")),
    })
TRACE_SPANS.extend(span_rows)
span_df = pd.DataFrame(TRACE_SPANS)

```

```

span_df.to_csv(TRACE_SPANS_PATH, index=False)
print("Saved OTEL-style spans:", TRACE_SPANS_PATH)

# Add compact JSONL events for key audit rows.
for _, r in rc1b_guard_audit_df.iterrows():
    log_event("rc1b_vs_true_guard_seed_audit", seed=safe_jsonable(r.
        ↳get("seed")), method="paired_context_gap_selected", phase="post_run_audit",
        ↳payload=r.to_dict(), level="WARN" if bool(r.get("weak_seed_flag")) else
        ↳"INFO")
for _, r in probe_gap_audit_df.iterrows():
    log_event("probe_gap_audit", method=safe_jsonable(r.get("from_method")),
        ↳phase="post_run_audit", payload=r.to_dict())

weak_rows = rc1b_guard_audit_df[rc1b_guard_audit_df.get("weak_seed_flag",
    ↳False) == True] if not rc1b_guard_audit_df.empty else pd.DataFrame()
weak_seed_report = {
    "n_weak_seeds": int(len(weak_rows)),
    "weak_seeds": safe_jsonable(weak_rows.to_dict("records")) if not weak_rows.
        ↳empty else [],
    "diagnostic_interpretation": "Weak seeds are candidates for localized
        ↳inspection: check context_selection_trace, head_repair_trace, memory_audit,
        ↳class_diagnostics, and probe_gap_audit.",
}
weak_seed_report_path = AUDIT_DIR / f"weak_seed_report_{RUN_MODE}.json"
with open(weak_seed_report_path, "w", encoding="utf-8") as f:
    json.dump(weak_seed_report, f, indent=2, ensure_ascii=False)
print("Saved weak seed report:", weak_seed_report_path)
log_event("observability_audit_completed", phase="post_run_audit",
    ↳payload={"audit_files": [str(AUDIT_DIR / f"{k}_{RUN_MODE}.csv") for k in
    ↳exports] + [str(weak_seed_report_path)]})

# Write/update manifest after audit outputs are available.
manifest = write_manifest(extra={
    "audit_exports": list(exports.keys()),
    "weak_seed_report": str(weak_seed_report_path),
    "trace_jsonl": str(TRACE_JSONL_PATH),
    "trace_spans": str(TRACE_SPANS_PATH),
})
print("Saved manifest:", MANIFEST_PATH)

print("\n=== Weak seed audit preview ===")
display(rc1b_guard_audit_df)
print("\n=== Probe-gap audit ===")
display(probe_gap_audit_df)

```

	seed	rc1b_final_avg_acc	rc1b_min_task_acc	rc1b_avg_forgetting	\
0	0	0.84125	0.75000	0.111563	

1	1	0.91150	0.81875	0.052187
2	2	0.90450	0.80375	0.067813
	rc1b_class2_acc	rc1b_class5_acc	rc1b_pred4_bias	guard_final_avg_acc \
0	0.82625	0.9925	0.147708	0.87150
1	0.86125	0.9975	0.172708	0.91275
2	0.84125	0.9975	0.185625	0.90275
	guard_min_task_acc	guard_avg_forgetting	guard_class2_acc \	
0	0.7500	0.080000	0.88375	
1	0.8300	0.043125	0.88250	
2	0.8375	0.063750	0.89250	
	guard_class5_acc	guard_pred4_bias	delta_final_avg_acc_rc1b_minus_guard \	
0	0.9925	0.131458	-0.03025	
1	0.9975	0.165625	-0.00125	
2	0.9975	0.178333	0.00175	
	delta_min_task_acc_rc1b_minus_guard	delta_class2_acc_rc1b_minus_guard \		
0	0.00000	-0.05750		
1	-0.01125	-0.02125		
2	-0.03375	-0.05125		
	delta_class5_acc_rc1b_minus_guard \			
0	0.0			
1	0.0			
2	0.0			
	delta_forgetting_reduction_rc1b_vs_guard	weak_seed_flag		
0	-0.031562	True		
1	-0.009062	True		
2	-0.004063	False		
	gap_name	from_method \		
0	deployed_to_proto_zf_probe	paired_context_gap_selected		
1	proto_to_oracle_context_gap	probe_proto__zf_proto		
2	deployed_to_oracle_zf_probe	paired_context_gap_selected		
	to_method	gap		
0	probe_proto__zf_proto	0.035500		
1	probe_proto__zf_oracle	0.022667		
2	probe_proto__zf_oracle	0.058167		

1.26 16c. RC2d / Hydro policy-selection audit

This table summarizes the Hydro diagnostics that were computed before final-test evaluation, using only the policy-validation memory split. It is the main Hydro audit layer beside RC2d.

```

[29]: # -----
# RC2d Hydro policy-selection audit summary.
# -----
if "selection_log_df" in globals() and not selection_log_df.empty:
    hydro_cols = [
        "hydro_route_drift_Dr", "hydro_latent_drift_Dz",
        ↪ "hydro_output_drift_Dy",
        "hydro_adaptive_viscosity_nu", "hydro_reynolds_like",
        ↪ "hydro_turbulence_score",
        "hydro_wave_entropy", "hydro_selection_penalty",
    ]
    available_hydro_cols = [c for c in hydro_cols if c in selection_log_df.
        ↪ columns]
    group_cols = ["candidate_variant", "candidate_method", "policy_family",
        ↪ "is_selected_candidate"]
    agg_spec = {c: (c, "mean") for c in available_hydro_cols}
    agg_spec.update({
        "n_rows": ("candidate_variant", "size"),
        "mean_proxy_acc": ("proxy_acc", "mean"),
        "mean_proxy_score": ("proxy_score", "mean"),
    })
    rc3_hydro_policy_audit_df = selection_log_df.groupby(group_cols,
        ↪ as_index=False).agg(**agg_spec)
    rc3_hydro_policy_audit_df["hydro_warning"] = False
    if "hydro_turbulence_score" in rc3_hydro_policy_audit_df:
        rc3_hydro_policy_audit_df["hydro_warning"] =
        ↪ rc3_hydro_policy_audit_df["hydro_warning"] |
        ↪ (rc3_hydro_policy_audit_df["hydro_turbulence_score"] >
        ↪ RC3_HYDRO_MECHANISTIC_TURBULENCE_WARNING)
    if "hydro_latent_drift_Dz" in rc3_hydro_policy_audit_df:
        rc3_hydro_policy_audit_df["hydro_warning"] =
        ↪ rc3_hydro_policy_audit_df["hydro_warning"] |
        ↪ (rc3_hydro_policy_audit_df["hydro_latent_drift_Dz"] >
        ↪ RC3_HYDRO_MECHANISTIC_LATENT_WARNING)
    hydro_policy_audit_path = AUDIT_DIR /
    ↪ f"rc2d_hydro_policy_selection_audit_{RUN_MODE}.csv"
    rc3_hydro_policy_audit_df.to_csv(hydro_policy_audit_path, index=False)
    print("Saved RC2d Hydro policy-selection audit:", hydro_policy_audit_path)
    display(rc3_hydro_policy_audit_df.sort_values(["is_selected_candidate",
        ↪ "mean_proxy_score"], ascending=[False, False]))
    log_event("rc2d_hydro_policy_audit_completed", phase="post_run_audit",
        ↪ payload={"audit_file": str(hydro_policy_audit_path), "hydro_policy_mode":
        ↪ RC3_HYDRO_POLICY_MODE})
else:
    rc3_hydro_policy_audit_df = pd.DataFrame()
    print("No selection_log_df found; RC3 Hydro audit summary not generated.")

```

	candidate_variant \			
5	context_temp_blend_selected_global			
1	context_blend_selected_global			
3	context_gap_selected			
0	context_blend_selected_global			
6	context_temp_blend_selected_head_guard_035			
4	context_temp_blend_selected_global			
7	proto_global_head_ce_kl_guard_035			
2	context_gap_selected			
8	proto_global_no_repair			

	candidate_method \			
5	paired_context_temp_blend_selected_global			
1	paired_context_blend_selected_global			
3	paired_context_gap_selected			
0	paired_context_blend_selected_global			
6	paired_context_temp_blend_selected_head_guard_035			
4	paired_context_temp_blend_selected_global			
7	paired_proto_global_head_ce_kl_guard_035			
2	paired_context_gap_selected			
8	paired_proto_global_no_repair			

	policy_family	is_selected_candidate	hydro_route_drift_Dr \	
5	context_only_global	True	0.004591	
1	context_only_global	True	0.061231	
3	rc1b_guarded_context_gap	True	0.006940	
0	context_only_global	False	0.007096	
6	rc1b_guarded_context_gap	False	0.006100	
4	context_only_global	False	0.010846	
7	proto_true_global_guard	False	0.000000	
2	rc1b_guarded_context_gap	False	0.000640	
8	proto_no_repair	False	0.000000	

	hydro_latent_drift_Dz	hydro_output_drift_Dy	hydro_adaptive_viscosity_nu \	
5	0.056952	2.159853e-01	1.184340	
1	0.609901	2.363864e+00	1.555147	
3	0.167339	6.688634e-01	1.668031	
0	0.166160	4.884491e-01	1.633481	
6	0.146247	6.371811e-01	1.628259	
4	0.199784	6.173126e-01	1.659968	
7	0.000000	2.623860e-01	1.628259	
2	0.009150	4.312460e-01	1.369744	
8	0.000000	-2.434914e-09	1.628259	

	hydro_reynolds_like	hydro_turbulence_score	hydro_wave_entropy \	
5	0.240394	0.379326	0.561810	
1	0.363549	3.811667	0.006034	
3	0.129194	0.996371	0.052044	

0	0.145674	0.908238	0.086818
6	0.121252	0.876935	0.045143
4	0.147942	1.124289	0.048758
7	0.000000	0.000000	0.217442
2	0.069626	0.100601	0.000283
8	0.000000	0.000000	0.217442

	hydro_selection_penalty	n_rows	mean_proxy_acc	mean_proxy_score \
5	0.0	1	0.955339	1.803555
1	0.0	1	0.909063	1.690517
3	0.0	13	0.925853	1.542663
0	0.0	14	0.926774	1.562885
6	0.0	15	0.923925	1.522950
4	0.0	14	0.923087	1.506341
7	0.0	15	0.922497	1.498423
2	0.0	2	0.911393	1.394812
8	0.0	15	0.908257	-0.291035

	hydro_warning
5	True
1	True
3	True
0	True
6	True
4	True
7	False
2	False
8	False

1.27 16. Export results

This cell zips all CSV outputs for upload and external analysis. The ZIP includes history, losses, repair logs, probe losses, forgetting, confusion, scorecards, paired deltas, per-seed RC1b comparisons, and the validation-gate table.

1.28 16b. Notebook/PDF evidence archival

This cell stores notebook-level evidence inside the same Google Drive results directory.

It always creates a compact **run-summary PDF** from the tables already produced by the notebook.

It also tries to copy the source `.ipynb` and any already-generated full notebook PDF from Google Drive into `notebook_exports/`.

Optional full notebook PDF export is supported, but disabled by default because LaTeX/Pandoc installation can be slow in Colab.

```
[30]: # -----
# Notebook/PDF evidence archival helper.
```

```

# -----
# By default this cell creates a compact run-summary PDF that is always stored
# in RESULTS_DIR/notebook_exports and included in the final ZIP package.
#
# If you want to export the full notebook to PDF from inside Colab, set both:
#   INSTALL_FULL_PDF_EXPORT_DEPS = True
#   TRY_FULL_NOTEBOOK_PDF_EXPORT = True
# The dependency installation can take several minutes.
# -----
from matplotlib.backends.backend_pdf import PdfPages
import subprocess

INSTALL_FULL_PDF_EXPORT_DEPS = False
TRY_FULL_NOTEBOOK_PDF_EXPORT = False
COPY_EXISTING_NOTEBOOK_PDF_IF_FOUND = True

NOTEBOOK_EXPORT_DIR.mkdir(parents=True, exist_ok=True)
EXPECTED_FULL_NOTEBOOK_PDF = _
    ↪f"{NOTEBOOK_BASENAME}_{DATASET_NAME}_{RUN_MODE}_{RUN_ID}.pdf"
RUN_SUMMARY_PDF_PATH = NOTEBOOK_EXPORT_DIR / _
    ↪f"{NOTEBOOK_BASENAME}_{DATASET_NAME}_{RUN_MODE}_{RUN_ID}_summary.pdf"

def find_file_by_name(filename: str, max_hits: int = 5):
    roots = []
    for p in [Path("/content/drive/MyDrive"), Path("/content"), Path(".")]:
        if p.exists():
            roots.append(p)
    hits = []
    for root in roots:
        try:
            hits.extend(list(root.rglob(filename))[:max_hits])
        except Exception:
            pass
    # Prefer files outside RESULTS_DIR to avoid copying the same file onto_
    ↪itself.
    unique = []
    seen = set()
    for h in hits:
        try:
            hp = h.resolve()
            if str(hp) not in seen:
                seen.add(str(hp))
                unique.append(h)
        except Exception:
            pass
    return unique[:max_hits]

```

```

def copy_if_exists(src: Path, dst: Path):
    try:
        if src.exists():
            dst.parent.mkdir(parents=True, exist_ok=True)
            if src.resolve() != dst.resolve():
                shutil.copy2(src, dst)
            return dst
    except Exception as e:
        print(f"Copy skipped for {src}: {repr(e)}")
    return None

def text_page(pdf, title, lines, fontsize=10):
    fig = plt.figure(figsize=(8.27, 11.69)) # A4 portrait
    fig.text(0.06, 0.96, title, fontsize=15, weight="bold", va="top")
    y = 0.91
    for line in lines:
        if y < 0.06:
            pdf.savefig(fig, bbox_inches="tight")
            plt.close(fig)
            fig = plt.figure(figsize=(8.27, 11.69))
            y = 0.96
        fig.text(0.06, y, str(line), fontsize=fontsize, va="top",
        ↪family="monospace")
        y -= 0.026
    pdf.savefig(fig, bbox_inches="tight")
    plt.close(fig)

def write_run_summary_pdf():
    lines = [
        f"Notebook: {NOTEBOOK_FILENAME}",
        f"Run ID: {RUN_ID}",
        f"Dataset: {DATASET_NAME}",
        f"Pair mode: {PAIR_MODE}",
        f"Rotation angles: {ROTATION_ANGLES if DATASET_NAME == 'RotatedMNIST'
        ↪else 'n/a'}",
        f"Permutation seeds: {PERMUTATION_SEEDS if DATASET_NAME ==
        ↪'PermutedMNIST' else 'n/a'}",
        f"RC2b candidates: {RC2B_CANDIDATE_VARIANTS}",
        f"Run mode: {RUN_MODE}",
        f"Seeds: {SEEDS}",
        f"Results dir: {RESULTS_DIR}",
        f"Trace dir: {TRACE_DIR}",
        f"Audit dir: {AUDIT_DIR}",
        f"Config hash: {CONFIG_HASH}",
        "",
        "Strict selected candidate: paired_rc2b_validation_selected_policy",
        "Historical fixed candidate: paired_context_gap_selected",
    ]

```

```

        "Architecture: proto-first MMALS + validation-memory-selected_
↳deployable policy; no final-test leakage for policy selection",
        "",
        "This PDF is a compact evidence summary generated by the notebook.",
        "For the full executed notebook PDF, use the optional nbconvert export_
↳or Colab browser PDF export.",
    ]

    top_methods = []
    try:
        if "scorecard_df" in globals() and isinstance(scorecard_df, pd.
↳DataFrame) and not scorecard_df.empty:
            sort_col = "final_avg_acc_mean" if "final_avg_acc_mean" in_
↳scorecard_df.columns else None
            cols = [
                "method_role", "method", "final_avg_acc_mean",_
↳"min_task_acc_mean",
                "avg_forgetting_mean", "class2_acc_mean", "class5_acc_mean"
            ]
            cols = [c for c in cols if c in scorecard_df.columns]
            if sort_col:
                top = scorecard_df[cols].sort_values(sort_col, ascending=False).
↳head(15)
            else:
                top = scorecard_df[cols].head(15)
            top_methods = top.to_string(index=False).splitlines()
    except Exception as e:
        top_methods = [f"Could not render scorecard table: {repr(e)}"]

    gate_lines = []
    try:
        if "validation_gate_df" in globals() and isinstance(validation_gate_df,_
↳pd.DataFrame) and not validation_gate_df.empty:
            gate_lines += ["", "RC1b validation gate:"]
            gate_lines += validation_gate_df.to_string(index=False).splitlines()
        if "benchmark_gate_df" in globals() and isinstance(benchmark_gate_df,_
↳pd.DataFrame) and not benchmark_gate_df.empty:
            gate_lines += ["", "Benchmark gate:"]
            gate_lines += benchmark_gate_df.to_string(index=False).splitlines()
    except Exception as e:
        gate_lines += [f"Could not render gate tables: {repr(e)}"]

    artifact_lines = []
    try:
        artifact_lines = [str(p.relative_to(RESULTS_DIR)) for p in_
↳sorted(RESULTS_DIR.rglob("*")) if p.is_file()]

```

```

except Exception:
    artifact_lines = []

    with PdfPages(RUN_SUMMARY_PDF_PATH) as pdf:
        text_page(pdf, "MMALS v1.0-RC1b Rotated-MNIST Cross-Benchmark - Run_
↳Summary", lines)
        text_page(pdf, "Top scorecard rows", top_methods if top_methods else_
↳["Scorecard not available yet."])
        text_page(pdf, "Validation / benchmark gates", gate_lines if gate_lines_
↳else ["Gate tables not available yet."])
        text_page(pdf, "Evidence artifacts stored in this run folder",_
↳artifact_lines if artifact_lines else ["No artifacts listed yet."],_
↳fontsize=8)

    return RUN_SUMMARY_PDF_PATH

# Always create a compact summary PDF inside the evidence folder.
summary_pdf_path = write_run_summary_pdf()
print("Run-summary PDF stored at:", summary_pdf_path)

# Try to archive the source notebook if it is saved in Drive or /content.
archived_notebook_path = None
notebook_hits = find_file_by_name(NOTEBOOK_FILENAME)
if notebook_hits:
    archived_notebook_path = copy_if_exists(notebook_hits[0],_
↳NOTEBOOK_EXPORT_DIR / NOTEBOOK_FILENAME)
    print("Notebook source archived:", archived_notebook_path)
else:
    print("Notebook source not found automatically. Save the notebook in Google_
↳Drive with this exact filename if you want it archived:")
    print(" -", NOTEBOOK_FILENAME)

# Optionally copy an existing full notebook PDF into the evidence folder.
archived_full_pdf_path = None
if COPY_EXISTING_NOTEBOOK_PDF_IF_FOUND:
    candidate_pdf_names = [
        EXPECTED_FULL_NOTEBOOK_PDF,
        f"{NOTEBOOK_BASENAME}.pdf",
        f"{NOTEBOOK_BASENAME}_{DATASET_NAME}_{RUN_MODE}.pdf",
    ]
    for pdf_name in candidate_pdf_names:
        hits = find_file_by_name(pdf_name)
        if hits:
            archived_full_pdf_path = copy_if_exists(hits[0],_
↳NOTEBOOK_EXPORT_DIR / pdf_name)

```



```

        print("Existing full notebook PDF archived:",
↪archived_full_pdf_path)
        break
    if archived_full_pdf_path is None:
        print("No existing full notebook PDF found automatically. Expected one
↪of:")
        for pdf_name in candidate_pdf_names:
            print(" -", pdf_name)

# Optional: install dependencies and try full notebook PDF export from inside
↪Colab.
nbconvert_status = {"attempted": False, "success": False, "message": "not
↪attempted"}
if INSTALL_FULL_PDF_EXPORT_DEPS:
    print("Installing full PDF export dependencies. This can take several
↪minutes.")
    subprocess.run("apt-get update -qq && apt-get install -y texlive
↪texlive-xetex texlive-latex-extra pandoc", shell=True, check=False)
    subprocess.run([sys.executable, "-m", "pip", "install", "-q", "nbformat"],
↪check=False)

if TRY_FULL_NOTEBOOK_PDF_EXPORT:
    nbconvert_status["attempted"] = True
    source_for_nbconvert = archived_notebook_path
    if source_for_nbconvert is None and notebook_hits:
        source_for_nbconvert = notebook_hits[0]
    if source_for_nbconvert is None:
        nbconvert_status["message"] = "No notebook source found for nbconvert."
    else:
        try:
            cmd = [
                "jupyter", "nbconvert",
                "--to", "pdf",
                "--output-dir", str(NOTEBOOK_EXPORT_DIR),
                "--PDFExporter.latex_command=['xelatex', '{filename}'],
↪'-interaction=nonstopmode']",
                str(source_for_nbconvert),
            ]
            print("Running:", " ".join(cmd))
            proc = subprocess.run(cmd, text=True, capture_output=True,
↪check=False)
            nbconvert_status["returncode"] = proc.returncode
            nbconvert_status["stdout_tail"] = proc.stdout[-2000:]
            nbconvert_status["stderr_tail"] = proc.stderr[-2000:]
            nbconvert_status["success"] = proc.returncode == 0

```

```

        nbconvert_status["message"] = "nbconvert completed" if proc.
↳returncode == 0 else "nbconvert failed"
        except Exception as e:
            nbconvert_status["message"] = repr(e)

with open(NOTEBOOK_EXPORT_DIR / "notebook_pdf_export_status.json", "w",
↳encoding="utf-8") as f:
    json.dump({
        "notebook_filename": NOTEBOOK_FILENAME,
        "expected_full_notebook_pdf": EXPECTED_FULL_NOTEBOOK_PDF,
        "summary_pdf_path": str(summary_pdf_path),
        "archived_notebook_path": str(archived_notebook_path) if
↳archived_notebook_path else None,
        "archived_full_pdf_path": str(archived_full_pdf_path) if
↳archived_full_pdf_path else None,
        "nbconvert_status": nbconvert_status,
    }, f, indent=2)

log_event("notebook_pdf_evidence_archived", phase="export", payload={
    "summary_pdf_path": str(summary_pdf_path),
    "archived_notebook_path": str(archived_notebook_path) if
↳archived_notebook_path else None,
    "archived_full_pdf_path": str(archived_full_pdf_path) if
↳archived_full_pdf_path else None,
    "nbconvert_status": nbconvert_status,
})
write_manifest(extra={"notebook_pdf_evidence": str(NOTEBOOK_EXPORT_DIR)})

```

```

[30]: {'run_id': 'MMALS_v11_RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT_FashionMNIST_e
vidence_20260526T183427Z_seeds3',
      'trace_id': '49d3678f2bbf0046',
      'created_utc': '2026-05-26T18:53:52.409383+00:00',
      'notebook_version': 'MMALS_v1.1_RC2h_RC1b_Anchored_Conservative_Selector',
      'dataset': 'FashionMNIST',
      'run_mode': 'evidence',
      'device': 'cpu',
      'python': '3.12.13 (main, Mar  4 2026, 09:23:07) [GCC 11.4.0]',
      'platform': 'Linux-6.6.122+-x86_64-with-glibc2.35',
      'config_hash':
'7e545c393f83086ae0b9e2a6b04aca0d5048aee8de5aa098bcf789226705a2a5',
      'config': {'RUN_MODE': 'evidence',
                  'DATASET_NAME': 'FashionMNIST',
                  'PAIR_MODE': 'overlap_chain',
                  'N_TASKS': 5,
                  'N_CONTEXTS': 5,
                  'N_HOSTS': 5,

```

```

'N_CLASSES': 6,
'TRAIN_PER_CLASS': 1200,
'TEST_PER_CLASS': 400,
'SEEDS': [0, 1, 2],
'EPOCHS_PER_TASK': 8,
'MEMORY_PER_TASK': 256,
'MEMORY_PER_CLASS': 256,
'LABEL_NOISE_RATE': 0.1,
'INPUT_NOISE_STD': 0.05,
'CONTEXT_NOISE_RATE': 0.2,
'LATENT_DIM': 64,
'HIDDEN_DIM': 256,
'READOUT_TRAIN_EPOCHS': 5,
'READOUT_TRAIN_STEPS': 50,
'CONTEXT_TEMP_SELECTION_GRID': [0.35, 0.6, 0.8, 1.0, 1.25, 1.5],
'CONTEXT_BLEND_SELECTION_GRID': [['base_proto_first', 0.05, 0.75, 0.2],
['latent_only', 0.0, 1.0, 0.0],
['feature_only', 0.0, 0.0, 1.0],
['latent_dominant', 0.05, 0.9, 0.05],
['single_mid', 0.3, 0.55, 0.15],
['balanced_all', 0.34, 0.33, 0.33]],
'SELECTED_CONTEXT_HEAD_VARIANTS': ['proto_global_no_repair',
'proto_kl_only_true_noop',
'proto_global_head_ce_kl_guard_035',
'oracle_global_no_repair_reference',
'oracle_global_head_ce_kl_guard_035_reference',
'context_blend_selected_global',
'context_temp_blend_selected_global',
'context_blend_selected_head_guard_035',
'context_temp_blend_selected_head_guard_035',
'context_gap_selected'],
'SELECTED_BASELINE_METHODS': ['finetune',
'ewc',
'lfw',
'experience_replay',
'balanced_replay',
'sparse_moe',
'pnn',
'joint_upper_bound'],
'RC2B_CANDIDATE_VARIANTS': ['proto_global_no_repair',
'context_blend_selected_global',
'context_temp_blend_selected_global',
'proto_global_head_ce_kl_guard_035',
'context_temp_blend_selected_head_guard_035',
'context_gap_selected'],
'RC2B_METHOD': 'paired_rc2h_rc1b_anchored_conservative_policy',
'RC3_METHOD': 'paired_rc2h_rc1b_anchored_conservative_policy',

```

```

'RC3_HYDRO_AUDIT_ENABLED': True,
'RC3_HYDRO_POLICY_MODE': 'audit_only',
'RC3_HYDRO_SELECTION_TURBULENCE_WEIGHT': 0.0,
'RC3_HYDRO_SELECTION_LATENT_DRIFT_WEIGHT': 0.0,
'RC2B_STRICT_NO_FINAL_TEST_SELECTION': True,
'NOTEBOOK_FILENAME': 'MMALS_Prototype_1_1_RC2h_RC1b_Anchored_Conservative_Selector_Harness_Colab.ipynb',
'ROTATION_ANGLES': [-30, -15, 0, 15, 30],
'PERMUTATION_SEEDS': [100, 101, 102, 103, 104]},
'persistence_root': '/content/drive/MyDrive/MMALS/v1_1_RC2h_rc1b_anchored_conservative_selector/RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT',
'results_dir': '/content/drive/MyDrive/MMALS/v1_1_RC2h_rc1b_anchored_conservative_selector/RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT/MMALS_v11_RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT_FashionMNIST_evidence_20260526T183427Z_seeds3',
'drive_available': True,
'artifacts': [{'path': 'audit/context_selection_trace_evidence.csv',
'size_bytes': 333005,
'sha256':
'99f691eb03e43481546fe84d4af128ab0a040ccdd7e914041a68c9d1234020a1'},
{'path': 'audit/head_repair_trace_evidence.csv',
'size_bytes': 411599,
'sha256':
'6032335875dd2ac722e20c886308c8fa6a9801263f91f397f7ab1643f3e600b0'},
{'path': 'audit/memory_audit_evidence.csv',
'size_bytes': 7374,
'sha256':
'43981869e6201ab3db7134bc2fa7a0bf27eab5ea7c748fbee09100645635ced1'},
{'path': 'audit/probe_gap_audit_evidence.csv',
'size_bytes': 324,
'sha256':
'cf9bc5dc0442829cfd1369429b920597a41527fddcbaadf95a18e0ae8db3b257'},
{'path': 'audit/rc1b_vs_true_guard_seed_audit_evidence.csv',
'size_bytes': 1095,
'sha256':
'd7054aea1dc46573c824731f881b6f1f56b423192649e064341323cba46c011f'},
{'path': 'audit/rc2d_hydro_policy_selection_audit_evidence.csv',
'size_bytes': 2722,
'sha256':
'0baff21b78d981210c72686af66d05df4fee80d8609e973770cbd24e325af15e'},
{'path': 'audit/seed_diagnostics_evidence.csv',
'size_bytes': 9742,
'sha256':
'add6ebb10819fc232293519fb17b1b1025ad6af1a05d0b9150e496c5b7d057e7'},
{'path': 'audit/weak_seed_report_evidence.json',
'size_bytes': 1915,
'sha256':
'978726a62afa129a8df159ac50bcf03eea29ec8427860ea22af73bf330f33a58'}],

```

```

    {'path': 'benchmark_gate_evidence.csv',
      'size_bytes': 396,
      'sha256':
'073ec6239a9901f19b4754c0dcbf4f094ba6302f72509fd74aa5973c6eeb589f'},
    {'path': 'benchmark_leaderboard_evidence.csv',
      'size_bytes': 8470,
      'sha256':
'08c7646af36bac15c73d92da8dae1c2e3dd45f930b7f08562a9eb2c93c6f2acf'},
    {'path': 'benchmark_method_groups_evidence.csv',
      'size_bytes': 3941,
      'sha256':
'71f25f899f3f52b5f9204bd7542beacdc20e6dbf1c09243539d35b2514025541'},
    {'path': 'benchmark_role_summary_evidence.csv',
      'size_bytes': 196,
      'sha256':
'00a6f6b74cbdcdfc2abfcfdd68782ec4409f16753ea9bb38bc2214430e4d070a'},
    {'path': 'class_confusion_evidence.csv',
      'size_bytes': 258297,
      'sha256':
'4e295fdf9ae2d8d7f5a8635aa229eb9945a39025eff5a68becd7391212f546db'},
    {'path': 'class_diagnostics_evidence.csv',
      'size_bytes': 8304,
      'sha256':
'198221e06d3f30506a7d0dfe59078093e838741f4579bf800c2062ecd0bb70a9'},
    {'path': 'context_gap_repair_logs_evidence.csv',
      'size_bytes': 1857077,
      'sha256':
'91219c645dd2e92884dfcffd7b0b6118f605325db039544819f767bc56b75bcc'},
    {'path': 'forgetting_evidence.csv',
      'size_bytes': 10073,
      'sha256':
'191bac0848dd5065165da3afc97fa3bce68c19d12d746e7060f46885ca8256bf'},
    {'path': 'history_evidence.csv',
      'size_bytes': 294288,
      'sha256':
'd7835cbcd311a62b1dcf3b803b909eecebc4c4bcef9ed2b6d27826d4e3cd2'},
    {'path': 'losses_evidence.csv',
      'size_bytes': 101947,
      'sha256':
'22795e0696da5a7e325507ff22627c2bdce035262bcfe1262b28d877ee1d0a4c'},
    {'path': 'notebook_exports/MMALS_Prototype_1_1_RC2h_RC1b_Anchored_Conservative
_Selector_Harness_Colab.ipynb',
      'size_bytes': 2518108,
      'sha256':
'5932d66061567d1388f31c2904e9e9b48632016f4b4b6b8307d1447c029e55c4'},
    {'path': 'notebook_exports/MMALS_Prototype_1_1_RC2h_RC1b_Anchored_Conservative
_Selector_Harness_Colab.pdf',

```

```

    'size_bytes': 1965233,
    'sha256':
'814d4be96687e6cfb378d412c44cf0cbe9c09a399c3df4e2b7c41a914e176ed4'},
    {'path': 'notebook_exports/MMALS_Prototype_1_1_RC2h_RC1b_Anchored_Conservative
_Selector_Harness_Colab_FashionMNIST_evidence_MMALS_v11_RC2H_RC1B_ANCHORED_CONSE
RVATIVE_HYDRO_AUDIT_FashionMNIST_evidence_20260526T183427Z_seeds3_summary.pdf',
    'size_bytes': 39284,
    'sha256':
'98f50a4c4b0859bd05c552f93861be1f32fa45b1a12ab7e4a903f7118b1b67a8'},
    {'path': 'notebook_exports/notebook_pdf_export_status.json',
    'size_bytes': 1615,
    'sha256':
'618317a3e17b714731ec6e7d18693d79df12df032a3127be76a2644e122abb79'},
    {'path': 'paired_context_deltas_evidence.csv',
    'size_bytes': 1159,
    'sha256':
'3ef594b83c5fcff828ffbd16441b35e0d53f48991dc2cd8c56eb1c281a431d8f'},
    {'path': 'per_seed_scorecard_evidence.csv',
    'size_bytes': 28298,
    'sha256':
'4a47c49d7925737e3b1c535c990d6241b5c4491b63a2c284478f29954d809db5'},
    {'path': 'probe_losses_evidence.csv',
    'size_bytes': 55001,
    'sha256':
'47db9430d35a88098e413beeab055e7869f5ad3f6114cadffefb9a65d258dedf'},
    {'path': 'raw_logs/console_evidence.log',
    'size_bytes': 33235,
    'sha256':
'9ea7c60916fe7e9c04e52c018be5441c8890c141e3617d5ae9df41a63457355d'},
    {'path': 'rc1b_per_seed_vs_true_guard_evidence.csv',
    'size_bytes': 845,
    'sha256':
'03718ea3c06832a9f55a0151c4c954b0e0a12960b82b2bfc013e6c75a5409706'},
    {'path': 'rc1b_validation_gate_evidence.csv',
    'size_bytes': 1146,
    'sha256':
'2526047e68d354f1453859ea7ea06d1e7771fa858a7f15549836df2dd20b3bdc'},
    {'path': 'rc2b_policy_selection_trace_evidence.csv',
    'size_bytes': 103482,
    'sha256':
'e27082b959ec72a64549176fe98ccba4897b02be347db7eb72af378d6d4261b5'},
    {'path': 'scorecard_evidence.csv',
    'size_bytes': 11543,
    'sha256':
'12776b5eae5271653d41cec5314c9e0c3684fc5fd1dc30477f0b2f7b97d24105'},
    {'path': 'traces/otel_style_events_evidence.jsonl',
    'size_bytes': 74176,

```

```

    'sha256':
'10b7d16384e1e9c8c5084705ceba663282fe8ca8f35eebd868de73dc3dcdbd8cc'},
    {'path': 'traces/otel_style_spans_evidence.csv',
     'size_bytes': 13192,
     'sha256':
'80b5cf94f9aedfbb01f476d773f0de725d57cf233bb8b423867ced0af208d759']},
    'extra': {'notebook_pdf_evidence': '/content/drive/MyDrive/MMALS/v1_1_RC2h_rc1b_anchored_conservative_selector/RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT/MMALS_v11_RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT_FashionMNIST_evidence_20260526_T183427Z_seeds3/notebook_exports'}}}

```

1.29 15.3 RC2d gate-aligned no-leakage policy-selection gate

This gate evaluates the selected policy row `paired_rc2d_gate_aligned_policy`.

It checks both scientific performance and the strict process claim: policy selection must be performed from validation memory before final-test evaluation.

```

[31]: # -----
# v1.1-RC2h RC1b-anchored conservative strict policy-selection gate.
# -----
RC2C_PRIMARY = RC2B_METHOD
RC3_PRIMARY = RC2C_PRIMARY
RC2B_PRIMARY = RC3_PRIMARY
RC1B_PRIMARY = "paired_context_gap_selected"

def _score_row(method):
    return get_row(scorecard_df, method)

rc2b_row = _score_row(RC2B_PRIMARY)
rc1b_row = _score_row(RC1B_PRIMARY)
balanced_replay_row = _score_row("baseline_balanced_replay")
experience_replay_row = _score_row("baseline_experience_replay")

replay_candidates = [r for r in [balanced_replay_row, experience_replay_row] if
    ↪ r is not None]
best_replay_row = max(replay_candidates, key=lambda r: metric(r,
    ↪ "final_avg_acc_mean")) if replay_candidates else None

non_diag = non_diagnostic_scorecard(scorecard_df)
mmals_deployable = non_diag[non_diag.method.str.startswith("paired_",
    ↪ na=False)] if not non_diag.empty else pd.DataFrame()
# Best deployable MMALS row, but exclude oracle references and probes by
    ↪ construction.
best_mmals_row = mmals_deployable.sort_values("final_avg_acc_mean",
    ↪ ascending=False).iloc[0] if not mmals_deployable.empty else None

```

```

selection_expected_rows = len(SEEDS) * N_TASKS * len(RC2B_CANDIDATE_VARIANTS)
selected_rows = selection_log_df[selection_log_df.get("is_selected_candidate",
    ↪pd.Series(dtype=bool)).astype(bool)] if "selection_log_df" in globals() and
    ↪not selection_log_df.empty else pd.DataFrame()
selected_per_checkpoint = selected_rows.groupby(["seed", "after_task"]).size().
    ↪reset_index(name="n_selected") if not selected_rows.empty else pd.DataFrame()

rc2b_gate_rows = []
def add_rc2b_gate(name, passed, detail):
    rc2b_gate_rows.append({"check": name, "passed": bool(passed), "detail":
    ↪str(detail)})

add_rc2b_gate(
    "policy_level_validation_proxy_available_all_candidates",
    (not selection_log_df.empty) and len(selection_log_df) >=
    ↪selection_expected_rows and selection_log_df["proxy_acc"].notna().all(),
    f"selection_rows={len(selection_log_df) if 'selection_log_df' in globals()
    ↪else 0}; expected>={selection_expected_rows}",
)
add_rc2b_gate(
    "exactly_one_policy_selected_per_seed_task",
    (not selected_per_checkpoint.empty) and
    ↪bool((selected_per_checkpoint["n_selected"] == 1).all()) and
    ↪len(selected_per_checkpoint) == len(SEEDS) * N_TASKS,
    f"selected_checkpoints={len(selected_per_checkpoint)}; expected={len(SEEDS)
    ↪* N_TASKS}",
)
add_rc2b_gate(
    "no_final_test_used_for_selection_declared",
    (not selection_log_df.empty) and
    ↪bool(selection_log_df["no_final_test_used_for_selection"].astype(bool).
    ↪all()),
    "selection trace marks no_final_test_used_for_selection=True for every
    ↪candidate row.",
)
if rc2b_row is not None and rc1b_row is not None:
    add_rc2b_gate(
        "rc2b_at_least_fixed_rc1b_with_tolerance",
        metric(rc2b_row, "final_avg_acc_mean") >= metric(rc1b_row,
    ↪"final_avg_acc_mean") - RC2B_SELECTION_TIE_TOL,
        f"RC2b={metric(rc2b_row, 'final_avg_acc_mean'):.4f};
    ↪RC1b={metric(rc1b_row, 'final_avg_acc_mean'):.4f}; delta={metric(rc2b_row,
    ↪'final_avg_acc_mean') - metric(rc1b_row, 'final_avg_acc_mean'):.4f}",
    )
if rc2b_row is not None and best_replay_row is not None:
    add_rc2b_gate(

```



```

        "rc2b_beats_best_replay_margin",
        metric(rc2b_row, "final_avg_acc_mean") >= metric(best_replay_row,
↪ "final_avg_acc_mean") + 0.005,
        f"RC2b={metric(rc2b_row, 'final_avg_acc_mean'):.4f}";
↪ best_replay={metric(best_replay_row, 'final_avg_acc_mean'):.4f}",
    )
if rc2b_row is not None and best_mmals_row is not None:
    add_rc2b_gate(
        "rc2b_within_tolerance_of_best_deployable_mmals",
        metric(rc2b_row, "final_avg_acc_mean") >= metric(best_mmals_row,
↪ "final_avg_acc_mean") - max(0.005, RC2B_SELECTION_TIE_TOL),
        f"RC2b={metric(rc2b_row, 'final_avg_acc_mean'):.4f}";
↪ best_mmals={metric(best_mmals_row, 'final_avg_acc_mean'):.4f}";
↪ best={best_mmals_row.method}",
    )
if rc2b_row is not None:
    add_rc2b_gate("rc2b_min_task_floor", metric(rc2b_row, "min_task_acc_mean")
↪ >= 0.80, f"min_task={metric(rc2b_row, 'min_task_acc_mean'):.4f}")
    add_rc2b_gate("rc2b_class2_floor", metric(rc2b_row, "class2_acc_mean") >= 0.
↪ 80, f"class2={metric(rc2b_row, 'class2_acc_mean'):.4f}")
    add_rc2b_gate("rc2b_class4_boundary_floor", metric(rc2b_row,
↪ "class4_acc_mean") >= 0.20, f"class4={metric(rc2b_row, 'class4_acc_mean'):.
↪ 4f}")
    add_rc2b_gate("rc2b_class5_floor", metric(rc2b_row, "class5_acc_mean") >= 0.
↪ 90, f"class5={metric(rc2b_row, 'class5_acc_mean'):.4f}")
    add_rc2b_gate("rc2b_forgetting_controlled", metric(rc2b_row,
↪ "avg_forgetting_mean") <= 0.08, f"forgetting={metric(rc2b_row,
↪ 'avg_forgetting_mean'):.4f}")

# RC2c-specific risk-aware selector audit gates.
if RC2C_RISK_AWARE_SELECTOR_ENABLED:
    add_rc2b_gate(
        "rc2c_risk_aware_selector_enabled",
        True,
        "RC2c risk-aware validation selector is enabled for this profile.",
    )
    if "selection_log_df" in globals() and not selection_log_df.empty:
        required_rc2c_cols = [
            "proxy_min_task",
            "proxy_old_task_min",
            "proxy_task_acc_std",
            "selection_reason",
            "raw_best_variant",
            "best_guarded_variant",

```

```

    ]
    present_rc2c_cols = [c for c in required_rc2c_cols if c in
↪selection_log_df.columns]
    add_rc2b_gate(
        "rc2c_selection_audit_columns_present",
        len(present_rc2c_cols) == len(required_rc2c_cols),
        f"present={present_rc2c_cols}; required={required_rc2c_cols}",
    )
    if "selection_reason" in selection_log_df.columns:
        reason_counts = selection_log_df.loc[selection_log_df.
↪get("is_selected_candidate", False).astype(bool), "selection_reason"].
↪value_counts().to_dict()
        add_rc2b_gate(
            "rc2c_selection_reason_recorded",
            len(reason_counts) > 0,
            f"selected_reason_counts={reason_counts}",
        )

# RC2c / Hydro audit gates.
if RC3_HYDRO_AUDIT_ENABLED:
    hydro_cols_required = ["hydro_latent_drift_Dz", "hydro_turbulence_score",
↪"hydro_audit_memory_only"]
    hydro_cols_present = [c for c in hydro_cols_required if ("selection_log_df"
↪in globals() and not selection_log_df.empty and c in selection_log_df.
↪columns)]
    add_rc2b_gate(
        "rc3b_hydro_audit_metrics_present",
        len(hydro_cols_present) == len(hydro_cols_required),
        f"present={hydro_cols_present}; required={hydro_cols_required}",
    )
    if "selection_log_df" in globals() and not selection_log_df.empty and
↪"hydro_audit_memory_only" in selection_log_df.columns:
        add_rc2b_gate(
            "rc3b_hydro_audit_uses_validation_memory_only",
            bool(selection_log_df["hydro_audit_memory_only"].fillna(False).
↪astype(bool).any()),
            "Hydro audit rows are marked memory-only; final test is not used
↪for selection/audit metrics.",
        )
    if "selection_log_df" in globals() and not selection_log_df.empty and
↪"hydro_turbulence_score" in selection_log_df.columns:
        selected_hydro = selected_rows if "selected_rows" in globals() and not
↪selected_rows.empty else pd.DataFrame()
        selected_turbulence = float(selected_hydro["hydro_turbulence_score"].
↪mean()) if not selected_hydro.empty and "hydro_turbulence_score" in
↪selected_hydro else np.nan

```

```

        add_rc2b_gate(
            "rc3b_selected_policy_hydro_turbulence_reported",
            not np.isnan(selected_turbulence),
            f"selected_policy_mean_hydro_turbulence={selected_turbulence}",
        )
    else:
        add_rc2b_gate(
            "rc3b_hydro_audit_disabled_by_profile",
            True,
            f"EXPERIMENT_PROFILE={EXPERIMENT_PROFILE}; Hydro audit intentionally_
↳ disabled.",
        )

# RC2H-specific anchor/selector/final-alignment gates.
if rc2b_row is not None and best_mmals_row is not None:
    add_rc2b_gate(
        "rc2h_selector_final_alignment_to_best_deployable",
        metric(rc2b_row, "final_avg_acc_mean") >= metric(best_mmals_row,
↳ "final_avg_acc_mean") - RC2D_SELECTOR_FINAL_ALIGNMENT_TOL,
        f"RC2H={metric(rc2b_row, 'final_avg_acc_mean'):.4f};
↳ best_deployable={metric(best_mmals_row, 'final_avg_acc_mean'):.4f};
↳ tol={RC2D_SELECTOR_FINAL_ALIGNMENT_TOL}",
    )
if rc2b_row is not None:
    add_rc2b_gate(
        "rc2h_class2_robustness_floor",
        metric(rc2b_row, "class2_acc_mean") >= 0.80,
        f"class2={metric(rc2b_row, 'class2_acc_mean'):.4f}; target>=0.80",
    )
    add_rc2b_gate(
        "rc2h_class5_preservation_floor",
        metric(rc2b_row, "class5_acc_mean") >= 0.90,
        f"class5={metric(rc2b_row, 'class5_acc_mean'):.4f}; target>=0.90",
    )

    tri_vals = [metric(rc2b_row, "class2_acc_mean"), metric(rc2b_row,
↳ "class4_acc_mean"), metric(rc2b_row, "class5_acc_mean")]
    tri_vals = [v for v in tri_vals if not np.isnan(v)]
    tri_min = float(np.min(tri_vals)) if tri_vals else np.nan
    tri_gap = float(np.max(tri_vals) - np.min(tri_vals)) if tri_vals else np.nan
    tri_target = 0.45 if RUN_MODE == "smoke" else 0.75
    add_rc2b_gate(
        "rc2h_tri_boundary_min_floor",
        (not np.isnan(tri_min)) and tri_min >= tri_target,
        f"tri_min={tri_min:.4f}; tri_gap={tri_gap:.4f}; target>={tri_target:.
↳ 2f}; classes=(2,4,5)",
    )

```

```

    )
    add_rc2b_gate(
        "rc2h_tri_boundary_balance_gap",
        (not np.isnan(tri_gap)) and tri_gap <= (0.55 if RUN_MODE == "smoke"
↪else 0.30),
        f"tri_gap={tri_gap:.4f}; target<= {(0.55 if RUN_MODE == 'smoke' else 0.
↪30):.2f}",
    )
if not selection_log_df.empty:
    add_rc2b_gate(
        "rc2h_selection_reason_audit_present",
        "selection_reason" in selection_log_df.columns and
↪selection_log_df["selection_reason"].notna().any(),
        "selection_reason is present in policy-selection trace.",
    )

rc2b_gate_df = pd.DataFrame(rc2b_gate_rows)
rc2b_gate_path = RESULTS_DIR /
↪f"rc2h_rc1b_anchored_conservative_policy_gate_{RUN_MODE}.csv"
rc2b_gate_df.to_csv(rc2b_gate_path, index=False)
print("Saved RC2H RC1b-anchored conservative policy gate:", rc2b_gate_path)
display(rc2b_gate_df)

RC2D_GATE_ALIGNED_STRICT_PASSED = bool(rc2b_gate_df["passed"].all()) if not
↪rc2b_gate_df.empty else False
RC2B_STRICT_PASSED = RC2D_GATE_ALIGNED_STRICT_PASSED
print("RC3B_CONFIGURABLE_STRICT_PASSED:", RC2D_GATE_ALIGNED_STRICT_PASSED)

if not selected_rows.empty:
    selected_policy_summary_df = selected_rows.groupby([
        "selected_candidate_variant", "selected_candidate_method",
↪"selected_policy_family", "selected_policy_label"
    ], as_index=False).agg(
        n_selected=("is_selected_candidate", "sum"),
        mean_selected_proxy_acc=("selected_proxy_acc", "mean"),
        mean_selected_proxy_score=("selected_proxy_score", "mean"),
    ).sort_values("n_selected", ascending=False)
    selected_policy_summary_path = RESULTS_DIR /
↪f"rc2b_selected_policy_summary_{RUN_MODE}.csv"
    selected_policy_summary_df.to_csv(selected_policy_summary_path, index=False)
    print("Saved selected policy summary:", selected_policy_summary_path)
    display(selected_policy_summary_df)
else:
    selected_policy_summary_df = pd.DataFrame()

```

```

                                check passed \
0  policy_level_validation_proxy_available_all_ca...  True

```

1	exactly_one_policy_selected_per_seed_task	True
2	no_final_test_used_for_selection_declared	True
3	rc2b_at_least_fixed_rc1b_with_tolerance	False
4	rc2b_beats_best_replay_margin	True
5	rc2b_within_tolerance_of_best_deployable_mmals	False
6	rc2b_min_task_floor	False
7	rc2b_class2_floor	False
8	rc2b_class4_boundary_floor	True
9	rc2b_class5_floor	True
10	rc2b_forgetting_controlled	False
11	rc2c_risk_aware_selector_enabled	True
12	rc2c_selection_audit_columns_present	True
13	rc2c_selection_reason_recorded	True
14	rc3b_hydro_audit_metrics_present	True
15	rc3b_hydro_audit_uses_validation_memory_only	True
16	rc3b_selected_policy_hydro_turbulence_reported	True
17	rc2h_selector_final_alignment_to_best_deployable	False
18	rc2h_class2_robustness_floor	False
19	rc2h_class5_preservation_floor	True
20	rc2h_tri_boundary_min_floor	True
21	rc2h_tri_boundary_balance_gap	True
22	rc2h_selection_reason_audit_present	True

```

                                detail
0      selection_rows=90; expected>=90
1      selected_checkpoints=15; expected=15
2  selection trace marks no_final_test_used_for_s...
3      RC2b=0.8788; RC1b=0.8857; delta=-0.0069
4      RC2b=0.8788; best_replay=0.8361
5  RC2b=0.8788; best_mmals=0.8957; best=paired_pr...
6      min_task=0.7358
7      class2=0.7696
8      class4=0.8533
9      class5=0.9942
10     forgetting=0.0958
11  RC2c risk-aware validation selector is enabled...
12  present=['proxy_min_task', 'proxy_old_task_min...
13  selected_reason_counts={'keep_rc1b_anchor_reje...
14  present=['hydro_latent_drift_Dz', 'hydro_turbu...
15  Hydro audit rows are marked memory-only; final...
16  selected_policy_mean_hydro_turbulence=1.142921...
17      RC2H=0.8788; best_deployable=0.8957; tol=0.005
18      class2=0.7696; target>=0.80
19      class5=0.9942; target>=0.90
20  tri_min=0.7696; tri_gap=0.2246; target>=0.75; ...
21      tri_gap=0.2246; target<= 0.30
22  selection_reason is present in policy-selectio...

```

	selected_candidate_variant \	
1	context_gap_selected	
0	context_blend_selected_global	
2	context_temp_blend_selected_global	

	selected_candidate_method	selected_policy_family \
1	paired_context_gap_selected	rc1b_guarded_context_gap
0	paired_context_blend_selected_global	context_only_global
2	paired_context_temp_blend_selected_global	context_only_global

	selected_policy_label	n_selected \
1	RC1b guarded context-gap	13
0	validation-selected context-only global head	1
2	validation-selected context-only global head	1

	mean_selected_proxy_acc	mean_selected_proxy_score
1	0.925853	1.542663
0	0.909063	1.690517
2	0.955339	1.803555

```
[32]: # Finalize persistent logs, manifest, and ZIP package.
log_event("packaging_started", phase="export", payload={"results_dir":
↳str(RESULTS_DIR)})
write_manifest(extra={"packaging_started": True})
stop_persistent_logging()

output_base = RESULTS_DIR.parent / f"{RESULTS_DIR.name}_package"
zip_path = Path(shutil.make_archive(str(output_base), "zip", RESULTS_DIR))
zip_sha = file_sha256(zip_path)

package_sha_path = zip_path.with_name(zip_path.stem + "_sha256.txt")
package_sha_path.write_text(f"{zip_sha} {zip_path.name}\n", encoding="utf-8")

release_manifest_path = zip_path.with_name(zip_path.stem + "_release_manifest.
↳json")
release_manifest = {
    "run_id": RUN_ID,
    "trace_id": TRACE_ID,
    "dataset": DATASET_NAME,
    "experiment_profile": EXPERIMENT_PROFILE,
    "policy_branch": POLICY_BRANCH,
    "activate_hydro_audit": ACTIVATE_HYDRO_AUDIT,
    "activate_hydro_regularizer": ACTIVATE_HYDRO_REGULARIZER,
    "activate_hydro_selection": ACTIVATE_HYDRO_SELECTION,
    "hydro_regularizer_mode": HYDRO_REGULARIZER_MODE,
    "run_mode": RUN_MODE,
    "pair_mode": PAIR_MODE,
```

```

    "rotation_angles": ROTATION_ANGLES if DATASET_NAME == "RotatedMNIST" else_
↪None,
    "permutation_seeds": PERMUTATION_SEEDS if DATASET_NAME == "PermutedMNIST"
↪else None,
    "rc2f_method": RC3_METHOD,
    "rc2b_compat_method_name": RC2B_METHOD,
    "rc2f_candidate_variants": RC2B_CANDIDATE_VARIANTS,
    "rc3_hydro_audit_enabled": RC3_HYDRO_AUDIT_ENABLED,
    "rc3_hydro_policy_mode": RC3_HYDRO_POLICY_MODE,
    "rc2g_tri_boundary_strict_passed": bool(globals().
↪get("RC2D_GATE_ALIGNED_STRICT_PASSED", False)),
    "notebook_filename": NOTEBOOK_FILENAME,
    "results_dir": str(RESULTS_DIR),
    "zip_package": str(zip_path),
    "zip_sha256": zip_sha,
    "package_sha256_file": str(package_sha_path),
    "run_manifest": str(MANIFEST_PATH),
    "notebook_export_dir": str(NOTEBOOK_EXPORT_DIR),
    "created_utc": datetime.now(timezone.utc).isoformat(),
}
release_manifest_path.write_text(json.dumps(safe_jsonable(release_manifest),
↪indent=2, ensure_ascii=False), encoding="utf-8")

# Keep a human-friendly evidence bundle inside the run folder.
TRUSTED_EVIDENCE_DIR.mkdir(parents=True, exist_ok=True)
for evidence_file in sorted(RESULTS_DIR.glob("*.csv")):
    if any(token in evidence_file.name for token in ["scorecard",
↪"leaderboard", "gate", "selection", "summary"]):
        try:
            shutil.copy2(evidence_file, TRUSTED_EVIDENCE_DIR / evidence_file.
↪name)
        except Exception as e:
            print("TrustedEvidence CSV copy skipped:", evidence_file, repr(e))
for evidence_file in [MANIFEST_PATH, release_manifest_path, package_sha_path]:
    try:
        if evidence_file.exists():
            shutil.copy2(evidence_file, TRUSTED_EVIDENCE_DIR / evidence_file.
↪name)
        except Exception as e:
            print("TrustedEvidence manifest/hash copy skipped:", evidence_file,
↪repr(e))
    try:
        shutil.copy2(zip_path, TRUSTED_EVIDENCE_DIR / zip_path.name)
    except Exception as e:
        print("TrustedEvidence ZIP copy skipped:", repr(e))

```

```

print(f"Results zipped to: {zip_path}")
print(f"ZIP SHA-256: {zip_sha}")
print(f"Package SHA file: {package_sha_path}")
print(f"Release manifest: {release_manifest_path}")

# Add ZIP metadata to the run manifest after the ZIP exists.
write_manifest(extra={
    "zip_package": str(zip_path),
    "zip_sha256": zip_sha,
    "package_sha256_file": str(package_sha_path),
    "release_manifest": str(release_manifest_path),
})
print("Final manifest:", MANIFEST_PATH)

```

1.30 17. Next scientific step after RC3

If RC3 strict passes across the MNIST-family benchmarks, the next step is not simply to declare Hydro a performance winner.

The next step is:

MMALS v1.1-RC4 / v1.2 - RC2c / Hydro-audited policy selection across harder visual and class-i

Decision logic:

- If RC3 selects the same policy as RC2c but reports low Hydro turbulence, Hydro is validated as an audit layer.
- If RC3 changes the selected policy only when stability-aware weights are enabled, run an ablation before claiming improvement.
- If Hydro audit warns on the selected policy while accuracy remains high, treat this as a debuggability finding rather than a failure.
- Only after an aligned inferred-context/global-head protocol should Hydro be considered as a default learning regularizer.

1.31 17. Explanation for a 12-year-old kid

Before, the mushroom model was tested on normal images.

Now we can make the game harder in two ways:

1. rotate the handwritten digits (`RotatedMNIST`);
2. scramble the pixels differently for each lesson (`PermutedMNIST`).

The lessons still overlap:

- lesson 0: digits 0 and 1;
- lesson 1: digits 1 and 2;
- lesson 2: digits 2 and 3;
- lesson 3: digits 3 and 4;
- lesson 4: digits 4 and 5.

For `RotatedMNIST`, each lesson is turned a little differently:

- -30 degrees;
- -15 degrees;
- 0 degrees;
- +15 degrees;
- +30 degrees.

For `PermutedMNIST`, each lesson uses a different secret pixel shuffle:

- seed 100;
- seed 101;
- seed 102;
- seed 103;
- seed 104.

So the mushroom must not only remember old digits. It must also understand that the situation changed.

In this observability patch, we keep a black box recorder, like in an airplane.

It records:

- which situation the mushroom thought it was in;
- which memory/prototype it used;
- whether the head/readout was stable;
- whether one seed was weaker than the others;
- whether the model beat replay and other baselines.

The important idea is:

We are not changing the mushroom again. We are testing whether the same mushroom stays reliable on harder tracks.

1.32 18. Recommended usage

Use this progression:

1. Run `RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT` in `smoke` mode to verify execution, anchor audit, Drive archive and artifact export.
2. Run `RC2H_RC1B_ANCHORED_CONSERVATIVE_NO_HYDRO` if you need to isolate selector effects without Hydro audit overhead.
3. Run `RC2B_STRICT` only as a reproducibility/control branch.
4. Run `RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_STABILITY_ABLATION` only after audit-only evidence shows meaningful drift/turbulence differences.
5. Move to `evidence`, then `robust`, only if smoke preserves the RC1b anchor and avoids accidental no-repair or weak-class overrides.

A result should always report the profile name. Do not compare audit-only and Hydro-stability-selected runs as if they had the same selection objective.

1.33 Resources

This notebook is intended to be saved as:

MMALS_Prototype_1_1_RC2h_RC1b_Anchored_Conservative_Selector_Harness_Colab.ipynb

Recommended profile for first RC2H execution:

```
EXPERIMENT_PROFILE = "RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT"  
RUN_MODE = "smoke"  
DATASET_NAME = "FashionMNIST"
```

Expected Drive structure:

```
MMALS/  
  v1_1_RC2h_rc1b_anchored_conservative_selector/  
    RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT/  
      MMALS_v11_RC2H_RC1B_ANCHORED_CONSERVATIVE_HYDRO_AUDIT_FashionMNIST_<mode>_<timestamp>_se  
        TrustedEvidence/  
        audit/  
        notebook_exports/  
        plots/  
        raw_logs/  
        traces/  
        history_<mode>.csv  
        scorecard_<mode>.csv  
        per_seed_scorecard_<mode>.csv  
        rc2b_policy_selection_trace_<mode>.csv  
        package.zip
```

Scientific claim for RC2H:

RC2H is not a new backbone. It is a conservative, no-leakage deployment selector anchored on the historically robust RC1b context-gap policy. Candidate overrides are permitted only when validation-memory evidence is strong and weak-class/tri-boundary safety gates are satisfied.

```
[33]: from google.colab import drive  
      drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
[ ]: import os  
  
!apt-get update  
!apt-get install texlive texlive-xetex texlive-latex-extra pandoc  
!pip install py pandoc  
!pip -q install nbformat
```

```
0% [Working]          Get:1 https://cli.github.com/packages stable  
InRelease [3,917 B]  
Get:2 https://cloud.r-project.org/bin/linux/ubuntu jammy-cran40/ InRelease  
[3,632 B]  
Get:3 https://cloud.r-project.org/bin/linux/ubuntu jammy-cran40/ Packages [95.6  
kB]
```

```

Get:4 http://security.ubuntu.com/ubuntu jammy-security InRelease [129 kB]
Hit:5 http://archive.ubuntu.com/ubuntu jammy InRelease
Get:6 http://archive.ubuntu.com/ubuntu jammy-updates InRelease [128 kB]
Get:7 http://security.ubuntu.com/ubuntu jammy-security/main amd64 Packages
[3,955 kB]
Get:8 http://security.ubuntu.com/ubuntu jammy-security/universe amd64 Packages
[1,295 kB]
Get:9 http://security.ubuntu.com/ubuntu jammy-security/multiverse amd64 Packages
[77.8 kB]
Get:10 http://security.ubuntu.com/ubuntu jammy-security/restricted amd64
Packages [7,084 kB]
Get:11 https://r2u.stat.illinois.edu/ubuntu jammy InRelease [6,555 B]
Hit:12 https://ppa.launchpadcontent.net/deadsnakes/ppa/ubuntu jammy InRelease
Get:13 https://r2u.stat.illinois.edu/ubuntu jammy/main amd64 Packages [3,021 kB]
Hit:14 https://ppa.launchpadcontent.net/ubuntugis/ppa/ubuntu jammy InRelease
Get:15 http://archive.ubuntu.com/ubuntu jammy-backports InRelease [127 kB]
Get:16 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 Packages
[1,602 kB]
Get:17 http://archive.ubuntu.com/ubuntu jammy-updates/multiverse amd64 Packages
[86.4 kB]
Get:18 http://archive.ubuntu.com/ubuntu jammy-updates/restricted amd64 Packages
[7,361 kB]
Get:19 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 Packages [4,296
kB]
Get:20 https://r2u.stat.illinois.edu/ubuntu jammy/main all Packages [10.2 MB]
Fetched 39.5 MB in 4s (9,468 kB/s)
Reading package lists... Done
W: Skipping acquire of configured file 'main/source/Sources' as repository
'https://r2u.stat.illinois.edu/ubuntu jammy InRelease' does not seem to provide
it (sources.list entry misspelt?)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
pandoc is already the newest version (2.9.2.1-3ubuntu2).
pandoc set to manually installed.
The following additional packages will be installed:
  dvisvgm fonts-droid-fallback fonts-lato fonts-lmodern fonts-noto-mono
  fonts-texgyre fonts-urw-base35 libapache-pom-java libcommons-logging-java
  libcommons-parent-java libfontbox-java libgs9 libgs9-common libidn12
  libijs-0.35 libjbig2dec0 libkpathsea6 libpdfbox-java libptexenc1 libruby3.0
  libsyntax2 libteckit0 libtexlua53 libtexluajit2 libwoff1 libzip-0-13
  lmodern poppler-data preview-latex-style rake ruby ruby-net-telnet
  ruby-rubygems ruby-webrick ruby-xmlrpc ruby3.0 rubygems-integration t1utils
  teckit tex-common tex-gyre texlive-base texlive-binaries
  texlive-fonts-recommended texlive-latex-base texlive-latex-recommended
  texlive-pictures texlive-plain-generic tipa xfonts-encodings xfonts-utils
Suggested packages:
  fonts-noto fonts-freefont-otf | fonts-freefont-ttf libavalon-framework-java

```

```
libcommons-logging-java-doc libexcalibur-logkit-java liblog4j1.2-java
poppler-utils ghostscript fonts-japanese-mincho | fonts-ipafont-mincho
fonts-japanese-gothic | fonts-ipafont-gothic fonts-arphic-ukai
fonts-arphic-uming fonts-nanum ri ruby-dev bundler debhelper gv
| postscript-viewer perl-tk xpdf | pdf-viewer xzdec
texlive-fonts-recommended-doc texlive-latex-base-doc python3-pygments
icc-profiles libfile-which-perl libspreadsheet-parseexcel-perl
texlive-latex-extra-doc texlive-latex-recommended-doc texlive-luatex
texlive-pstricks dot2tex prerex texlive-pictures-doc vprerex
default-jre-headless tipa-doc
```

The following NEW packages will be installed:

```
dvisvgm fonts-droid-fallback fonts-lato fonts-lmodern fonts-noto-mono
fonts-texgyre fonts-urw-base35 libapache-pom-java libcommons-logging-java
libcommons-parent-java libfontbox-java libgs9 libgs9-common libidn12
libijs-0.35 libjbig2dec0 libkpathsea6 libpdfbox-java libptexenc1 libruby3.0
libsyntax2 libteckit0 libtexlua53 libtexluajit2 libwoff1 libzip-0-13
lmodern poppler-data preview-latex-style rake ruby ruby-net-telnet
ruby-rubygems ruby-webrick ruby-xmlrpc ruby3.0 rubygems-integration t1utils
teckit tex-common tex-gyre texlive texlive-base texlive-binaries
texlive-fonts-recommended texlive-latex-base texlive-latex-extra
texlive-latex-recommended texlive-pictures texlive-plain-generic
texlive-xetex tipa xfonts-encodings xfonts-utils
```

0 upgraded, 54 newly installed, 0 to remove and 20 not upgraded.

Need to get 182 MB of archives.

After this operation, 571 MB of additional disk space will be used.

Get:1 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 fonts-droid-fallback all 1:6.0.1r16-1.1build1 [1,805 kB]

Get:2 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 fonts-lato all 2.0-2.1 [2,696 kB]

Get:3 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 poppler-data all 0.4.11-1 [2,171 kB]

Get:4 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 tex-common all 6.17 [33.7 kB]

Get:5 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 fonts-urw-base35 all 20200910-1 [6,367 kB]

Get:6 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libgs9-common all 9.55.0~dfsg1-0ubuntu5.13 [753 kB]

Get:7 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libidn12 amd64 1.38-4ubuntu1 [60.0 kB]

Get:8 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 libijs-0.35 amd64 0.35-15build2 [16.5 kB]

Get:9 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 libjbig2dec0 amd64 0.19-3build2 [64.7 kB]

Get:10 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libgs9 amd64 9.55.0~dfsg1-0ubuntu5.13 [5,032 kB]

Get:11 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libkpathsea6 amd64 2021.20210626.59705-1ubuntu0.3 [60.6 kB]

Get:12 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 libwoff1 amd64

1.0.2-1build4 [45.2 kB]
Get:13 http://archive.ubuntu.com/ubuntu jammy/universe amd64 dvisvgm amd64
2.13.1-1 [1,221 kB]
Get:14 http://archive.ubuntu.com/ubuntu jammy/universe amd64 fonts-lmodern all
2.004.5-6.1 [4,532 kB]
Get:15 http://archive.ubuntu.com/ubuntu jammy/main amd64 fonts-noto-mono all
20201225-1build1 [397 kB]
Get:16 http://archive.ubuntu.com/ubuntu jammy/universe amd64 fonts-texgyre all
20180621-3.1 [10.2 MB]
Get:17 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libapache-pom-java
all 18-1 [4,720 B]
Get:18 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libcommons-parent-
java all 43-1 [10.8 kB]
Get:19 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libcommons-logging-
java all 1.2-2 [60.3 kB]
Get:20 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libptexenc1
amd64 2021.20210626.59705-1ubuntu0.3 [39.1 kB]
Get:21 http://archive.ubuntu.com/ubuntu jammy/main amd64 rubygems-integration
all 1.18 [5,336 B]
Get:22 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 ruby3.0 amd64
3.0.2-7ubuntu2.12 [50.1 kB]
Get:23 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 ruby-rubygems
all 3.3.5-2ubuntu1.2 [228 kB]
Get:24 http://archive.ubuntu.com/ubuntu jammy/main amd64 ruby amd64 1:3.0~exp1
[5,100 B]
Get:25 http://archive.ubuntu.com/ubuntu jammy/main amd64 rake all 13.0.6-2 [61.7
kB]
Get:26 http://archive.ubuntu.com/ubuntu jammy/main amd64 ruby-net-telnet all
0.1.1-2 [12.6 kB]
Get:27 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 ruby-webrick
all 1.7.0-3ubuntu0.2 [52.5 kB]
Get:28 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 ruby-xmlrpc all
0.3.2-1ubuntu0.1 [24.9 kB]
Get:29 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libruby3.0
amd64 3.0.2-7ubuntu2.12 [5,113 kB]
Get:30 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libsynchronet2
amd64 2021.20210626.59705-1ubuntu0.3 [55.8 kB]
Get:31 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libteckit0 amd64
2.5.11+ds1-1 [421 kB]
Get:32 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libtexlua53
amd64 2021.20210626.59705-1ubuntu0.3 [120 kB]
Get:33 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libtexluajit2
amd64 2021.20210626.59705-1ubuntu0.3 [267 kB]
Get:34 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libzzip-0-13 amd64
0.13.72+dfsg.1-1.1 [27.0 kB]
Get:35 http://archive.ubuntu.com/ubuntu jammy/main amd64 xfonts-encodings all
1:1.0.5-0ubuntu2 [578 kB]
Get:36 http://archive.ubuntu.com/ubuntu jammy/main amd64 xfonts-utils amd64

```

1:7.7+6build2 [94.6 kB]
Get:37 http://archive.ubuntu.com/ubuntu jammy/universe amd64 lmodern all
2.004.5-6.1 [9,471 kB]
Get:38 http://archive.ubuntu.com/ubuntu jammy/universe amd64 preview-latex-style
all 12.2-1ubuntu1 [185 kB]
Get:39 http://archive.ubuntu.com/ubuntu jammy/main amd64 t1utils amd64
1.41-4build2 [61.3 kB]
Get:40 http://archive.ubuntu.com/ubuntu jammy/universe amd64 teckit amd64
2.5.11+ds1-1 [699 kB]
Get:41 http://archive.ubuntu.com/ubuntu jammy/universe amd64 tex-gyre all
20180621-3.1 [6,209 kB]
Get:42 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 texlive-
binaries amd64 2021.20210626.59705-1ubuntu0.3 [9,861 kB]
Get:43 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-base all
2021.20220204-1 [21.0 MB]
Get:44 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-fonts-
recommended all 2021.20220204-1 [4,972 kB]
Get:45 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-latex-base
all 2021.20220204-1 [1,128 kB]
Get:46 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-latex-
recommended all 2021.20220204-1 [14.4 MB]
Get:47 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive all
2021.20220204-1 [14.3 kB]
Get:48 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libfontbox-java all
1:1.8.16-2 [207 kB]
Get:49 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libpdfbox-java all
1:1.8.16-2 [5,199 kB]
Get:50 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-pictures
all 2021.20220204-1 [8,720 kB]
Get:51 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-latex-extra
all 2021.20220204-1 [13.9 MB]
Get:52 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-plain-
generic all 2021.20220204-1 [27.5 MB]
Get:53 http://archive.ubuntu.com/ubuntu jammy/universe amd64 tipa all 2:1.3-21
[2,967 kB]
Get:54 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-xetex all
2021.20220204-1 [12.4 MB]
Fetched 182 MB in 1s (123 MB/s)
Extracting templates from packages: 100%
Preconfiguring packages ...
Selecting previously unselected package fonts-droid-fallback.
(Reading database ... 118212 files and directories currently installed.)
Preparing to unpack .../00-fonts-droid-fallback_1%3a6.0.1r16-1.1build1_all.deb
...
Unpacking fonts-droid-fallback (1:6.0.1r16-1.1build1) ...
Selecting previously unselected package fonts-lato.
Preparing to unpack .../01-fonts-lato_2.0-2.1_all.deb ...
Unpacking fonts-lato (2.0-2.1) ...

```

```

Selecting previously unselected package poppler-data.
Preparing to unpack .../02-poppler-data_0.4.11-1_all.deb ...
Unpacking poppler-data (0.4.11-1) ...
Selecting previously unselected package tex-common.
Preparing to unpack .../03-tex-common_6.17_all.deb ...
Unpacking tex-common (6.17) ...
Selecting previously unselected package fonts-urw-base35.
Preparing to unpack .../04-fonts-urw-base35_20200910-1_all.deb ...
Unpacking fonts-urw-base35 (20200910-1) ...
Selecting previously unselected package libgs9-common.
Preparing to unpack .../05-libgs9-common_9.55.0~dfsg1-0ubuntu5.13_all.deb ...
Unpacking libgs9-common (9.55.0~dfsg1-0ubuntu5.13) ...
Selecting previously unselected package libidn12:amd64.
Preparing to unpack .../06-libidn12_1.38-4ubuntu1_amd64.deb ...
Unpacking libidn12:amd64 (1.38-4ubuntu1) ...
Selecting previously unselected package libijs-0.35:amd64.
Preparing to unpack .../07-libijs-0.35_0.35-15build2_amd64.deb ...
Unpacking libijs-0.35:amd64 (0.35-15build2) ...
Selecting previously unselected package libjbig2dec0:amd64.
Preparing to unpack .../08-libjbig2dec0_0.19-3build2_amd64.deb ...
Unpacking libjbig2dec0:amd64 (0.19-3build2) ...
Selecting previously unselected package libgs9:amd64.
Preparing to unpack .../09-libgs9_9.55.0~dfsg1-0ubuntu5.13_amd64.deb ...
Unpacking libgs9:amd64 (9.55.0~dfsg1-0ubuntu5.13) ...
Selecting previously unselected package libkpathsea6:amd64.
Preparing to unpack .../10-libkpathsea6_2021.20210626.59705-1ubuntu0.3_amd64.deb
...
Unpacking libkpathsea6:amd64 (2021.20210626.59705-1ubuntu0.3) ...
Selecting previously unselected package libwoff1:amd64.
Preparing to unpack .../11-libwoff1_1.0.2-1build4_amd64.deb ...
Unpacking libwoff1:amd64 (1.0.2-1build4) ...
Selecting previously unselected package dvisvgm.
Preparing to unpack .../12-dvisvgm_2.13.1-1_amd64.deb ...
Unpacking dvisvgm (2.13.1-1) ...
Selecting previously unselected package fonts-lmodern.
Preparing to unpack .../13-fonts-lmodern_2.004.5-6.1_all.deb ...
Unpacking fonts-lmodern (2.004.5-6.1) ...
Selecting previously unselected package fonts-noto-mono.
Preparing to unpack .../14-fonts-noto-mono_20201225-1build1_all.deb ...
Unpacking fonts-noto-mono (20201225-1build1) ...
Selecting previously unselected package fonts-texgyre.
Preparing to unpack .../15-fonts-texgyre_20180621-3.1_all.deb ...
Unpacking fonts-texgyre (20180621-3.1) ...
Selecting previously unselected package libapache-pom-java.
Preparing to unpack .../16-libapache-pom-java_18-1_all.deb ...
Unpacking libapache-pom-java (18-1) ...
Selecting previously unselected package libcommons-parent-java.
Preparing to unpack .../17-libcommons-parent-java_43-1_all.deb ...

```

```

Unpacking libcommons-parent-java (43-1) ...
Selecting previously unselected package libcommons-logging-java.
Preparing to unpack .../18-libcommons-logging-java_1.2-2_all.deb ...
Unpacking libcommons-logging-java (1.2-2) ...
Selecting previously unselected package libptexenc1:amd64.
Preparing to unpack .../19-libptexenc1_2021.20210626.59705-1ubuntu0.3_amd64.deb
...
Unpacking libptexenc1:amd64 (2021.20210626.59705-1ubuntu0.3) ...
Selecting previously unselected package rubygems-integration.
Preparing to unpack .../20-rubygems-integration_1.18_all.deb ...
Unpacking rubygems-integration (1.18) ...
Selecting previously unselected package ruby3.0.
Preparing to unpack .../21-ruby3.0_3.0.2-7ubuntu2.12_amd64.deb ...
Unpacking ruby3.0 (3.0.2-7ubuntu2.12) ...
Selecting previously unselected package ruby-rubygems.
Preparing to unpack .../22-ruby-rubygems_3.3.5-2ubuntu1.2_all.deb ...
Unpacking ruby-rubygems (3.3.5-2ubuntu1.2) ...
Selecting previously unselected package ruby.
Preparing to unpack .../23-ruby_1%3a3.0~exp1_amd64.deb ...
Unpacking ruby (1:3.0~exp1) ...
Selecting previously unselected package rake.
Preparing to unpack .../24-rake_13.0.6-2_all.deb ...
Unpacking rake (13.0.6-2) ...
Selecting previously unselected package ruby-net-telnet.
Preparing to unpack .../25-ruby-net-telnet_0.1.1-2_all.deb ...
Unpacking ruby-net-telnet (0.1.1-2) ...
Selecting previously unselected package ruby-webrick.
Preparing to unpack .../26-ruby-webrick_1.7.0-3ubuntu0.2_all.deb ...
Unpacking ruby-webrick (1.7.0-3ubuntu0.2) ...
Selecting previously unselected package ruby-xmlrpc.
Preparing to unpack .../27-ruby-xmlrpc_0.3.2-1ubuntu0.1_all.deb ...
Unpacking ruby-xmlrpc (0.3.2-1ubuntu0.1) ...
Selecting previously unselected package libruby3.0:amd64.
Preparing to unpack .../28-libruby3.0_3.0.2-7ubuntu2.12_amd64.deb ...
Unpacking libruby3.0:amd64 (3.0.2-7ubuntu2.12) ...
Selecting previously unselected package libsyntax2:amd64.
Preparing to unpack .../29-libsyntax2_2021.20210626.59705-1ubuntu0.3_amd64.deb
...
Unpacking libsyntax2:amd64 (2021.20210626.59705-1ubuntu0.3) ...
Selecting previously unselected package libteckit0:amd64.
Preparing to unpack .../30-libteckit0_2.5.11+ds1-1_amd64.deb ...
Unpacking libteckit0:amd64 (2.5.11+ds1-1) ...
Selecting previously unselected package libtexlua53:amd64.
Preparing to unpack .../31-libtexlua53_2021.20210626.59705-1ubuntu0.3_amd64.deb
...
Unpacking libtexlua53:amd64 (2021.20210626.59705-1ubuntu0.3) ...
Selecting previously unselected package libtexluajit2:amd64.
Preparing to unpack

```



```

.../32-libtexluajit2_2021.20210626.59705-1ubuntu0.3_amd64.deb ...
Unpacking libtexluajit2:amd64 (2021.20210626.59705-1ubuntu0.3) ...
Selecting previously unselected package libzip-0-13:amd64.
Preparing to unpack .../33-libzip-0-13_0.13.72+dfsg.1-1.1_amd64.deb ...
Unpacking libzip-0-13:amd64 (0.13.72+dfsg.1-1.1) ...
Selecting previously unselected package xfonts-encodings.
Preparing to unpack .../34-xfonts-encodings_1%3a1.0.5-0ubuntu2_all.deb ...
Unpacking xfonts-encodings (1:1.0.5-0ubuntu2) ...
Selecting previously unselected package xfonts-utils.
Preparing to unpack .../35-xfonts-utils_1%3a7.7+6build2_amd64.deb ...
Unpacking xfonts-utils (1:7.7+6build2) ...
Selecting previously unselected package lmodern.
Preparing to unpack .../36-lmodern_2.004.5-6.1_all.deb ...
Unpacking lmodern (2.004.5-6.1) ...
Selecting previously unselected package preview-latex-style.
Preparing to unpack .../37-preview-latex-style_12.2-1ubuntu1_all.deb ...
Unpacking preview-latex-style (12.2-1ubuntu1) ...
Selecting previously unselected package t1utils.
Preparing to unpack .../38-t1utils_1.41-4build2_amd64.deb ...
Unpacking t1utils (1.41-4build2) ...
Selecting previously unselected package teckit.
Preparing to unpack .../39-teckit_2.5.11+ds1-1_amd64.deb ...
Unpacking teckit (2.5.11+ds1-1) ...
Selecting previously unselected package tex-gyre.
Preparing to unpack .../40-tex-gyre_20180621-3.1_all.deb ...
Unpacking tex-gyre (20180621-3.1) ...
Selecting previously unselected package texlive-binaries.
Preparing to unpack .../41-texlive-
binaries_2021.20210626.59705-1ubuntu0.3_amd64.deb ...
Unpacking texlive-binaries (2021.20210626.59705-1ubuntu0.3) ...
Selecting previously unselected package texlive-base.
Preparing to unpack .../42-texlive-base_2021.20220204-1_all.deb ...
Unpacking texlive-base (2021.20220204-1) ...
Selecting previously unselected package texlive-fonts-recommended.
Preparing to unpack .../43-texlive-fonts-recommended_2021.20220204-1_all.deb ...
Unpacking texlive-fonts-recommended (2021.20220204-1) ...
Selecting previously unselected package texlive-latex-base.
Preparing to unpack .../44-texlive-latex-base_2021.20220204-1_all.deb ...
Unpacking texlive-latex-base (2021.20220204-1) ...
Selecting previously unselected package texlive-latex-recommended.
Preparing to unpack .../45-texlive-latex-recommended_2021.20220204-1_all.deb ...
Unpacking texlive-latex-recommended (2021.20220204-1) ...
Selecting previously unselected package texlive.
Preparing to unpack .../46-texlive_2021.20220204-1_all.deb ...
Unpacking texlive (2021.20220204-1) ...
Selecting previously unselected package libfontbox-java.
Preparing to unpack .../47-libfontbox-java_1%3a1.8.16-2_all.deb ...
Unpacking libfontbox-java (1:1.8.16-2) ...

```

```

Selecting previously unselected package libpdfbox-java.
Preparing to unpack .../48-libpdfbox-java_1%3a1.8.16-2_all.deb ...
Unpacking libpdfbox-java (1:1.8.16-2) ...
Selecting previously unselected package texlive-pictures.
Preparing to unpack .../49-texlive-pictures_2021.20220204-1_all.deb ...
Unpacking texlive-pictures (2021.20220204-1) ...
Selecting previously unselected package texlive-latex-extra.
Preparing to unpack .../50-texlive-latex-extra_2021.20220204-1_all.deb ...
Unpacking texlive-latex-extra (2021.20220204-1) ...
Selecting previously unselected package texlive-plain-generic.
Preparing to unpack .../51-texlive-plain-generic_2021.20220204-1_all.deb ...
Unpacking texlive-plain-generic (2021.20220204-1) ...
Selecting previously unselected package tipa.
Preparing to unpack .../52-tipa_2%3a1.3-21_all.deb ...
Unpacking tipa (2:1.3-21) ...
Selecting previously unselected package texlive-xetex.
Preparing to unpack .../53-texlive-xetex_2021.20220204-1_all.deb ...
Unpacking texlive-xetex (2021.20220204-1) ...
Setting up fonts-lato (2.0-2.1) ...
Setting up fonts-noto-mono (20201225-1build1) ...
Setting up libwoff1:amd64 (1.0.2-1build4) ...
Setting up libtexlua53:amd64 (2021.20210626.59705-1ubuntu0.3) ...
Setting up libijs-0.35:amd64 (0.35-15build2) ...
Setting up libtexluaajit2:amd64 (2021.20210626.59705-1ubuntu0.3) ...
Setting up libfontbox-java (1:1.8.16-2) ...
Setting up rubygems-integration (1.18) ...
Setting up libzip-0-13:amd64 (0.13.72+dfsg.1-1.1) ...
Setting up fonts-urw-base35 (20200910-1) ...
Setting up poppler-data (0.4.11-1) ...
Setting up tex-common (6.17) ...
update-language: texlive-base not installed and configured, doing nothing!
Setting up libjbig2dec0:amd64 (0.19-3build2) ...
Setting up libteckit0:amd64 (2.5.11+ds1-1) ...
Setting up libapache-pom-java (18-1) ...
Setting up ruby-net-telnet (0.1.1-2) ...
Setting up xfonts-encodings (1:1.0.5-0ubuntu2) ...
Setting up t1utils (1.41-4build2) ...
Setting up libidn12:amd64 (1.38-4ubuntu1) ...
Setting up fonts-texgyre (20180621-3.1) ...
Setting up libkpathsea6:amd64 (2021.20210626.59705-1ubuntu0.3) ...
Setting up ruby-webrick (1.7.0-3ubuntu0.2) ...
Setting up fonts-lmodern (2.004.5-6.1) ...
Setting up fonts-droid-fallback (1:6.0.1r16-1.1build1) ...
Setting up ruby-xmlrpc (0.3.2-1ubuntu0.1) ...
Setting up libsynchronet2:amd64 (2021.20210626.59705-1ubuntu0.3) ...
Setting up libgs9-common (9.55.0~dfsg1-0ubuntu5.13) ...
Setting up teckit (2.5.11+ds1-1) ...
Setting up libpdfbox-java (1:1.8.16-2) ...

```

```

Setting up libgs9:amd64 (9.55.0~dfsg1-0ubuntu5.13) ...
Setting up preview-latex-style (12.2-1ubuntu1) ...
Setting up libcommons-parent-java (43-1) ...
Setting up dvisvgm (2.13.1-1) ...
Setting up libcommons-logging-java (1.2-2) ...
Setting up xfonts-utils (1:7.7+6build2) ...
Setting up libptexenc1:amd64 (2021.20210626.59705-1ubuntu0.3) ...
Setting up texlive-binaries (2021.20210626.59705-1ubuntu0.3) ...
update-alternatives: using /usr/bin/xdvi-xaw to provide /usr/bin/xdvi.bin
(xdvi.bin) in auto mode
update-alternatives: using /usr/bin/bibtex.original to provide /usr/bin/bibtex
(bibtex) in auto mode
Setting up lmodern (2.004.5-6.1) ...
Setting up texlive-base (2021.20220204-1) ...
/usr/bin/ucfr
/usr/bin/ucfr
/usr/bin/ucfr
/usr/bin/ucfr
mktexlsr: Updating /var/lib/texmf/ls-R-TEXLIVEDIST...
mktexlsr: Updating /var/lib/texmf/ls-R-TEXMFMAIN...
mktexlsr: Updating /var/lib/texmf/ls-R...
mktexlsr: Done.
tl-paper: setting paper size for dvips to a4:
/var/lib/texmf/dvips/config/config-paper.ps
tl-paper: setting paper size for dvipdfmx to a4:
/var/lib/texmf/dvipdfmx/dvipdfmx-paper.cfg
tl-paper: setting paper size for xdvi to a4: /var/lib/texmf/xdvi/XDvi-paper
tl-paper: setting paper size for pdftex to a4: /var/lib/texmf/tex/generic/tex-
ini-files/pdftexconfig.tex
Setting up tex-gyre (20180621-3.1) ...
Setting up texlive-plain-generic (2021.20220204-1) ...
Setting up texlive-latex-base (2021.20220204-1) ...
Setting up texlive-latex-recommended (2021.20220204-1) ...
Setting up texlive-pictures (2021.20220204-1) ...
Setting up texlive-fonts-recommended (2021.20220204-1) ...
Setting up tipa (2:1.3-21) ...
Setting up texlive (2021.20220204-1) ...
Setting up texlive-latex-extra (2021.20220204-1) ...
Setting up texlive-xetex (2021.20220204-1) ...
Setting up rake (13.0.6-2) ...
Setting up libruby3.0:amd64 (3.0.2-7ubuntu2.12) ...
Setting up ruby3.0 (3.0.2-7ubuntu2.12) ...
Setting up ruby (1:3.0~exp1) ...
Setting up ruby-rubygems (3.3.5-2ubuntu1.2) ...
Processing triggers for man-db (2.10.2-1) ...
Processing triggers for mailcap (3.70+nmulubuntu1) ...
Processing triggers for fontconfig (2.13.1-4.2ubuntu5) ...
Processing triggers for libc-bin (2.35-0ubuntu3.13) ...

```

```

/sbin/ldconfig.real: /usr/local/lib/libtcm_debug.so.1 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libhwloc.so.15 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbmalloc_proxy.so.2 is not a symbolic
link

/sbin/ldconfig.real: /usr/local/lib/libur_adapter_level_zero.so.0 is not a
symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbbind_2_0.so.3 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbb.so.12 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libur_adapter_opencl.so.0 is not a symbolic
link

/sbin/ldconfig.real: /usr/local/lib/libur_adapter_level_zero_v2.so.0 is not a
symbolic link

/sbin/ldconfig.real: /usr/local/lib/libumf.so.1 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbmalloc.so.2 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbbind.so.3 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtcm.so.1 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libur_loader.so.0 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbbind_2_5.so.3 is not a symbolic link

```

```

Processing triggers for tex-common (6.17) ...
Running updmap-sys. This may take some time... done.
Running mktexlsr /var/lib/texmf ... done.
Building format(s) --all.
    This may take some time...

```

```

[ ]: # Explicitly setting the notebook filename for this execution,
# as there isn't a robust automatic detection method in Colab.
notebook_filename =
    ↪"MMALS_Prototype_1_1_RC2h_RC1b_Anchored_Conservative_Selector_Harness_Colab.
    ↪ipynb"
notebook_name_without_ext = os.path.splitext(notebook_filename)[0]

# Define the full path to the notebook in Google Drive

```

```

full_notebook_path_drive = f"/content/drive/MyDrive/Colab Notebooks/
↳{notebook_filename}"
local_notebook_path = f"/content/{notebook_filename}"

# Copy the notebook to the local /content/ directory to avoid path issues with
↳nbconvert
print(f"Executing: cp \"{full_notebook_path_drive}\" \"{local_notebook_path}\"")
!bash -c "cp \"{full_notebook_path_drive}\" \"{local_notebook_path}\""

```

```

[ ]: !jupyter nbconvert \
    --to pdf \
    --output-dir='/content' \
    --PDFExporter.latex_command=["'xelatex', '{filename}',
↳'-interaction=nonstopmode']" \
    "/content/
↳MMALS_Prototype_1_1_RC2h_RC1b_Anchored_Conservative_Selector_Harness_Colab.
↳ipynb"

```